Introduction to Programming Logic

I.S.W.B. Prasetya
Dept. of Information and Computing Sciences
Utrecht University
P.O.Box 80.089, 3508 TB Utrecht, the Netherlands

email: `wishnu@cs.uu.nl`
URL: `www.cs.uu.nl\staff\wishnu.html`

Contents

1 Introduction **7**

2 Formulas **11**

3 Logic and Proofs **17**
3.1 Style . . . 18
3.1.1 The Implicit G Abbreviation . . . 20
3.2 Inference Rule . . . 20
3.2.1 Substitution . . . 21
3.3 Definition . . . 22
3.4 Rewriting . . . 22
3.5 Subproof . . . 24
3.6 Equational Proof . . . 24
3.7 Handling ∀ and ∃ . . . 26
3.7.1 Inferring a ∀-Formula . . . 26
3.7.2 Eliminating ∃ . . . 27
3.8 Some Useful Proof Techniques . . . 28
3.9 Exercise . . . 32
3.10 Solution . . . 34

4 Lists **37**
4.1 Basic Notation and Properties . . . 38
4.2 Proofs . . . 41
4.3 Exercises . . . 43
4.4 Solution . . . 44

5 Program and Specification **47**
5.1 Program . . . 47
5.2 Specification . . . 50
5.3 Exercises . . . 55
5.4 Solution . . . 56

6 The Basic Proof System **59**
6.1 Abstract Model . . . 59
6.1.1 State . . . 60
6.1.2 Modelling Program as Relation . . . 60
6.1.3 Predicate as a Set of States . . . 61
6.1.4 The Formal Meaning of Hoare Triples . . . 62
6.1.5 Expressing Non-termination . . . 63
6.2 Weakest Pre-condition . . . 63
6.3 Loop Reduction Strategy . . . 70
6.4 Organizing Your Proof . . . 73
6.5 Example: Fibonacci Problem . . . 76
6.5.1 Step 1: Proof Plan . . . 78
6.5.2 Step 3: Cross Checking with `Init` . . . 79
6.5.3 Step 4: Verifying Invariance . . . 80
6.5.4 Step 5: Loop Termination . . . 83
6.5.5 Step 6: Initialization . . . 84
6.5.6 Summarizing . . . 84
6.6 On Iteratively Constructing `I` . . . 85
6.7 Linear Scanning . . . 86
6.7.1 Constructing Invariant . . . 86
6.7.2 Loop Termination . . . 87
6.7.3 Proving Invariance . . . 89
6.7.4 Initialization . . . 93
6.8 Invariants . . . 94
6.8.1 Head and Tail Invariant . . . 94
6.8.2 Loop for Searching a Solution . . . 96
6.8.3 Iterative Implementation of Tail Recursion . . . 96
6.9 Proof of the Distributivity of `wp` . . . 98
6.10 Simultant Assignment . . . 99
6.11 Reducing Program's Specification . . . 100
6.12 Updating Compound Structures . . . 101
6.12.1 Updating Record . . . 103
6.13 Advanced Features . . . 104
6.13.1 `assert` and `assume` . . . 104
6.13.2 Non-deterministic Assignment . . . 105
6.13.3 Non-deterministic Choice . . . 105
6.13.4 goto . . . 105
6.13.5 Inner block . . . 106
6.13.6 Expression with side effect . . . 106
6.13.7 Exception . . . 106
6.13.8 Alias . . . 108
6.14 Exercise . . . 110
6.15 Solution . . . 115

7 Program Call **133**
7.1 Restrictions . . . 134
7.2 Formal Treatment of Program Call . . . 135
7.2.1 Transforming Program Call . . . 136
7.2.2 Black Box Reduction . . . 137
7.3 Functional Black Boxes . . . 140
7.4 Recursion . . . 142
7.5 Exercise . . . 145
7.6 Solution . . . 146

8 Abstract Data Type **151**
8.1 ADT with uPL . . . 152
8.2 Extending ADT . . . 153
8.3 Implementing ADT . . . 153
8.4 Validation . . . 156
8.5 Exercise . . . 160
8.6 Solution . . . 161

9 Incremental Proof **165**
9.1 Splitting Specifications . . . 166
9.2 Superposition . . . 167
9.3 Breaking a Loop . . . 171
9.4 Exercise . . . 173
9.5 Solution . . . 175

10 Exercises Collection **179**

A Standard Rules and Facts **191**
A.1 Predicate Logic Inference Rules . . . 191
A.2 Rewriting . . . 192
A.2.1 Rewrite with a Valid Equality . . . 192
A.3 Inductions . . . 192
A.4 Predicate Logic Rewrite Theorems . . . 193
A.4.1 Boolean Connectors . . . 193
A.4.2 Conditional . . . 193
A.4.3 Quantification . . . 193
A.4.4 Domains . . . 194
A.5 Lists . . . 195

B The uPL Logic **197**
B.1 Inference Rules . . . 197
B.2 Weakest Pre-condition . . . 198
B.3 Program Call . . . 199
B.4 Program Transformation . . . 200

C uPL Grammar **203**

CHAPTER 1

Introduction

Computers are probably the most revolutionary and successful invention in our history. In the last few decenia, they have played a dominant role in creating the modern world that we know now. A world that in terms of productivity and comfort far exceeds the imagination of our ancestors. Increasingly, computers are becoming an integral part of our modern culture. They are everywhere, aiding us in every activity. What most people do not realize, is that dependency always has its other side. More and more aspects of our life are now controlled by computers. Some mistakes or failures made by the computers may just be annoying, but some may have a disastrous consequence for someone's life. The danger comes from the way people produce software. The behavior of even a simple program is often very complex. The depth of its complexity far exceeds that of more traditional engineering products, like radios, cars, or bridges. On the other hand, for more than a century we have known a golden age of our industry. People have been successful in manufacturing and building almost anything, from paper clips to skyscrapers. This develops a certain style of engineering, and not surprisingly the same style is used in developing software.

Unfortunately, the traditional engineering approach is not suitable for dealing with the magnitude of complexity that we have to deal with in the software world. This often results in unreliable software. It is true that there are many applications where occasional failures are acceptable, but there are also applications where this is highly unacceptable. Think of the control systems of traffic lights, aeroplanes, or nuclear reactors. Think also of the protocols that allow internet access to bank accounts. These are applications whose failure may cause huge material or financial damage or even loss of human life. Such an application or system is often called *critical system*. It is obvious that such a system has to be developed with a lot of care.

To safe guard the quality of a program, what people do is 'testing' it. That is, we run the program on a number of real inputs and observe whether its behavior conforms to its specification. The approach is fast and cheap. You can find a lot of errors with testing, but unfortunately you cannot find them all. This is simply because in general you cannot test *all* possible inputs. E.g. the set of all possible inputs of a small program that takes a single integer as a parameter is theoretically infinitinely large. Even if we use the fact that integers are in the computer represented by a string

of e.g. 32 bits, there are still 10^{32} possible integers! It is not feasible to test all these possibilities. The fact is, testing is very incomplete. For critical systems this may not be a satisfactory solution.

This book introduces you to an alternative technique, called *program verification*. It uses mathematical reasoning to verify the correctness of a program. Its use requires a lot more effort than testing, but on the other hand the coverage is complete, which means that using this technique guarantees that a program to behave as specified with respect to all inputs.

This is not to say that program verification is the ultimate solution for writing correct software. Real applications are huge. Writing formal specifications for them and subsequently verifying them will be very expensive, even with the help of computers. However the technique discussed here can still be effectively applied to slices or models of real programs. Because, within its range, verification is always thorough, it often finds bugs that ordinary testing misses.

An example

To give you a glimpse to some of the issues we are going to discuss, let us consider the following small program to calculate the greatest common divisor of two positive integers.

An integer d is said to be a *divisor* of another integer x if it can divide x (so, 3 is a divisor of 6, but not of 7). This d is a *common* divisor of x and y if it is a divisor of x and of y. Let us denote the *greatest* common divisor of two integers x and y by `gcd` x y. For example, `gcd` $2\ 6 = 2$ and `gcd` $6\ 9 = 3$.

The straight forward way to calculate `gcd` x y is to try out all possible integers d between 2 and $x/2$ (assuming $x < y$) to find out which ones are common divisors, and then compute the maximum. However, there is a faster way to do it. This is shown in Figure 1.1 —this solution has been around for more than 2000 years. It is invented by an ancient Greek mathematician, Euclides in ± 320 BC. You may have seen this program before, but let us pretend that this is the first time we see it and we are given the task to check if it is correct.

```
Euclid(x,y : int) : int {
   while x<>y do if x>y then x:=x-y else y:=y-x ;
   return x
}
```

Figure 1.1: A program to compute the greatest common divisor.

Specification

Well, what we can do is to test the program. But even to test we first need a *specification*, which is a description of what the program is expected to do. When we test a program, we test if it meets its specification.

Ideally, a specification should *not be ambiguous*, or else testers and other programmers may misunderstand how to use your program and the consequence may be undesirable. Human languages (like English or Japanese) are unfortunately often ambiguous. So, they are actually not suitable for writing specifications. Instead, you should use a so-called *specification language*. Like programming languages, such a language have a strict syntax. This helps to avoid most ambiguity. Some even have formal descriptions of their semantics which exclude ambiguity. Here we will use a very simple specification language. Its original style is invented by C.A.R. Hoare back in 1969 [13]. In the Hoare style, we write a specification like this:

```
{* x>0 ∧ y>0 *}

Euclid(x,y)

{* return = gcd x y *}
```

It says that if the value of `x` and `y` are positive, then the call `Euclid(x,y)` will return a value which is equal to `gcd x y`.

The expressions between between `{*` and `*}` are called *assertions*. The one preceding the program is called the *pre-condition* of the program, and the one after the program is called the *post-condition*. The triple of pre-condition, program, and post-condition is often called *Hoare triple*. This style of specification is used in many verification approaches.

Proof

In program verification, rather than testing we try to *prove* that the `Euclid` is correct. So, how can we do that?

We proceed in steps. First, rather than proving that `return` = `gcd x y` we can equivalently try to prove that the loop in `Euclid` terminates in this condition:

$$\texttt{x} = \texttt{gcd X Y} \tag{1.1}$$

where `X` and `Y` represent the initial values of `x` and `y`. The strategy now is to come up with some general property about `x` and `y` which will be maintained at the end of every iteration; e.g. this property:

$$\texttt{x>0} \ \wedge \ \texttt{y>0} \tag{1.2}$$

Such a property is also called *loop invariant* or simply *invariant*. Because it holds at the end of every iteration, it also holds when the loop terminates. So we may be able to use it to prove what we want, namely (1.1). We need to find a good invariant though. Unfortunately (1.2) does not contain enough information to prove (1.1). In general, finding the right invariant is not trivial.

We are not going to prove it here, but you can actually show that every iteration also maintains this:

$$\texttt{gcd x y} \ = \ \texttt{gcd X Y} \tag{1.3}$$

Note that when the loop terminates `x=y` holds. Now, what would be the `gcd` of `x` and `y` if both are positive and equal? Well, `x` itself of course! (e.g. `gcd 6 6` = `6`) So, after the last iteration we have:

$$\texttt{gcd x y} = \texttt{x} \tag{1.4}$$

Together with (1.3) this now implies that `x` = `gcd X Y` holds when the loop ends. So, we are done.

We did skip few things in the proof above, such as the proof of (1.3) and the loop will actually terminate. We will show you the complete proof of `Euclid` later. The important thing to note here is that a proof such as the one above made no particular assumption about the initial values of `x` and `y`, other than, in this case, that they have to be positive. Consequently, the proven correctness is guaranteed with respect to *all* possible (positive) inputs of the program!

Proof: Informal or Formal?

The proof in the previous section is an *informal proof*, because it is mostly written in plain English. Such a proof is often ambigous and incomplete. Because we can just express anything in plain English, an informal proof is also error prone. Nevertheless, it allows us to present a proof abstractly, thus making it easy to understand. Proofs of rules and theorems in this book are presented in a mix of formal and informal parts. The main purpose of those proofs is to improve your understanding.

When verifying a real program we would have a very different purpose. The main purpose of verification is to check the program's correctness, and not so much to try to understand why the program is correct. Of course if it does help your understanding then it is also good. Anyway, because a proof itself is also prone to errors, for verification we need a to construct a *formal proof*. Such a proof must follow a certain set of rules that define precisely which proof steps you are allowed to use. This set of rules is called *logic*. It is usually also straight forward to write a program to check if a formal proof follows its logic; thus, it contains no error. Of course having those rules do mean that we are no longer free to write a proof in any way that is convenient to us; so, formal proofs do tend to be longer, more verbose, and are more difficult to write.

Can we also program a computer to construct (formal) proofs for us? Unfortunately, it has been shown that in general this is impossible. So, our human insight is still needed. In various limited cases, automatic proof construction is possible. Proof automation is also a very active research field. The number of tools to do it keeps growing and they are also getting smarter. The use of proof tools is however outside the scope of this book. We will focus more on learning the basic of programming logics. For this purpose the use of tools is not really necessary.

To use a logic effectively it is not enough to know its rules: you also have to understand their principles. If your attemp to prove the correctness of a program fails, it can be because the program is incorrect, or because your proof strategy is not good. In either case, you will have to analyze the proof to discover where it starts to go wrong, why it goes wrong, and what options you have to fix it. You cannot do this without a good understanding of how the logic works.

A skill that you also need to develop here is the skill to write a proof *formally*, because this is the kind of proofs that you have to deal with in an actual setup. Training it requires some discipline. However, as a programmer this should not be new to you: writing Java programs also requires discipline. You cannot deliver a program with a bad syntax. The compiler won't accept it, so it won't run. The same kind of discipline applies to formal proofs. If you write a formula in the wrong way, or use the logic in a wrong way, your proof is flawed, which is just as useless as a program with a bad syntax.

Despite all the above emphasis on formal proofs it is a mistake to dismiss the value of informal proofs. Despite the ambiguity and lack of precision, an informal proof is a good tool to abstractly *document* our insight and high level proof strategies. For verification there is no way around: you need a formal proof.

□□□

New Vocabulary

ambigous/unambigous; assertion; formal/informal proof; Hoare triple; invariant; post-condition; pre-condition; specification; specification language; testing; verification

CHAPTER 2

Formulas

Expressions are texts like x=0 and `x + 1>0`. In programming expressions are used in assignments and to guard loops and conditional statements. In logics they are also called *formulas*. We have seen in Chapter 1 that formulas are used to specify the allowed initial and final states of a program. Simple formulas like above may be sufficient to specify simple programs. A more complicated program, such as a program to check if an array contains a zero, may however need more sophisticated formulas to specify. This chapter will introduce you to the language we will use to write our formulas. We will simply call it Form. Form generally follows the style common in the presentation of predicate logic e.g. [7, 11, 10, 6, 16]. It is a small language, but is sufficient for our purpose. For industrial use people use larger specification languages such as Z [8] or JML [18].

Operators

In the simplest form, a formula[1] is just a constant like 1, 2, 3, or a variable like `x` and `y`. Boolean constants are denoted by `true` and `false`. A formula can also be constructed with operators, like in `x + 1`. Allowed binary operators are shown in Table 2.1. They are listed in the decreasing order of their priority. For example, $\wedge$ has a higher priority (binds stronger) than $\vee$. So, $\mathtt{p} \wedge \mathtt{q} \vee \mathtt{r}$ is read as $(\mathtt{p} \wedge \mathtt{q}) \vee \mathtt{r}$. Operators listed in the same row has the same priority. We also have the unary operator $\neg$ (negation); it has a higher priority than any binary operator.

Note that the symbol = denotes the operator to test equality and that = has the lowest priority[2]. We also allow some list operators: `:`, $+\!\!+$, and $\in$. They will be explained in Chapter 4.

The L and R flags in Table 2.1 indicate the 'associativity' of an operator. L means that the operator is *left associative*, and R means that it is *right associative*. For example, `-` is left

[1] In literature e.g. [15] people distinguish, for a technical reason, between Boolean and non-Boolean formulas. The latter are called *terms*. We will ignore this distinction as it is not necessary for our discussions.

[2] In particular, this is unlike in some programming languages, e.g. Java, where = denotes assignment and where the operator used to test equality (usually `==`) has a higher priority.

^ (L)
* (LR)
+ (LR), - (L), ++ (LR)
mod, div, max (LR), min (LR), : (R)
$<, >, \leq, \geq, \in$
$\wedge$ (LR)
$\vee$ (LR)
$\Rightarrow$ (R)R)
$=, \neq$

Table 2.1: *Binary operators in* Form *and their associativity. The operators are listed in decreasing priority.*

associative, meaning that $\mathtt{x}-\mathtt{y}-\mathtt{z}$ is read as $(\mathtt{x}-\mathtt{y})-\mathtt{z}$. On the other hand, $\Rightarrow$ is right associative, meaning that $\mathtt{p} \Rightarrow \mathtt{q} \Rightarrow \mathtt{r}$ is read as $\mathtt{p} \Rightarrow (\mathtt{q} \Rightarrow \mathtt{r})$. Operators which are marked with LR, like +, are both left and right associative. It means that $\mathtt{x}+\mathtt{y}+\mathtt{z}$ can be read as either $(\mathtt{x}+\mathtt{y})+\mathtt{z}$ or $\mathtt{x}+(\mathtt{y}+\mathtt{z})$. The operators with no associativity flag are *not* associative. For example, = is not associative in Form, which means that you cannot write `x = y = z`, because this is not meaningful in Form.

Note that in $\Rightarrow$ is right associative and its priority is less than $\wedge$ and $\vee$. So, for example:

1. $\mathtt{p} \Rightarrow \mathtt{q} \Rightarrow \mathtt{r}$ means $\mathtt{p} \Rightarrow (\mathtt{q} \Rightarrow \mathtt{r})$.

2. $\mathtt{p} \wedge \mathtt{q} \Rightarrow \mathtt{r}$ means $(\mathtt{p} \wedge \mathtt{q}) \Rightarrow \mathtt{r}$.

3. $\mathtt{p} \vee \mathtt{q} \Rightarrow \mathtt{r}$ means $(\mathtt{p} \vee \mathtt{q}) \Rightarrow \mathtt{r}$.

The mod and div Operators

For integers `x` and `y`, the expression `x mod y` returns the integer remainder of dividing `x` by `y`; whereas `x div y` returns the quotient. If `x` and `y` are both positives the meaning is quite clear. When they are negatives different languages may define the resuls differently. Here, our interpretation is as follows.

If $\mathtt{y} \neq 0$, then `x mod y` and `x div y` are defined to be integers `r` respectively `q`, such that:

$$0 \leq \mathtt{r} < |\mathtt{y}| \quad \wedge \quad \mathtt{x} = \mathtt{q}\mathtt{y} + \mathtt{r} \tag{2.1}$$

So, -5 `mod` 3 is 1, and -5 `mod` -3 is also 1. However, 5 `mod` -3 is 2.

Functions

You can define your own functions in Form, like:

$$\mathtt{xor\ x\ y} \quad = \quad (\mathtt{x} \vee \mathtt{y}) \wedge (\mathtt{x} \neq \mathtt{y})$$

However functions in Form are pure mathematical functions; they are *not* regarded as programs. They are introduced purely for the purpose of specifying programs, for example to express that a program implements a certain mathematical function. Functions application is written like `f x y` instead of the usual $\mathtt{f}(\mathtt{x},\mathtt{y})$, and has the highest priority. For example, `f x y` $+$ 1 is read as (`f x y`) $+$ 1.

Array

In imperative programming we often deal with arrays so we also want to able to specify properties about them. We will use the usual array notation: if `a` is an array `a[i]` denotes the `i`-th element of `a`. Just to simplify reasoning we will use idealized arrays. They are *of infinite size*; the indices run from $-\infty$ to $+\infty$. So, you we do not have to worry about the meaning of `a[i]` if i turns out to be beyond `a`'s domain of indices. When dealing with real programs we should not of course naively ignore this aspect.

Integer domain expressions

An integer domain expression, or simply a *domain expression*, is the term we will use for an expression like `a≤i<b` and all its variations with different numeric relations; `a`, `i`, and `b` are of the integer type. They are abbreviations; for example:

1. `a≤b<c` abbreviates `a≤b ∧ b<c`.

2. `a≤i, j<c` abbreviates `a≤i<c ∧ a≤j<c`.

The same abbreviation scheme is used for other combinations of $<$, $>$, $\leq$, and $\geq$.

Conditional

A conditional formula is a formula of the form $g \rightarrow e_1 \mid e_2$. The g has to of type Boolean. If g is true then the meaning of this formula is equal to e_1 and else equal to e_2.

You may also write a conditional like this: `if` g `then` e_1 `else` e_2. Some people prefer this notation because of its familiar construct. If you use it, don't confuse it with the `if-then-else` used in imperative programming languages. In the latter `if-then-else` is an action that can change the state of a program. In Form `if-then-else` is a formula; it specifies a value and does not change anything.

Quantified formula

A *quantified formula* is a formula like (∀`k : 0≤k<N : a[k]>0`). It means that for all `k` such that `0≤k<N` the formula `a[k]>0` holds. A formula of this form is also called a *universally quantified* expression.

Another example of a quantified expression is (∃`k : 0≤k<N : a[k]>0`), which means that there exists a `k` in the domain `0≤k<N` such that `a[k]>0` holds. An expression of this form is also called an *existentially quantified* formula.

The symbols ∀ and ∃ are called *quantifiers* (also called *quantors* or *binders*). The formula between the two colons in a quantified formula is called the *domain* of the formula. For example the domain of (∀`k : 0≤k<N : a[k]>0`) is `0≤k<N`. A domain is said to be *full* when it is equivalent to `true`. It is said to be *empty* when it is equivalent to `false`. For example, (∀`k : 0≤k<0 : a[k] > 0`) has an empty domain. When its domain is empty a quantified expression has a 'special' meaning:

```
(∀i : false : P i) = true

(∃i : false : P i) = false
```

It actually makes sense. For example in the case of ∃: there is of course no `i` that can satisfy `false`. In other words, the domain allows no `i`, let alone an `i` satisfying `P i`.

We also write (∀`i :: P i`) (notice that the domain formula is omitted) as a shorthand for (∀`i : true : P i`). Similarly, (∃`i :: P i`) means (∃i : `true` : $P\ i$). Do keep in mind that omitting the domain specification in a quantified formula is *not* the same as quantifying over an empty domain.

You can also quantify over multiple 'bound variables', e.g. $(\forall \mathtt{i}, \mathtt{j} : \mathtt{0} \leq \mathtt{i}, \mathtt{j} < \mathtt{n} : \mathtt{a}[\mathtt{i}] = \mathtt{a}[\mathtt{j}])$, which says that for all `i` and `j` in the specified domain the formula $\mathtt{a}[\mathtt{i}] = \mathtt{a}[\mathtt{j}]$ holds. You can also have nested quantifications, e.g:

$$(\forall \mathtt{i} : \mathtt{0} \leq \mathtt{i} < \mathtt{n} : (\forall \mathtt{j} : \mathtt{i} \leq \mathtt{j} < \mathtt{n} : \mathtt{a}[\mathtt{i}] \leq \mathtt{a}[\mathtt{j}]))$$

(so what does the formula above say in plain English?)

The scope of quantification

In a quantified formula like $(\forall \mathtt{k} : \mathtt{0} \leq \mathtt{k} < \mathtt{N} : \mathtt{a}[\mathtt{k}] > \mathtt{x})$, the 'variable' `k` is called a *bound variable*. Its scope is limited within the quantified formula. So, if we have an expression like:

$$(\forall \mathtt{k} : \mathtt{0} \leq \mathtt{k} < \mathtt{N} : \mathtt{a}[\mathtt{k}] > \mathtt{x}) \ \wedge \ \mathtt{k}$$

the `k` outside the quantification refers to something different than the `k` inside the quantification. Variables which are not bound are called *free*. For example, `x` and `N` in the above formula are free.

The term 'variable' is indeed a bit confusing. We actually have two concepts of 'variable' here: 'variable' in mathematics and 'variable' in programming. In a mathematical formula such as $x > 0 \Rightarrow x \geq 0$, the x is commonly called a variable. It is just a word mathematicians use for a place holder representing an arbitrary value. In most mathematical books, such a variable represents some abstract concept that has nothing to do with programs.

In formal specifications, we use mathematical formulas to specify what a program is supposed to do. So, you can expect that some variables in a specification represent actual variables of a program. For example `x` and `y` in the specification of the program `Euclid` from the previous chapter refer to `Euclid`'s variables of the same names. So, they are program variables. From the mathematical point of view, a program variable occurring in a specification is just a special kind of a mathematical variable that also represents a physical place in the computer's memory where one can store an arbitrary value. When writing a specification we are thus not limited to just using program variables. E.g. the variables `X` and `Y` we introduced in the informal proof of `Euclid` are not program variables. They are variables that refer to arbitrary integers. A bounded variable is also an example of a variable which is not a program variable. We are allowed to use these kinds of variables so that we can conveniently express our intent in a specification.

Shifted domain

When you have a universally or existentially quantified expression you can shift some part of its domain like this:

1. $(\forall i : P\ i \wedge Q\ i : R\ i)$ is equivalent to $(\forall i : P\ i : Q\ i \Rightarrow R\ i)$
2. $(\exists i : P\ i \wedge Q\ i : R\ i)$ is equivalent to $(\exists i : P\ i : Q\ i \wedge R\ i)$

You can also shift the entire domain like this:

1. $(\forall i : P\ i : Q\ i)$ is equivalent to $(\forall i :: P\ i \Rightarrow Q\ i)$
2. $(\exists i : P\ i : Q\ i)$ is equivalent to $(\exists i :: P\ i \wedge Q\ i)$

Example

As an example, below we show a program. Form formulas are used to specify the program's pre- and post-conditions. The specification says that when the the program `ELEM` is called in a state satisfying $\mathtt{n} \geq \mathtt{0}$, it will terminate in a state where:

$$\mathtt{return} \ = \ (\exists\, \mathtt{k} : \mathtt{0} \leq \mathtt{k} < \mathtt{n} : \mathtt{a}[\mathtt{k}] = \mathtt{x})$$

In other words, `ELEM` tests whether `x` is an element of the array `a` in the domain $\mathtt{0} \leq \mathtt{k} < \mathtt{n}$.

```
{* n≥0 *}

ELEM(x,n:int, a:int[]) : bool
  { var i   : int ;
    var found : bool ;
    i:=0
    found:=false ;
    while i<n ∧ ¬found do { found:=(a[i]=x) ; i:=i+1 } ;
    return found
  }

{* return = (∃ k : 0≤k<n : a[k] = x) *}
```

□□□

New Vocabulary

associativity: left,right,non-; conditional; domain expression; quantified formula: universal, existential, domain, empty domain, full domain, without domain; quantifier; shifting domain; variable: bound, free

CHAPTER 3

Logic and Proofs

A logic is essentially just a collection of rules for inferring a formula from a set of other formulas. If a formula q can be inferred from another formula p by using a one rule or a composition of rules then we say that q *follows from* p. The sequence of the rules used to do so is our *proof*. A logic is also a *formal system*: it describes its derivation rules *precisely*, so that there is no ambiguity as to when a rule can be applied, and what the results are. Proposition and predicate logics are examples of logics. There are many more logics —some are very general, others are specialized to specific purposes, e.g. to abstractly reason about security protocols. Even proposition and predicate logics have many variants. Later we will also use a logic to prove the correctness of a program. Such a logic is called *programming logic*.

There are also many programming logics, but often they are built on top of a *predicate logic*. The pure programming logic part is used to prove properties about executions; the underlying predicate logic is used to prove properties about data. An example of a data-level property is this:

> *The first element of ascendingly sorted array, is also its minimum.*

This property holds for any array, and has nothing to do with program execution. The technique that we will use (Chapter 6) for proving program correctness works in fact by reducing a pair of program and specification to a set of data-level formulas. After the reduction we can fall back to the predicate logic to finish the proof.

This chapter will show you the style used in this book to prove predicate logic formulas. When you look in the literature, don't be surprised to find many different proof styles. Which one to use is often a matter of taste; however, some styles are less suitable for introductory training (e.g. too low level, or use difficult notations). We will use a variation of the flag-pole style used in [21]. We will explain some of the basic proof rules. However, this book is not intended to be a complete introduction to the predicate logic —we assume you have been sufficiently exposed to it. For your convenience, the Appendix lists commonly used inference rules and equalities in predicate logic.

Basic Terminology

In mathematics a function that returns a Boolean value is called *predicate*. Here the notion corresponds to Form -formula of type Boolean, since such a formula can be seen as defining a function over its free variables as parameters.

A predicate p is *universally true*, also called *valid*, when it is true for all possible values (of the correct types) of its *free* variables. For example, this predicate:

$$\mathtt{x} > 0 \Rightarrow \mathtt{x} \geq 0 \tag{3.1}$$

is true for all possible values of x, and therefore it is valid. On the other hand, $\mathtt{x} > 0$ is not a valid predicate, because it is not true when $\mathtt{x} = 0$.

A valid predicate usually captures a useful fact. To explicitly indicate that an expression p is valid, we write it like this: $\vdash p$. When we loosely say "*proving a formula p*" we actually mean proving that p is valid. So, it as if p has an implicit outmost-level $\forall$-quantifier over its free variables. We also say "*proving q follows from p*" to mean proving $\vdash p \Rightarrow q$.

Other frequently used notions are the notions of stronger and weaker. If $\vdash p \Rightarrow q$ holds, then we say that p is *stronger* than q. Conversely, q is *weaker* than p. Notice that when p is valid, so is any q which is weaker than p. Therefore, if we know that if p is stronger than q, then proving $\vdash p$ is sufficient to prove $\vdash q$.

3.1 Style

The proof style that we are going to use is most suitable for proving expressions of this form:

$$(p_1 \wedge \ldots \wedge p_n) \Rightarrow q \tag{3.2}$$

or this form:

$$(\forall v_i, \ldots v_k : p_1 \wedge \ldots \wedge p_n : q) \tag{3.3}$$

Notice that (3.3) can also be written as $(\forall v_i, \ldots v_k :: \ p_1 \wedge \ldots \wedge p_n \Rightarrow q)$. So it is just a quantified form of (3.2). Any predicate can be converted to one of those forms.

As an example, let us prove that this formula is valid:

$$\begin{array}{l} (\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{n} : \mathtt{a}[\mathtt{i}] > 0) \\ \wedge\ \mathtt{n} > 0 \\ \wedge\ \mathtt{x} > 0 \\ \Rightarrow \\ \mathtt{x} * \mathtt{a}[0] > 0 \end{array} \tag{3.4}$$

Intuitively, this is quite easy:

> The universal quantification at the left hand side of the $\Rightarrow$ says that $\mathtt{a}[\mathtt{i}] > 0$, for all i in the specified range. Since $\mathtt{n} > 0$, this range is not empty. In particular, this implies that $\mathtt{a}[0] > 0$. Since $\mathtt{x} > 0$, then it follows that $\mathtt{x} * \mathtt{a}[0] > 0$.

This is still an informal proof. Figure 3.1 shows how we write this more formally. As you can see, a proof consists of three parts: its name, declaration, and body. The proof in Figure 3.1 is named `top`. Name is optional. The part between the name and the keyword `BEGIN` is the declaration part, and the part between `BEGIN` and `END` is the body.

In the declaration part you list what your assumptions are, and the formulas that you want to prove. The latter are called *goals*. Entries in the declaration can be labelled (e.g. `A1`, `A2`, `G` and so on) so that we can refer to them. An entry whose label begins with an `A` is an assumption. In the proof above you see three assumptions: `A1`, `A2`, and `A3`. An entry whose label begins with a `G` is a goal.

```
PROOF top
[A1:] (∀i : 0 ≤ i < n : a[i] > 0)
[A2:] n > 0
[A3:] x > 0
[G :] x ∗ a[0] > 0
BEGIN ____________________________________________________
  1 { trivially follows from A2 }
    0 ≤ 0 < n
  2 { ∀-elimination on A1 using 1, taking i = 0 }
    a[0] > 0
  3 { trivially follows from A3 and 2 }
    x ∗ a[0] > 0
END ______________________________________________________
```

Figure 3.1: *This is how we write a formal proof in this book.*

Consider a proof with $A_1 \ldots A_n$ specified as the assumptions and g as a goal; the total formula you try to prove is:

$$A_1 \wedge \ldots \wedge A_n \Rightarrow g$$

If the proof is successful, that is what it proves[1]. You can also specify multiple goals. A proof is then successfull if *all* of its goals are proven.

Recall that a logic is a system to infer new formulas from other formulas. This is what you do in a proof's body. So, in the body you can write down new 'facts' (that is, formulas) which are inferred from the assumptions, or from the formulas you derived previously. In this way, all formulas you write down in a proof are logical consequences of the assumptions. A proof is therefore sucessfull if you manage to derive all its goals somewhere in the proof body. Later, you may also open a subproof within a proof; things become a bit more complicated then, but we will return to this later.

We will call the set of assumptions plus the facts derived so far within a proof the *proof-context* or simply *context* of the proof. This context grows as you add more facts into a proof. What we said in the previous paragraph can also be rephrased in terms of proof-context: *we can only extend a proof with a new formula if it can be inferred from the current proof-context.* Note that if you have subproofs, the context does not include the formulas derived in subproofs. It does include formulas derived previously in the ancestor proofs.

It is convenient to number the inferred formulas in a proof body, so that we can refer to them by their numbers. The text between { and } (printed blue, if you have a color version of this book) is a hint of how the formula below it is inferred. For example, in the proof in Figure 3.1 the first expression derived is $0 \leq 0 < n$. The hint above it says that this follows trivially from A2.

The next question is, how do we infer a new fact? Strictly speaking we have to use a rule from whatever logic we use. Here it would be the predicate logic. But since we also use integers, floats, strings, and lists, we additionally also need logics about each of those data types. Since you would be familiar with simple reasoning about e.g. integers and simple boolean operators, we will allow some degree of informality. E.g. it is acceptable to consider the fact that $n = 0$ implies $n>0$ to be trivial and use this to justify a proof step as we do in step 1 and 3 in Figure 3.1. Afterall, proving these kind of little facts are not what what we mainly want to learn here. In the real setup you of

[1] Be careful: if you have assumptions, a successful proof does not imply the validity of the goal. What you prove is that the assumptions imply the goal.

course cannot say this to a proof tool. If it wants a proof, then you have to deliver it. There are however many tools that can help you to automate the proving of simple facts.

3.1.1 The Implicit G Abbreviation

The declaration of a goal, e.g. as in Figure 3.1:

```
[G:] x * a[0] > 0
```

also implicitly introduces the variable `G` as an abbreviation for the specified goal. If you have a large goal, this is convenient to avoid having to copy the entire formula in your proof steps. E.g. the proof in Figure 3.1 could also be written:

```
PROOF top
...
[G:] x * a[0] > 0
BEGIN ____________________________________________________
  1 ...
  2 ...
  3 { trivially follows from A3 and 2 }   G
END ______________________________________________________
```

3.2 Inference Rule

Another way to infer a formula, is to use an *inference rule*. Predicate logic has a number of rules Here is one called the Conjunction Rule (also called $\wedge$ Introduction Rule):

$$\frac{\begin{array}{l} P \\ Q \end{array}}{P \wedge Q} \tag{3.5}$$

It is read like this: if P and Q are in the current proof-context, then you can infer $P \wedge Q$ (and add it in your proof). The formulas above the line are also called the *premises* of the rule. The formula under the line is called the *conclusion*; it tells you which formula can be derived from the premises. You can also view an inference rule as a rule to *reduce* the conclusion to the premises: in order to prove the conclusion, it is sufficient to prove the premises.
Here is another example:

$$\frac{\begin{array}{l} P\ e \\ (\forall i : P\ i : Q\ i) \end{array}}{Q\ e} \tag{3.6}$$

This rule can be explained more easily with an example. Suppose we already have ($\forall$`i : 0≤i<n : a[i]>0`) in the current proof-context. So, if we can infer that `e` it is inside the $\forall$-quantification's domain, that is: `0≤e<n`, then it follows that `a[e]>0` holds.

The rule is used in step 2 in the proof in Figure 3.1: in step 1 we have concluded that 0 is inside the quantification's range; hence by the rule above we can infer `a[0]>0`.

Since using the rule above we can infer a new formula as if by eliminating a $\forall$ symbol, the rule is called $\forall$-Elimination. It is also called $\forall$-Specialization. The act of eliminating $\forall$ in this way is called *specializing* or *instantiating* a $\forall$-formula. Be careful: there are some conditions in invoking this rule. They will be explained in Subsection 3.2.1.

Sometimes, we names the premises, just to make it convenient to refer to them, like in:

$$\begin{array}{lcl} \mathbf{C_1} & : & P\ e \\ \mathbf{C_2} & : & (\forall i : P\ i : Q\ i) \\ \hline & & Q\ e \end{array}$$

The names are printed in bold.

You can find a list of predicate logic's inference rules in the Appendix.

3.2.1 Substitution

Recall the ∀-Elimination rule in (3.6). We show it again below, next to an alternative formulation of the rule:

$$\begin{array}{l} P\ e \\ (\forall i : P\ i : Q\ i) \\ \hline Q\ e \end{array} \qquad\qquad \begin{array}{l} P[e/i] \\ (\forall i : P : Q) \\ \hline Q[e/i] \end{array} \tag{3.7}$$

You see a new notation here, namely $P[e/i]$. It means the formula that you obtain by replacing all *free* occurrences of i in P by e. This operation is called *substitution*. Be careful not to confuse it with the array or list notations, which also use square brackets. Also, do not confuse the use of '/' with the division operator, which is often denoted with the same symbol.

The rule to the left implicitly assumes that i is free in the formulas P and Q. We implicitly treat P and Q as functions parameterized with i. So, $P\ e$ is the predicate we obtain by replacing i in P with e. In other words, it is the same as $P[e/i]$ in the second rule. The second rule on the right is more precise in the sense that it does not make the implicit assumption about treating formulas as functions. While more explicit, the second rule is also more cluttered. So, in this book we will stick to the style used in the first rule.

We will also need substitution to handle assignments in imperative programs —we will discuss this in Chapter 6.

Note that in a substitution $P[e/i]$ only free occurrences of `i` in P should be replaced! So, for example:

$$((\exists \mathtt{i} :: \mathtt{b}[\mathtt{i}])\ \wedge\ \mathtt{i} \geq \mathtt{0})\ \ [\mathtt{0}/\mathtt{i}]$$

results in $(\exists \mathtt{i} :: \mathtt{b}[\mathtt{i}])\ \wedge\ \mathtt{0} \geq \mathtt{0}$, and *not* $(\exists \mathtt{i} :: \mathtt{b}[\mathtt{0}])\ \wedge\ \mathtt{0} \geq \mathtt{0}$.

A substitution $P[e/i]$ is also *not* allowed if after the substitution there is a free variable in e that becomes bound, because the resulting formula would have a different meaning —one which is not your intention. So, for example, this substitution is not allowed:

$$(\exists \mathtt{i} :: \mathtt{a}[\mathtt{i}] > \mathtt{x})\ [\mathtt{i}+\mathtt{1}/\mathtt{x}]$$

because it would result in $(\exists \mathtt{i} :: \mathtt{a}[\mathtt{i}] > \mathtt{i}+\mathtt{1})$, where `i`, which occurs free in the substituting expression `i+1` , becomes bound in the result.

You can however first rename the bound variable `i` in $(\exists \mathtt{i} :: \mathtt{a}[\mathtt{i}] > \mathtt{x})$, for example to $\mathtt{i}'$. So, we obtain: $(\exists \mathtt{i}' :: \mathtt{a}[\mathtt{i}'] > \mathtt{x})$. Renaming bound variables does not change the meaning of a quantified expression. After the renaming it is now safe to apply the substitution $[\mathtt{i}+\mathtt{1}/\mathtt{x}]$, producing: $(\exists \mathtt{i}' :: \mathtt{a}[\mathtt{i}'] > \mathtt{i}+\mathtt{1})$.

Because of the above restriction, note that when using an inference rule like those in (3.7), no free variables in the substituting expression, i.e. the e in (3.7), should become bound in the inferred formula.

```
PROOF
[D1:] Q n = (∀i : i<n : b[i])
[A :] Q n ∧ b[n]
[G :] Q (n+1)
BEGIN ____________________________________________________
  1 { rewrite the 2nd conjuct of A with Quantification over Singleton (T A.4.10) }
    Q n ∧ (∀i : i = n : b[i])
  2 { rewrite 1 with D1} }
    (∀i : i<n : b[i]) ∧ (∀i : i = n : b[i])
  3 { rewrite 2 with Domain Split (T 3.8.1) }
    (∀i : i<n ∨ (i = n) : b[i])
  4 { rewrite 2 with i<n ∨ (i = n) = i<n+1 }
    (∀i : i<n+1 : b[i])
  5 { rewrite 4 with D1 }
    Q (n+1)
END ______________________________________________________
```

Figure 3.2: *Well chosen definitions can improve the readability of your proof.*

3.3 Definition

You can also introduce definitions in the declaration part of a proof. They are convenient for abbreviating long expressions that we expect to occur several times in the proof. Well chosen definitions can make a complicated proof much easier to read. As an example, consider the formula below:

$$(\forall \mathtt{i} : \mathtt{i}<\mathtt{n} : \mathtt{b}[\mathtt{i}]) \ \wedge \ \mathtt{b}[\mathtt{n}] \ \Rightarrow \ (\forall \mathtt{i} : \mathtt{i}<\mathtt{n}+1 : \mathtt{b}[\mathtt{i}]) \tag{3.8}$$

Intuitively, the formula is clearly valid. Figure 3.2 shows the formal proof. Entries in the declaration whose labels begin with a `D` are definitions. In Figure 3.2 `D1` introduces the notation `Q n` to abbreviate $(\forall \mathtt{i} : \mathtt{i}<\mathtt{n} : \mathtt{b}[\mathtt{i}])$.

The equality that specifies a definition can be thought to be implicitly universally quantified over all parameters of the defined name. So, in the definition of `Q` in Figure 3.2, the equation does not only specify the meaning of `Q n` for a single `n`, but also for all instances of `n`. So, it should be read:

$$(\forall \mathtt{n} :: \mathtt{Q}\ \mathtt{n} \ = \ (\forall \mathtt{i} : \mathtt{i}<\mathtt{n} : \mathtt{b}[\mathtt{i}]))$$

3.4 Rewriting

Rewriting is a frequently used operation in proofs. If you know that $e_1 = e_2$, then you can replace some or all occurrences of e_1 in E with e_2. We use $E[e_2/e_1]$ to denote the resulting formula. Within the current proof, the result of a rewrite is equivalent with the original formula. Rewriting is captured by the following rule:

$$\frac{\begin{array}{l} e_1 \ = \ e_2 \\ P \end{array}}{P[e_2/e_1]} \tag{3.9}$$

Rewriting is a generalization of substitution (Sebsection 3.2.1). A substitution replaces occurences of a single variable. In rewriting you can replace arbitrary expressions. Like in substitution, there are a number of things that you should keep in mind:

1. As in substitution, when performing a rewrite $E[e_2/e_1]$, no free variables in e_2 should become bound after the rewrite.

2. When performing a rewrite $E[e_2/e_1]$, you can only replace an occurence of e_1 in E if in this occurrence no free variable of e_1 is bound. For example, supposed we have derived this expression:

   $$(\forall \mathtt{i} : 0 \leq \mathtt{i} : \mathtt{b}[\mathtt{i}]) \ \wedge \ \mathtt{b}[\mathtt{i}]$$

   If we also know that $\mathtt{b}[\mathtt{i}] = \mathtt{x}$, we can replace $\mathtt{b}[\mathtt{i}]$ with $\mathtt{x}$. So, we infer this:

   $$((\forall \mathtt{i} : 0 \leq \mathtt{i} : \mathtt{b}[\mathtt{i}]) \ \wedge \ \mathtt{b}[\mathtt{i}]) \ [\mathtt{x}/\mathtt{b}[\mathtt{i}]]$$

   which gives:

   $$(\forall \mathtt{i} : 0 \leq \mathtt{i} : \mathtt{b}[\mathtt{i}]) \ \wedge \ \mathtt{x}$$

   Notice that only the outer $\mathtt{b}[\mathtt{i}]$ is replaced. The $\mathtt{b}[\mathtt{i}]$ inside the quantification has its $\mathtt{i}$ bound: the scope of this $\mathtt{i}$ is limited to within the quantified expression. So, it does not mean the same thing as the $\mathtt{i}$ in the derived fact $\mathtt{b}[\mathtt{i}]{=}\mathtt{x}$. Therefore, using this equality to rewrite the inner $\mathtt{b}[\mathtt{i}]$ would be incorrect. So, this is *not* a correct result of the above rewrite:

   $$(\forall \mathtt{i} : 0 \leq \mathtt{i} : \mathtt{x}) \ \wedge \ \mathtt{x}$$

If you have derived P, you can also replace its occurences in Q with `true`. Similarly, if you have derived $\neg P$, you can replace the occurences of P in Q with `false`. These are captured by the following rules:

$$\frac{\begin{array}{l} P \\ Q \end{array}}{Q[\mathtt{true}/P]} \qquad \frac{\begin{array}{l} \neg P \\ Q \end{array}}{Q[\mathtt{false}/P]} \tag{3.10}$$

In the Appendix you can find a list of standard equalities in predicate logic. You can use them to rewrite the formulas in your proof. Using them needs a little explanation. Here is an example of the listed equalities:

$$\vdash \ \neg(\forall i : P\ i : Q\ i) = (\exists i : P\ i : \neg(Q\ i)) \tag{3.11}$$

Implicitly, we mean that the equality holds for all instances of P and Q. We could formulate it more precisely:

$$\vdash \ (\forall P, Q :: \neg(\forall i : P\ i : Q\ i) = (\exists i : P\ i : \neg(Q\ i)))$$

but the outermost $\forall$ clutters the formula. So, we will stick with the first formulation, and uses as a convention that the theorems in this book are implicitly universally quantified over their free variables (a quite common convention in mathematics).

In effect, we can use (3.11) not only to rewrite a expression that is exactly equal to either sides of (3.11), but also to rewrite any expression whose pattern matches either sides of (3.11). For example, using (3.11) we can rewrite:

$$\neg(\forall \mathtt{i} : \mathtt{i}{\geq}0 : \mathtt{i}{>}0) \quad \text{to} \quad (\exists \mathtt{i} : \mathtt{i}{\geq}0 : \neg(\mathtt{i}{>}0))$$

and vice versa.

Sometimes, an equality only holds under certain circumstances. For example, a theorem called Domain Merging (Theorem 3.8.3 —see Section 30) states the following equality:

$$\vdash\ a \leq b\ \Rightarrow\ (a{\leq}i{<}b{+}1\ \ =\ \ a{\leq}i{<}b\ \vee\ (i = b)) \tag{3.12}$$

Note that implicitly, as we have agreed above, the formula holds for all a, b, and i. So, it allows you to rewrite e.g.:

$$\mathtt{0{\leq}i{<}k{+}1} \quad \text{to} \quad \mathtt{0{\leq}i{<}k} \vee (\mathtt{i} = \mathtt{k})$$

but only if you know that $\mathtt{0{\leq}k}$. This kind of rewriting is called *conditional rewriting*, because you can only do it under certain circumstances.

3.5 Subproof

A proof may also contains subproofs. As an example, let us prove the following:

$$\neg\mathtt{b}\ \wedge\ (\mathtt{x}^2{>}\mathtt{0} \Rightarrow \mathtt{y}{<}\mathtt{0})\ \wedge\ (\mathtt{y}{\leq}\mathtt{0} \Rightarrow \mathtt{b})\ \Rightarrow\ (\mathtt{x} = \mathtt{0}) \tag{3.13}$$

Informally this can be argued by contradiction as follows. Assume $\mathtt{x} \neq \mathtt{0}$. It follows that $\mathtt{x}^2 > \mathtt{0}$. By the assumption $\mathtt{x}^2{>}\mathtt{0} \Rightarrow \mathtt{y}{<}\mathtt{0}$ it follows that $\mathtt{y}{<}\mathtt{0}$. Next, by the assumption $\mathtt{y}{\leq}\mathtt{0} \Rightarrow \mathtt{b}$ it follows that $\mathtt{b}$ holds. But this is in contradiction with the assumption $\neg\mathtt{b}$! So, the original assumption $\mathtt{x} \neq \mathtt{0}$ must be wrong, and therefore $\mathtt{x} = \mathtt{0}$.

The formal proof is shown in Figure 3.3. Notice that the first expression derived in `top` is justified by the subproof that follows below it.

The nesting in a proof can be several levels. For example, a proof $\mathcal{P}$ may contain a subproof $\mathcal{Q}$, which in turn contains a subproof $\mathcal{R}$.

Recall that a new formula can only be added to a proof if it is a logical consequence of the current proof-context. When we start a subproof $\mathcal{Q}$, its starting context consists of $\mathcal{Q}$'s own assumptions plus the context of $\mathcal{Q}$'s parent proof at the moment $\mathcal{Q}$ is opened.

Anything you derive inside $\mathcal{Q}$ potentially relies on $\mathcal{Q}$'s assumptions. Keep in mind that the scope of these assumptions is limited within $\mathcal{Q}$. Therefore information derived *inside* a subproof *cannot* be used in the parent proof. For example, in Figure 3.3 within `top` we are not allowed to refer to any formula derived inside `pot`. However, we can (of course) in `top` refer to the *conclusion* of `pot`, which is formula 1 in `top`.

For labelling the proof elements (e.g. assumptions and derived formulas) in $\mathcal{Q}$ you can reuse labels already used in the parent proofs. We will use qualified notation when it is necessary to distinguish them. For example, in the subproof `pot` (Figure 3.3), `A1` refers to `pot`'s own assumption `A1`, whereas `top.A1` refers to the `A1` of `top`. Similarly, `top.1` refers to the formula 1 in `top`.

3.6 Equational Proof

The flag pole proof style we use so far works fine for proving a formula of the form $p \Rightarrow q$, but is less convenient for proving formulas of the form $e_1 = e_2$. A more convenient way to do it is by using an equational proof. The idea is that we rewrite e_1 to an equivalent formula and repeat this until we obtain e_2. As an example, consider this:

$$\neg(\forall\mathtt{i} : \mathtt{0} \leq \mathtt{i} : \mathtt{b}[\mathtt{i}] \Rightarrow \mathtt{c}[\mathtt{i}])\ =\ (\exists\mathtt{i} : \mathtt{0} \leq \mathtt{i} : \mathtt{b}[\mathtt{i}] \wedge \neg\mathtt{c}[\mathtt{i}]) \tag{3.14}$$

One way to prove this in a pure flag pole style is by proving the double implications equivalent of the above equality. It can be shorter and nicer as shown in the following equational style:

```
PROOF top
[A1:] ¬b
[A2:] x² > 0 ⇒ y < 0
[A3:] y ≤ 0 ⇒ b
[G :] x = 0
BEGIN ____________________________________________________
  1 { see proof pot }   x ≠ 0 ⇒ false

    PROOF pot
    [A1:] x ≠ 0
    [G :] false
    BEGIN ________________________________________________
      1 { follows from A1 }   x² > 0
      2 { Modus Ponens (R A.1.2) on 1 and top.A2 }   y < 0
      3 { follows from 2 }   y ≤ 0
      4 { Modus Ponens (R A.1.2) on 3 and top.A3 } }   b
      5 { conjunction of 4 and top.A1 }   false
    END ___________________________________________________

  2 { Contradiction Rule (R A.1.3) on 1 }   x = 0
END ______________________________________________________
```

Figure 3.3: *An example showing the use of a subproof.*

```
EQUATIONAL PROOF top ______________________________________
¬(∀i : 0 ≤ i : b[i] ⇒ c[i])
= { Negate ∀ (T A.4.7) }
(∃i : 0 ≤ i : ¬(b[i] ⇒ c[i]))
= { basic equalities of boolean connectors (T A.4.1) }
(∃i : 0 ≤ i : b[i] ∧ ¬c[i])
END ______________________________________________________
```

In an equational proof the only thing you can do is to rewrite the current formula in the proof to another equivalent formula. So, every formula in the sequence above is equivalent to each other[2]

An equational proof may have a declaration part containing assumptions. You don't have to specify the goal. If f_1 and f_n are the first and the last formulas in an equational proof, the proof justifies the validity of $f_1 = f_n$. However, if the proof has declared assumptions $A_1, \ldots, A_k$, then it proves this instead:

$$A_1 \wedge \ldots \wedge A_k \;\Rightarrow\; (f_1 = f_n)$$

[2]Notice that if you are in the body of an `EQUATIONAL PROOF`, the formula in k+1-th step is *equivalent* with the previous one. In contrast, in the body of a flag pole `PROOF` each formula you infer is, by agreement, only a *consequence* of the current proof-context. For example if you infer e_1 in k-th step (in a flag pole proof) and e_2 in the next step, you cannot then claim that $e_1 = e_2$. If this is what you want, you have to explicitly try to infer (and write down) $e_1 = e_2$ in your k+1-th step.

3.7 Handling ∀ and ∃

The quantors ∀ and ∃ are used quite often. Section 3.2 already explains the rule (3.6) used to instantiate (eliminate) a ∀-quantification . The following rule can be used to infer a ∃-quantification. It is called ∃-Introduction Rule; in a sense it is the dual of the ∀-Elimination Rule (3.6). Informally, suppose we have already inferred `a[i]=0`. So, there is (obviously) a `k` such that `a[k]=0`. In other words, we have $(\exists \mathtt{k} :: \mathtt{a}[\mathtt{k}] = \mathtt{0})$. Formally, in two variants:

$$\frac{Q\ e}{(\exists i :: Q\ i)} \qquad \frac{\begin{array}{l} P\ e \\ Q\ e \end{array}}{(\exists i :\ P\ i\ \ : Q\ i)} \tag{3.15}$$

Inferring a ∀-quantification and eliminating ∃ are a bit more complicated.

3.7.1 Inferring a ∀-Formula

Suppose we have a top level proof with A as an assumption and g as a goal. If successful, it proves $\vdash A \Rightarrow g$. That is, that $A \Rightarrow g$ is valid. But we can infer more. Suppose i occurs free in $A \Rightarrow g$, then by the meaning of 'validity' the same proof also implies $\vdash (\forall i :: A \Rightarrow g)$. Or equivalently: $\vdash (\forall i : A : g)$. For example, consider again the proof in Figure 3.3. It proves this formula:

$$\neg b\ \wedge\ (x^2 > 0 \Rightarrow y < 0)\ \wedge\ (y \leq 0 \Rightarrow b)\ \Rightarrow\ (x = 0)$$

However, by quantifying over the free variables x and y, it also proves this:

$$(\forall x, y, b :\ \neg b\ \wedge\ (x^2 > 0 \Rightarrow y < 0)\ \wedge\ (y \leq 0 \Rightarrow b)\ :\ x = 0)$$

Therefore, to prove a formula of the form:

$$(\forall x_1, \ldots, x_k : A_1 \wedge \ldots \wedge A_n : g)$$

we can instead just prove:

$$A_1 \wedge \ldots \wedge A_n \Rightarrow g$$

Doing this in a subproof is more tricky. Consider a proof $\mathcal{P}$ containing a subproof $\mathcal{Q}$. Suppose $\mathcal{Q}$ proves $A \Rightarrow g$ and suppose x is free in $A \Rightarrow g$. Can we also infer $(\forall x : A : g)$? Well, it depends. If x is used in the parent proof $\mathcal{P}$[3] the proof in $\mathcal{Q}$ may refer to some proven fact about x in $\mathcal{P}$. The latter means that g depends on a fact which is not stated in A. So, it would be incorrect to conclude $(\forall x : A : g)$; otherwise it is a sound conclusion. Since you have to keep track of references, this kind of inference can be quite error prone.

To guard against mistakes, we will introduce a so-called `ANY` flag. You can start a subproof $\mathcal{Q}$ with a keyword `ANY` followed by a list of variables. For example `ANY` x, y. It means that we start the proof $\mathcal{Q}$ by assuming nothing about x and y, other than the assumptions stated in $\mathcal{Q}$ itself. In particular, this limits the scope of x and y to $\mathcal{Q}$ only. It follows, when concluding the proof $\mathcal{Q}$ we can also universally quantify the conclusion over x and y.

There is one condition that you have to remember: since the scope of x and y is limited to $\mathcal{Q}$, you are not allowed to refer to any formula in the higher level proofs that contains x or y as a free variable. It follows that using fresh variables every time you use the `ANY` flag is always safe (that you don't actually need the flag anymore). When you conclude a subproof, you can only ∀-quantify over `ANY` variables. To do otherwise is potentially unsound, thus is forbidden.

As an example, consider the following formula:

$$\begin{array}{l} (\forall \mathtt{i} : \mathtt{0} \leq \mathtt{i} : \mathtt{a}[\mathtt{i}] > \mathtt{0})\ \wedge\ \mathtt{x}^2 > \mathtt{0} \\ \Rightarrow \\ (\forall \mathtt{i} : \mathtt{0} \leq \mathtt{i} : \mathtt{x} * \mathtt{a}[\mathtt{i}] \neq \mathtt{0}) \end{array} \tag{3.16}$$

[3] More precisely: x occurs free in one of the formulas in the proof-context of $\mathcal{P}$ at the moment $\mathcal{Q}$ is opened.

```
PROOF top
[A1:] (∀i : 0≤i : a[i]>0)
[A2:] x²>0
[G :] (∀i : 0≤i : x ∗ a[i] ≠ 0)
BEGIN ____________________________________________________________
  1. { follows from A2 }    x ≠ 0
  2. { see proof pot }    (∀i : 0≤i : x ∗ a[i] ≠ 0)

     PROOF pot
     ANY i
     [A1:] 0≤i
     [G :] x ∗ a[i] ≠ 0
     BEGIN ____________________________________________________
       1 { ∀ elimination (R A.1.10) on top.A1 and A1 }    a[i]>0
       2 { follows from top.1 and 1 }    x ∗ a[i] ≠ 0
     END ______________________________________________________

END ______________________________________________________________
```

Figure 3.4: *Proving a universally quantified goal.*

A proof of its validity is shown in Figure 3.4. The subproof `pot` proves that $0 \leq i$ implies $x * a[i] \neq 0$. However it also has an `ANY i` flag, which means we can also ∀-quantify over `i`, thus drawing this conclusion instead (in step-2 of `top`):

$$(\forall i : 0 \leq i : x * a[i] \neq 0)$$

Notice that the flagged variable `i` in the proof `pot` *does* occur in the parent proof `top`, namely in `top.A1`. However it does not occur free in `top.A1`, so it won't pose any problem. If it does occur free in some formula f in `top` it is still not a problem as long as you do *not* refer to f from within `pot`.

3.7.2 Eliminating ∃

Sometimes you have to prove a goal with a ∃-formula as one of its assumptions. The information inside the ∃-formula is often more useful if we can somehow get rid of the ∃. However, we have to be careful, because doing so may lead to an unsound conclusion. Consider the following example:

$$(\exists i : 0 \leq i : b[i]) \ \wedge \ \neg b[k] \ \wedge \ (\forall i : b[i] : c[i]) \qquad (3.17)$$
$$\Rightarrow$$
$$(\exists i :: c[i])$$

The proof is shown in Figure 3.5.

Look at the first step in Figure 3.5. If you have a ∃-formula in the current proof-context, you can infer a new formula by replacing the ∃ quantor with a `SOME` flag. You can also rename the bound variable. In Figure 3.5, `A1` states the existence of an `i` satisfying $0 \leq i$ and $b[i]$. The `SOME` flag in step-1 simply re-asserts this fact:

$$[\texttt{SOME}\ i'] \ \ 0 \leq i' \ \wedge \ b[i']$$

This may seem a bit pointless, but there is one important difference between ∃ and `SOME`: subsequent proof steps can now directly use the formula after the `SOME` flag. Before, it was not possible because it is 'packed' inside an ∃ quantifier.

```
PROOF
[A1:] (∃i : 0≤i : b[i])
[A2:] ¬b k
[A3:] (∀i :: b[i] ⇒ c[i])
[G :] (∃i :: c[i])
BEGIN ____________________________________________________________
  1 { ∃ elimination on A1 }
    [SOME i′] 0≤i′ ∧ b[i′]
  2 { ∧ Elimination (R A.1.5) on 1 }
    b[i′]
  3 { ∀ Elimination (R A.1.10) on A3 and 2 }
    c[i′]
  4 { ∃ Introduction (R A.1.11) on 3 }
    (∃i′ :: c[i′])
END ______________________________________________________________
```

Figure 3.5: *An example of how to eliminate* ∃.

The flag `SOME i′` introduces however a scope. From this point on, all references to `i′` refers to the `i′` that is introduced by the flag. In particular, if the current proof-context already had a variable `i′`, after the `SOME i′` flag you are not allowed to use any assumed or derived facts on the 'old' `i′` —doing so would mean that you illegally assume something that is not there for the flagged `i′`. If you consider this to be error prone, then just introduce a fresh variable on every `SOME`. For example, in Figure 3.5 introducing `SOME i′` in step-1 is allowed; so are `SOME i` or `SOME j`. Even this is allowed:

[SOME k] 0≤k ∧ b[k]

But be careful! Any `k` after this refers to the above `k`. Now, notice that `k` already appears free in `A2`. So invoking `A2` would now be an invalid step (in fact, `A2` asserts something that is contradictive to the above asserted property of b[k]).

3.8 Some Useful Proof Techniques

Proof by contradiction

Sometimes, it is easier to prove a goal Q by contradiction. That is, we assume $\neg Q$, and then show that this is not possible. Formally, this is done by invoking the following rule:

$$\frac{\neg Q \Rightarrow \texttt{false}}{Q} \tag{3.18}$$

The premise $\neg Q \Rightarrow$ `false` captures the fact that $\neg Q$ is not possible. An example showing the use of this rule has been shown in Figure 3.3.

Case split

Consider the formula below:

$$\begin{array}{l}(\texttt{a}[0] = 0) \ \wedge \ (\forall \texttt{i} : 0{<}\texttt{i} : 0 < \texttt{a}[\texttt{i}]) \ \wedge \ 0{\leq}\texttt{j} \\ \Rightarrow \\ 0{\leq}\texttt{a}[\texttt{j}]\end{array} \tag{3.19}$$

```
PROOF top
[A1:] a[0] = 0
[A2:] (∀i : 0<i : 0<a[i])
[A3:] 0≤j
[G :] 0 ≤ a[j]
BEGIN ____________________________________________________
  1 { follows trivially from A3 }    0<j ∨ (j = 0)
  2 { see subproof pot }    0<j ⇒ 0≤a[j]

    PROOF pot
    [A1:] 0<j
    [G :] 0≤a[j]
    BEGIN ____________________________________________
      1 { ∀-Elimination on top.A2 using A1 }    0 < a[j]
      2 { follows trivially from 1 }    0 ≤ a[j]
    END ______________________________________________

  3 { follows trivially from A1 }    (j = 0) ⇒ 0≤a[j]
  4 { Case Split on 1,2,3 }    0≤a[j]
END ______________________________________________________
```

Figure 3.6: *An example of a proof using case split.*

Informally this can be argued as follows. We only need to prove the implication for `j` satisfying `0≤j`. We split this in two cases: (1) if `0<j` then it is in the domain of the assumption (∀`i` : `0<i` : `0` < `a[i]`), thus implying `0<a[j]`; (2) if `j` = `0`, the first assumption above says `a[0]` = `0`. Both cases imply `0≤a[j]`.

The proof relies on analyzing two possibles cases of `j`. Distinguishing the two cases is necessary, because the ways the conclusion `0` ≤ `a[j]` is inferred are different for each case. Formally, to do this kind of split-cases proof we invoke the following rule:

$$\frac{\begin{array}{l} P \vee Q \\ P \Rightarrow R \\ Q \Rightarrow R \end{array}}{R} \tag{3.20}$$

So, we can infer R by first identifying two cases P and Q and proving $P \vee Q$ —the latter is to show that P and Q sufficiently cover all possible cases. Then we prove that each of the cases implies R. The proof in Figure 3.6 shows an example. It is the formal proof of (3.19).

Splitting a quantified formula

Quantified formulas occur quite often in the specifications of programs that use arrays. During the proof, quite often we have to prove that an equality the form $x \oplus e = (\forall i : P_1\ i : Q\ i)$ follows from another equality: $x = (\forall i : P_2\ i : Q\ i)$; here $\oplus$ is an operator or a binary function. A strategy that is often used to first prove:

$$(\forall i : P_2\ i : Q\ i) \oplus e \ = \ (\forall i : P_1\ i : Q\ i) \tag{3.21}$$

Then it follows that:

$x \oplus e$
$= \{$ assumption $\}$
$(\forall i : P_2\ i : Q\ i)\ \oplus\ e$
$= \{$ using (3.21) $\}$
$(\forall i : P_1\ i : Q\ i)$

Notice that the strategy works even if we have another quantor instead of $\forall$. There are a number of theorems that are often convenient for proving equalities between quantified formulas such as (3.21). Here are some examples (of the theorems):

Theorem 3.8.1 : Domain split

1. $\vdash (\forall i : P\ i \vee Q\ i : R\ i)\ =\ (\forall i : P\ i : R\ i) \wedge (\forall i : Q\ i : R\ i)$
2. $\vdash (\exists i : P\ i \vee Q\ i : R\ i)\ =\ (\exists i : P\ i : R\ i) \vee (\exists i : Q\ i : R\ i)$

□

The theorem above is called Domain Split because the equalities above can be used to 'split' a quantified formula on its domain. We can also split on its range, which is stated by the theorem below:

Theorem 3.8.2 : Range Split

1. $\vdash (\forall i : P\ i : Q_1\ i \wedge Q_2\ i)\ =\ (\forall i : P\ i : Q_1\ i) \wedge (\forall i : P\ i : Q_2\ i)$
2. $\vdash (\exists i : P\ i : Q_1\ i \vee Q_2\ i)\ =\ (\exists i : P\ i : Q_1\ i) \vee (\exists i : P\ i : Q_2\ i)$

□

Merging integer domains

Recall that an integer domain expression is a formula of the form, for example, $a{\le}i{<}b$. Such a formula is used quite often, especially when specifying and proving the correctness of a program that uses arrays. Quite often, in a proof, such as in Figure 3.2, we need to convert a domain expression of the form $a{\le}i{<}b{+}1$ to $a{\le}i{<}b \vee (i = b)$. These two formulas are equal, but be *careful*, they are only equal if a, b, and i are all integers and if $a \le b$. This is expressed by the following theorem:

Theorem 3.8.3 : Domain Merging
Let a, b, and i be integers:

$$\vdash\ a \le b\ \Rightarrow\ (a{\le}i{<}b{+}1\ =\ a{\le}i{<}b\ \vee\ (i = b))$$

□

In a more general case, we may want to split $a{\le}i{<}c$ to:

$$a{\le}i{<}b\ \vee\ b{\le}i{<}c$$

The two formulas are equivalent, but as before, they are only equivalent under a certain condition:

Theorem 3.8.4 : Domain Merging
Let a, b, c, and i be integers:

$$\vdash\ a \le b\ \wedge\ b \le c\ \Rightarrow\ (a{\le}i{<}c\ =\ a{\le}i{<}b\ \vee\ b{\le}i{<}c)$$

□

Both equalities are also listed in the Appendix.

Simplifying conditional formulas

Recall that a conditional formula is a formula of the form $P \to e_1 \mid e_2$. You may encounter this kind of formulas when proving the correctness of a program that contains an `if-then-else` statement. The theorem below lists a number of equalities which can be used to simplify a conditional formula.

Theorem 3.8.5 : COND Conversion

1. $\vdash \quad P \quad \Rightarrow \quad (P \to e_1 \mid e_2 \quad = \quad e_1)$
2. $\vdash \quad \neg P \quad \Rightarrow \quad (P \to e_1 \mid e_2 \quad = \quad e_2)$
3. $\vdash \quad P \to e \mid e \quad = \quad e$
4. $\vdash \quad f\ (P \to e_1 \mid e_2) \quad = \quad P \to f\ e_1 \mid f\ e_2$
5. $\vdash \quad P \to e_1 \mid e_2 \quad = \quad \neg P \to e_2 \mid e_1$

Sometimes, we have to prove a formula of the form $P \to Q \mid R$. We can use the first two equalities above to simplify $P \to Q \mid R$ to either Q or R. Unfortunately this does not always work, since we may not be able to derive either P or $\neg P$. The following theorem states that we can equivalently prove $P \Rightarrow Q$ and $\neg P \Rightarrow R$ instead (but note that the theorem requires that P, Q, and R are all *predicates*):

Theorem 3.8.6 : COND Split
Let P, Q, and R be predicates:

$$\vdash \quad P \to Q \mid R \quad = \quad (P \Rightarrow Q) \wedge (\neg P \Rightarrow R)$$

□

□□□

New Vocabulary

conclusion; COND coversion; COND split; conditional rewrite; domain merging; domain split; eliminiating ∃; goal; inference rule; introducing ∀, ∃; predicate; premise; proof-context; range split; rewriting; specialiazing (eliminating, instantiating) ∀; stronger; subproof; substitution; universally true; valid; weaker

3.9 Exercise

1. Explain in words what the following formulas say:

   (a) $(\forall \mathtt{i}, \mathtt{j} :: (\mathtt{a}[\mathtt{i}] = \mathtt{a}[\mathtt{j}]) \Rightarrow (\mathtt{i} = \mathtt{j}))$
   (b) $\neg(\exists \mathtt{i} : 0 \leq \mathtt{i} : \mathtt{a}[\mathtt{i}] \neq \mathtt{a}[\mathtt{i}+1])$
   (c) $(\exists \mathtt{i} :: (\forall \mathtt{j} :: \mathtt{a}[\mathtt{i}] \leq \mathtt{a}[\mathtt{j}]))$
   (d) $(\forall \mathtt{i} :: \neg(\exists \mathtt{j} :: \mathtt{a}[\mathtt{i}] = \mathtt{b}[\mathtt{j}]))$

2. Write formulas expressing the following informal texts:

   (a) Whithin some domain `D` the array `a`, when viewed as a set, is a 'subset' of array `b`.
   (b) There is a unique index `i` in the domain `D` such that $\mathtt{f}\ (\mathtt{a}[\mathtt{i}]) = 0$.
   (c) The array `a` contains at least two distinct elements.
   (d) The array `a` has at most two elements which are equal to each other.

3. Explain in words what the following formula says, then prove it.

$$
\begin{array}{l}
(\forall \mathtt{i}, \mathtt{j} :\ 0 \leq \mathtt{i} < \mathtt{n}\ \wedge\ \mathtt{i} \leq \mathtt{j} < \mathtt{n}\ :\ \mathtt{a}[\mathtt{i}] \leq \mathtt{a}[\mathtt{j}]) \\
\wedge \\
0 < \mathtt{n} \\
\Rightarrow \\
(\exists \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{n} : (\forall \mathtt{j} : 0 \leq \mathtt{j} < \mathtt{n} : \mathtt{a}[\mathtt{i}] \leq \mathtt{a}[\mathtt{j}]))
\end{array}
$$

   Actually, the implication is still valid even if we drop the $(\forall i, j : \ldots)$ conjunct at the left side of the $\Rightarrow$, though the proof is now more complicated and would require induction (see below).

4. Explain in words what this formula says then prove it.

$$
\begin{array}{l}
(\forall \mathtt{i} : 0 \leq \mathtt{i} : \mathtt{a}[\mathtt{i}] = \mathtt{a}[\mathtt{i}+1]) \\
\Rightarrow \\
(\forall \mathtt{i} : 0 \leq \mathtt{i} : \mathtt{a}[\mathtt{i}] = \mathtt{a}[0])
\end{array}
$$

   To prove you will need to use induction over positive integers. This is expressed by the following rule:

$$
\frac{\begin{array}{c} P\ 0 \\ (\forall n : n \geq 0 : P\ n \Rightarrow P\ (n+1)) \end{array}}{(\forall n : n \geq 0 : P\ n)}
$$

   where n in the above rule in assumed to be of the integer type.

5. Suppose at some point during an execution of a program we know that this condition holds:

$$\mathtt{r}\ =\ (\forall \mathtt{k} : 0 \leq \mathtt{k} < \mathtt{i} : \mathtt{b}[\mathtt{k}])$$

   A subsequent statement S requires however that another condition should hold before it executes, namely:

$$\mathtt{r} \wedge \mathtt{b}[\mathtt{i}]\ =\ (\forall \mathtt{k} : 0 \leq \mathtt{k} < \mathtt{i}+1 : \mathtt{b}[\mathtt{k}])$$

   It is safe to continue with S if the first condition above actually implies the second. Can you prove this? You will discover that your proof also need an additional assumption $\mathtt{i} \geq 0$, why?

6. Suppose we mark some integers as 'blue' integers. We know the following facts about an array `a`:

   (a) If `a` contains a blue integer in the domain $0 \leq \mathtt{i} < \mathtt{n}$, then this integer is 2.

   (b) In the domain mentioned above, `a` does *not* contain any even integer.

   Prove that `a` does not contain (in the above domain) any blue integer. You can assume that you have a function `blue` to test whether an integer is blue.

7. What does the following formula say? Prove it.

$$\neg(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}]) \quad \wedge \quad (\forall \mathtt{i} : \mathtt{j} \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}])$$
$$\Rightarrow$$
$$\neg(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{j} : \mathtt{b}[\mathtt{i}])$$

   Notice that all occurences of `i` are bound, whereas `j` is free.

CHAPTER 4

Lists

In Chapter 2 we have seen that the existential quantifier $\exists$ is needed to specify a program that tests whether an array contains a value that satisfies a certain property. Similarly, the universal quantifier $\forall$ is needed to specify a program that tests whether *all* elements of an array satisfy a certain property. Unfortunately, with only $\exists$ and $\forall$ we still cannot specify a program that, for example, computes the sum of all elements in an array. To solve this problem we need to extend our expression language with more quantifiers. The most common notation used to denote the sum over some specified domain is, for example:

$$(\Sigma i : 0 \leq i < n : i) \quad \text{, or this: } \Sigma_{0 \leq i < n}\ i$$

where i is assumed to be of type integer. Both denote the sum of all (integer) values i over the domain $\mathtt{0} \leq \mathtt{i} < \mathtt{n}$. These notations has a drawback. An operation that we often do during a proof is splitting the domain of a quantification. For example, we may want to rewrite:

$$(\Sigma i : P \vee Q : i) \ \text{ to } \ (\Sigma i : P : i) \ + \ (\Sigma i : Q : i)$$

Unfortunately, the two expressions are *not* in general equal. They are only equal if P and Q specify disjoint domains, else the two separate sums in the formula to the right will add some i twice. This often causes confusion and mistakes by students, so we decide to use a different notation. We will denote the sum of all integer values in the domain $0 \leq i < n$ as follows:

$$\mathtt{SUM}\ [0 \ldots n)$$

where SUM is an ordinary list function (rather than a quantifier) which can be defined recursively as follows:

$$\begin{array}{lcl} \mathtt{SUM}\ [] & = & 0 \\ \mathtt{SUM}\ (a : s) & = & a + \mathtt{SUM}\ s \end{array}$$

stating that the sum of an empty list is 0, and the sum of a list whose first element is a is equal to a plus the sum of the rest of the list. The function SUM satisfies this property:

$$\texttt{SUM}\,(s +\!\!+ t) \;=\; \texttt{SUM}\,s + \texttt{SUM}\,t$$

where $s +\!\!+ t$ denotes the list obtained by concatenating s in front of t. As before, the above equality expresses how to 'split the domain' of SUM. But now the equality holds without restriction, so one is less likely to make a mistake.

To use the above kind of notation we will need to extend Form, the language we use to write formulas, with a list notation. We will use a slightly modified notation borrowed from Haskell [26]. Haskell is a functional programming language. Lists are used a lot in functional programming. Do keep in mind that here we do *not* use the list notation to program, as you would do in Haskell. Instead, we use the notation purely for expressing specifications.

4.1 Basic Notation and Properties

Basic operators

The operator : is called the *cons* operator. For example $x : s$ denotes the list obtained by adding x in front of s. The operator $+\!\!+$ is called *append*. For example $s +\!\!+ t$ denotes the list obtained by appending s in front of t. The formula $x \in s$ means that x is an element of the list s. Append and $\in$ can be defined recursively as follows:

Definition 4.1.1 : $+\!\!+$ AND $\in$

$$\begin{array}{lcl} [\,] +\!\!+ t & = & t \\ (x : s) +\!\!+ t & = & x : (s +\!\!+ t) \\ \\ x \in [\,] & = & \texttt{false} \\ x \in (y : s) & = & (x = y) \;\vee\; x{\in}s \end{array}$$

□

The $+\!\!+$ operator has a higher priority than :, and : has a higher priority than $\in$; $+\!\!+$ is left and right associative[1], whereas : is only right associative —see also Table 2.1 in Chapter 2.

Concrete list and enumeration

The notation like $[i, j, k]$ is used to denote a concrete list, in this case the list contains three elements: i, j, and k. Empty list is denoted by $[\,]$. You can also specify lists using expressions of the form $[i \ldots j)$, where i and j are of the type integer. The notation is called *list enumeration*; it denotes the list of all integers from i up to, but not including, j. Below are a number of basic theorems about list enumeration.

Theorem 4.1.2 : EMPTY ENUMERATION

$$\vdash\; j \leq i \;\Rightarrow\; ([i \ldots j) = [\,])$$

Theorem 4.1.3 : SINGLETON ENUMERATION

$$\vdash\; [i \ldots i{+}1) = [i]$$

Theorem 4.1.4 : ENUMERATION MEMBERSHIP

$$\vdash\; j \in [i \ldots k) \;=\; i{\leq}j{<}k$$

[1] As a functional language $+\!\!+$ is usually only left associative. This is for an efficiency reason. However here we use $+\!\!+$ for expressing specifications where effeciency is not an issue.

Theorem 4.1.5 : Enumeration Split

1. $\vdash\ i \leq j\ \Rightarrow\ ([i \ldots j+1)\ =\ [i \ldots j) +\!\!+ [j])$
2. $\vdash\ i \leq j \leq k\ \Rightarrow\ ([i \ldots k)\ =\ [i \ldots j) +\!\!+ [j \ldots k))$

List comprehension

You may be familiar with this notation from set theory: $\{f\ x \mid x \in A\}$ denoting the set of all $f\ x$ where x is taken from the set A. The notation is called set comprehension. We will also use something similar for list, called *list comprehension*. We will write, for example:

$$[x * x \mid x\ \texttt{from}\ s,\ x > 0]$$

to denote the list consisting of all $x * x$ values, where x is taken from the list s, and moreover x should satisfy $x > 0$. This is a quite convenient notation. For example, to specify the sum of all positive elements of an array `a` in the domain `0≤i<n` we can write:

$$\texttt{SUM}\ [\texttt{x} \mid \texttt{x from a}[\texttt{0} \ldots \texttt{n}),\ \texttt{x} > \texttt{0}]$$

The list s in $[e\ i \mid i\ \texttt{from}\ s,\ P\ i]$ is also called *generator*. The P is called the *condition* of the comprehension, and e is called the *transformer* of the comprehension.

Sometimes, we also write a comprehension without the P part, e.g.: $[e\ i \mid i\ \texttt{from}\ s]$, which means that we have no condition to constrain the i's. Formally, you can consider $[e\ i \mid i\ \texttt{from}\ s]$ to be an abbreviation of $[e\ i \mid i\ \texttt{from}\ s, \texttt{true}]$. Below you can find some basic theorems about list comprehension.

Theorem 4.1.6 : Empty Comprehension

1. $[e\ i \mid i\ \texttt{from}\ [],\ P] \quad = \quad []$
2. $[e\ i \mid i\ \texttt{from}\ s,\ \texttt{false}] \quad = \quad []$
3. $([e\ i \mid i\ \texttt{from}\ s,\ \texttt{P i}] = []) \quad = \quad (\forall i : i \in s : \neg(P\ i))$

Theorem 4.1.7 : Singleton Comprehension

1. $[e\ i \mid i\ \texttt{from}\ [x]] \quad = \quad [e\ x]$
2. $[e\ i \mid i\ \texttt{from}\ [x],\ P\ i] \quad = \quad P\ x \rightarrow [e\ x] \mid []$

Theorem 4.1.8 : Comprehension Membership

1. $x \in [e\ i \mid i\ \texttt{from}\ s,\ P\ i] \quad = \quad (\exists i : i \in s \wedge P\ i : x = e\ i)$
2. $x \in [i \mid i\ \texttt{from}\ s,\ P\ i] \quad = \quad x \in s \wedge P\ x$

Theorem 4.1.9 : Comprehension Split

$$[e \mid i\ \texttt{from}\ (s +\!\!+ t),\ P]\ =\ [e \mid i\ \texttt{from}\ s,\ P] +\!\!+ [e \mid i\ \texttt{from}\ t,\ P]$$

□

A comprehension whose generator is another comprehension is called *nested*. The following theorem tells us how to 'collapse' a nested comprehension.

Theorem 4.1.10 : Nested Comprehension

1. $[e_1\ i \mid i\ \texttt{from}\ [e_2\ j \mid j\ \texttt{from}\ s,\ P\ j],\ Q\ i]\ =\ [e_1\ (e_2\ j) \mid j\ \texttt{from}\ s,\ P\ j\ \wedge\ Q\ (e_2\ j)]$
2. $[e_1\ i \mid i\ \texttt{from}\ [e_2\ j \mid j\ \texttt{from}\ s,\ P\ j]]\ =\ [e_1\ (e_2\ j) \mid j\ \texttt{from}\ s,\ P\ j]$

Specifying array properties via list

If a is an array and s is a list, the notation a s denotes the list of all elements of a whose indices is taken from s. It is called *listed array*. More precisely:

$$\mathtt{a}\ s \ = \ [\mathtt{a}[i] \mid i\ \mathtt{from}\ s]$$

For example $\mathtt{a}[\mathtt{0} \ldots \mathtt{n})$ denotes the list consisting of $\mathtt{a}[\mathtt{0}], \ldots, \mathtt{a}[\mathtt{n}-\mathtt{1}]$. The notation allows list functions like SUM to be used for specifying properties over arrays. For example, as in $\mathtt{SUM}(\mathtt{a}[\mathtt{0} \ldots \mathtt{n})) > \mathtt{0}$.

The analogous of Theorems 4.1.6 ... 4.1.5 for enumeration also hold for listed array. For example, for splitting a listed array we have:

Theorem 4.1.11 : Array Split

1. $\vdash\ i \leq j \ \Rightarrow\ (a[i \ldots j+1) \ = \ a[i \ldots j) +\!\!+ [a[j]])$
2. $\vdash\ i \leq j \leq k \ \Rightarrow\ (a[i \ldots k) \ = \ a[i \ldots j) +\!\!+ a[j \ldots k))$

Some standard list functions

We have mentioned SUM, which is a function used to specify the sum of all elements in a list. We can also define COUNT to specify the number of elements in a list; MAX to specify the greatest element in a list; and MIN to specify the smallest element in a list. Their definitions are shown below.

Definition 4.1.12 : SUM, COUNT, MAX, and MIN

$$\begin{array}{lcl}
\mathtt{SUM}\ [] & = & 0 \\
\mathtt{SUM}\ (x : s) & = & x + \mathtt{SUM}\ s \\
 & & \\
\mathtt{COUNT}\ [] & = & 0 \\
\mathtt{COUNT}\ (x : s) & = & 1 + \mathtt{COUNT}\ s \\
 & & \\
\mathtt{MAX}\ [x] & = & x \\
\mathtt{MAX}\ (x : s) & = & x\ \mathtt{max}\ \mathtt{MAX}\ s \qquad \text{, provided } s \text{ is non-empty.} \\
 & & \\
\mathtt{MIN}\ [x] & = & x \\
\mathtt{MIN}\ (x : s) & = & x\ \mathtt{min}\ \mathtt{MIN}\ s \qquad \text{, provided } s \text{ is non-empty.}
\end{array}$$

Notice that MAX and MIN are not defined when the list is empty.

Splitting function over list

A function like SUM distributes over $+\!\!+$. That is:

$$\mathtt{SUM}\ (s +\!\!+ t) \ = \ \mathtt{SUM}\ s + \mathtt{SUM}\ t$$

This is a useful properties when implementing SUM. For example, it allows us to compute SUM recursively by splitting the input list. The property above suggests that it should not matter, in principle, where we split the list.

In Algebra people have a nice name for a function with a SUM-like property above: *homomorphism*[2] More precisely, given a function f and two operators $\oplus$ and $\otimes$, the function f is a *homomorphism* if it satisfies:

$$f\ (x \oplus y) = f\ x \otimes f\ y$$

Many useful list functions are homomorphisms over $+\!\!+$, for example:

[2] In Algebra a homomorphism is a structure perserving map from one set of values to another. In the case of SUM, it maps lists to integers. The property above can be also interpreted as follows: SUM consistently maps lists constructed with $+\!\!+$ to integers constructed with $+$.

Theorem 4.1.13 : Homomorphism of List Functions

1. $\vdash \quad x \in (s +\!\!+\, t) \quad = \quad x \in s \;\vee\; x \in t$
2. $\vdash \quad \texttt{SUM}\,(s +\!\!+\, t) \quad = \quad \texttt{SUM}\, s + \texttt{SUM}\, t$
3. $\vdash \quad \texttt{COUNT}\,(s +\!\!+\, t) \quad = \quad \texttt{COUNT}\, s + \texttt{COUNT}\, t$

□

Some functions are only homomorphic under a certain condition. For example $\texttt{MAX}\ (s +\!\!+\, t)$ is equal to $\texttt{MAX}\ s\ \texttt{max}\ \texttt{MAX}\ t$, but only if both s and t are non-empty (since MAX is undefined on an empty list):

Theorem 4.1.14 : Homomorphism of MAX and MIN

1. $\vdash \quad s \neq [] \wedge t \neq [] \quad \Rightarrow \quad (\texttt{MAX}\,(s +\!\!+\, t) \quad = \quad \texttt{MAX}\ s\ \texttt{max}\ \texttt{MAX}\ t)$
2. $\vdash \quad s \neq [] \wedge t \neq [] \quad \Rightarrow \quad (\texttt{MIN}\,(s +\!\!+\, t) \quad = \quad \texttt{MIN}\ s\ \texttt{min}\ \texttt{MIN}\ t)$

□

4.2 Proofs

We will show several examples of how to prove properties about list. Consider a list s. We will prove that if s contains an element x satisfying P, then $[e\ i \mid s\ \texttt{from}\ s,\ P\ i]$ is not empty. In fact, the two statements are equivalent, as formulated by the theorem below:

Theorem 4.2.1 : Non-empty Comprehension

$\vdash \quad (\exists i : i \in s : P\ i) \;=\; \neg([e\ i \mid i\ \texttt{from}\ s,\ P\ i] = [])$

□

The proof is quite simple:

```
EQUATIONAL PROOF ____________________________________________
¬([e i | i from s,  P i] = [])
= { rewrite with Theorem Empty Comprehension (T 4.1.6) }
¬(∀i : i ∈ s : ¬(P i))
= { rewrite with Theorem Negate ∀ (T A.4.7) }
(∃i : i ∈ s : ¬¬(P i))
= { trivial }
(∃i : i ∈ s : P i)
END ____________________________________________________
```

In the next example we will prove that $\texttt{COUNT}\ s$ cannot be negative. Stated formally, we want to prove this:

$$(\forall s :: \texttt{COUNT}\ s \geq 0)$$

Intuitively, the property is obvious. To prove it formally is a bit more complicated. It cannot be proven using basic equalities like in the previous proof. It requires something else, namely induction.

The induction principle over lists (also called *list induction*) is analogous to the natural number induction. It says that to prove that $P\ s$ holds for all lists s, it is sufficient to show $P\ []$ and $P\ s \Rightarrow P\ (x : s)$, for arbitrary x and s. Formally:

```
PROOF
[G:] (∀s :: COUNT s ≥ 0)
BEGIN ____________________________________________________________

  1 { follows from the definition of COUNT }   COUNT [] ≥ 0
  2 { see the proof below }   (∀x, s :: COUNT s ≥ 0 ⇒ COUNT (x : s) ≥ 0)

    PROOF
    ANY x, s
    [A:] COUNT s ≥ 0
    [G:] COUNT (x : s) ≥ 0
    BEGIN ____________________________________________________________
      1 { from the defition of COUNT }   COUNT (x : s) = 1 + COUNT s
      2 { follows from 1 }   COUNT (x : s) ≥ COUNT s
      3 { follows from 2 and A }   COUNT (x : s) ≥ 0
    END ______________________________________________________________
  3 { list induction on 1 and 2 }   (∀s :: COUNT s ≥ 0)

END ______________________________________________________________
```

Figure 4.1: *An example of a proof using induction.*

Rule 4.2.2 : List Induction

$$\frac{\begin{array}{l} P\ [] \\ (\forall x, s :: P\ s \Rightarrow P\ (x : s)) \end{array}}{(\forall s :: P\ s)}$$

□

The $P\ s$ in the second premise is also called *induction hypothesis*.

To use induction to prove that COUNT $s \geq 0$, we have to prove two things: (1) COUNT $[] \geq 0$, which is quite trivial, and (2) COUNT $s \geq 0$ implies COUNT $(x : s)$ is also at least 0. The complete proof is shown in Figure 4.1. The step marked with (*) is where we use the induction hypothesis (A1).

□□□ _____

New Vocabulary

distributivity; homomorphism; lambda notation; list comprehension (generator, condition, transformer, nested); list enumeration; list induction

4.3 Exercises

1. Write formulas to express the following texts:

   (a) `k` is the index pointing to a maximum element of an array `a` in the domain `0≤i<n`.

   (b) In the domain `0≤i<n`, the array `a` has at least two maximum elements.

   (c) `k` is the length of the longest prefix of an array `a` within the domain `0≤i<n` that consists only of zeros.

2. Prove that if s contains only even integers then `SUM` s is also even.

3. The functions `map` and `filter` are defined as follows:

$$\begin{array}{lcl} \texttt{map}\ f\ [] & = & [] \\ \texttt{map}\ f\ (x : s) & = & f\ x : \texttt{map}\ f\ s \\ \\ \texttt{filter}\ p\ [] & = & [] \\ \texttt{filter}\ p\ (x : s) & = & (p\ x \rightarrow [x] \mid []) \mathbin{+\!\!+} \texttt{filter}\ p\ s \end{array}$$

   Function composition $\circ$ is defined as follows:

$$(f \circ g)\ x \ = \ f\ (g\ x)$$

   Prove the following properties:

   (a) $\texttt{map}\ f\ (\texttt{map}\ g\ s) \ = \ \texttt{map}\ (f \circ g)\ s$

   (b) $\texttt{map}\ f\ (\texttt{filter}\ p\ s) \ = \ [f\ x \mid x\ \texttt{from}\ s,\ p\ x]$

4. Consider a function f defined recursively as follows:

$$\begin{array}{lcl} f\ [] & = & e \\ f\ (x : s) & = & x \oplus f\ s \end{array}$$

   Prove that if $\oplus$ is associative with e as its unit, then f is a homomorphism:

$$f\ (s \mathbin{+\!\!+} t) \ = \ f\ s\ \oplus f\ t$$

   Based on this, give some examples of a useful f which is homomorphic over $\mathbin{+\!\!+}$.

5. Theorem 4.1.11 is used to split an 'enumerated' array. Prove it.

CHAPTER 5

Program and Specification

To discuss programs we will assume a small and simple programming language which we will just call uPL. It has the basic features of an imperative language, but lacks sophisticated features such as pointers, exception, and concurency. This is a deliberate choice to let you focus on the basic principles. Of course in practice, when you deal with real programs, you also have to deal with advanced features, for which you will need a programming logic which is more advanced than uPL's logic. You can find such a logic in the literature, e.g. [23] to handle C programs, or [14] for Java programs.

All programs we will discuss in this book are toy programs; these are small programs designed for exposing the principles and the use of our proof techniques. They are not specifically made to solve real problems, but you should not immediately dismiss their value because of this. In fact, proving the correctness of some toy programs is more challenging than that of many real programs.

5.1 Program

As an example here is again the program `Euclid`; it is already in of a uPL :

```
Euclid(x,y : int) : int {
   while x<>y do if x>y then x:=x-y else y:=y-x ;
   return x
}
```

Available statements in uPL are assignments, sequence, `while` loop, `if-then-else`, and a limited form of program call; `goto` and exception are excluded. Available types are basic types and compound types. Basic types are `()` (void), `int`, `bool`, `char`, and `string`. Compound types are array and record. In Appendix C you can find uPL's EBNF grammar.

Array

Arrays can be one dimensional or multi-dimensional. For example, `int[]` is the type of one dimensional arrays of integers and `int[][]` is the type of two dimensional arrays of integers.

The `i`-th element of a one dimensional array `a` is denoted by `a[i]`. If `a` is two dimensional, the notation `a[i][j]` is used to refer to its elements.

To keep uPL's logic simple we will assume that all uPL arrays are *infinitely large*. So, there can be no error because an array is accessed outside its range. Of course, in a real setup you should not rely on such an assumption.

Record

A record type can be defined like this:

```
record Person = {name:string; age:int}
```

The notation used to access the fields of a record is quite standard: if `r` is a value of type `Person`, `r.name` and `r.age` refer to the values stored in `r`'s fields.

Operators

Most imperative programming languages only allow simple expressions in their programs. In particular, they do not typically allow us to write a quantified expression like ($\exists$`i` :: `a[i]` = `0`). So, we will not assume it either in uPL. Available uPL operators are listed below —most of them are quite standard. The operators $\wedge$, $\vee$, and $\neg$ are used to write respectively the Boolean *and*, *or*, and *not* operators. Other languages often denote them by `&&`, `||`, and `!`. All uPL operators are also Form operators, and all uPL expressions are also Form expressions. However, not all Form expressions are also uPL expressions. In particular, as said above, quatified expressions are not allowed in uPL.

`^` (L)
`*` (LR)
`+` (LR), `-` (L)
`mod`, `div`, `max` (LR), `min` (LR)
$<$, $>$, $\neq$, $\leq$, $\geq$
$\wedge$ (LR)
$\vee$ (LR)
`=`

The operators are listed in the decreasing order of priority. The flag L and R specify their associativity —see Chapter 2 for their meaning.

Note that the symbol `=` denotes the operator used to test equality. It does not denote assignment as for example in Java. It is *not* associative, and unlike in many programming languages, `=` in uPL has a low priority.

Local Variables

uPL allows you to declare local variables. In the example below `h` is declared as a local variable:

```
SWAP (OUT x,y:int) : ()
  { var h : int ;
    h := x  ; x := y ; y := h }
```

Aliasing

A computer's memory can be thought to consists of data cells. Program variables represent data stored in these cells. To access a specific cell, the computer must know which cell it is. So, each cell has an *address*, also called *reference* or *pointer*. This address is however specified in a binary string, which is too difficult for human to remember. So, we use symbolic names like `x` and `y` which are easier to remember. A compiler maps these names to binary addresses pointing to where the corresponding data are stored in the actual memory.

In Java an assignment x=y will make the value of x equal to y. However when x and y represent objects the actual values of these variables are pointers to their respective objects. The assignment will actually just copy the pointer to y's object to x. So now both point to the same object. We also say that x becomes an *alias* of y. Aliasing improves efficiency and is a powerful feature, but it also greatly complicates reasoning. If you change y, the logic will have to take into account that the change will also influence all aliases of y. In general the aliases cannot be determined statically (that is, just by analyzing the syntax of a program).

For simplicity in uPL we do *not* have aliasing. More specifically: two *distinct variables names*, e.g. x and y, represent two totally disjoint sets of memory cells. So, an assignment to one will not affect the other. It also follows that a uPL assignment x:=y does not copy a pointer as in Java. Instead, it copies the value of y to x, even if y is a large data structure.

Of course for languges where aliasing is pervasive, like Java, this non-aliasing assumption is unrealistic —you would need a more powerful logic to deal with the issue.

Parameter Passing

Parameters are used to pass data to and from a program. The most common ways to pass parameters to a program are the so-called *pass-by-value* and *pass-by-reference*. Consider a program with header `P(a)`. Suppose we want to pass an expression `x + y` as a parameter to `P` in the call `P(x+y)`. To pass `x+y` by value means that it is first evaluated (computed), and the resulting value is copied to `a`, which acts as a local variable to `P`. Since `a` is local to `P`, whatever `P` does to `a` will not influence any variable of its caller.

Now consider a program with header `Q(b)`, which is called within another program `R`:

```
R() {var x:int ... ; Q(x) ; ... }
```

Rather than passing the value of `x` to `Q` we can also pass its reference (address). This is called *pass-by-reference*. However, unlike in the pass-by-value scheme, changes made by `Q` on this `x` will affect the `x` in `R`. In this respect pass-by-reference is more error prone. On the other hand, it is more efficient, especially if you have to pass large data.

There are some technical subtleties involved in the logical treatment of pass-by-reference, in particular in conjunction with aliasing and concurrency issues. In uPLwe decide not to have pass-by-reference parameters. Instead we have pass-by-copy-restore.

When a parameter x is passed by copy-restore to a program P, it is first evaluated. The result is copied to a local variable of P, as in pass-by-value. However, when P is about to return, the final value of this local variable is copied over to x —this is called, perhaps a bit misleading, *restore* action. This scheme is just as inefficient as pass-by-value. However, under some conditions it can be equivalently replaced by pass-by-reference.

Furthermore, in uPL a parameter of an array type is by default passed by copy-restore, except when it is marked with the `READ` keyword. Parameters of other types are by default passed by value, unless if they are marked with the `OUT` keyword.

A uPL program can return a value. The usual `return` statement is used to return a value. However, to simplfy the logic `return` can only appear as the last statement in the body of a uPL program.

Specification: this program searches an element in `a`*, returning the position where it can be found.*

```
BS(READ a:int[], i,s:int) : int
  { var m,mid,k : int ;
    k := 0 ;
    m := s ;
    while m ≠ k + 1 do
      { mid := (k+m) div 2 ;
         if a[mid]<=i then k:=mid else m:=mid } ;
    if a[k]=i then skip else k=-1 ;
    return k
  }
```

Figure 5.1: *An example of an informal specification*

Program Call

To simplify the logic the syntax of program call will be restricted. We will discuss this in Chapter 7.

Scoping

To simplify the logic, any variable in a uPL program should be either a local variable or one of the program's parameters. This means that a uPL program is not allowed to access a global variable, unless it is passed to it as a parameter.

5.2 Specification

A specification of a program is just a piece of text describing what we expect the program will do. Many people just use plain English to write a specification, e.g. as in Figure 5.1. Such a specification is called *informal*. Most specifications in practice are informal, which is not surprising because informal specifications are quite easy to write. However, we cannot verify a program with such a specification because it is often imprecise and ambiguous. For example, in the specification in Figure 5.1 it is not clear, unless we read the code: where does 'it' refer to? Does it mean `i` or perhaps `s`? And what to expect when the searched element cannot be found.

Rather than plain English, we can use a formal language like Form to express specifications. Sentences of a formal language has a well-defined meaning. A specification expressed in a formal language is called a *formal specification*.

You can express a lot with Form ; but it is still a a small specification language. In practice you may prefer more sophisticated languages, such as OCL (Object Constraint Language) [28], JML (Java Modelling Language) [18], or Z [8]. OCL is part of the UML methodology. JML is used to specify Java programs. Z is based on a set theory and is more generic.

There is by the way a trade-off to be made in the choice of the specification language. A more expressive specification language allows more delicate properties of a program to be expressed. But usually these properties are also more difficult to prove. The situation is reversed if we choose for a less expressive language.

We will specify programs and statements with so-called Hoare triples [13]. Let S be a uPL program or statement, and P, Q be Form predicates. A Hoare triple is a triple of the form:

$$\{* \, P \, *\} \;\; S \;\; \{* \, Q \, *\}$$

In the context of such a specification, P is called the *pre-condition* and Q is called the *post-condition*. Such a specification states that if S is executed in a state that satisfies P, it will terminate in a state that satisfy Q. The pre-condition can be thought to specify allowed initial states to execute S. The post-condition specifies its possible final states. Trivial examples of valid specifications are these:

$$\{*\ \mathtt{x} = 0\ *\}\ \ \mathtt{x} := \mathtt{x}{+}1\ \ \{*\ \mathtt{x} = 1\ *\}$$

$$\{*\ \mathtt{x}{\geq}0\ *\}\ \ \mathtt{y} := \mathtt{x}\ \ \{*\ \mathtt{y}{\geq}0\ *\}$$

An example of a specification which is not valid:

$$\{*\ \mathtt{true}\ *\}\ \mathtt{skip}\ \{*\ \mathtt{x} = 1\ *\}$$

We have used the term 'valid specification'. More precisely, a specification is *valid* if all possible executions (runs) of the specified program satisfy the specification. Otherwise it is *invalid*. Validity is usually shown by giving a mathematical proof. Invalidity is shown by giving a *counter example*, which in this case is a run that does not satisfy the specification. In terms of programming, such a counter example is called a *bug*. Note that *testing*, even if we do that with tools, can show invalidity, but it cannot prove validity.

We also say that a program *satisfies* a specification to mean that the specification is a valid specification. The *correctness* of a program is always expressed with respect to some specification. So, saying "program S is correct" does not make sense unless you also give a specification which S has to satisfy. As a side note, any program S satisfies this specification:

$$\{*\ \mathtt{false}\ *\}\ S\ \{*\ Q\ *\}$$

So, any S is trivially 'correct' in this sense (so, it is also not a useful specification, except for a theoretical purpose).

In addition to using Form predicates to specify initial and final states of a program, we can also use it to specify its possible states at any control point. Here is an example:

$$\{*\ \mathtt{x}{>}0\ *\}\ \mathtt{y} := \mathtt{x};\ \ \{*\ \mathtt{y}{>}0\ *\}\ \ \mathtt{y} := \mathtt{y} - 1\{*\ \mathtt{y}{\geq}0\ *\}$$

With the predicate $\mathtt{y}{>}0$ between the two assignments we mean to say that when the program is in that control location, its state should satisfy the predicate. So, in the example above the specification is quite obviously valid.

A Form predicate used inside $\{*\ -\ *\}$ is also called an *assertion*: it asserts some constraint on the program's state at that point. Pre- and post-conditions are assertions too.

A specification can be extended by inserting new assertions in it. For example, suppose we have this specification (it does not say much about the final state, but it does say that $S_1;\ S_2$ terminates):

$$\{*\ \mathtt{true}\ *\}\ S_1;\ S_2\ \{*\ \mathtt{true}\ *\}$$

We can extend it to:

$$\{*\ \mathtt{true}\ *\}\ S_1;\ \{*\ \mathtt{x} = 0\ *\}\ S_2\ \{*\ \mathtt{true}\ *\}$$

which also demands that after S_1 we have $\mathtt{x} = 0$. Inserting a new assertion is safe in the sense that it cannot make an invalid specification valid. However, it may make your specification 'stronger'. That is, it demands more, and thus is also more difficult, perhaps even impossible, to satisfy.

Total and Partial Correctness

Remember that $\{*\ P\ *\}\ S\ \{*\ Q\ *\}$ also means that S *terminates* when executed in a state satisfying P. In some setups we may deliberately choose to *assume* termination rather than requiring the proof of it. In the literature, such an interpretation of Hoare triple is also called *partial correctness* interpretation. It may be accceptable when, for example, we are dealing with programs whose termination is obvious. The default notion of correctness used by uPL includes termination, and is called *total correctness*. As an example, the following specification is not always valid under total correctness:

$$\{*\ P\ *\}\ S\ \{*\ \texttt{true}\ *\}$$

because S may not terminate. It is however valid under partial correctness: S is assumed to terminate; so it will terminate in some state s; and any state satisfies the `true` predicate.

Black Box and White Box

Some specification may not reveal the code of the program being specified. Such a specification is called *black box* specification. In this context, the term *white box* is used to refer to a specification that does reveal the program code. An example of a black box specification:

$$\{*\ \texttt{0}\leq\texttt{n}\ *\}\ \ \texttt{SUMN}(\texttt{n}:\texttt{int})\ \ \{*\ \texttt{return} = \texttt{n} * (\texttt{n}-\texttt{1})\texttt{div2}\ *\} \tag{5.1}$$

We only see the header of `SUMN`, but not its body. Despite this fact, the specification still reveals useful information: we know that when it is called with a non-negative `n` it will *terminate*, and we know exactly what the return value will be.

Obviously you cannot prove the validity of a black box specification —for that you need its full code. However, once a specification is proven, there are a number of good reasons to use it as a black box:

- By relying only on the specification rather than the code, you force yourself to reason more abstractly.
- If you only rely on the specification you can still change the program code, even after the program is deployed in a running application, as long as its specification is maintained.
- A black box informs your customers what the functionality of your program is, without having to expose the source code.

Initial Values

In a specification we sometimes need to refer to the initial values of some variables. Consider a program `P` with the following header:

```
P (OUT x:int) :  ()
```

We want to specify that `P` should increase the value of `x`. There is no way we can specify this in the post-condition without referring to the initial value of `x`. One way to do it is to allow a notation like this:

$$\{*\ \texttt{true}\ *\}\ \ \texttt{P}(\texttt{x})\ \ \{*\ \texttt{x} > \texttt{old}\ \texttt{x}\ *\}$$

where `old x` refers to the value of `x` when it is passed to `P`. Although convenient, the use of the `old` operator complicates the logic [24]. To keep it simple, here we will use another trick. We add so-called *auxiliary variables*. These are fresh variables added to the program. We can subsequently extend the program, for example with assignments to these variables. However you can only extend the program in a way that does not change the the original control flow. With respect to

the original set of variables, the new and the old program should be equivalent. Auxiliary variables are used to record the values of actual program variables at certain control points.

Let us now introduce an auxiliary variable X to record the initial value of x. Here is now how we specify the program P:

$$\{*\ \texttt{true}\ *\}\ \ \texttt{X} := \texttt{x};\ \texttt{P(x)}\ \ \{*\ \texttt{x>X}\ *\}$$

As another example, here is how you can specify a program for swapping the value of two variables. X and Y below are auxiliary variables.

$$\{*\ \texttt{true}\ *\}\ \ \texttt{X,Y} := \texttt{x,y};\ \texttt{SWAP(OUT x,y : int)}\ \ \{*\ (\texttt{x} = \texttt{Y}) \wedge (\texttt{y} = \texttt{X})\ *\}$$

Specifying a Program

When specifying a *statement*, a (free) variable x in the post-condition refers to the value of the program variable x when the statement terminates. When specifying a full program we use a slightly different convention. A program is different from a statement (consult uPL grammar in Appendix C) because it has a name, it may have parameters, and it can be called by another program. Firstly, in a program's post-condition we can use the keyword `return` to refer to the value returned by the program. We have seen an example of this in the specification of SUMN in (5.1).

The second difference can be better explained with an example. Consider this contrived program:

```
copy(x:int, OUT y:int) : int
   { y:=y+x
   ; x:=y-x
   ; y:=y-x
   ; return x }
```

The program copies the value of x to y, and return the original value of y. Notice that x is passed by value. So, even though copy changes its value, it does so only on its own copy of x. The caller will see that x is unchanged. For this reason we use the convention that if x appears in the post-condition of copy, it refers to its value when it is passed to copy. The same convention applies to read-only pass-by-copy-restore parameter.

With the above convention, we can specify copy as follows:

$$\{*\ \texttt{true}\ *\}\ \ \texttt{Y} := \texttt{y};\ \texttt{copy(x,OUT y)}\ \ \{*\ (\texttt{y} = \texttt{x}) \wedge (\texttt{return} = \texttt{Y})\ *\}$$

Notice that the x in the post-condition refers to the value of x when it is passed, whereas y in the post-condition refers to the final value of y.

Typically, we also require that a program's specification is *closed*. That is, it does not mention any variable other than the variables in the program's formal parameters or the auxilliary variables used to remember the parameters' initial values. This makes sure that such a specification can be reused in any context.

ASSUMING Section

Consider again the code of BS in Figure 5.1. The program is used to check if the value of i occurs in a[0 ... s). If it does, a non-negative value is returned, and otherwise a negative value is returned. We can easily express this formally:

$$\texttt{return}<0\ =\ \texttt{i} \in \texttt{a}[0\ldots\texttt{s})$$

However this does not cover all functionalities of the program. If i can be found, the return value also points to where i can be found in a. This can be expressed as follows:

$$\texttt{return}\geq 0\ \ \Rightarrow\ \ (\texttt{a}[\texttt{return}] = \texttt{i})$$

You may notice that `BS` implements the *binary search* strategy. It only works if `n>0` and if `a` is sorted ascendingly. These facts seem to be 'forgotten' in BS's informal specification. We now want to make it explicit in our formal specification. The following formula can be used to express that a is *sorted ascendingly* :

$$(\forall \mathtt{p}, \mathtt{q} : 0 \leq \mathtt{p} < \mathtt{s} \wedge 0 \leq \mathtt{q} < \mathtt{s} : \mathtt{p} \leq \mathtt{q} \;\Rightarrow\; \mathtt{a}[\mathtt{p}] \leq \mathtt{a}[\mathtt{q}])$$

Figuring out the intuition behind a quantified formula like this can be quite hard. This can be improved with abbreviations. For example, we can introduce the abbreviation `sorted a s` to stand for the formula above. You can choose an abbreviation's name that gives a hint about its meaning. This often improves the readability of a specification. If you need to document the abbreviations explicitly in your specification, you can do so in the `ASSUMING` section —see the example below.

$\{* \; \mathtt{n}>0 \;\wedge\; \mathtt{sorted\ a\ s} \; *\}$

`BS(READ a:int[], i,s:int) : int` { *program body* }

$\{* \; (\mathtt{return}<0 \;=\; \mathtt{found\ a\ s\ i}) \;\wedge\; (\mathtt{return} \geq 0 \;\Rightarrow\; (\mathtt{a}[\mathtt{return}] = \mathtt{i})) \; *\}$

`ASSUMING`

$\mathtt{sorted\ a\ s} \;=\; (\forall \mathtt{p}, \mathtt{q} : \; 0 \leq \mathtt{p} < \mathtt{s} \;\wedge\; 0 \leq \mathtt{q} < \mathtt{s} \;:\; \mathtt{p} \leq \mathtt{q} \;\Rightarrow\; \mathtt{a}[\mathtt{p}] \leq \mathtt{a}[\mathtt{q}])$

$\mathtt{found\ a\ s\ i} \;=\; \mathtt{i} \in \mathtt{a}[0 \ldots \mathtt{s})$

□□□

New Vocabulary

address; alias; assertion; ASSUMING section; auxiliary variable; non-aliasing assumption; partial correctness; pass by copy-restore; pass by reference; pass by value; pointer; reference; specification: black box, white box, valid, invalid, satisfy, correctness, counter example; total correctness

5.3 Exercises

1. Consider the black box specifications below. What does each program do according to its specification? Express it in plain english.

   (a) $\{*$ true $*\}$
   `X:=x; inc(d:int, OUT x:int)`
   $\{*\ (\mathtt{x} = \mathtt{X} + \mathtt{d})\ \wedge\ (\mathtt{return} = \mathtt{X})\ *\}$

   (b) $\{*$ true $*\}$
   `zero(READ a:int[], n,k:int)`
   $\{*\ \mathtt{return}\ =\ (\forall \mathtt{i} :\ 0 \leq \mathtt{i} < \mathtt{n} :\ (\mathtt{a}[\mathtt{i}] = 0) \Rightarrow (\mathtt{i} = \mathtt{k}))\ *\}$

   (c) $\{*$ true $*\}$
   `find(READ a:int[], n,x:int)`
   $\{*\ \mathtt{x} \in \mathtt{a}[0 \ldots \mathtt{n})\ \Rightarrow\ (\mathtt{a}[\mathtt{return}] = \mathtt{x})\ *\}$

   (d) $\{*$ true $*\}$
   `isINC(READ a:int[], n:int)`
   $\{*\ \mathtt{return}\ =\ (\forall \mathtt{i}, \mathtt{j} :\ 0 \leq \mathtt{i}, \mathtt{j} < \mathtt{n} :\ \mathtt{a}[\mathtt{i}] \geq \mathtt{a}[\mathtt{j}]\ \Rightarrow\ \mathtt{i} \geq \mathtt{j})\ *\}$

   (e) $\{*$ true $*\}$
   `maxTprefLen(READ b:bool[], n:int)`
   $\{*\ \mathtt{return}\ =\ \mathtt{MAX}\ [\mathtt{i}\ \mid\ \mathtt{i}\ \mathtt{from}\ [0..\mathtt{n}],\ \mathtt{allT}\ \mathtt{b}\ \mathtt{i}]\ *\}$
   `ASSUMING`
   $\mathtt{allT}\ \mathtt{b}\ \mathtt{right}\ =\ (\forall \mathtt{j} :\ 0 \leq \mathtt{j} < \mathtt{right}\ :\ \mathtt{b}[\mathtt{j}])$

2. What do you think the following uPL program does? Capture it in a formal specification.

```
iP (n:int) : bool
  { var d : int ;
    var b : bool ;
    b := true ; d := 2 ;
    while b ∧ d<n do
      { b := n mod d ≠ 0
      ; d := d+1 } ;
    return b
  }
```

3. Write a specification for a program `getMAX`. When given an array `a` and a positive integer `n`, the program should return the greatest element of `a` in the domain $0 \leq \mathtt{i} < \mathtt{n}$.

4. We decide to extend `getMAX` so that it also checks if the computed maxium is unique. That is, if `m` is the returned value, then the program also checks if `m` only occurs once in `a` within the the domain $0 \leq \mathtt{i} < \mathtt{n}$. Extend your specification to also reflect this additional functionality.

5. Give a specification for a program that reverses the order of the elements of an array in the domain $0 \leq \mathtt{i} < \mathtt{n}$.

CHAPTER 6

The Basic Proof System

In Chapter 5 you have learned how to precisely specify what a program is required to do. Once specified, we still have to show that the program is correct with respect to its specification. Informal reasoning is useful to motivate why the program is correct (or otherwise), but it is not something that we really can rely on, since it is ambiguous and lacks in precision.

In formal reasoning you will have to deal with formulas. Typical operations that you want to do are rearranging a formula, eliminating some parts of it, or adding new parts to it. In principle, you can do these operations without being aware of what your formulas mean. However, trying out different operations without having an idea of what you are doing or where you want to go is like walking in a thick forest without a compass —it is unlikely that you can succeed in finding a proof in this way. The compass that you need is *abstraction*. Literally it means a visionary idea[1]. Abstraction is the mental picture that you have in your mind of what the formulas in your proof mean. This mental picture is much simpler than the formulas they represent, and thus is easier to comprehend. It allows you to make a mental plan or strategy of how to solve a problem. So, you will need your skill to think abstractly. Once you have your strategy, you can work it out in the form of a formal proof. A formal proof is very precise. It is much more reliable. However, being very precise also means that you have to deal with a lot more details than in informal reasoning. You will have to learn the skill to handle these details.

6.1 Abstract Model

Before we proceed with discussing our programming logic, it is useful to first look at models of programs. A *model* is a simplified representation of a real thing. Models are thus much easier to understand. For us they help in explaining some of the more technical aspects of the programming logic.

[1] In computer science, the term *abstraction* is also used technically to mean a simplified, and usually formal, representation of something. In a sense, a visionary idea is a also simplified view of the implementation of the idea.

Because a model and the program it represents are two different things, you may want to denote them differently. For example, if S is a program, then perhaps we can write $[|S|]$ to denote the model of S. However, this clutters our formulas and hinders our discussion. So we decide not to do so. That is, we write S to refer to both the program S and its model. Hopefully from the context it would be obvious which one is meant in a particular use.

6.1.1 State

A program have states. At any moment it is in a particular state, and each step of the program causes it to go from one state to another. The actual state of a program is very complex, comprising of the content of the memories of your computer, the content of various registers of your computer's processors, etc. However, abstractly the *state* of a program comprises of the value of all its variables at a particular moment. For example if we recall the program `Euclid`:

```
Euclid(x,y : int) : int {
   while x<>y do if x>y then x:=x-y else y:=y-x ;
   return x
}
```

the state of this program is the value `x` and `y` at a given moment. Let us write e.g. {`x=0, y=1`} to denote a state where the value of `x` is 0, and that of `y` is 1.

So, we can say for example that if {`x=4, y=8`} is the state of `Euclid` as we enter its body, then {`x=2, y=2`} would be its state just before it returns.

Each statement in a program can be thought to move the program from one state to another. The state where a program or statement is executed on is called its *initial state* or *starting state*. The state at which it stops is called *terminal state* or *final state*.

6.1.2 Modelling Program as Relation

In mathematics, a *relation* on a domain A is a function of type $A \to Pow(A)$, where $Pow(A)$ denotes the set of all subsets of A (it is the so-called *power set* of A). So, if R is a relation on A, $y \in R\ x$ means that R relates x to y. A relation can also be interpreted as a function that returns multiple possible results. So if $R\ x = U$ then firstly U is a set. It is also a subset of A. Furthermore, U can be seen as specifying all possible results of applying the 'function' R to x.

We can abstractly model a program as a relation on states. It is a simple and easy to understand model. For simplicity, we will for now *assume that all our programs terminate*.

Let $\sum$ be the set of all possible states that a program S may have. The program S can be modelled as a relation on $\sum$. So, it is a function of type:

$$\sum \to Pow(\sum)$$

So, if s is a state in $\sum$, then $S\ s = V$ means that V is the set of all possible final states of S when it is executed on the state s. Since we assumed S to terminate, V will contain at least one element. For example, in terms of the above kind of models the `skip` statement is defined like this:

$$\texttt{skip s} \quad = \quad \{\texttt{s}\}$$

It says that if we execute `skip` on a state s, then there is only one possible final state, which is s itself.

Other statements, e.g. an assignment is also a relation on $\sum$. So, we can apply it to a state to get its possible final states, e.g.:

$$(\texttt{x} := \texttt{x+1})\ \{\texttt{x=0, y=1}\} \quad = \quad \{\{\texttt{x=1, y=1}\}\}$$

Most (non-interactive) sequential programs are *deterministic*. If we repeatedly started such a program in the same state s, the program will always behave in the same way; thus ending in the same state. Programs that do not have this property are called *non-deterministic*. In terms of the above model, S is deterministic if $S\ s$ contains exactly one state. Otherwise, if for some s there are more that one (terminal) state in $S\ s$, then S is non-deterministic.

Interactive or concurrent programs programs are typically non-deterministic. Non-determinism may simplify or complicate analysis. In the case of concurrent programs, it is usually the latter.

We can now define the meaning of some standard statements in terms of models as explained above:

Definition 6.1.1 : Models of Skip, Conditional, and Sequence

1. $\texttt{skip}\ s\ =\ \{s\}$
2. $(\ \texttt{if}\ g\ \texttt{then}\ S_1\ \texttt{else}\ S_2)\ s\ \ =\ \ g\ s \rightarrow S_1\ s \mid S_2\ s$
3. $(S_1\ ;\ S_2)\ s\ \ =\ \ \cup\{S_2\ t \mid t \in S_1\ s\}$

□

where $\cup X$ means the union of all sets in X. So, $\cup\{U, V\} = U \cup V$.

Notice that the above definition excludes exceptions. You can see this in e.g. the definition of $S_1; S_2$, which simply passes on the final state t after S_1 to S_2, whereas if we are to allow exception then S_2 may be skipped if S_1 throws an exception. We will not have exception in uPL . In practice you will of course have to deal with that, and would need a more powerful logic that that of uPL to deal with it.

Because assignments affects states, defining their models requires us to first come up with a model of states. If we for example model a state as a function from variable names to their value, and an expression as a function from state to value, then we can model assignents as follows:

Definition 6.1.2 : Model of Assignment

$$(x := e)\ s\ \ =\ \ (\lambda v.\ v = x \rightarrow e\ s \mid s\ x)$$

□

6.1.3 Predicate as a Set of States

When we write a predicate like `x>y` as e.g. a pre-condition, we intend to interpret it over states. That is, when given a particular state the value of the predicate would be either true or false. Because we will use this concept frequently later, let us also introduce a notation for that.

For a predicate P and a state s we will write $P\ s$ to mean the value of the predicate P when interpreted on the state s. For example we can write:

$$(\texttt{x>y})\ \{\texttt{x} = \texttt{1},\ \texttt{y} = \texttt{0}\}$$

By this we mean the value of the predicate `x>y` interpreted on the state $\{\texttt{x} = \texttt{1},\ \texttt{y} = \texttt{0}\}$. This value is in thus true.

The term 'a state s satisfies a predicate P' is thus the same as '$P\ s$ is true'.

A predicate, such as `x>y`, can be thought to specify a *set of states* of a program. So if we talk about a program S whose set of possible states is $\sum$, a predicate P specifies a set containing all state s in $\sum$ such that s satisfies P. So, this set:

$$\{s \mid s \in \sum,\ P\ s\}$$

This is called the *set interpretation* of the predicate P. Such an interpretation explains why we use predicates to specify programs. E.g. in $\{*\ P\ *\}\ S\ \{*\ Q\ *\}$ the predicate P is interpreted as specifying the *set of allowed initial states*, and Q the *set of all possible final states*.

The set interpretation of a predicate is an *equivalent* interpretation. It is sometimes useful to make certain things easier to understand. In fact, in programming logic predicates are often treated as if they are just sets. For example, let P be a predicate, people often say "*the set* P" to actually mean the set of states specified by the predicate P. People also say "*a state* s *is in* P" to actually mean that s is in the set of states specified by the predicate P (or equivalently, s is a state such that P is true).

Recall that we have introduced the notation $P\ s$ to mean that that the value of the predicate P is true when interpreted on the state s. When P is interpreted as a set, then $P\ s = s \in P$.

Theorem 6.1.3 :
Let P be a predicate and s a state:

$$P\ s \ = \ s \in P$$

□

Notice that implication corresponds to set inclusion. That is, $\vdash P \Rightarrow Q$ is equivalent with the fact that P, interpreted as a set, is a subset of (included in) Q. Hence, a weaker predicate is larger, and a stronger one is smaller. Analogously, the operator $\vee$ and $\wedge$ correspond to respectively set union and set intersection.

6.1.4 The Formal Meaning of Hoare Triples

Now we can give a meaning to specifications in terms of our models. However, since our models assume termination, with them we can only express the meaning of specifications under the *partial correctness* interpretation. Recall that under partial correctness, this:

$$\{*\ P\ *\}\ S\ \{*\ Q\ *\}$$

means that if S is executed on a state in P then, if it terminates, its final state will be in Q. Recall that we model S as a relation on states. In terms of our models, the above specification means this:

Definition 6.1.4 : Hoare Triple in Partial Correctness

$$\{*\ P\ *\}\ S\ \{*\ Q\ *\} \quad = \quad (\forall s : s \in P : S\ s \subseteq Q)$$

□

The programming logic that we will introduce later will be in total correctness. To give you a deeper understanding of some of the logic's inference rules sometimes we also show their proofs. However, to make these proofs easy to follow they will be given in terms of partial correctness using the models discussed in this section. So, keep in mind that formally those proofs are not complete (we have yet to prove termination).

From the above definition, we can for example already infer the following 'principles':

1. Suppose $\{*\ P\ *\}\ S\ \{*\ Q\ *\}$. This implies that for any state s in P we have $S\ s \subseteq Q$. If Q' is weaker than Q, then $Q \subseteq Q'$. It follows that $S\ s \subseteq Q'$. Hence, S also satisfies Q' as a post-condition. This principle is also called *post-condition weakening*.

2. Suppose $\{*\ P'\ *\}\ S\ \{*\ Q\ *\}$. This implies that *any* state $s' \in P'$ we have $S\ s' \subseteq Q$. If P is stronger than P', then $P \subseteq P'$. So, any state $s \in P$ will also be in P', and thus we also have $S\ s \subseteq Q$. Therefore S also satisfies P as a pre-condition. This principle is also called *pre-condition strengthening*.

3. A bit more sophisticated, suppose S satisfies these two specifications:

   (a) $\{*\ P_1\ *\}\ S\ \{*\ Q_1\ *\}$

Rule 6.1.5 : Post-condition Weakening

$$\frac{\models Q \Rightarrow Q' \qquad \{*\ P\ *\}\ S\ \{*\ Q\ *\}}{\{*\ P\ *\}\ S\ \{*\ Q'\ *\}}$$

Rule 6.1.6 : Pre-condition Strengthening

$$\frac{\models P \Rightarrow P' \qquad \{*\ P'\ *\}\ S\ \{*\ Q\ *\}}{\{*\ P\ *\}\ S\ \{*\ Q\ *\}}$$

Rule 6.1.7 : Hoare Triple Disjunction

$$\frac{\{*\ P_1\ *\}\ S\ \{*\ Q_1\ *\} \qquad \{*\ P_2\ *\}\ S\ \{*\ Q_2\ *\}}{\{*\ P_1 \vee P_2\ *\}\ S\ \{*\ Q_1 \vee Q_2\ *\}}$$

Rule 6.1.8 : Hoare Triple Conjunction

$$\frac{\{*\ P_1\ *\}\ S\ \{*\ Q_1\ *\} \qquad \{*\ P_2\ *\}\ S\ \{*\ Q_2\ *\}}{\{*\ P_1 \wedge P_2\ *\}\ S\ \{*\ Q_1 \wedge Q_2\ *\}}$$

Figure 6.1: *Some basic rules for Hoare triples.*

(b) $\{*\ P_2\ *\}\ S\ \{*\ Q_2\ *\}$.

You can also prove the following:

(a) From any state s in $P_1 \cup P_2$, the set of possible final states, $S\ s$, is a subset of $Q_1 \cup Q_2$. In other words, S satisfies $\{*\ P_1 \vee P_2\ *\}\ S\ \{*\ Q_1 \vee Q_2\ *\}$.

(b) From any state s in $P_1 \cap P_2$, the set of possible final states, $S\ s$, is a subset of $Q_1 \cap Q_2$. In other words, S satisfies $\{*\ P_1 \wedge P_2\ *\}\ S\ \{*\ Q_1 \wedge Q_2\ *\}$.

The principles are called *Hoare triples disjunction* respectively conjunction.

Though these are simple principles, we can already use them to formally reason about programs and specifications. However, as you can see in the above explanation, we have to keep translating predicates to sets. While this is useful when trying to explain and justify a 'principle' or an inference rule, we do not want to keep doing it when reasoning about programs. For the latter, we will keep our proofs at the predicate level; it is more abstract that way.

The above 'principles' are formally expressed by the inference rules shown in Figure 6.1.

6.1.5 Expressing Non-termination

Just for completeness, we will briefly discuss here how to extend the models above so that we can express total correctness. This discussion is theoretical in nature. We prefer not to, and we are not going to, write the correctness proofs of our programs in terms of those models.

The models have to be as such that we can distinguish between terminating and non-terminating programs. We can do this by extending $\sum$ with a special element, let's denote it by $\perp$, to denote non-termination. Let us denote this extended $\sum$ with $\sum_\perp$. If S is a statement, it is now modelled by a function from $\sum_\perp$ to $Pow(\sum_\perp)$. For any t, $S\ t$ is never empty (either it terminates in some state, or it yields $\perp$). Furthermore, for any S, $S\ \perp = \{\perp\}$. So, the models of some statements, in particular `skip` and assignment given before will have to be fixed:

$$\begin{array}{lcl} \texttt{skip}\ s & = & s{=}\perp \rightarrow \{\perp\} \mid \{s\} \\ (x := e)\ s & = & s{=}\perp \rightarrow \{\perp\} \mid (\lambda v.\ v = x \rightarrow e\ s \mid s\ x) \end{array}$$

The definitions of sequential composition and if-then-else do not have to be changed. For the latter we do assume that the evaluation of the guard always terminate.

6.2 Weakest Pre-condition

We will show you two complementary verification approaches. This section discusses the first one, which is called `wp` reduction. Given a specification $\{*\ P\ *\}\ S\ \{*\ Q\ *\}$, the idea is to somehow

calculate the *weakest pre-condition* P' that guarantees that S will always terminate in Q. After obtaining P', it is then sufficient to prove that P implies P'. Unfortunately, calculating the weakest pre-condition is not always possible, e.g. when S contains a loop or recursion. This is where the second verification approach comes in; we will discuss this later.

Take a look at the second rule in Figure 6.1; it is called the Pre-condition Strengthening Rule. It says that in order to prove $\{*\ P\ *\}\ S\ \{*\ Q\ *\}$ we can instead try to prove another specification, say $\{*\ P'\ *\}\ S\ \{*\ Q\ *\}$. This can be useful if the second specification is easier to prove. The rule allows this provided P is stronger than P'. The disadvantage of this approach is that if $\{*\ P'\ *\}\ S\ \{*\ Q\ *\}$ turns out to be invalid, we do not know whether it is because the chosen P' is too weak, or because the original specification is simply invalid (and hence there is no P' such that $\{*\ P'\ *\}\ S\ \{*\ Q\ *\}$).

A more clever strategy is to construct the weakest P'. We will denote this by $\mathsf{wp}\ S\ Q$ which is pronounced: "*the weakest pre-condition of S with respect to (the post-condition) Q*". The meaning of this is somewhat different for partial and total correctness. Under the *partial correctness* interpretation it can be seen as the set of *all* initial states of S, such that the corresponding final states are all in Q. We can express this in terms of our set models (Section 6.1):

Definition 6.2.1 : WP UNDER PARTIAL CORRECTNESS

$$\mathsf{wp}\ S\ Q \quad = \quad \{s \mid S\ s \subseteq Q\}$$

□

An equivalent way to express this, which we will use several times later, is the following:

Corollary 6.2.2 :

$$s \in \mathsf{wp}\ S\ Q \quad = \quad S\ s \subseteq Q$$

□

Under the *total correctness* interpretation $\mathsf{wp}\ S\ Q$ specifies the set of *all* initial states from which S *will terminate* in Q. Rather than giving a direct formal definition of this we will instead provide you with the following characterization for wp. This characterization is good for both partial and total correctness:

Definition 6.2.3 : CHARACTERIZATION OF WP

$$\models\ P \Rightarrow \mathsf{wp}\ S\ Q \quad = \quad \{*\ P\ *\}\ S\ \{*\ Q\ *\}$$

□

Firstly, the definition implies that $\mathsf{wp}\ S\ Q$ itself, whatever it is, is at least also a 'good' pre-condition for realizing Q. This is formally expressed by the following theorem (we leave out the proof for you; it's really trivial):

Corollary 6.2.4 :

$$\{*\ \mathsf{wp}\ S\ Q\ *\}\ \ S\ \ \{*\ Q\ *\}$$

□

Secondly, the above definition of wp implies that if $P' = \mathsf{wp}\ S\ Q$, then it will be *weaker* than any pre-condition P from which S can realize the post-condition Q. Since this holds for *any* P, this indirectly implies that P' must be the weakest one.

Notice also that we have an equality relation above, which is bi-directional. This implies, that if $\models P \Rightarrow \mathsf{wp}\ S\ Q$ turns out to be *not* valid (there exists a counter example against its validity), the above definition says that the specification $\{*\ P\ *\}\ S\ \{*\ Q\ *\}$ is thus invalid as well.

The above definition is also very powerful: it allows you *to reduce* the problem of proving that a program satisfies a specification into a problem of proving an ordinary predicate logic formula. Of course this is only useful if we can actually calculate $\mathsf{wp}\ S\ Q$. In some cases it is possible, even trivial, e.g:

$$\mathsf{wp}\ \mathtt{skip}\ Q \ = \ Q \tag{6.1}$$

Unfortunately, there are also cases where no general algorithm is known for calculating the weakest pre-condition —we will return to this issue later.

The following theorem shows how to calculate the `wp` of `if-then-else`.

Theorem 6.2.5 : `wp` OF IF-THEN-ELSE

$$\mathsf{wp}\ (\mathtt{if}\ g\ \mathtt{then}\ S_1\ \mathtt{else}\ S_2)\ Q \quad = \quad g \rightarrow \mathsf{wp}\ S_1\ Q \mid \mathsf{wp}\ S_2\ Q$$

□

By the way, the `wp` is also equivalent with the following:

$$(g \Rightarrow \mathsf{wp}\ S_1\ Q)\ \wedge\ (\neg g \Rightarrow \mathsf{wp}\ S_2\ Q)$$

or this one:

$$(g \wedge \mathsf{wp}\ S_1\ Q)\ \vee\ (\neg g \wedge \mathsf{wp}\ S_2\ Q)$$

PROOF

We will only prove this for partial correctness. If we interpret the predicates at the left and right sides of '=' above as sets, then it is sufficient to show that for any state s:

$$s \ \in \ \mathsf{wp}\ (\mathtt{if}\ g\ \mathtt{then}\ S_1\ \mathtt{else}\ S_2)\ Q$$

if and only if:

$$g\ s \ \rightarrow \ s \in \mathsf{wp}\ S_1\ Q \ \mid \ s \in \mathsf{wp}\ S_2\ Q$$

This is quite easy to prove:

$$s \ \in \ \mathsf{wp}\ (\mathtt{if}\ g\ \mathtt{then}\ S_1\ \mathtt{else}\ S_2)\ Q$$

$= \{$ Corollary 6.2.2 $\}$

$$(\mathtt{if}\ g\ \mathtt{then}\ S_1\ \mathtt{else}\ S_2)\ s \ \subseteq \ Q$$

$= \{$ Def. 6.1.1 of if-statement $\}$

$$(g\ s \ \rightarrow \ S_1\ s \mid S_2\ s) \ \subseteq \ Q$$

$= \{$ using COND Conversion $\}$

$$g\ s \ \rightarrow \ S_1\ s \subseteq Q \mid S_2\ s \subseteq Q$$

$= \{$ Corollary 6.2.2 $\}$

$$g\ s \ \rightarrow \ s \in \mathsf{wp}\ S_1\ Q \mid s \in \mathsf{wp}\ S_2\ Q$$

□

From the theorem we can prove this inference rule. We leave out the proof for you.

Rule 6.2.6 : IF-THEN-ELSE

$$\frac{\begin{array}{c}\{*\ P \wedge g\ *\}\ S_1\ \{*\ Q\ *\} \\ \{*\ P \wedge \neg g\ *\}\ S_2\ \{*\ Q\ *\}\end{array}}{\{*\ P\ *\} \quad \texttt{if}\ g\ \texttt{then}\ S_1\ \texttt{else}\ S_2 \quad \{*\ Q\ *\}}$$

□

And here is how to calculate the wp of a sequence of statements:

Theorem 6.2.7 : wp OF STATEMENTS SEQUENCE

$$\mathsf{wp}\ (S_1;S_2)\ Q \quad = \quad \mathsf{wp}\ S_1\ (\mathsf{wp}\ S_2\ Q)$$

□

PROOF
We will only prove this for partial correctness:

$s\ \in\ (\mathsf{wp}\ (S_1;S_2)\ Q)$

$=$ { Corollary 6.2.2 }

$(S_1;S_2)\ s\ \subseteq\ Q$

$=$ { Def. 6.1.1 of sequence }

$\cup\{S_2\ t \mid t \in S_1\ s\}\ \subseteq\ Q$

$=$ { set theory }

$(\forall t : t \in S_1\ s : S_2\ t \subseteq Q)$

$=$ { Corollary 6.2.2 }

$(\forall t : t \in S_1\ s : t \in \mathsf{wp}\ S_2\ Q)$

$=$ { set theory }

$S_1\ s\ \subseteq\ \mathsf{wp}\ S_2\ Q$

$=$ { Corollary 6.2.2 }

$s\ \in\ \mathsf{wp}\ S_1\ (\mathsf{wp}\ S_2\ Q)$

□
We also have the analogous of the if-the-else Rule for sequence of statements, but to prove that we first need the following result. Recall Rule 6.1.5: it states that you can weaken a post-condition. The rule below rephrases it in terms of wp:

Rule 6.2.8 : WP POST-CONDITION WEAKENING

$$\frac{\models\ Q \Rightarrow Q'}{\models\ \mathsf{wp}\ S\ Q\ \Rightarrow\ \mathsf{wp}\ S\ Q'}$$

□

PROOF
Interpreting predicates as sets, it is sufficient to prove that $s \in \mathsf{wp}\ S\ Q$ implies $s \in \mathsf{wp}\ S\ Q'$. By Corollary 6.2.2, $s \in \mathsf{wp}\ S\ Q$ is equal to:

$$S\ s \subseteq Q$$

Now in terms of set, $Q \Rightarrow Q'$ means $Q \subseteq Q'$. So with the above this implies:

$$S\ s \subseteq Q'$$

Applying Corollary 6.2.2 again, then we have $s \in \mathsf{wp}\ S\ Q'$
□
Now we can have the following inference rule about sequence of statements:

Rule 6.2.9 : SEQ

$$\frac{\begin{array}{c}\{*\ P\ *\}\ S_1\ \{*\ P'\ *\} \\ \{*\ P'\ *\}\ S_2\ \{*\ Q\ *\}\end{array}}{\{*\ P\ *\}\quad (S_1;\ S_2)\quad \{*\ Q\ *\}}$$

□

PROOF
The second assumption implies $P' \Rightarrow \mathsf{wp}\ S_2\ Q$. Now we use Rule 6.2.8 (instantiating the S in the rule with S_1, and Q with P'), and conclude:

$$\mathsf{wp}\ S_1\ P' \quad \Rightarrow \quad \mathsf{wp}\ S_1\ (\mathsf{wp}\ S_2\ Q)$$

The first assumption implies $P \Rightarrow \mathsf{wp}\ S_1\ P'$. So, together with the above we have:

$$P \quad \Rightarrow \quad \mathsf{wp}\ S_1\ (\mathsf{wp}\ S_2\ Q)$$

Now applying Theorem 6.2.7 on the right hand side of $\Rightarrow$ above we obtain:

$$P \quad \Rightarrow \quad \mathsf{wp}\ (S_1\ ;\ S_2)\ Q$$

which is equivalent to:

$$\{*\ P\ *\}\ \ S_1\ ;\ S_2\ \ \{*\ Q\ *\}$$

□

Assignment

The weakest pre-condition of an assignment requires some explanation. Consider the following simple example.

$$\{*\ ?\ *\} \quad \texttt{x := x+10} \quad \{*\ \texttt{x} > 10\ *\}$$

For example, $\texttt{x} = 1$ is a sufficient pre-condition for the assignment above to realize $\texttt{x}>10$. But it is not the weakest one, because $\texttt{x} = 2$ will also do.

Since the value of $\texttt{x}$ after the assignment is equal to the value of $x + 10$ *before* the assignment, the condition $\texttt{x} > 10$ *after* the assignment is *equivalent* with the condition $\texttt{x} + 10 > 10$ *before* the assignment. In other words, the requirement $\texttt{x} > 10$ *after* the assignment is equivalent to the requirement $\texttt{x} + 10 > 10$ as a pre-condition. The fact that they are equivalent implies that the pre-condition we just come up with is also the weakest one!

By the same argument, if we now have an arbitrary expression E at the right hand side of the assignment:

$$\{*\ ?\ *\} \quad x{:=}E \quad \{*\ x > 10\ *\}$$

the weakest pre-condition for this assignment to realize the post-condition $x > 10$ would be $E > 10$. Note that the argument above suggests we should replace x in the post-condition by E. So, we obtain $E > 10$ by the substitution $(x > 10)[E/x]$.

In general, consider a post-condition Q of the assignment $x{:=}E$. Any x (occuring free) in Q refers to the final value of x. That is, the value of x in the state *after* the assignment. However since x gets its value from E, its final value is equal to the value of E, but latter is interpreted on the state *before* the assignment. So, it follows that the value of Q interpreted on the state after the assignment is equal to the value of $Q[E/x]$ interpreted on the state before the assignment. Because of the equilance this implies that $Q[E/x]$ would be a good pre-condition for Q, and furthermore it must also be the weakest one. Formally:

Theorem 6.2.10 : wp OF ASSIGNMENT

$$\mathsf{wp}\ (v{:=}E)\ Q \quad = \quad Q[E/v]$$

□

In the theorem above, the target of the assignment (v) should a program variable. Assignments to an array element or a record field requires a slightly different treatment —this will be discussed later in Section 6.12.

There is an important assumption which is implicit in the above theorem, namely that changing the value of a variable named x only affects x. This is the case in uPL because uPL does not allow aliases —see also the discussion about alias in Section 5.1. This assumption is made to simplify reasoning. For languages like Java where aliasing is pervasive it is not realistic; a more powerful logic will be needed.

As an example, consider again the following (a bit contrived) example. We want to prove:

```
{* x > 0 *}   y := -1-x ; z := x*y    {* z < 0 *}
```

The validity of this specification is not intuinively clear. So a formal proof would be helpful here. We can do it by first calculating the wp of the statement above with respect to its post-condition:

$$\begin{aligned}
&\mathsf{wp}\ \ (\mathtt{y} := -1 - \mathtt{x};\ \mathtt{z} := \mathtt{x} * \mathtt{y})\ \ (\mathtt{z} < 0)\\
&= \{\ \text{Theorem 6.2.7}\ \}\\
&\mathsf{wp}\ \ (\mathtt{y} := -1 - \mathtt{x})\ \ (\mathsf{wp}\ (\mathtt{z} := \mathtt{x} * \mathtt{y})\ (\mathtt{z} < 0))\\
&= \{\ \text{Theorem 6.2.10}\ \}\\
&\mathsf{wp}\ \ (\mathtt{y} := -1 - \mathtt{x})\ \ ((\mathtt{z} < 0)[\mathtt{x} * \mathtt{y}/\mathtt{z}])\\
&= \{\ \text{applying the substitution}\ \}\\
&\mathsf{wp}\ \ (\mathtt{y} := -1 - \mathtt{x})\ \ (\mathtt{x} * \mathtt{y} < 0)\\
&= \{\ \text{Theorem 6.2.10}\ \}\\
&(\mathtt{x} * \mathtt{y} < 0)[-1 - \mathtt{x}/\mathtt{y}]\\
&= \{\ \text{applying the substitution}\ \}\\
&\mathtt{x} * (-1 - \mathtt{x}) < 0
\end{aligned}$$

```
{* x>0,  justified by Definition 6.2.3 and  PROOF top *}
y := -1 - x ;
z := x*y ;
{* z<0 *}
```

PROOF top

[A:] $x > 0$
[G:] $\mathsf{wp}\ (y := -1 - x;\ z := x * y)\ (z < 0)$

BEGIN

1. { see the wp calculation in page 68 }
   $\mathsf{wp}\ (y := -1 - x;\ z := x * y)\ (z < 0) \quad = \quad x * (-1 - x) < 0$
2. { follows from A }
   $x^2 > 0$
3. { follows from A }
   $-x < 0$
4. { follows from 2 and 3 }
   $-x < x^2$
5. { follows from 4 }
   $-x - x^2 < 0$
6. { follows from 5 }
   $x * (-1 - x) < 0$
7. { rewrite 6 with 1 }
   $\mathsf{wp}\ (y := -1 - x; z := x * y)\ (z < 0)$

END

Figure 6.2: *An example of a formal proof of a specification.*

So, we obtain $x * (-1 - x) < 0$ as the wp. This is equal to $-x < x^2$, which is indeed implied by the pre-condition $x > 0$. This proves the validity of the specification given above. The last step in the proof (that the pre-condition implies the calculated wp) is however given informally. In this example it is quite harmless, since it is a quite simple argumentation. But as an example of a full formal proof see Figure 6.2. The pre-condition is augmented with a comment, saying that the specification is proven through the Definition 6.2.3 of wp; recall that the latter implies that it is sufficient to prove that the given pre-condition implies the wp. The proof of the latter is given in proof top.

Looking at top, it sets the wp of the corresponding statement as its goal, and the pre-condition as an assumption (A). You see that the first step is to prove that:

$$\mathsf{wp}\ (y := -1 - x; z := x * y)\ (z < 0) \ = \ x * (-1 - x) < 0$$

The right hand side of this equation is just the result of calculating the wp. In other words, the equation above tells you what the resulting wp is, which is what you must show to be implied by the assumption A. The proof then proceeds by trying to derive the right hand side of the equation above. Step 6 finally succeeds in deriving it, and hence, based on the equation above the goal is proven.

In the above example we have shown the sf wp calculation. However, such a calculation is usually quite straight forward (you only have to apply the corresponding wp theorems, such as

```
PROOF
[A:] x > 0
[D:] P = wp (y := −1−x; z := x ∗ y) (z < 0)
[G:] P
BEGIN ____________________________________________________________
  1. { calculating wp }
     P = x ∗ (−1 − x) < 0
  2. { follows from A }
     x^2 > 0
  3. { follows from A }
     −x < 0
  4. { follows from 2 and 3 }
     −x < x^2
  5. { follows from 4 }
     x ∗ (−1 − x) < 0
  6. { rewrite 5 with 1 }
     P
END ______________________________________________________________
```

Figure 6.3: *Using abbreviation to improve the readability of a proof.*

theorems 6.2.10, 6.2.7, and 6.2.5). For this reason in most proofs in this book we usually ommit the wp calculation.

You can use abbreviations to improve the readability of a proof. You see that in the proof in Figure 6.2 the expression wp (y := −1−x; z := x ∗ y) (z < 0) is repeated three times. Figure 6.3 shows a bit more readable version of top. It introduces a new name P to abbreviate wp (y := −1−x; z := x ∗ y) (z < 0), and hence avoiding its annoying duplications.

The return statement

Recall that when specifying a program, we can use the keyword **return** in the post-condition to refer to the value returned by the program. This is also the only place where **return** can appear in the specification. For the purpose of reasoning we can treat the **return** in the post-condition as if it is a variable and the statement **return** e as if it is an assignment **return** := e. Then we can handle it as explained above.

6.3 Loop Reduction Strategy

The proof strategy in the previous section is to convert the problem of proving a specification $\{* \ P \ *\} \ S \ \{* \ Q \ *\}$ to the problem of showing that P implies wp S Q. This works as long as we can calculate the wp. Unfortunately, there is no general algorithm known for calculating the wp of a loop and of a recursive program. However, if you recall the Pre-condition Strengthening Rule (Rule 6.1.6), to prove $\{* \ P \ *\} \ S \ \{* \ Q \ *\}$ it is sufficient to find a P' which is *weaker* than P such that $\{* \ P' \ *\} \ S \ \{* \ Q \ *\}$. So, it is not necessary to find the weakest P'! However you have to keep in mind that if we later discover that $\{* \ P' \ *\} \ S \ \{* \ Q \ *\}$ is invalid, we do not know whether it is because the original specification $\{* \ P \ *\} \ S \ \{* \ Q \ *\}$ is invalid, or because the choice of P' is too weak. A closer analysis will be needed to determine which of these two is the problem and how to deal with it.

Consider now a specification:

$$\{* \, P \, *\} \;\; \texttt{while}\; g \;\texttt{do}\; S \;\; \{* \, Q \, *\}$$

Using the idea above, we can prove it by finding a weaker pre-condition I such that:

$$\{* \, I \, *\} \;\; \texttt{while}\; g \;\texttt{do}\; S \;\; \{* \, Q \, *\} \tag{6.2}$$

The next question is: what kind of I would satisfy the above specification? A trivial candidate `false` will do. However, we will then have a problem in proving that P implies it.

To find a more reasonable I, consider an I such that it holds at the end of every iteration. This is nice, because then I will also hold after the last iteration. So, if I also implies Q then it follows that from the pre-condition I the loop, if it terminates, will terminate in Q. We can express this with a rule as follows:

$$\frac{\begin{array}{l} \models \; I \Rightarrow Q \\ I \text{ holds at the end of every iteration} \end{array}}{\{* \, I \, *\} \;\; \texttt{while}\; g \;\texttt{do}\; S \;\; \{* \, Q \, *\}}$$

The rule is still a bit informal; we will make it formal in a minute. But the important thing here is that this is a sufficient rule. It tells us how we can prove that a loop, if it terminates, will terminate in Q. The rule says thus that we have to find an I satisfying the above two premines.

Another important thing to note here is that the argument that leads to the above rule *has assumed termination*. The premise "I holds at the end of every iteration" does not by itself implies that the loop above will actually terminate. So, the rule is only good for the partial correctness interpretation.

The first premise, $I \Rightarrow Q$ can actually be weakened (thus easier to prove). When the loop terminate we know something else, namely that g must be false. So, it is actually sufficient to prove $I \wedge \neg g \Rightarrow Q$.

The second premise, namely that I holds at the end of every iteration, can be formally expressed by:

$$\{* \; \texttt{true} \; *\} \;\; S \;\; \{* \, I \, *\}$$

However this is actually a stronger requirement. Requiring S to establish I from `true` as the pre-condition means that S has to establish I from *any* initial state. But in reality, S will not start from an arbitrary state. At the i-th iteration, it will start from whatever state, which is the result of the previous iteration. Now, if I is to holds at the end of every iteration, we actually also know that it therefore also hold and the end of the previous iteration. And therefore we can assume it as the pre-condition of S. In other words, it is sufficient to require this instead:

$$\{* \, I \, *\} \;\; S \;\; \{* \, I \, *\}$$

Furtheremore, since an iteration can only execute if g is true, we can further weaken the above to this requirement:

$$\{* \, I \wedge g \, *\} \;\; S \;\; \{* \, I \, *\}$$

which may be easier to prove because we have a stronger pre-condition.

With all the refinements above we now formally express the inference rule to prove the correctness of a loop (under the partial correctness interpretation):

Rule 6.3.1 : Loop Reduction in Partial Correctness

$$\frac{\begin{array}{l} \models \; I \wedge \neg g \Rightarrow Q \\ \{* \, I \wedge g \, *\} \;\; S \;\; \{* \, I \, *\} \end{array}}{\{* \, I \, *\} \;\; \texttt{while}\; g \;\texttt{do}\; S \;\; \{* \, Q \, *\}}$$

□

A predicate I satisfying the second premise above is also called *invariant*, because it is preserved by each iteration. Note that an iteration may temporarily destroy an invariant (cause it to become false), but in the end it must re-establish it. An invariant can also be thought to specify the goal of every iteration, since it should hold at the end of all iterations.

If you have an arbitrary specification $\{* \ P \ *\}$ `while` g `do` S $\{* \ Q \ *\}$ to prove, the above rule seems to require you prove that P is invariant. In most cases the pre-condition that you have will actually not be an invariant. However, recall that we can weaken a pre-condition. It is sufficient to find a *weaker* predicate than P, which is an invaraint. This is expressed by the following version of the Loop Reduction Rule:

Rule 6.3.2 : Loop Reduction in Partial Correctness

$$\frac{\begin{array}{l} \models \ P \Rightarrow I \\ \models \ I \wedge \neg g \Rightarrow Q \\ \{* \ I \wedge g \ *\} \ \ S \ \ \{* \ I \ *\} \end{array}}{\{* \ P \ *\} \ \ \texttt{while}\ g\ \texttt{do}\ S \ \ \{* \ Q \ *\}}$$

□

Let us now take a look at what would be needed to also prove the termination of a loop. This is particularly important for programs whose termination are not obvious, e.g. as in the Euclides' program for computing the greatest common divisor you saw in Chapter 1. One way to show termination is to first come up with an integer expression, say, m whose value is decreased by every iteration. Subsequently, we show that at the beginning of every iteration m has a fixed lower bound, say 0. In other words, at beginning of every iteration $m > 0$ holds. Suppose the loop does not terminate. Every iteration decreases the value of m. As the loop keeps iterating, at some point m will become negative. However, this is a contradiction, since we have shown that at the beginning of every iteration $m > 0$. So, the assumption that the loop does not terminate must be false. In other words, in this way we have shown that the loop terminates.

Since g holds at the beginning of every iteration, showing $g \Rightarrow m > 0$ will imply that $m > 0$ holds at the beginning of every iteration. If you have shown that I is invariant (that is, it is preserved by every iteration), then you know that I also holds at the beginning of every iteration. Hence it is sufficient to show $I \wedge g \Rightarrow m > 0$. The requirement that every iteration decreases m can be captured by:

$$\{* \ I \wedge g \ *\} \ \ (C := m; S) \ \ \{* \ m < C \ *\}$$

where C is a fresh auxiliary variable introduced to record the value of m at the beginning of S. Recall that an auxiliary variable is a variable introduced just for the purpose of specifying a property of a program. It can be added to a program but only in a way that does not influence the behavior of the original program.

This is formalized in the following inference rules. This one is for total correctness of course, because it contains premises implying termination.

Rule 6.3.3 : Loop Reduction for Total Correctness

$$\frac{\begin{array}{lcl} \textbf{TC1} & : & \{* \ I \wedge g \ *\} \ \ (C := m; S) \ \ \{* \ m < C \ *\} \\ \textbf{TC2} & : & \models I \wedge g \Rightarrow m > 0 \\ \textbf{EC} & : & \models I \wedge \neg g \Rightarrow Q \\ \textbf{IC} & : & \{* \ I \wedge g \ *\} \ \ S \ \ \{* \ I \ *\} \end{array}}{\{* \ I \ *\} \ \texttt{while}\ \ g\ \texttt{do}\ S \ \ \{* \ Q \ *\}}$$

In **TC1** `C` should be a fresh variable.

□

The integer expression m used to argue termination is also called *termination metric*. In a more general setup you may need a more sophisticated termination metric, e.g. a lexicographically ordered tuple of integers. The general requirement remains the same: it has to be strictly decreasing, and has a lower bound. In this book, we will only use integer-typed termination metrics.

For completeness below we also show the variant of the rule above that allows you to directly prove a specification with an arbitrary pre-condition P:

Rule 6.3.4 : Loop Reduction for Total Correctness

$$\begin{array}{lcl}
\textbf{InitC} & : & \models P \Rightarrow I \\
\textbf{TC1} & : & \{* \, I \wedge g \, *\} \;\; (C := m; S) \;\; \{* \, m < C \, *\} \\
\textbf{TC2} & : & \models I \wedge g \Rightarrow m > 0 \\
\textbf{EC} & : & \models I \wedge \neg g \Rightarrow Q \\
\textbf{IC} & : & \{* \, I \wedge g \, *\} \;\; S \;\; \{* \, I \, *\} \\
\hline
 & & \{* \, P \, *\} \; \texttt{while} \;\; g \; \texttt{do} \; S \;\; \{* \, Q \, *\}
\end{array}$$

In **TC1** `C` should be a fresh variable.
□

There is unfortunately no algorithm to (automatically) construct a right I. In fact, finding one can be a challenging puzzle. E.g. choosing a weak I may cause **EC**, **TC1**, or **TC2** to become invalid. For these you would prefer a strong I. However, if I is too strong **InitC** may become invalid. Proving **IC** poses a similar dilemma because I appears in both pre- and post-conditions.

6.4 Organizing Your Proof

Before we show you examples involving concrete programs with loops, let us first discuss how to organize your top level proof so that it gives you a better overview.

If a program is too large to be proven in a single step we can split its specification by adding assertions. Recall that adding assertions is safe. That is, it cannot turn an invalid specification to valid. For example, suppose we have this specification to prove:

$$\{* \, P \, *\} \; S_1; S_2 \; \{* \, R \, *\} \tag{6.3}$$

By inserting a new assertion Q, we can instead try to prove:

$$\{* \, P \, *\} \; S_1 \; \{* \, Q \, *\} \; ; S_2 \; \{* \, R \, *\} \tag{6.4}$$

The new specification is more helpful as it provides you with more information, though in exchange now you have two goals to prove:

$$\{* \, P \, *\} \; S_1 \; \{* \, Q \, *\} \quad \text{and} \quad \{* \, Q \, *\} \; S_2 \; \{* \, R \, *\}$$

A frequently appearing program shapes are what we call *IL* and *ILF* shapes: IL stands for 'init+loop', and ILF for 'init+loop+finalization'. The first is a program with a single loop preceeded by an initialization code, like this:

$$S_i; \; \texttt{while} \; g \; \texttt{do} \; S$$

A ILF-shaped program is an IL program followed by another piece of code that we call the finalization code. Let us first consider IL-shaped programs. Consider this specification:

$$\{* \, P \, *\} \; S_i; \; \texttt{while} \; g \; \texttt{do} \; S \; \{* \, Q \, *\}$$

Inspired by the previous discussion about splitting up a specification, we add an intermediate assertion I, obtaining this:

$$\{* \, P \, *\} \; S_i; \; \{* \, I \, *\} \; \texttt{while} \; g \; \texttt{do} \; S \; \{* \, Q \, *\}$$

which requires you to prove two things:

C1: prove that the loop initialization part S_i can establish I.

C2: prove $\{* \, I \, *\}$ `while` g `do` S $\{* \, Q \, *\}$

C2 can be proven using one of the Loop Reduction rules from the previous section. You can choose I such that it is an invariant of the loop.

Figure 6.4 shows the global structure of the correctness proof of a specification of an IL shaped program. It consists of five separate proofs. `Init` proves the condition **C1** above. `PTC1`, `PTC2`, `PEC`, and `PIC` prove the conditions required by Rule 6.3.3, which in turns implies **C2** above. The rule requires a termination metric. This is specified as a comment next to the assertion specifying the loop's invariant.

A variant of the IL shape is the ILF shape. An ILF-shaped program has an IL shape followed by a piece of so-called finalization code, like this:

$$\{* \, P \, *\} \; S_i; \; \texttt{while} \; g \; \texttt{do} \; S; \; S_f \; \{* \, R \, *\}$$

S_f is the finalization code. If it does not contain any loop then we can calculate its wp with respect to R, and thus reducing the above specification to the following:

$$\{* \, P \, *\} \; S_i; \; \texttt{while} \; g \; \texttt{do} \; S \; \{* \; \mathsf{wp} \; S_f \; R \, *\}$$

which has an IL shape and can thus be handled in the way we discussed above.

```
{* P  , see PROOF Init *}

S_i;

{* I ,  see PROOF PTC1, PTC2, PEC, PIC.  Termination metric: ...  *}

while g do S

{* Q *}

ASSUMING

I = ...
```

PROOF Init

Here you prove the initialization part: $\{* P *\}\ S_i\ \{* I *\}$. If S_i does not contain any loop, then you can do this by showing that P implies wp S_i I.

PROOF PTC1

Here you prove the **TC1** premise (see the Loop Reduction Rule). This premise, together with **TC2**, is required to prove that the loop terminates. Basically, what you prove here is that every iteration will decrease the value of the termination metric.

PROOF PTC2

Here you prove the **TC2** premise, namely that at the beginning of every iteration, the value of the termination metric is positive.

PROOF PEC

Here you prove the **EC** premise, namely that at the end of the loop, the invariant and the negation of the guard imply the specified post-condition of the loop.

PROOF PIC

Here you prove the **IC** premise, namely that body of the loop re-establishes the invariant I.

Figure 6.4: *The global structure of the proof of an IL shaped program.*

```
{* n > 0 *}

FIB (n:int) : int

{ var nNow,nBefore,newPairs,t : int ;

  t:=1 ; nBefore:=0 ; nNow:=1 ;
  while t ≠ n do
    { newPairs := nBefore ;
      nBefore  := nNow ;
      nNow := nNow + newPairs ;
      t := t + 1
    } ;
  return nNow }

{* return = fib n *}
```

Figure 6.5: *An imperative implementation of Fibonacci's rabbit farm.*

6.5 Example: Fibonacci Problem

You have now been introduced to all the basic inference rules to verify the correctness of imperative sequential programs. You have also seen one example, but it was a rather trivial example. Let us now consider a slightly larger example.

This is a classical example. In 1202 Leonardo Fibonacci investigated a model to calculate how fast a rabbit population grows. The model is perhaps overly simple, but it still makes an interesting programming example. In the model the population starts with a newly born pair of rabbits (one male and one female). The rabbits are assumed to live forever. Every pair will become mature after two months and will then produce a new pair every month. The environment in which the rabbits live is assumed to be sufficient and stable. The table below shows how many pairs of rabbits this model will produce at the end of the nth month:

month (n):	0	1	2	3	4	5	6	7	8	9	10	11	12
#rabbits (in pairs)	0	1	1	2	3	5	8	13	21	34	55	89	144

The sequence of numbers in the second row is popularly called the Fibonacci sequence. Let us use the notation `fib` n to denote the n-th number in the sequence. Looking at the sequence above, you may observe that it seems that it can produced by the following recursive function:

$$\begin{aligned} \texttt{fib}\ 0 &= 0 \\ \texttt{fib}\ 1 &= 1 \\ \texttt{fib}\ (\texttt{n}+2) &= \texttt{fib}\ (\texttt{n}+1) + \texttt{fib}\ \texttt{n} \end{aligned} \tag{6.5}$$

for all $\texttt{n} \geq 0$. This (recursive) function can be directly implemented as a program. Unfortunately the program would be very ineffecient. For example, `fib 5` recursively calls `fib 3` and `fib 4`. The latter also calls `fib 3`. So, `fib 3` is unnacessarily called twice. Each call to `fib 3` in turn calls `fib 1` and `fib 2`. A call to `fib 2` also calls `fib 1`, so again we have duplicated calls; and so on.

Alternatively, we can also construct a program that implements the rabbit model more directly. An example is shown Figure 6.5. As a correctness criterion we can require that it should produce the same result as the function `fib` above.

The program reads an integer `n`, and will return the number of rabbits (in pairs) after `n` months. The variable `t` models time in months. The program starts with `t=1`. The variable `nNow` maintains

```
  {* n  >  0  , see proof Init *}

  t := 1; nBefore := 0; nNow := 1 ;

  {* I  see proofs PTC1, PTC2, PEC, PIC.  Termination metric: m = ...*}

  while t ≠ n do
    { newPairs := nBefore ; nBefore  := nNow ;
      nNow := nNow + newPairs ; t := t + 1
    } ;

  {* nNow  =  fib n  (by calculating  wp)*}

  return nNow

  {* return  =  fib n *}

  ASSUMING
  I = ...

PROOF Init
... (here you prove {* n > 0 *} t := 1; nBefore := 0; nNow := 1 {* I *}).
PROOF PTC1
... (here you prove {* I ∧ t ≠ n *} (C := m; "loop's body") {* m < C *})
PROOF PTC2 ... (here you prove I ∧ t ≠ n ⇒ m > 0)
PROOF PEC ... (here you prove I ∧ (t = n) ⇒ (nNow = fib n))
PROOF PIC ... (here you prove {* I ∧ t ≠ n *} "loop's body" {* I *})
```

Figure 6.6: *The global structure of the proof of* `FIB` *(the program header and local declaration are removed just to save space). This kind of global structure of a proof can also be seen as a proof plan as it provides an overview of all top level proof obligations.*

the number of rabbits (in pairs) that we currently have after `t` months. The variable `nBefore` maintains the number of rabbits we have the month before.

It starts of course with `nNow=1`, since the model starts with one pair of rabbits. The value of `nBefore` is set to 0, because before we have no rabbit at all. As we move from time `t` to `t+1`, we will get new pairs of rabbits. Only mature pairs can produce new pairs. A pair become mature after two months. So, as we enter $\mathtt{t}+1$-th month, the number of mature pairs is equal to `nBefore`, and these will give an equal number of new pairs. So, at the end of every iteration, the new number of pairs is equal to the current number of pairs (`nNow`) plus `nBefore`. Compare this explanation with the program code.

The function `fib` in (6.5) provides an abstract and ellegant way to characterize the Fibonacci sequence. However, to actually produce the numbers in the sequence, `FIB` (the imperative version in Figure 6.5) is more efficient. The loop will only iterates n times to compute `fib` n, whereas the function `fib` itself requires in the order of 2^n recursive calls. However, we have yet to show that `FIB`, as its specification claims, is indeed a consistent implementation of `fib`. Let us now see how the theory you have seen so far can be used to do so.

Although it is quite obvious that `FIB` terminates, for the sake of example we will still prove its termination. So, the given specification in Figure 6.5 is to be interpreted under the total correctness interpretation.

6.5.1 Step 1: Proof Plan

The program has an ILF shape. Following the strategy discusses in the previous section we draw an outline for our proof —see Figure 6.6. The outline is not the complete proof. Its purpose is to line out the global structure of our proof and to give an overview of the formulas we have to prove (also called *proof obligations*). We will also call such an outline a *proof plan*. We will gradually fill it in with details until it become a complete proof.

Notice that a new assertion $\texttt{nNow} = \texttt{fib n}$ is inserted between the loop and the final assignment in the program. However, the proof plan in Figure 6.6 does not include a proof for:

$$\{*\ \texttt{nNow} = \texttt{fib n}\ *\}\ \ \texttt{return nNow}\ \ \{*\ \texttt{return} = \texttt{fib n}\ *\}$$

As commented in the proof plan, the assertion $\texttt{nNow} = \texttt{fib n}$ is obtained by calculating the wp of return nNow with respect to the post-condition $\texttt{return} = \texttt{fib n}$. Being the weakest pre-condition it automatically satisfies the above specification.

For the purpose of computing wp, the statement return nNow is treated as if it is an assignment $\texttt{return} := \texttt{nNow}$. This yields indeed $\texttt{nNow} = \texttt{fib n}$ as the weakest pre-condition.

Step 2: Constructing I

As said before, there is no general algorithm to construct I. There are however some strategies. For example we can try to construct I by incrementally adding details to it. The final I has this form: $\texttt{I}_1 \wedge \ldots \wedge \texttt{I}_\texttt{n}$. We start by giving a guess for $\texttt{I}_1$ and assume that $\texttt{I} = \texttt{I}_1 \wedge \texttt{h}_1$ where $\texttt{h}_1$ is unspecified and is used as a place holder for further refinement. As we work out the proofs we may find out that $\texttt{I}_1$ alone is not sufficient to close all the proofs. We fix I by instantiating (refining) $\texttt{h}_1$ to $\texttt{I}_2 \wedge \texttt{h}_2$ for some $\texttt{I}_2$. So, now we have $\texttt{I} = \texttt{I}_1 \wedge \texttt{I}_2 \wedge \texttt{h}_2$, with $\texttt{h}_2$ as a new place holder for further refinement The process is repeated until you can close all the proofs.

Note that the strategy above fails if you start with an $\texttt{I}_1$ which is already too strong, and similarlly if at some point you choose an $\texttt{I}_i$ which is overly strong. So, it requires some wisdom in making the choices.

As you gain experience, you will become more skilled in guessing a right I. Many programs are also quite similar to each other. Similar programs tend to have similar sorts of invariants.

For the first guess usually PEC provides a useful clue. Recall that PEC concerns the situation when the loop terminates. We have to show that I and the guard being false (implying the end of the loop) implies the specified post-condition of the loop. This is the only condition in the Loop Reduction rules that directly relates I with the loop's specified goal, which is why it often gives useful insight for the choice of I. Now, this is what we have to prove in PEC:

$$\texttt{I} \wedge (\texttt{t} = \texttt{n}) \ \Rightarrow\ \texttt{nNow} = \texttt{fib n} \tag{6.6}$$

A trivial choice of I is false. It would satisfy the above implication, but unfortunately Init, for example, will then become imposible to prove. So false is a useless choice.

Another trivial I that would satisfy the above implication is $\texttt{nNow} = \texttt{fib t}$. Is this a good choice? The only way to really verify it is of course by proving it. But we can first give it some further informal consideration. Recall that nNow is supposed to contain the current number of the rabbits (in pairs). That is, the number of rabbits at time t. This is supposedly equal to fib t, suggesting that $\texttt{nNow} = \texttt{fib t}$ is an invariant property, and hence indeed a part of I. So, let us indeed put it in our I:

$$\texttt{I} \ \ = \ \ (\texttt{nNow} = \texttt{fib t}) \wedge \texttt{h}_1 \tag{6.7}$$

where $\texttt{h}_1$ is still unknown and is a place holder for further refinement. We add this new information in the proof plan. The changes are shown in below:

```
{* n > 0 *}
FIB's body
{* return = fib n *}

ASSUMING

I = (nNow = fib t) ∧ h₁

PROOF Init ...
...
PROOF PEC ...
...
```

As said PEC is trivial, but for the sake of completeness it is shown below:

PROOF PEC
[A:] $\mathtt{I} \wedge (\mathtt{t} = \mathtt{n})$
[G:] $\mathtt{nNow} = \mathtt{fib\ n}$

BEGIN

1. { follows from A and the def. of I } $\mathtt{nNow} = \mathtt{fib\ t}$
2. { follows from A and the def. of I } $\mathtt{t} = \mathtt{n}$
3. { rewrite 1 with 2 } $\mathtt{nNow} = \mathtt{fib\ n}$

END

Note that the proof above does not depend on what $\mathtt{h}_1$ is. Consequently, instantiating it will not invalidate the conclusion of PEC.

6.5.2 Step 3: Cross Checking with Init

Init is where we prove that the initialization code can setup I for the loop as the loop is entered for the first time. Obviously an I which is not realizable in this sense is not very useful. Checking Init is usually easy. Each time you refine I, you may want to cross check it against Init before you start working on PIC, which is usually much more involved. Let us do so with our example. We cannot complete Init yet, because I is only partially known. However we can make this calculation:

$\mathsf{wp}\ (\mathtt{t} := 1;\ \mathtt{nBefore} := 0;\ \mathtt{nNow} := 1)\ \mathtt{I}$

$= \{$ definition of I $\}$

$\mathsf{wp}\ (\mathtt{t} := 1;\ \mathtt{nBefore} := 0;\ \mathtt{nNow} := 1)\ ((\mathtt{nNow} = \mathtt{fib\ t}) \wedge \ldots)$

$= \{$ calculating the wp $\}$

$(1 = \mathtt{fib}\ 1) \wedge \ldots$

$= \{$ definition of fib $\}$

$(1 = 1) \wedge \ldots$

This shows that at least the known part of I will not cause a problem in Init, because after the wp calculation it is reduced to $1 = 1$ which is trivially true.

6.5.3 Step 4: Verifying Invariance

So now we know that our choice of I so far is strong enough to prove PEC's goal and does not seem to cause a problem in Init. The next thing to do is to verify the invariance of I. This is done in PIC (see the proof plan). More precisely, you have to show:

$$\{*\ \mathtt{I}\ \wedge\ (\mathtt{i} \neq \mathtt{n})\ *\}\ \textit{"loop's body"}\ \{*\ \mathtt{I}\ *\} \tag{6.8}$$

Notice that the definition of I specifies a property of nNow. Each iteration updates nNow using nBefore. So, in order to prove the property about nNow you can expect that you need to know something about nBefore. However, the pre-condition of the loop's body, which is $\mathtt{I} \wedge \mathtt{i} < \mathtt{n}$, does not contain any information about nBefore. So you can expect that the proof will fail. The calculation below will show you. To prove (6.8) we can equivalently prove:

$$\mathtt{I}\ \wedge\ (\mathtt{i} \neq \mathtt{n})\quad \Rightarrow\quad \mathsf{wp}\,\textit{"loop's body"}\ \mathtt{I} \tag{6.9}$$

We calculate the wp:

$$\begin{aligned}
&\mathsf{wp}\ \textit{"loop's body"}\ \mathtt{I}\\
&= \{\ \text{definition of } \mathtt{I}\ \}\\
&\mathsf{wp}\ \textit{"loop's body"}\ ((\mathtt{nNow} = \mathtt{fib\ t})\ \wedge\ \ldots)\\
&= \{\ \text{calculating } \mathsf{wp}\ \}\\
&(\mathtt{nNow} + \mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}+1))\ \wedge\ \ldots
\end{aligned}$$

There is no way to prove the last formula above from $\mathtt{I} \wedge (\mathtt{t} \neq \mathtt{n})$ because I does not say anything about nBefore.

If you recall the explanation given for the code of FIB (page 77), the variable nBefore is used to hold the number of rabbits (in pairs) at time $\mathtt{t}-1$. The loop seems to maintain this relation. In other words, it seems to be an invariant property —perhaps we can use this fact. To do so, we add it to our I. This is done by instantiating $\mathtt{h}_1$ in the previous definition of I (6.7) to:

$$\mathtt{h}_1\quad =\quad (\mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}-1)) \wedge \mathtt{h}_2 \tag{6.10}$$

where $\mathtt{h}_2$ is unknown and serves as a new place holder. So, I is now:

$$\mathtt{I}\quad =\quad (\mathtt{nNow} = \mathtt{fib\ t}) \wedge (\mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}-1)) \wedge \mathtt{h}_2 \tag{6.11}$$

With this new information now we redo the calculation:

$$\begin{aligned}
&\mathsf{wp}\ \textit{"loop's body"}\ \mathtt{I}\\
&= \{\ \text{definition of } \mathtt{I}\ \}\\
&\mathsf{wp}\ \textit{"loop's body"}\ ((\mathtt{nNow} = \mathtt{fib\ t})\ \wedge\ (\mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}-1))\ \wedge\ \mathtt{h}_2)\\
&= \{\ \text{calculating } \mathsf{wp}\ \}\\
&(\mathtt{nNow} + \mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}+1))\ \wedge\ (\mathtt{nNow} = \mathtt{fib\ t})\ \wedge\ \ldots
\end{aligned}$$

As before, we have to prove that $\mathtt{I} \wedge (\mathtt{t} \neq \mathtt{n})$ implies the last formula above. Assuming $\mathtt{I} \wedge (\mathtt{t} \neq \mathtt{n})$ we continue the calculation we did above:

$$\begin{aligned}
&(\mathtt{nNow} + \mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}+1))\ \wedge\ (\mathtt{nNow} = \mathtt{fib\ t})\ \wedge\ \ldots\\
&= \{\ \mathtt{I} \text{ is assumed, and it says that } \mathtt{nNow} = \mathtt{fib\ t}\ \}\\
&(\mathtt{nNow} + \mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}+1))\ \wedge\ \mathtt{true}\ \wedge\ \ldots\\
&= \{\ \text{using the assumed } \mathtt{I} \text{ again; it says } \mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}-1)\ \}\\
&(\mathtt{fib\ t} + \mathtt{fib}\ (\mathtt{t}-1) = \mathtt{fib}\ (\mathtt{t}+1))\ \wedge\ \mathtt{true}\ \wedge\ \ldots
\end{aligned}$$

$= \{$ simplifying $\}$

$(\texttt{fib t} + \texttt{fib}\,(\texttt{t} - 1) = \texttt{fib}\,(\texttt{t} + 1)) \;\wedge\; \ldots$

$= \{$ definition of $\texttt{fib}$ $\}$

$(\texttt{fib}\,(\texttt{t} + 1) = \texttt{fib}\,(\texttt{t} + 1)) \;\wedge\; \ldots$

The equality in the last expression is trivially true. Are we done with PIC? Well, not really. The last step above is not entirely correct.

In the last step we use the definition of fib to rewrite $\texttt{fib t} + \texttt{fib}\,(\texttt{t} - 1)$ to $\texttt{fib}\,(\texttt{t} + 1)$. However, this two are only equal if $\texttt{t}>0$, which cannot be inferred from $\texttt{I} \wedge (\texttt{k} \neq \texttt{n})$ using the above choice of I. So, we still need to extend I with further detail. An easy solution is to simply extend I with $\texttt{t}>0$ itself. This is reasonable since the loop starts with $\texttt{t} = 1$ and increases it at each iteration. So, $\texttt{t}>0$ is an invariant property. It is also not too difficult to see that the iterations will also maintain $\texttt{t} \leq \texttt{n}$. We do not need this knowledge now, but we will need it later as we prove the loop's termination. So, we patch the definition of $\texttt{h}_1$ (6.10) to include the above details:

$$\texttt{h}_1 \;=\; (\texttt{nBefore} = \texttt{fib}\,(\texttt{t} - 1)) \;\wedge\; 0 < \texttt{t} \leq \texttt{n} \;\wedge\; \texttt{h}_2 \tag{6.12}$$

So, the new I is:

$$\begin{aligned} \texttt{I} \;=\; & (\texttt{nNow} = \texttt{fib t}) \;\wedge\; (\texttt{nBefore} = \texttt{fib}\,(\texttt{t} - 1)) \\ & \wedge\, 0 < \texttt{t} \leq \texttt{n} \\ & \wedge\, \texttt{h}_2 \end{aligned} \tag{6.13}$$

For later it is convenient to introduce J and write I like this:

$$\begin{aligned} \texttt{I} \;&=\; \texttt{J} \wedge \texttt{h}_2 \\ \texttt{J} \;&=\; (\texttt{nNow} = \texttt{fib t}) \;\wedge\; (\texttt{nBefore} = \texttt{fib}\,(\texttt{t} - 1)) \;\wedge\; 0 < \texttt{t} \leq \texttt{n} \end{aligned}$$

The new proof plan should now look like this (leaving out the parts which are not changed):

```
{* n > 0 *}
FIB's body
{* return = fib n *}

ASSUMING

I   =  J ∧ h2
J   =  (nNow = fib t) ∧ (nBefore = fib (t − 1)) ∧ 0<t≤n
...
PROOF PIC ...
...
```

PIC is shown in Figure 6.7. It is not complete yet: $\texttt{h}_2$ is unknown, so we cannot prove its invariance yet (the second goal in Figure 6.7).

The proof uses the following useful property of wp saying that the wp of some statement S with respect to $Q_1 \wedge Q_2$ can be constructed by *separately* calculating the wp with respect to Q_1 and Q_2, and then taking the conjunction of the results. A similar property holds for disjunction, provided S is deterministic. The proof can be found in Section 6.9.

Theorem 6.5.1 : wp Distributivity

1. $\models \mathsf{wp}\; S\; (Q_1 \wedge Q_2) \;=\; (\mathsf{wp}\; S\; Q_1) \wedge (\mathsf{wp}\; S\; Q_2)$
2. If S is deterministic, we have:

$$\models \mathsf{wp}\; S\; (Q_1 \vee Q_2) \;=\; (\mathsf{wp}\; S\; Q_1) \vee (\mathsf{wp}\; S\; Q_2)$$

□

PROOF PIC
[A :] $\mathtt{I} \wedge (\mathtt{t} \neq \mathtt{n})$
[D1:] $\mathtt{Q}_1 \;=\; \mathsf{wp}$ *"loop's body"* $\mathtt{J}$
[D2:] $\mathtt{Q}_2 \;=\; \mathsf{wp}$ *"loop's body"* $\mathtt{h}_2$
[G :] wp *"loop's body"* $(\mathtt{J} \wedge \mathtt{h}_2)$
BEGIN

1. { follows from A and def. of I }
   $(\mathtt{nNow} = \mathtt{fib\ t}) \;\wedge\; (\mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}-1)) \;\wedge\; 0<\mathtt{t}\leq\mathtt{n}$
2. { wp calculation and def. of J }
   $\mathtt{Q}_1 \;=\; (\mathtt{nNow} + \mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}+1)) \;\wedge\; (\mathtt{nNow} = \mathtt{fib\ t}) \;\wedge\; 0<\mathtt{t}+1\leq\mathtt{n}$
3. { see the equational proof below }
   $\mathtt{nNow} + \mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}+1)$

   EQUATIONAL PROOF
   $\mathtt{nNow} + \mathtt{nBefore}$
   $=$ { PIC.1 implies $\mathtt{nNow} = \mathtt{fib\ t}$ and $\mathtt{nBefore} = \mathtt{fib}\ (\mathtt{t}-1)$ }
   $\mathtt{fib\ t} + \mathtt{fib}\ (\mathtt{t}-1)$
   $=$ { PIC.1 implies $\mathtt{t} > 0$, use the def. of fib }
   $\mathtt{fib}\ (\mathtt{t}+1)$
   END

4. { $\wedge$ elimination on 1 } $\mathtt{nNow} = \mathtt{fib\ t}$
5. { follows triviallay from $0 < \mathtt{t}$, which is implied by 1 } $0 < \mathtt{t}+1$
6. { follows from 1 and A } $\mathtt{t} \leq \mathtt{n} \;\wedge\; \mathtt{t} \neq \mathtt{n}$
7. { follows from 6 } $\mathtt{t}+1 \leq \mathtt{n}$
8. { follows from 3,4,5,7, and 2 } $\mathtt{Q}_1$
9. { proof is delayed } $\mathtt{Q}_2$
10. { follows from 8,9, and wp Distributivity (Theorem 6.5.1) }
    wp *"loop's body"* $(\mathtt{J} \wedge \mathtt{h}_2)$

END

Figure 6.7: *The* PIC *part of the proof of* FIB. *The justification of step 9 is still missing.*

6.5.4 Step 5: Loop Termination

Intuitively, it is quite clear that FIB's loop will terminate. Nevertheless for the sake of example, we will still show the formal proof of this. Recall the discussion about the Loop Reduction rules, in particular Rule 6.3.4: to prove termination we need to find a termination metric m. This is an integer expression whose value is decreased by every iteration (proven in PTC1), but bounded below by some constant (proven in PTC2).

Now, each iteration in FIB's loop increases the value t. So, each iteration decreases $-$t. This is a good candidate for m. However, our Loop Reduction rule requires an m with 0 as its lower bound; $-$t does not have this property. This is quite easy to fix. We take n$-$t instead, which also decreases over the iterations. The invariant I (6.13) says that t$\leq$n. An iteration can only start if $\mathtt{t} \neq \mathtt{n}$. The last two facts imply that $\mathtt{t} < \mathtt{n}$, hence n$-$t$>$0.

So, to summerize this m will be our chosen termination metric:

$$\mathtt{m} \;=\; \mathtt{n} - \mathtt{t}$$

This new information should be added to the proof plan. PTC1 and PTC2 are quite trivial, but for the completeness are shown below.

PROOF PTC2

[A:] $\mathtt{I} \wedge \mathtt{t} \neq \mathtt{n}$
[G:] $\mathtt{n} - \mathtt{t} > 0$

BEGIN

1. { follows from the 1-st conjunct of A and definition of I } $\mathtt{t} \leq \mathtt{n}$
2. { follows from 1 and 2-nd conjunct of A } $\mathtt{t} < \mathtt{n}$
3. { follows from 2 } $\mathtt{n} - \mathtt{t} > 0$

END

PROOF PTC1

[A:] $\mathtt{I} \wedge \mathtt{t} \neq \mathtt{n}$
[D:] $\mathtt{Q} \;=\; \mathtt{wp}\ (\mathtt{C} := \mathtt{n} - \mathtt{t};\ \mathtt{loop'sbody})\ \ (\mathtt{n} - \mathtt{t} < \mathtt{C})$
[G:] Q

BEGIN

1. { calculating wp } $\mathtt{Q} \;=\; \mathtt{n} - (\mathtt{t} + 1) < \mathtt{n} - \mathtt{t}$
2. { trivial } $\mathtt{n} - \mathtt{t} - 1 < \mathtt{n} - \mathtt{t}$
3. { follows from 1 and 2 } Q

END

Are we done now? Well, not quite. We still have to do Init and PIC is also incomplete because the proof of the invariance of $\mathtt{h}_2$ is still missing; and $\mathtt{h}_2$ itself is still unspecified. What should we use for it? Well, in all the proofs we do so far we actually never use it. In other words, the proofs so far do not depend on what $\mathtt{h}_2$ is. So, take $\mathtt{h}_2 = \mathtt{true}$. With respect to the definition of I, this has the same effect as simply dropping $\mathtt{h}_2$ from the definition. It also means the still missing part of PIC in which we have to prove $\mathtt{h}_2$ can also be dropped.

So, we remove $\mathtt{h}_2$. The proof plan should be adjusted accordingly. This leaves us with one remaining proof obligation, namely Init.

6.5.5 Step 6: Initialization

In `Init` we have to prove that the initialization code can establish the invariant `I` as the program enters the loop for the first time. In our case, this is what we have to prove:

$$\{* \, \mathtt{n} > 0 \, *\} \; (\mathtt{t} := 1; \mathtt{nBefore} := 0; \mathtt{nNow} := 1) \; \mathtt{I} \tag{6.14}$$

The proof is below.

```
PROOF Init

[A:] n > 0
[G:] wp (t := 1; nBefore := 0; nNow := 1) I

BEGIN ____________________________________________________________

  1. { see the equational proof below }   wp (t := 1; nBefore := 0; nNow := 1) I  =  1 ≤ n

       EQUATIONAL PROOF ____________________________________________
       wp (t := 1; nBefore := 0; nNow := 1) I
       = { definition I }
       wp (t := 1; nBefore := 0; nNow := 1) ((nNow = fib t) ∧ (nBefore =
       fib (t − 1)) ∧ 0 < t ≤ n)
       = { calculating wp }
       (1 = fib 1) ∧ (0 = fib 0) ∧ 0 < 1 ≤ n
       = { def. of fib }
       (1 = 1) ∧ (0 = 0) ∧ 0 < 1 ≤ n
       = { simplification }
       1 ≤ n
       END _________________________________________________________

  2. { follows from A }   1 ≤ n

  3. { rewite 2 with 1 }   wp (t := 1; nBefore := 0; nNow := 1) I

END ______________________________________________________________
```

This is also a typical situation where program verification is often successful in flushing out errors. Since a program with a weaker a pre-condition is more versatile (the program accepts a larger set of inputs), a less experienced programmer may be tempted to weaken the pre-condition of `FIB`, for example to $0 \leq \mathtt{n}$. Unfortunately this will cause a problem if the program is called with $\mathtt{n} = 0$ (it does not terminate). Such an error may slip unnoticed through the testing phase, because it only happens to *one* input combination out of infinitely many of them. There is however no way the error can slip through program verification. If you set the pre-condition to $0 \leq \mathtt{n}$, there is simply no way you can prove `I` in `Init`.

6.5.6 Summarizing

Now we have completed all the proofs, and hence the program `FIB` is correct with respect to its specification. We have shown to you how to incrementally construct an invariant. During the process we extended `I` several times. For your overview Figure 6.8 shows the final proof plan. It shows the final `I` and the termination metric `m`. The proofs are not repeated in the Figure —we have shown them all.

The incremental approach is typically used when you are dealing with programs whose structures or algorithms may be new to you. As you gain in experience you will be getting better in 'guessing' invariants. You may even be able to immediately guess a correct invariant when given a simple program.

```
{* n > 0 , see proof Init *}

t := 1; nBefore := 0; nNow := 1 ;

{* I  see proofs PTC1, PTC2, PEC, PIC.  Termination metric: m = n-t  *}

while t ≠ n do
  { newPairs := nBefore ; nBefore  := nNow ;
    nNow := nNow + newPairs ; t := t + 1
  } ;

{* nNow = fib n  (by calculating wp)*}

return nNow

{* return = fib n *}

ASSUMING

I = (nNow = fib t) ∧ (nBefore = fib (t − 1)) ∧ 0 < t ≤ n

PROOF Init ...  done
PROOF PTC1 ...  done
PROOF PTC2 ...  done
PROOF PEC  ...  done
PROOF PIC  ...  done
```

Figure 6.8: *The final proof plan. The proofs are not shown. They have been shown before.*

6.6 On Iteratively Constructing I

Consider again the steps we took to construct the invariant in the Fibonacci example. We started with a simple I (6.7), namely:

$$\mathtt{I} \;=\; (\mathtt{nNow} = \mathtt{fib\ t}) \wedge \mathtt{h}_1$$

where $\mathtt{h}_1$ in unspecified. As we proceeded we discovered that with this I we could not close PIC. After some calculation, we ended up having to prove that I implies:

$$\mathtt{nNow} + \mathtt{nBefore} = \mathtt{fib}\,(\mathtt{t} + 1) \tag{6.15}$$

We could not prove this because I contains no information about nBefore. So, in (6.10) and (6.11) we decided to extend the candidate invariant I with:

$$\mathtt{nBefore} = \mathtt{fib}\,(\mathtt{t} - 1)$$

as a new conjunct. The discovery of this fact about nBefore was motivated by appealing to your programmer insight on the program's code. It worked. However programming insight or intuition will be much less effective when we have to deal with complicated programs.

Rather than relying on insight we can just extend I with the formula on which either PIC, PTC1, or PTC2 fail. We then rework the proofs, and iterate this process until we can close all the proofs. So, in the above case we would just add (6.15) to I. It is intuitively less obvious that it is an invariant property (it is!), but on the other hand discovering it does not require any thinking: if we cannot prove it from I, we add it.

This process will always drive you towards an `I` satisfying all the conditions required by Rule 6.3.3. But it is a costly procedure. It may also not terminate, and if it does you may still end up with an `I` which is too strong to be setup by your loop initialization code and its given pre-condition.

We will try this strategy in the next example. After you have studied it, as an exercise you can redo the proof of `FIB` using the same strategy.

6.7 Linear Scanning

In the following example we have a program that works on an array. Its specification is slightly more complicated than that of the previous example. So is its proof. Below is a simple program to sum all positive integers in the array `a`, in the domain $0 \leq i < n$.

Example 6.7.1 :

```
{* n ≥ 0 *}

SUMP (READ a:int[], n:int, OUT s:int) : ()

{ var i : int ;
  i := 0 ; s := 0 ;
  while i ≠ n do
    { if a[i]>0 then s:=s+a[i] else skip ;
      i := i + 1 }
}

{* s  =  SUM [x | x from a[0...n),  x > 0] *}
```

□

The program has an IL shape. Following the strategy in Subsection 6.4 we extend the specification as shown in Figure 6.9. The figure also shows our initial proof plan. To complete the proof you need to come up with a right `I` and `m`.

6.7.1 Constructing Invariant

Arbitrarily trying out different candidates for `I` is not a successful strategy. We will construct it incrementally. We first take a look at the post-condition for clues. In `PEC` (see the proof plan) you have to prove that $\mathtt{I} \wedge (\mathtt{i} = \mathtt{n})$, which is assumed to hold when the loop in `SUMP` terminates, implies the loop's specified post-condition. That is, we have to prove:

$$\mathtt{I} \wedge (\mathtt{i} = \mathtt{n}) \;\Rightarrow\; (\mathtt{s} \;=\; \mathtt{SUM}\,[\mathtt{x} | \mathtt{x}\ \mathtt{from}\ \mathtt{a}[0 \ldots \mathtt{n}),\ \mathtt{x} > 0]) \tag{6.16}$$

Let us first introduce the following abbreviation:

$$\mathtt{sum\ a\ n} \;=\; \mathtt{SUM}\,[\mathtt{x} | \mathtt{x}\ \mathtt{from}\ \mathtt{a}[0 \ldots \mathtt{n}),\ \mathtt{x} > 0] \tag{6.17}$$

So, now we can write (6.16) as:

$$\mathtt{I} \wedge (\mathtt{i} = \mathtt{n}) \;\Rightarrow\; (\mathtt{s} \;=\; \mathtt{sum\ a\ n}) \tag{6.18}$$

This is shorter. Moreover, it is now more obvious that a quite trivial choice for `I` is $\mathtt{s} = \mathtt{sum\ a\ i}$. Is this a reasonable choice? Well, at least initialization will not be a problem:

```
{* n ≥ 0 , see proof Init *}

i := 0 ; s := 0 ;

{* I , see proofs PTC1,PTC2,PEC,PIC. Termination metric: m = ... *}

while i ≠ n do
  { if a[i]>0 then s:=s+a[i] else skip ;
    i := i + 1 }

{* s = SUM [x | x from a[0...n), x > 0] *}

ASSUMING

I = ...

PROOF Init
... (here you prove {* n ≥ 0 *} i := 0; s := 0 {* I *})
PROOF PTC1
... (here you prove {* I ∧ i ≠ n *} (C := m; "loop's body") {* m < C *})
PROOF PTC2
... (here you prove I ∧ i ≠ n ⇒ m > 0)
PROOF PEC
... (here you prove I ∧ (i = n) ⇒ (s = SUM[x|x from a[0...n), x > 0]))
PROOF PIC
... (here you prove {* I ∧ i ≠ n *} "loop's body" {* I *})
```

Figure 6.9: *An initial proof plan of* SUMP

$$
\begin{aligned}
&\mathtt{wp}\ (\mathtt{i}{:=}0;\ \mathtt{s}{:=}0)\ (\mathtt{s}\ =\ \mathtt{sum\ a\ i})\\
&=\\
&0 = \mathtt{sum\ a}\ 0\\
&= \{\ \text{def. sum}\ \}\\
&0 = 0\\
&=\\
&\mathtt{true}
\end{aligned}
$$

We can further rationalize the choice by appealing to our programming insight on how SUMP's loop works. However, let us not do this now. We pretend that the loop is complicated, and try, this time, to just rely on our calculation. So, to summarize, for now we take this as I:

$$\mathtt{I}\ =\ (\mathtt{s}\ =\ \mathtt{sum\ a\ i}) \wedge \mathtt{h}_1 \tag{6.19}$$

where $\mathtt{h}_1$ is a place holder for further refinement of I. PEC, as already motivated above, is trivial and is not shown.

6.7.2 Loop Termination

To formally prove termination we need a termination metric m, which is an integer expression whose value is decreased by every iteration (proven in PTC1), but bounded below by 0 (proven in PTC2). Now, each iteration increases i, but leaves n unchanged. Therefore, it decreases $\mathtt{n}-\mathtt{i}$ —let us take this as our m. It has to be bounded below by 0. In this case, it means:

$$\mathtt{I} \wedge (\mathtt{i} \neq \mathtt{n})\ \Rightarrow\ 0 < \mathtt{n}-\mathtt{i} \tag{6.20}$$

In terms of implication from I we can equivalently express the above as:

$$\mathtt{I} \;\Rightarrow\; ((\mathtt{i} \neq \mathtt{n}) \Rightarrow 0 < \mathtt{n}-\mathtt{i})$$

If we simplify this a bit, it is equivalent to:

$$\mathtt{I} \Rightarrow \mathtt{i} \leq \mathtt{n} \tag{6.21}$$

Unfortunately, we cannot prove this $\mathtt{i} \leq \mathtt{n}$ from the information we have in I so far. So, according to the strategy in Section 6.6, we simply add $\mathtt{i} \leq \mathtt{n}$ to I. We can do so by instantiating the place holder $\mathtt{h}_1$ to $\mathtt{i} \leq \mathtt{n} \wedge \mathtt{h}_2$, where $\mathtt{h}_2$ is a new place holder for further refinement. So, now I is:

$$\mathtt{I} \;=\; (\mathtt{s} \;=\; \mathtt{sum\ a\ i}) \wedge \mathtt{i} \leq \mathtt{n} \wedge \mathtt{h}_2 \tag{6.22}$$

With this choice of I PTC2 is of course trivial. We leave it out for you.

That m is decreasing has been argued informally above. For completeness we show PTC1 below —it is quite trivial. Notice that since we have an `if-then-else` in the loop's body, the resulting wp is of the form $g \rightarrow f \mid g$. In this case it is actually of the form $g \rightarrow f \mid f$, which is of course equal to f (see the step marked with (*)). The assumption A in is not used in the proof, but is there for formality.

PROOF PTC1

[A:] $\mathtt{I} \wedge \mathtt{i} \neq \mathtt{n}$
[D:] $\mathtt{Q} \;=\; \mathtt{wp}\ (\mathtt{C} = \mathtt{n} - \mathtt{i}; \text{"}\textit{loop's body}\text{"})\ (\mathtt{n} - \mathtt{i} < \mathtt{C})$
[G:] Q

See the equational proof below:

EQUATIONAL PROOF

Q
= { wp calculation }
$\mathtt{a[i]{>}0} \;\rightarrow\; \mathtt{n} - (\mathtt{i}+1) < \mathtt{n} - \mathtt{i} \;\mid\; \mathtt{n} - (\mathtt{i}+1) < \mathtt{n} - \mathtt{i}$
= { (*) rewrite with COND Conversion (T A.4.4) }
$\mathtt{n} - (\mathtt{i}+1) < \mathtt{n} - \mathtt{i}$
= { trivial }
true
END

We can add the new information we gain over I and m to the the proof plan —the changes to the plan are shown below:

{* ... *}
initialization ;
{* I , see proofs PTC1,PTC2,PEC,PIC. Termination metric: n−i *}
while $\mathtt{i} \neq \mathtt{n}$ do { *loop's body* }
{* $\mathtt{s} \;=\; \mathtt{sum\ a\ n}$ *}

ASSUMING
$\mathtt{I} \;=\; (\mathtt{s} \;=\; \mathtt{sum\ a\ i}) \wedge \mathtt{i} \leq \mathtt{n} \wedge \mathtt{h}_2$

PROOF ...

Notice that PTC1 and PTC2 do not depend on what $\mathtt{h}_2$ is. So further extension of I obtained through instantiating $\mathtt{h}_2$ will not invalidate those proofs.

6.7.3 Proving Invariance

We have proven that the loop in SUMP terminates and that the state upon termination indeed satisfies SUMP's post-condition. However, the proofs assume the invariance of I. Now, in PIC, we are now going to verify this assumption. Given the definition of I, this is what we have to prove in PIC:

$$\{*\ \mathtt{I} \wedge \mathtt{i} \neq \mathtt{n}\ *\}\ \text{"}\textit{loop body}\text{"}\ \{*\ (\mathtt{s} = \mathtt{sum\ a\ i}) \wedge \mathtt{i} \leq \mathtt{n} \wedge \mathtt{h}_2\ *\}$$

Since the body does not contain further loop, we can prove this by showing that the pre-condition implies the wp of the body over the specified post-condition. Theorem 6.5.1 allows us to prove the implication separately for the wp of each of the conjunct in the post-condition. See the top level of PIC below:

PROOF PIC

[A:] $\mathtt{I} \wedge \mathtt{i} \neq \mathtt{n}$
[G:] wp "*loop's body*" $((\mathtt{s} = \mathtt{sum\ a\ i}) \wedge \mathtt{i} \leq \mathtt{n} \wedge \mathtt{h}_2)$

BEGIN

1. { justified by proof PI1 } wp "*loop's body*" $(\mathtt{s} = \mathtt{sum\ a\ i})$
2. { justified by PI2, left out for you } wp "*loop's body*" $\mathtt{i} \leq \mathtt{n}$
3. { justified by PI3, postponed } wp "*loop's body*" $\mathtt{h}_2$
4. { wp Distributivity (Theorem 6.5.1) on 1,2,3} }
   wp "*loop's body*" $((\mathtt{s} = \mathtt{sum\ a\ i}) \wedge \mathtt{i} \leq \mathtt{n} \wedge \mathtt{h}_2)$

END

PI2 proves wp *body* $\mathtt{i} \leq \mathtt{n}$. It is quite easy, and is left out for you. PI3 is supposed to prove wp *body* $\mathtt{h}_2$, but it cannot be given yet, since $\mathtt{h}_2$ is still unspecified. We will return to this later.

The proof of wp *body* $(\mathtt{s} = \mathtt{sum\ a\ i})$ is less trivial. It is shown below, but it is still incomplete. It proceeds as follows. In step 1 we show the result of calculating the wp. The resulting formula is of the form $P \rightarrow R_1 \mid R_2$, which can be proven by proving $P \Rightarrow R_1$ and $\neg P \Rightarrow R_2$ separately (formally, based on Theorem COND Split; see page 31). Finally, step 6 derives the goal.

PROOF PI1

[D:] $\mathtt{Q} = \mathtt{wp}\ \textit{loop body}\ (\mathtt{s} = \mathtt{sum\ a\ i})$
[G:] Q

BEGIN

1. { calculating wp }
   $\mathtt{Q} = \mathtt{a[i]} > 0 \rightarrow (\mathtt{s} + \mathtt{a[i]} = \mathtt{sum\ a}\ (\mathtt{i}+1)) \mid (\mathtt{s} = \mathtt{sum\ a}\ (\mathtt{i}+1))$
2. { rewrite 1 with $\mathtt{s} = \mathtt{sum\ a\ i}$, which follows from PIC.A and def. (6.22) of I }
   $\mathtt{Q} = \mathtt{a[i]} > 0 \rightarrow (\mathtt{sum\ a\ i} + \mathtt{a[i]} = \mathtt{sum\ a}\ (\mathtt{i}+1)) \mid (\mathtt{sum\ a\ i} = \mathtt{sum\ a}\ (\mathtt{i}+1))$
3. { justified by proof PCASE1, see Figure 6.10 }
   $\mathtt{a[i]} > 0 \Rightarrow (\mathtt{sum\ a\ i} + \mathtt{a[i]} = \mathtt{sum\ a}\ (\mathtt{i}+1))$
4. { justified by PCASE2, see Figure 6.11 }
   $\neg(\mathtt{a[i]} > 0) \Rightarrow (\mathtt{sum\ a\ i} = \mathtt{sum\ a}\ (\mathtt{i}+1))$
5. { rewrite with COND Split (T 3.8.6) on the conjunction of 3 and 4 }
   $\mathtt{a[i]} > 0 \rightarrow (\mathtt{sum\ a\ i} + \mathtt{a[i]} = \mathtt{sum\ a}\ (\mathtt{i}+1)) \mid (\mathtt{sum\ a\ i} = \mathtt{sum\ a}\ (\mathtt{i}+1))$
6. { rewrite 5 with 2 } Q

END

Let us now take a closer look at step 3 in the proof PI1. Assuming $\mathtt{a[i]} > \mathtt{0}$ you have to prove that $\mathtt{sum\ a\ i} + \mathtt{a[i]} = \mathtt{sum\ a\ (i+1)}$. This is a bit tricky. Unfolding the definition of sum, you have to prove that:

$$\mathtt{SUM\ [x|x\ from\ a[0 \ldots i),\ x{>}0]\ +\ a[i]} \tag{6.23}$$

is equal to:

$$\mathtt{SUM\ [x|x\ from\ a[0 \ldots i+1),\ x{>}0]} \tag{6.24}$$

We expect: $\mathtt{a[0 \ldots i+1)\ =\ a[0 \ldots i)}{+\!\!+}\mathtt{[a[i]]}$. Hence (6.24) is equal to:

$$\mathtt{SUM\ [x|x\ from\ (a[0 \ldots i)\ {+\!\!+}\ [a[i]]),\ x{>}0]} \tag{6.25}$$

which is equal to:

$$\mathtt{SUM\ [x|x\ from\ a[0 \ldots i),\ x{>}0]\ +\ (a[i]{>}0 \rightarrow a[i] \mid 0)} \tag{6.26}$$

And since $\mathtt{a[i]{>}0}$ is assumed, the above is equal to (6.23), and hence we have shown what we want, namely that (6.23) is equal to (6.24).

There is however a detail overlooked in the above argument. We have said that $\mathtt{a[0 \ldots i{+}1)}$ is equal to $\mathtt{a[0 \ldots i)}{+\!\!+}\mathtt{[a[i]]}$. However, this is only the case if $\mathtt{0} \leq \mathtt{i}$ —with a negative i, $\mathtt{a[0 \ldots i{+}1)}$ is empty whereas $\mathtt{a[0 \ldots i)}{+\!\!+}\mathtt{[a[i]]}$ is not, so they are not equal. See also Theorem 4.1.11. Splitting an array and a domain expression (e.g. $a{\leq}i{<}b$) has the same pitfall. In terms of bug, the above unsound reasoning can lead to the conclusion that it is safe to innitialize i with for example $-\mathtt{1}$, which of course leads to an incorrect result.

So, expression 3 in PI1 cannot be proven without first having the condition $\mathtt{0} \leq \mathtt{i}$ derived. Unfortunately, none of the assumptions of PI1 or its ancestors. Essentially, we cannot prove $\mathtt{0} \leq \mathtt{i}$ because I does not contain the needed information. So again, we apply the strategy in Section 6.6. We add $\mathtt{0} \leq \mathtt{i}$ to I by instantiating $\mathtt{h_2}$:

$$\mathtt{h_2\ =\ 0 \leq i \wedge h_3} \tag{6.27}$$

where $\mathtt{h_3}$ is a new place holder. So, I is now:

$$\mathtt{I\ =\ (s\ =\ sum\ a\ n) \wedge i \leq n \wedge 0 \leq i \wedge h_3} \tag{6.28}$$

Now we can prove expression 3 of PI1. The proof is shown in Figure 6.10. The step marked with (*) is where we use the new information $\mathtt{0} \leq \mathtt{i}$. Expression 4 (of PI1) can be proven in much the same way; the proof is shown in Figure 6.11.

We are almost done. PIC is still incomplete. PI3, where we have to prove wp *body* $\mathtt{h_2}$ is still missing. We now know more about $\mathtt{h_2}$, see (6.27), so perhaps now we can close the proof. See below:

PROOF PI3
[G:] wp *body* $(\mathtt{0{\leq}i} \wedge \mathtt{h_3})$
BEGIN

1. { wp calculation } wp *body* $\mathtt{0{\leq}i\ =\ 0{\leq}i{+}1}$
2. { follows from I in PIC.A } $\mathtt{0 \leq i}$
3. { follows from 2 } $\mathtt{0 \leq i{+}1}$
4. { rewrite 3 with 1 } wp *body* $\mathtt{0{\leq}i}$
5. { see the explanation below } wp *body* $\mathtt{h_3}$
6. { Theorem 6.5.1 } wp *body* $(\mathtt{0{\leq}i} \wedge \mathtt{h_3})$

END

PROOF PCASE1

[A:] $\mathtt{a[i]} > 0$

[G:] $\mathtt{sum\ a\ i} + \mathtt{a[i]} \ = \ \mathtt{sum\ a\ (i+1)}$

BEGIN

1. { follows from PIC.A, def. (6.28) of I }
   $0 \leq \mathtt{i}$
2. { see the equational proof below }
   $\mathtt{sum\ a\ i} + \mathtt{a[i]} = \mathtt{sum\ a\ (i+1)}$

   EQUATIONAL PROOF

   $\mathtt{sum\ a\ (i+1)}$

   = { PI1.D }

   $\mathtt{SUM\ [x|x\ from\ a[0 \ldots i+1),\ x > 0]}$

   = { Enumeration Split (T A.5.8), justified by 1 }

   $\mathtt{SUM\ [x|x\ from\ (a[0 \ldots i) +\!\!+ [a[i]]),\ x > 0]}$

   = { Comprehension Split (T A.5.12) }

   $\mathtt{SUM\ ([x|x\ from\ a[0 \ldots i),\ x > 0] +\!\!+ [x|x\ from\ [a[i]],\ x > 0])}$

   = { Singleton Comprehension (T A.5.10) }

   $\mathtt{SUM\ ([x|x\ from\ a[0 \ldots i),\ x > 0] +\!\!+ (a[i] > 0 \rightarrow [a[i]] \mid []))}$

   = { homomorphism of SUM (T A.5.16) }

   $\mathtt{SUM\ [x|x\ from\ a[0 \ldots i),\ x > 0]\ +\ SUM\ (a[i] > 0 \rightarrow [a[i]] \mid [])}$

   = { def. sum and A }

   $\mathtt{sum\ a\ i} + \mathtt{SUM\ (true} \rightarrow \mathtt{[a[i]] \mid [])}$

   = { COND conversion (T 3.8.5) }

   $\mathtt{sum\ a\ i} + \mathtt{SUM\ [a[i]]}$

   = { def. of SUM }

   $\mathtt{sum\ a\ i} + \mathtt{a[i]}$

   END

END

Figure 6.10: *Sub-proof* PCASE1.

PROOF PCASE2

[A:] $\neg(\mathtt{a}[\mathtt{i}] > 0)$
[G:] $\mathtt{sum\ a\ i} \ = \ \mathtt{sum\ a}\ (\mathtt{i} + 1)$

BEGIN

1. { follows from PIC.A, def. (6.28) of I }
   $0 \leq \mathtt{i}$
2. { see the equational proof below }
   $\mathtt{sum\ a\ i} = \mathtt{sum\ a}\ (\mathtt{i} + 1)$

   EQUATIONAL PROOF

   $\mathtt{sum\ a}\ (\mathtt{i} + 1)$

   $=$ { same steps as in PCASE1 }

   $\mathtt{SUM}\ [\mathtt{x}|\mathtt{x\ from\ a}[0 \ldots \mathtt{i}),\ \mathtt{x} > 0] \ + \ \mathtt{SUM}\ (\mathtt{a}[\mathtt{i}] > 0 \rightarrow [\mathtt{a}[\mathtt{i}]] \mid [])$

   $=$ { def. sum and A }

   $\mathtt{sum\ a\ i} + \mathtt{SUM}\ (\mathtt{false} \rightarrow [\mathtt{a}[\mathtt{i}]] \mid [])$

   $=$ { COND conversion (T 3.8.5) }

   $\mathtt{sum\ a\ i} + \mathtt{SUM}\ []$

   $=$ def.ofSUM

   $\mathtt{sum\ a\ i}$

   END

END

Figure 6.11: *Sub-proof* PCASE2.

The justification of wp *body* h_3 in step 5 is still missing. We cannot prove it because h_3 is still unknown. However, this and Init are the only proofs left to close. Taking the trivial solution true for h_3 will do: wp *body* true = true; so step 5 becomes trivial. In Init we prefer an I which is as weak as possible; taking true for h_3 will not cause us to fail Init. By taking true, we can just as well remove h_3 from the definition of I.

Below is the structure of the new proof plan. It now includes the full invariant, and also an overview of the proof obligations which have been fulfilled.

```
{* n≥0 *},  see proof Init
initialization  ;
{* I  , see proofs PTC1,PTC2,PEC,PIC.  Termination metric: n−i *}
while i ≠ n do  {  loop's body  }
{* s  =  sum a n *}

ASSUMING
I  =  (s  =  sum a i)  ∧  i≤n  ∧  0≤i

PROOF Init  to do
PROOF PTC1 ...  done
PROOF PTC2 ...  done
PROOF PIC  ...  done
PROOF PEC  ...  done
```

6.7.4 Initialization

It now remains to show, in Init, that the invariant that we have constructed can be established by the initialization code. More specifically, you have to prove:

$$\{* \, \mathtt{n} \geq 0 \, *\} \; (\mathtt{i} := 0; \mathtt{s} := 0) \; \{* \, \mathtt{I} \, *\}$$

The proof is quite simple:

```
PROOF Init

[A:] n ≥ 0
[G:] wp (i := 0; s := 0) I

BEGIN ____________________________________________________________

  1. { see the equational proof below }   wp (i :=0; s:=0) I  =  n≥0

        EQUATIONAL PROOF ____________________________________________
        wp (i := 0; s := 0) I
        = { def. of I, calculating wp }
        (0 = SUM [x|x from a[0...0), x > 0]) ∧ 0 ≤ n ∧ 0 ≤ 0
        = { Empty Enumeration (T A.5.5) }
        (0 = SUM [x|x from [], x > 0]) ∧ 0 ≤ n ∧ 0 ≤ 0
        = { Empty Comprehension (T A.5.9) }
        (0 = SUM []) ∧ 0 ≤ n ∧ 0 ≤ 0
        = { def. of SUM, simplification }
        0 ≤ n
        END _________________________________________________________

  2. { rewrite A with 1 }
     wp (i := 0; s := 0) I

END ______________________________________________________________
```

This completes the proof of SUMP.

6.8 Invariants

Quite often, the goal of a loop is to implement the calculation of a certain expression, e.g. to calculate $\sqrt{x}$, `fib t`, (`SUM` i : $0 \leq i < n$: `a`$[i]$). A loop is needed either because the expression cannot be directly expressed in the given programming language, or we simply want to have our own implementation of it, e.g. to improve performance. Let $f\ n$ be the expression we want to calculate, and the post-condition to realize is:

$$\texttt{while}\ g\ \texttt{do}\ S \quad \{*\ \texttt{x} = f\ n\ *\} \tag{6.29}$$

for some result-variable `x`. E.g. in our previous examples `FIB` and `SUMP` in Figures 6.5 and 6.7.1, the post-conditions are `nNow` = `fib n` and `s` = `sum a n`. The presence of some parameters like `n` is typical; it simply reflects of the fact that our program itself takes parameters.

What we have seen in the previous examples is that the key part of our invariants have the form of:

$$\texttt{x} = f\ i$$

where i is a variable or a sub-expression that will progress towards n, so that when the loop terminates with $i = n$, we then have the desired post-condition in (6.29). For example, in the program `FIB` the post-condition to realize is `nNow` = `fib n`, and the key part of its invariant is `nNow` = `fib t`, where `t` is its loop counter. In `SUMP` the post-condition is `s` = `sum a n`, and the key part of its invariant is `s` = `sum a i`. Both formulate the invariant in terms of *what has been calculated so far*. For example:

$$\texttt{s} \ = \ \texttt{sum a i}$$

expresses that so far *we have calculated* `sum a i`: the value of this expression is stored in `nNow`.

6.8.1 Head and Tail Invariant

Alternatively, we can also try to formulate an invariant in terms of *what still have to be computed*. For example, let us we define `sumr a i n` = (`SUM k` : `i`$\leq$`k`$<$`n` : `a`[`k`]). The following is also a good invariant for `SUMP`:

$$\texttt{0} \leq \texttt{i} \leq \texttt{n} \ \wedge \ (\texttt{s} + \texttt{sumr a i n} = \texttt{sum a n}) \tag{6.30}$$

Remember that `SUMP`'s loop terminates with `i` = `n`. This implies that `sumr a i n` = `0`. Hence `s` = `sum a n`.

Notice that the `sumr a i n` express the part that *still has to be computed*: its value is not stored in any variable. Termination should imply that this part would then become 'empty', and thus `s` will hold the desired goal.

Consider again the general problem in (6.29), with $x = f\ n$ as the post-condition. The two forms of invariants have the following general shapes:

$$I: \quad x = f\ i \tag{6.31}$$

$$I: \quad x \oplus \psi\ i = f\ n \tag{6.32}$$

The latter is also called a *tail invariant* (and thus some people call the first one a *head invariant*). In the first case, when the loop terminates with $i = n$, the post-condition is realized. In the second one, when the loop terminates it should imply that $\psi\ i$ would be equal to the unit element of $\oplus$; thus implying $x = f\ n$.

In the case of `SUMP` both head and tail invariants are equally natural. For `FIB` it is not straight forward to come up with a tail invariant. Then you can also imagine that there are situations where it is not straight forward to come up with a head invariant. An example of that is the program `Euclid` program you saw in the Introduction —see again Figure 1.1.

`Euclid` calculates the greatest common divisor of two positive integers `x` and `y`. Its loop's post-condition is $\mathtt{x} = \mathtt{gcd}(\mathtt{X}, \mathtt{Y})$, where `X` and `Y` represent the initial values of `x` and `y`. Its invariant is:

$$0<\mathtt{x} \wedge 0<\mathtt{y} \ \wedge \ (\mathtt{gcd}(\mathtt{x}, \mathtt{y}) = \mathtt{gcd}(\mathtt{X}, \mathtt{Y})) \tag{6.33}$$

When its loop terminates we have $\mathtt{x} = \mathtt{y}$. This implies $\mathtt{gcd}(\mathtt{x}, \mathtt{y}) = \mathtt{x}$. Hence, by the invariant above, we now have the post-condition.

The key part of the above invariant is the $\mathtt{gcd}(\mathtt{x}, \mathtt{y}) = \mathtt{gcd}(\mathtt{X}, \mathtt{Y})$ part. It is not a head invariant, because we have not calculated $\mathtt{gcd}(\mathtt{x}, \mathtt{y})$. We do have some intermediate results in `x` and `y`, but they are not $\mathtt{gcd}(\mathtt{x}, \mathtt{y})$. The invariant does not look to fit the pattern in (6.32) either. At least, not at the first glance. However, the $\mathtt{gcd}(\mathtt{x}, \mathtt{y})$ does represent the remaining work to do to get the value of `x` and `y` towards the goal $\mathtt{gcd}(\mathtt{X}, \mathtt{Y})$. If we do a little trick to rewrite the invariant to:

$$\mathtt{x} \oplus \mathtt{y} \ = \ \mathtt{gcd}(\mathtt{X}, \mathtt{Y}) \ \text{ where } \mathtt{x} \oplus \mathtt{y} = \mathtt{gcd}(\mathtt{x}, \mathtt{y})$$

Now it does fit to (6.32). It is just a 'cosmetical trick' of course; the important thing is to distinguish between expressing your invariant in terms of what you did or in terms of what is still to be done. The choice depends on which one can be expressed.

Generalizing

In the two examples discussed above, the programs `SUMP` and `Euclid`, the post-conditions take the form of $v = f\ n$ where f is some function whose value is to be calculated, and v is the variable that holds the result of this calculation —see the pattern in (6.29). Let us re-express the problem in a slightly more general way. Consider a variation of such a post-condition where goal is to calculate the value of x *such that* $\phi\ x = f\ n$. That is:

$$\mathtt{while}\ g\ \mathtt{do}\ S \ \ \{*\ \phi\ \mathtt{x} = f\ n\ *\} \tag{6.34}$$

If ϕ is invertible then we can indeed re-express the post-condition as $\mathtt{x} = \phi^{-1}\ (f\ n)$, which is an instance of the previous problem of "calculating the value of $f\ n$". If ϕ is not invertible then obviously we cannot formulate the problem in that way.

A candidate head invariant for the loop would have this form as its key part:

$$I: \ \ \phi\ x = f\ i \ \text{ with } \phi\ x = f\ n \text{ as the post-condition} \tag{6.35}$$

with termination, $I \wedge \neg g$, should be as such that it implies $i = n$.

A candidate tail invariant has this form as its key part:

$$I: \ \ \phi\ x \otimes \psi\ i = f\ n \tag{6.36}$$

For this to work, we need termination to imply that $\psi\ i$ is equal to the unit element of $\otimes$. Note that the above invariant is an instance of (6.32): we can write it as $x \oplus \psi\ i$ with $\oplus$ defined as $x \oplus y = \phi\ x \otimes y$.

As an example, consider a program called `digits` to destruct a non-negative integer x into a list of its digits. The list will be stored in the array `a`. If we have such a list, we can easily express what the integer value it represents, namely:

$$\mathtt{ival\ a\ n} \ = \ (\mathtt{SUM\ i} : 0 \le \mathtt{i} < \mathtt{n} \ : \ \mathtt{a}[\mathtt{i}] * 10^{\mathtt{i}})$$

With this we can specify `digits` as shown in Figure 6.12. The post-condition of its loop is also annotated there.

Here is a good invariant for the loop:

$$0 \le \mathtt{x} \le \mathtt{X} \ \wedge \ (\mathtt{ival\ a\ i} + \mathtt{x} * 10^{\mathtt{i}} \ = \ \mathtt{X}) \tag{6.37}$$

where `X` is the initial value of `x` (when it was passed to `digits`). When the loop terminates, we have $\mathtt{x} = 0$. So, the term $\mathtt{x} * 10^{\mathtt{i}}$ is also 0. Hence we realize the post-condition $\mathtt{ival\ a\ i} = \mathtt{X}$. Notice that the invariant is a tail invariant.

```
{* x = X ∧ X ≥ 0 *}

digits(x:int, a:int[]) : int {
   var i:int ;
   i := 0 ;
   while x>0 do {
     a[i] = x mod 10 ;
     x = x div 10
   }
    {* ival a i = X *}
   return i
}

{* ival a return = x *}
```

Figure 6.12: *A program to destruct an integer.*

6.8.2 Loop for Searching a Solution

Another typical use of a loop is to find a solution of some predicate Q, e.g. to find a minimum of a function (it may have multiple minimums), or a crossing point between two curves. The specification typically has this form:

$$\{*\ (\exists x : x \in \Sigma : Q\ x)\ *\}\ \ search(\Sigma)\ \ \{*\ Q\ \texttt{return}\ *\} \tag{6.38}$$

where Q is a parameterized predicate, whose solution is wanted. So, Q `return` means that the return value of the program above satisfies Q; so, it is a solution. Note that Q itself may have multiple solutions, but here we just want to have one solution.

The Σ in the pre-condition expresses the space within which we will do our search. The pre-condition furthermore says that a solution exists within this Σ. In practice you may want drop this condition and have the program to return a special value to signal if a solution cannot be found. For simplicity we will not do this here.

When searching, what you do is essentially to repeatedly and systematically shrink the search space, which you can do with a loop. So, if V is represents the part of Σ which we have visited, this will make a reasonable candidate of the invariant of that loop:

$$(\exists x : x \in \Sigma/V : Q\ x)\ =\ (\exists x : x \in \Sigma : Q\ x) \tag{6.39}$$

You can terminate the loop once you can trivially find an element in Σ/V that satisfies Q, e.g. if the set only contains one element. Returning this element will realize the post-condition in (6.38).

Notice that the above invariant is a tail invariant: Σ/V is the part of the search space which still have to be searched.

6.8.3 Iterative Implementation of Tail Recursion

Consider the following two functions to recursively calculate the sum of the elements in a list. You have seen the definition of the first one, `SUM`, before, in Chapter 4. The second function `summacc` works differently. It accumulates the result of the summing in its first parameter r, which it passes down through the recursion until it reaches the base case []; at that point r would contain the final result. You can prove that `sumacc` $0\ s =$ `SUM` s, and hence you can use the first to calculate the latter.

$$\begin{array}{lcl} \texttt{SUM}\ [] & = & 0 \\ \texttt{SUM}\ (x : s) & = & x + \texttt{SUM}\ s \end{array} \tag{6.40}$$

$$\begin{array}{lcl} \texttt{sumacc}\ r\ [] & = & r \\ \texttt{sumacc}\ r\ (x : s) & = & \texttt{sumacc}\ (r + x)\ s \end{array} \tag{6.41}$$

Notice that when `sumacc` recurses, after returning from the recursion the result of the recursion does not need further processing. It is immediately returned. In contrast, in `SUM` the result of a recursion does need further processing: it needs to be combined with $x+$ before being returned. The recursion pattern of `sumacc` is also called *tail recursion*.

Let us first consider a general form of tail recursive functions:

$$F\ x\ =\ g\ x \rightarrow base\ x \mid F\ (\Delta\ x) \tag{6.42}$$

Notice that when the recursion hits an x such that $g\ x$ is true, it stops recursing. In other words, $g\ x$ is the recursion's base case. The function *base* specifies the final processing upon reaching the base case. The function Δ specifies what calculation is to be accumulated into the parameter x at every recursion.

Such a function can be directly translated to a more efficient iterative solution:

```
{* true *}
 x := α ; while¬g x do x := Δ x ;
 r := base x
{* r = F α *}
```

The program above will calculate $F\ \alpha$, for some α that was given at the start of the program. We can prove the correctness of this loop through the invariant:

$$I:\ F\ x = F\ \alpha \tag{6.43}$$

Notice that it has the form of the tail invariant in (6.36).

When the loop terminate, we would have $I \wedge g\ x$. When $g\ x$ holds, the definition of F (6.42) says that $F\ x = base\ x$, and therefore $I \wedge g\ x$ would then imply $base\ x = F\ \alpha$. And therefore, after the final assignment $r := base\ x$, we will establish the post-condition $r = F\ \alpha$.

To prove that the invariant can be maintained we need to prove:

$$\{*\ \texttt{F x=F}\ \alpha\ \wedge\ \neg\texttt{g x}\ *\}\ \ \texttt{x} := \Delta\ \texttt{x}\ \ \{*\ \texttt{F x=F}\ \alpha\ *\}$$

By calculating the `wp` of $\texttt{x} := \Delta\ \texttt{x}$ the above is equivalent to:

$$\texttt{F x=F}\ \alpha \wedge \neg\texttt{g x}\ \ \Rightarrow\ \ \texttt{F}\ (\Delta\ \texttt{x})\ =\ \texttt{F}\ \alpha$$

But this follows from the definition of F that says that when $g\ x$ does not hold then $F\ x$ is equal to $F\ (\Delta\ x)$.

If we assume F to be of type $A \rightarrow A$, we can show that the above loop terminates by finding a termination metric in the form of a function $m : A \rightarrow \texttt{int}$, and showing that $\neg g\ x$ implies $m\ x > 0$ and $m\ (F\ x) < m\ x$.

Let us now return to the example `sumacc` in (6.41). Notice that it can be rewritten as shown below:

$$\mathsf{sumacc}\ (r, s)\ \ =\ \ s{=}[]\ \rightarrow\ r \mid \mathsf{sumacc}\ (r + \mathsf{head}\ s\ ,\ \mathsf{tail}\ s) \tag{6.44}$$

where for a non-empty list s, $\mathsf{head}\ s$ and $\mathsf{tail}\ s$ return respectively the first element of s, and the rest of s after its first element.

We can easily see that `sumacc` is an instance of the pattern in (6.42), which then gives us the corresponding iterative implementation, which in this case would be:

```
{* true *}
 r, s := 0, σ ; while s≠[] do x, s := Δ x, tail s ;
{* r = summacc (0, σ) *}
```

The general pattern above also specifies what the needed invariant for the loop, namely I : $\texttt{sumacc}(r, s) = \texttt{sumacc}(0, \sigma)$.

6.9 Proof of the Distributivity of wp

This section shows the proofs that (under some conditions) wp 'distributes' over conjunction and disjunction of post-conditions. What this means is that we can split the calculation of the wp of $Q_1 \wedge Q_2$ or $Q_1 \vee Q_2$ as the post-conditions to calculation of the wp of Q_1 and Q_2 separately. We have used these results in e.g. the proof of the Fibonacci program.

Theorem 6.9.1 : wp Distributivity over $\wedge$

$$\models \mathsf{wp}\ S\ (Q_1 \wedge Q_2) \quad = \quad (\mathsf{wp}\ S\ Q_1) \wedge (\mathsf{wp}\ S\ Q_2)$$

□

PROOF
Since $Q_1 \wedge Q_2 \Rightarrow Q_1$, by Rule 6.2.8 we have:

$$\mathsf{wp}\ S\ (Q_1 \wedge Q_2) \quad \Rightarrow \quad \mathsf{wp}\ S\ Q_1$$

By a similar argument we have:

$$\mathsf{wp}\ S\ (Q_1 \wedge Q_2) \quad \Rightarrow \quad \mathsf{wp}\ S\ Q_2$$

So:

$$\mathsf{wp}\ S\ (Q_1 \wedge Q_2) \quad \Rightarrow \quad (\mathsf{wp}\ S\ Q_1) \wedge (\mathsf{wp}\ S\ Q_2)$$

It remains to show that the implication in the reversed direction also holds. By Corollary 6.2.4 we have:

$$\{*\ \mathsf{wp}\ S\ Q_1\ *\}\ \ S\ \ \{*\ Q_1\ *\}$$

$$\{*\ \mathsf{wp}\ S\ Q_2\ *\}\ \ S\ \ \{*\ Q_2\ *\}$$

Now, by applying Hoare triple conjunction on the two above we obtain:

$$\{*\ (\mathsf{wp}\ S\ Q_1) \wedge (\mathsf{wp}\ S\ Q_2)\ *\}\ \ S\ \ \{*\ Q_1 \wedge Q_2\ *\}$$

So, the pre-condition above must imply $\mathsf{wp}\ S\ (Q_1 \wedge Q_2)$.
□.

Theorem 6.9.2 : wp Distributivity over $\vee$

$$\models \mathsf{wp}\ S\ (Q_1 \vee Q_2) \quad = \quad (\mathsf{wp}\ S\ Q_1) \vee (\mathsf{wp}\ S\ Q_2)$$

provided S is deterministic.
□

PROOF
The full proof of this is more complicated. We will only show it for partial correctness. Note first that it is sufficient to show that for all state s:

$$s \in \mathsf{wp}\ S\ (Q_1 \vee Q_2) \quad = \quad s \in \mathsf{wp}\ S\ Q_1\ \vee\ s \in \mathsf{wp}\ S\ Q_2$$

We derive:

$$s \in \mathsf{wp}\ S\ (Q_1 \vee Q_2)$$
$$= \{\ \text{Corollary 6.2.2}\ \}$$
$$S\ s \subseteq Q_1 \vee Q_2$$
$$= \{\ \text{in set interpretation } \vee \text{ corresponds to set union}\ \}$$
$$S\ s \subseteq Q_1 \cup Q_2$$

Since S is deterministic, $S\ s$ contains exactly one state; let's call it t. So, $S\ s = \{t\}$. We continue the derivation:

$$
\begin{array}{l}
S\ s \subseteq Q_1 \cup Q_2 \\
= \{\ S\ s = \{t\}\ \} \\
t \in Q_1 \cup Q_2 \\
= \{\ \text{set theory}\ \} \\
t \in Q_1\ \ \vee\ \ t \in Q_2 \\
= \{\ S\ s = \{t\}\ \} \\
S\ s \subseteq Q_1\ \ \vee\ \ S\ s \subseteq Q_2 \\
= \{\ \text{Corollary 6.2.2}\ \} \\
s \in \mathsf{wp}\ S\ Q_1\ \vee\ s \in \mathsf{wp}\ S\ Q_2
\end{array}
$$

□

6.10 Simultant Assignment

A simultant assignment is an assignment that simultaneously assigns values to multiple variables. We add it to uPL just for convenience. Here is an example:

```
x,y := 0,x+y
```

This is executed as follows. First *all* expressions at the right hand side are evaluated, then the results are assigned to the corresponding variables. Notice the difference with e.g. this:

```
x:=0; y:=x+y
```

This will two assignments in sequence. First 0 is evaluated then assigned to `x`, *then* $\mathtt{x} + \mathtt{y}$ is evaluated and then assigned to `y`. See the table below:

before		after
$\mathtt{x} = 1,\ \mathtt{y} = 2$	$\mathtt{x}, \mathtt{y} := 0, \mathtt{x} + \mathtt{y}$	$\mathtt{x} = 0,\ \mathtt{y} = 3$
$\mathtt{x} = 1,\ \mathtt{y} = 2$	$\mathtt{x} := 0;\ \mathtt{y} := \mathtt{x} + \mathtt{y}$	$\mathtt{x} = 0,\ \mathtt{y} = 2$

As said, simultant assignment is just for convenience. It can be implemented using ordinary assignments. For example, the assignment `x,y := 0,x+y` can be implemented by:

```
{var @x,@y ; @x:=x ; @y:=y ;
 x  := 0        ;
 y  := @x + @y  }
```

To calculate the wp of a simultant assignment we will need the corresponding notion of simultant substitution: the notation $Q[e_1/x, e_2/y]$ means the expression obtained by simultaneously replacing all free occurrences of x and y in Q with respectively e_1 and e_2. In particular, you should not perform the substitutions in this order $Q[e_1/x][e_2/y]$, nor $Q[e_2/y][e_1/x]$. For example:

$$(\mathtt{x} - \mathtt{y})[\mathtt{y}/\mathtt{x},\ \mathtt{x} + 1/\mathtt{y}] \quad = \quad (\mathtt{y} - (\mathtt{x} + 1))$$

whereas:

$$
\begin{array}{lcl}
(\mathtt{x} - \mathtt{y})[\mathtt{y}/\mathtt{x}][\mathtt{x} + 1/\mathtt{y}] & = & (\mathtt{x} + 1) - (\mathtt{x} + 1) \\
(\mathtt{x} - \mathtt{y})[\mathtt{x} + 1/\mathtt{y}][\mathtt{y}/\mathtt{x}] & = & \mathtt{y} - (\mathtt{y} + 1)
\end{array}
$$

A simultant assignment implicitly requires that the target variables are all *distinct*. So, $\mathtt{x}, \mathtt{x} := 0, 1$ is illegal. Similar requirement applies to simultant substitution: $\mathtt{Q}[0/\mathtt{x}, 1/\mathtt{x}]$ is illegal.

The Bar Notation

The notation $\overline{e}$ (pronounced "e bar") denotes a list of expressions: $e_1, \ldots, e_k$. The same notation is used to denote a list of variables.

Two lists of expressions or variables, $\overline{d}$ and $\overline{e}$, are *compatible* if they are of the same length and the type of d_i matches with the type of e_i for every i, $0 \leq i <$length of $\overline{d}$.

We will use the notation $\overline{x}{:=}\overline{e}$ to abbreviate $x_0, \ldots, x_k := e_1, \ldots, e_k$, and $P[\overline{e}/\overline{x}]$ to abbreviate $P\,[e_1/x_1, \ldots, e_k/x_k]$.

Theorem 6.10.1 : wp OF SIMULTANT ASSIGNMENT

$$\mathsf{wp}\ (\overline{x} := \overline{e})\ Q \quad = \quad Q[\overline{e}/\overline{x}]$$

□

6.11 Reducing Program's Specification

The logic given so far actually only deals with statements. When given the specification of a full program, what we did in the previous examples was to implicitly reduce it to a slightly different specification in terms of the program's body. Subsequently, we can handle it with the logic from the previous sections. So far we do the reduction from program to its body rather informally, but there are actually some technical details which you have to be aware of. Consider first this simple program without parameters:

$$\texttt{trivial}()\ \{\texttt{var z};\ \texttt{z} := \texttt{0};\ \texttt{return z}\ \}$$

You may expect that a specification like:

$$\{*\ P\ *\}\ \texttt{trivial}()\ \{*\ Q\ *\} \tag{6.45}$$

can be reduced to:

$$\{*\ P\ *\}\ \{\texttt{z} := \texttt{0};\ \texttt{return} := \texttt{z}\}\ \{*\ Q\ *\} \tag{6.46}$$

That is, proving the latter is sufficient to prove the first. Although seemingly trivial, there is already one complication that you have to consider here. The reduction is not correct if either P or Q in (6.45) contains a free occurence of `z`. This `z` would refer to the `z` in the context of `trivial`'s caller, and not to `trivial`'s local `z`. In (6.46) the `z` would refer to `trivial`'s local `z`.

Fortunately we have required that a program's specification must be *closed* (page 53). That is, it is not allowed to mention any variable other than the program's own formal parameters or the auxilliary variables used to remember the parameters' initial values. So, mentioning `z` in the pre- or post-condition in (6.45) is not allowed. Under this assumption (6.45) and (6.46) are equivalent.

Suppose we now have a program with parameters. Consider this specification:

$$\{*\ \texttt{true}\ *\}$$

$$\texttt{Y} := \texttt{y};\ \texttt{test}(\texttt{x}, \texttt{OUT y})\ \{\texttt{x} := \texttt{x+y};\ \texttt{y} := \texttt{0};\ \texttt{return x}\} \tag{6.47}$$

$$\{*\ (\texttt{return} = \texttt{x+Y}) \wedge (\texttt{y} = \texttt{0})\ *\}$$

Recall the agreement we made in page 53: `x` is a pass-by-value parameter; so `x` in the post-condition refers to its initial value rather than its final value just before the program `test` returns. Suppose we now naively translate the above specification to the one below, in terms of `test`'s body:

$$\{*\ \texttt{true}\ *\} \quad \{\texttt{Y} := \texttt{y};\ \texttt{x} := \texttt{x+y};\ \texttt{y} := \texttt{0};\ \texttt{return} := \texttt{x}\} \quad \{*\ (\texttt{return} = \texttt{x+Y}) \wedge (\texttt{y} = \texttt{0})\ *\}$$

The `x` in the post-condition now has a very different interpretation: it now refers to its final value. So, the above translation is not correct. We should have replaced `x` with `X` as follows:

$$\{*\ \texttt{true}\ *\}\quad \{\texttt{X},\texttt{Y} = \texttt{x},\texttt{y};\ \texttt{x} := \texttt{x}+\texttt{y};\ \texttt{y} := \texttt{0};\ \texttt{return} := \texttt{x}\}\quad \{*\ (\texttt{return} = \texttt{X}+\texttt{Y}) \wedge (\texttt{y} = \texttt{0})\ *\}$$

which is equivalent to (6.47). Formally, the reduction is captured by the following theorem. The case when the program have more or less parameters and local variables can be handled analogously.

Theorem 6.11.1 : Program to Statement Reduction
Let Pr be defined as below; x, y are its formal parameters; x is passed by-value and y by copy-restore; X, Y are auxilliary paremeters used to record x, y's initial values. Let this be a *closed* specification:

$$\{*\ P\ *\}\ \ X,Y{:=}x,y;\ \ Pr(x, \texttt{OUT}\ y)\ \{\texttt{var}\ z;\ S;\ \texttt{return}\ e\ \}\ \ \{*\ Q\ *\}$$

It is valid if and only if the following statement is valid:

$$\{*\ P\ *\}\ \ \{\ X,Y{:=}x,y;\ S;\ \texttt{return}{:=}e\ \}\ \ \{*\ Q[X/x]\ *\}$$

□

6.12 Updating Compound Structures

Assignments to coumpund structures, in particular arrays, have a little complication. Our method to calculate the `wp` of an assignment (Theorem 6.2.10) is:

$$\texttt{wp}\ (v := E)\ Q \quad = \quad Q[E/v]$$

Recall that we have specifically require that v should be a program variable. So, it does not apply for an assignment like $\texttt{a}[\texttt{i}] := \texttt{0}$. Let us see what will go wrong if we do that. Consider the post-condition $\texttt{a}[\texttt{i}] = \texttt{0}$. We now calculate the `wp` simply by replacing occurences of the the assignment's left hand side in the post-condition with the assignment's right hand side:

$$\begin{aligned}
&\texttt{wp}\ (\texttt{a}[\texttt{i}] := \texttt{0})\ (\texttt{a}[\texttt{i}] = \texttt{0})\\
&= \{\ \text{assume we calculate the } \texttt{wp} \text{ like this}\ \}\\
&(\texttt{a}[\texttt{i}] = \texttt{0})[\texttt{0}/\texttt{a}[\texttt{i}]]\\
&=\\
&\texttt{0} = \texttt{0}\\
&=\\
&\texttt{true}
\end{aligned}$$

This is as expected, since regardless the initial state $\texttt{a}[\texttt{i}] := \texttt{0}$ will of course establish $\texttt{a}[\texttt{i}] = \texttt{0}$ as the post-condition. Now consider a different post-condition:

$$\texttt{a}[\texttt{j}] = \texttt{0}$$

We do the calculation again:

$$\begin{aligned}
&\texttt{wp}\ (\texttt{a}[\texttt{i}] := \texttt{0})\ (\texttt{a}[\texttt{j}] = \texttt{0})\\
&= \{\ \text{assume we calculate the } \texttt{wp} \text{ like this}\ \}\\
&(\texttt{a}[\texttt{j}] = \texttt{0})[\texttt{0}/\texttt{a}[\texttt{i}]]\\
&=\\
&\texttt{a}[\texttt{j}] = \texttt{0}
\end{aligned}$$

However, this is not correct! The post-condition can also be realized if initially $\mathtt{i} = \mathtt{j}$. Since above we have concluded that $\mathtt{a}[\mathtt{j}] = \mathtt{0}$ is the *weakest* pre-condition, then it must be the case that:

$$(\mathtt{i} = \mathtt{j}) \Rightarrow (\mathtt{a}[\mathtt{j}] = \mathtt{0})$$

is valid. However, it is not.

What we have here is an aliasing problem at the array level. If `i` and `j` turns out to point to the same index in the array `a`, then changing the value of $\mathtt{a}[\mathtt{i}]$ will of course impact $\mathtt{a}[\mathtt{j}]$. In the above derivation, the substitution $[\mathtt{0}/\mathtt{a}[\mathtt{i}]]$ that we did essentially ignores this fact. Although we have said that, for simplicity, aliasing is not allowed in uPL (see page 5.1), we will still allow aliasing of array indices. Our logic will need a little extension though.

Let us introduce some notation first. Let $a(i \ \mathtt{repby}\ e)$ denote a new array whose content is the same as that of array a, except that at index i its value is e. Formally this can be characterized by the following equality:

Definition 6.12.1 : `repby`
Let a be an array:

$$a(i \ \mathtt{repby}\ e)[j] \quad = \quad (i = j) \rightarrow e \mid a[j]$$

□

Notice that assignments $a[i] := e$ and $a := a(i \ \mathtt{repby}\ e)$ have the same effect. Of course the latter is very inefficient, but for the purpose of reasoning we equivalently convert $a[i] := e$ to:

$$a \ := \ a(i \ \mathtt{repby}\ e)$$

Subsequently, we can calculate the `wp` in the usual way (as in Theorem 6.2.10). So:

Theorem 6.12.2 : `wp` OF ASSIGNMENT TO ARRAY

$$\mathtt{wp}\ (a[i]{:=}e)\ Q \ = \ Q[a(i \ \mathtt{repby}\ e)/a]$$

□

Notice that in effect the substitution:

$$Q[a(i \ \mathtt{repby}\ e)/a]$$

replaces not only $a[i]$ but also *every* sub-expression of the form $a[j]$ in Q with $i = j \rightarrow e \mid a[j]$. The is done because j *may* point to the same element as i in the array a. However, at this point we do not know this for sure. The formula $i = j \rightarrow e \mid a[j]$ takes both possibilities into account. If we later can prove that $i = j$, hence they are aliases, then $i = j \rightarrow e \mid a[j]$ can be reduced to e. Similarly, if we can prove $i \neq j$, so they are not aliases, we can reduce the formula to $a[j]$. Here is an example:

$$\mathtt{wp}\ (\mathtt{a}[\mathtt{i}] := \mathtt{0})\ (\mathtt{a}[\mathtt{j}] = \mathtt{0})$$
$$= \{ \text{ Theorem 6.12.2 } \}$$
$$(\mathtt{a}(\mathtt{i}\ \mathtt{repby}\ \mathtt{x}))[\mathtt{j}] \ = \ \mathtt{0}$$
$$= \{ \text{ Definition 6.12.1 } \}$$
$$(\mathtt{i} = \mathtt{j} \ \rightarrow \ \mathtt{0} \mid \mathtt{a}[\mathtt{j}]) \ = \ \mathtt{0}$$

The last formula can be proven if the pre-condition implies either $\mathtt{i} = \mathtt{j}$ or $\mathtt{a}[\mathtt{j}] = \mathtt{0}$.

6.12.1 Updating Record

Assignments to record fields are in some sense similar to assignments to array elements. We can see a record as an array, and its fields are the array's indices. We will not get the aliasing problem with records however, because the field names of a record are static (they do not change during the execution). So if two field names fn_1 and fn_2 are distinct, they will refer to different fields.

Still, it is convenient to handle assignments to a record field in the same way as we handle assignments to an array element. We will extend the `repby` notation to records:

Definition 6.12.3 : `repby`
Let r be a record. Let fn_1 and fn_2 be *distinct* field names:

$$\begin{array}{lcl} r(fn_1 \texttt{ repby } e).fn_1 & = & e \\ r(fn_1 \texttt{ repby } e).fn_2 & = & r.fn_2 \end{array}$$

□

Logically, the effect of $r.fn := e$ is the same as $r := r(fn \texttt{ repby } e)$. The `wp` of the latter is calculated as standard. Formally:

Theorem 6.12.4 : `wp` OF ASSIGNMENT TO RECORD

$$\texttt{wp } (r.fn{:=}e)\ Q \ = \ Q[r(fn \texttt{ repby } e)/r]$$

□

As an example consider the record type `person` defined as follows:

```
record person {key:int, name:string}
```

Suppose `a` is a variable of type `person`. The `wp` of the assignment `a.key:=0` with respect to the post-condition $\texttt{a.key} = \texttt{b.key}$ is calculated as follows:

$$\begin{array}{l} \texttt{wp } (\texttt{a.key} := 0)\ (\texttt{a.key} = \texttt{b.key}) \\ = \{ \text{ Theorem 6.12.4 } \} \\ (\texttt{a}(\texttt{key repby } 0)).\texttt{key} = \texttt{b.key} \\ = \{ \text{ Definition 6.12.3 } \} \\ 0 = \texttt{b.key} \end{array}$$

A post-condition that contains an expression on a record as a whole, e.g. a post-condition like $\texttt{a} = \texttt{b}$, causes a little complication. For example:

$$\begin{array}{l} \texttt{wp } (\texttt{a.key} := 0)\ (\texttt{a} = \texttt{b}) \\ = \{ \text{ Theorem 6.12.4 } \} \\ (\texttt{a}(\texttt{key repby } 0)) = \texttt{b} \end{array}$$

The resulting `repby` expression cannot be further simplified. However, we do know that two records r_1 and r_2 of type `person` are equal if and only if $r_1.\texttt{key} = r_2.\texttt{key}$ and $r_1.\texttt{name} = r_2.\texttt{name}$. With this information we can simplify further:

$$\begin{array}{l} (\texttt{a}(\texttt{key repby } 0)) = \texttt{b} \\ = \\ ((\texttt{a}(\texttt{key repby } 0)).\texttt{key} = \texttt{b.key}) \ \wedge \ ((\texttt{a}(\texttt{key repby } 0)).\texttt{name} = \texttt{b.name}) \\ = \{ \text{ Definition 6.12.3 } \} \\ (0 = \texttt{b.key}) \ \wedge \ (\texttt{a.name} = \texttt{b.name}) \end{array}$$

6.13 Advanced Features

We have kept uPL simple because of its introductory purpose. Below, we will shortly discuss some more advanced features which are not covered in uPL . A good source for further reading is [3, 4, 19].

6.13.1 assert and assume

Many languages e.g. Java allow us to write a statement like `assert` P. It checks if P holds. If so, then everything is fine. It will just proceed to the next statement. If P does not hold it will throw an exception. So, it is just a syntactic sugar of

$$\texttt{if}\ P\ \texttt{then skip else throw}\ an{-}exception$$

In practice people use `assert` to help them during testing, namely to express general properties that are expected to hold at certain points in the execution. But for verification it is also has another use. Let us define it such that it satisfies this property:

Definition 6.13.1 : WP OF ASSERT

$$\texttt{wp}\ (\texttt{assert}\ P)\ Q\ =\ P \wedge Q \quad \text{, if } Q \not\equiv \texttt{true}$$

□

The definition implies that to prove $\{*\ O\ *\}$ `assert` P $\{*\ Q\ *\}$, it is sufficient to prove that the pre-condition O implies both P and Q. So, the logic wants us to prove that we will not violate P in the first place. In theory this is incomplete. It may still be ok to violate P if you catch the exception elsewhere. However, this is not captured by the above definition. So, as said, the definition above is incomplete. But in return, we can use it without having to deal with its exception effect.

We can exploit `assert` to deal with e.g. normal arrays without having to change our logic. Unlike uPL arrays, normal arrays have finite size. For example, our inference rules for assignment do not apply on `a[i] := 0` if `i` turns out to be a value outside the allowed range of `a`'s indices. However, we can convert the assignment to:

```
assert 0≤i<a.length ; a[i] := 0
```

This can be treated with our uPL logic as is, without any extension.

Interestingly, we can also introduce the dual of `assert`, namely `assume` P. As the name suggests, this assumes that P holds when the execution reaches the statement. If P does not hold, then we do not care anymore. Its semantic can be characterized as follows:

Definition 6.13.2 : WP OF ASSUME

$$\texttt{wp}\ (\texttt{assume}\ P)\ Q\ =\ P \Rightarrow Q$$

□

So, $\{*\ O\ *\}$ `assume` P $\{*\ Q\ *\}$ is valid only if $O \Rightarrow (P \Rightarrow Q)$. Notice that the implication is valid if $O \wedge P$ implies Q. But it is also valid if O implies $\neg P$, in which case it does not matter what Q is.

This statement `assume` is just a theoretical concept. It cannot be implemented, because `assume false` would then be a magic statement that can realize any post-condition. However, it is useful for inserting assumptions in our verification. Imagine we have a program that contains `a[i] := 0` and `a[z] := 0`. For the first one we are not sure if `i` will be within the allowed range, so we want to verify that. This means that we then translate `a[i] := 0` to the `assert`-construct as discussed above.

But suppose for the second one, a[z] := 0, we are sure that z will be in the allowed range. Well, then we can just leave the assignment a[z] := 0 as it is. However, we can also convert it to:

assume 0≤z<a.length ; a[z] := 0

This will now explicitly insert the assumption 0≤z<a.length. If at some point latter you have another statement that exploits this assumption, then having it explicitly in the proof does matter.

6.13.2 Non-deterministic Assignment

A *non-deterministic assignment* is an assignment of this form: $\mathtt{x}, \mathtt{y} \leftarrow P$. This will magically set the variables x and y to some values satisfying P. Since there are typically multiple values that satisfy a given P, this statement is non-deterministic. Its semantic can be characterized as follows:

Definition 6.13.3 : ND Assignment

$$\mathsf{wp}\ (x \leftarrow P)\ Q \ = \ (\forall x :: P \Rightarrow Q)$$

□

Such an assignment is not in general implementable, but can be useful to be used as an equivalent abstraction to replace a concrete statement. Such an abstraction can be useful for verification.

For example, consider a statement S. Its *write frame* is the set of variables it writes. Suppose that this is just $\{x, y\}$, and suppose we know that S satisfies $\{*\ P\ *\}\ S\{*\ Q\ *\}$. Then for the purpose of verification we can just replace the entire occurrence(s) of S in the program by the statement:

$$\mathtt{assert}\ P;\ \mathtt{x}, \mathtt{y} \leftarrow Q$$

6.13.3 Non-deterministic Choice

Let $S \Box T$ be a statement that non-deterministically chooses to either execute S or T. It can be defined as follows:

Definition 6.13.4 : ND Choice

$$\mathsf{wp}\ (S \Box T)\ Q \ = \ (\mathsf{wp}\ S\ Q) \wedge (\mathsf{wp}\ T\ Q)$$

□

This can be used to define non-deterministic if-then-else and non-deterministic loop, as shown below. This also means that we get their inference rules as well.

$$\left.\begin{array}{lll} \mathtt{if} & g_1 & \rightarrow & S_1 \\ & g_2 & \rightarrow & S_2 \end{array}\right\} = \mathtt{assert}\ g_1 \vee g_2\ ;\ ((\mathtt{assume}\ g_1\ ;\ S_1) \Box (\mathtt{assume}\ g_2\ ;\ S_2))$$

$$\left.\begin{array}{lll} \mathtt{while} & g_1 & \rightarrow & S_1 \\ & g_2 & \rightarrow & S_2 \end{array}\right\} = \mathtt{while}\ g_1 \vee g_2\ \mathtt{do}\ ((\mathtt{assume}\ g_1\ ;\ S_1) \Box (\mathtt{assume}\ g_2\ ;\ S_2))$$

6.13.4 goto

Our uPL logic is nicely syntax driven. That is, it constructs a proof by following the syntactical structure of the given program. This relies on the assumption that a program's control flow follows its syntactical structure. For example, in a statement $S_1;\ S_2$ the program will really proceed with S_2 after S_1. A syntax driven logic can be straight forwardly automated. A goto statement as in BASIC allows a jump to an arbitrary control location in a program. If goto is allowed, the above assumption no longer holds and the entire uPL logic collapses. A more powerful logic is needed, e.g. a proof-outline [9] based logic. Unfortunately, such a logic is no longer syntax driven.

6.13.5 Inner block

So far we only introduce local variables at the beginning of a program's body. In general, we can have an inner statement block that declares its own local variables. Here is how you can caluculate the wp of such a block:

Theorem 6.13.5 : WP OF BLOCK

$$\mathsf{wp}\ \{\mathtt{var}\ x;\ S\ \}\ Q \quad = \quad (\forall x' :: (\mathsf{wp}\ S\ (Q[@x/x]))[x'/x])[x/@x]$$

where $@x$ and x' are fresh variables.
□

The first substitution $Q[@x/x]$ replaces x in Q with a fresh variable. This is only temporary. The last substitution $[x/@x]$ will replace $@x$ back with x. Recall that this x does not refer to the locally declared x in the block $\{\mathtt{var}\ x;\ S\ \}$. We (temporarily) replace x with $@x$ to prevent it from being affected by assignments to the local x in S.

So far, we have not said anything about the initial value of a variable when it is declared. In e.g. Java newly declared variables will assume pre-defined default values. To make it simple, we will assume no such mechanism in uPL. So, when declared a variable may be in any state.

Now, $\mathsf{wp}\ S\ (Q[@x/x])$ gives the weakest pre-condition just before S starts. However, since initially the x declared in $\mathtt{var}\ x$ can take any value, the weakest pre-condition just before the block is entered is $(\forall x :: \mathsf{wp}\ S\ (Q[@x/x]))$. To allow us to restore $@x$ back to x we rename this quantification so that it quantifies over x'. This explains the second substitution $[x'/x]$.

If we add a syntax to immediately innitialize a local variable at its declaration, the formula for calculating wp can be simplified:

Theorem 6.13.6 : WP OF BLOCK WITH INNITIALIZATION

$$\mathsf{wp}\ \{\mathtt{var}\ x = e;\ S\ \}\ Q \ = \ (\mathsf{wp}\ (x{:=}e\ ;\ S)\ (Q[@x/x]))[x/@x]$$

where $@x$ is a fresh variable.
□

6.13.6 Expression with side effect

Expression like x++ in C has a side effect. Allowing program call inside an expression may also cause a side effect. The logic of uPL will incorrectly calculate the wp of, for example, y := x++, as it assumes that only y is affected. We can however transform such an expression to an equivalent code that is side effect free. For example, y := x++ is logically equivalent to y := x; x := x + 1. Once translated to a side effect free statement, we can handle it with the ordinary uPL logic. To write such a translation we have to know the precise semantics of expressions. For many real imperative languages, e.g. C, it can be quite tricky —see for example [22].

6.13.7 Exception

Exception is a form of goto jump. However, the jump made by exception is never cyclic. This makes a big difference. A more powerful logic is still needed, but we can still keep it syntax driven, e.g. as in [19, 5].

Let's add this statement: raise throws an exception. To make it simple, let us just one sort of exception, and it does not carry any data as exceptions in e.g. Java do. We also add a try S catch T construct. It executes S; if it throws an exception the control will jump to the handler T.

We extend Hoare triples to look like this:

$$\{*\ P\ *\}\ S\ \{*\ Q,\ R\ *\}$$

Q specifies the expected final state of S if it terminates normally, whereas R specifies it if S terminates by exception. So, Q corresponds to the normal post-condition that we have before. We will have to fix our inference rules; we will show some of them below.

1. A `skip` does not any side effect, nor does it throw any exception. So:

$$\frac{-}{\{*\ Q\ *\}\ \texttt{skip}\ \{*\ Q, R\ *\}}$$

   Notice that any exceptional post-condition R is good, because `skip` will not terminates exceptionally anyway.

2. If an assignment never throws an exception, then its inference rule is simply:

$$\frac{-}{\{*\ Q[e/x]\ *\}\ x := e\ \{*\ Q, R\ *\}}$$

   Like in `skip`, the exceptional post-condition does not matter.

   However, what if the assignment can throw an exception, e.g. `x := x/y` if `y` turns out to be 0? A solution is to first convert this to:

```
if y=0 then throw else x = x/y
```

   This can be handled by our logic.

3. The only effect of `throw` is to throw an exception. So, if R is the required exceptional post-condition, this has to hold as well before the `throw`:

$$\frac{-}{\{*\ R\ *\}\ \texttt{throw}\ \{*\ Q, R\ *\}}$$

4. The rule for `catch` is somewhat complicated:

$$\frac{\begin{array}{c}\{*\ P\ *\}\ S\ \{*\ Q, Q'\ *\}\\ \{*\ Q'\ *\}\ T\ \{*\ Q, R\ *\}\end{array}}{\{*\ P\ *\}\ \texttt{try}\ S\ \texttt{catch}\ T\ \{*\ Q, R\ *\}}$$

   Note that if S terminates normally, then T is not executed. So, S must realize the normal post-condition Q. This is captured by the first premise. However, S may also terminate exceptionally. Suppose when this happens the state satisfies some intermediate predicate Q'. The execution then proceeds to the exception handler T. So, from this pre-condition Q' T will have to realize the specified goal $\{*\ Q, R\ *\}$. This is reflected in by the second premise.

5. And the rule for sequential composition:

$$\frac{\begin{array}{c}\{*\ P\ *\}\ S\ \{*\ Q', R\ *\}\\ \{*\ Q'\ *\}\ T\ \{*\ Q, R\ *\}\end{array}}{\{*\ P\ *\}\ S;T\ \{*\ Q, R\ *\}}$$

   Notice that the rule looks very similar to that of `try` − `catch`. This is because in a way their behavior is also similar. In both, the execution of T is conditional. In `try` S `catch` T the control continues to T if S terminates exceptionally, and else not. In $S;T$ is this the other way around. So, it is not so surprising the their inference rules look the same, except for some inversions, namely the position of the intermediate Q'.

6.13.8 Alias

If aliasing is alowed, an assignment to x will also affects all its aliases. The logic of uPL will again incorrectly calculates the wp of $x := e$ with respect to a post-condition Q, because it assumes that the assignment only affects x. A more powerful way of calculating wp is needed. See for example [17].

New Vocabulary

alias; deterministic; EC; exceptional post-condition; Hoare triples disjunction/conjunction; IC; IL/ILF; invariant; loop reduction; non-deterministic; non-deterministic assignment; non-determinstic choice; post-condition weakening; pre-condition strengthening; proof plan; repby; simultant assignment; state; state: initial, teminal; tail invariant; TC1/TC2; termination metric; weakest pre-condition; wp

6.14 Exercise

1. Let P and Q be some predicates. What do the following specifications say if we interpret them with the partial correctness interpretation? What do they say under total correctness? What kind of statements S satisfy each of them?

   (a) $\{* \, P \, *\} \;\; S \;\; \{* \, \texttt{true} \, *\}$

   (b) $\{* \, P \, *\} \;\; S \;\; \{* \, \texttt{false} \, *\}$

   (c) $\{* \, \texttt{false} \, *\} \;\; S \;\; \{* \, Q \, *\}$,

   Prove your conclusions.

2. A statement S 'always' terminates if it terminates from any state. Express the property "S always terminates" using a Hoare triple.

   Prove that if $S;T$ always terminates, so does S.

   Does it imply that T always terminate?

3. In partial correctness, the characterization of wp we give in Definition 6.2.3 is actually a theorem. It follows from Definition 6.2.1 of wp and Definition 6.1.4 of Hoare triples in partial correctness. Prove this.

4. We want to add a new statement to uPL. If S_2 and S_2 are statements, $S_1 \sqcup S_2$ is a statement that non-deterministically chooses S_1 or S_2 and then executes the choice.

   (a) Define the meaning of this statement in terms of models discussed in Section 6.1 (so this would be a definition that assumes termination).

   (b) Give a formula for calculating its wp.

   (c) Prove that your formula for calculating its wp is correct with respect to your definition. That is, that you can show that it follows from your definition above.

5. Let `even x` and `odd x` mean that `x` is an even respectively odd integer. Prove that the validity of this specification:

   $\{* \, \texttt{odd x} \;\wedge\; \texttt{odd y} \, *\}$

   ```
   y := y+1 ; if x>y then x:=x-y else skip
   ```

   $\{* \, \texttt{odd (x+y)} \, *\}$

6. Let `cntEven a i` $=$ `COUNT` $[\texttt{x} | \texttt{x from a}[0 \ldots \texttt{i}), \texttt{even x}]$. Prove this:

   $\{* \, 0 \leq \texttt{i} \;\wedge\; (\texttt{c} = \texttt{cntEven a i}) \, *\}$

   ```
   if even (a[i]) then c:=c+1 else skip ;
   i:=i+1
   ```

   $\{* \, \texttt{c} = \texttt{cntEven a i} \, *\}$

7. If two programs satisfy exactly the same set of Hoare triple specifications then they will behave the same way and thus implement the same functionality. We can say that they are equivalent (but keep in mind that we do not pay attention to aspects like performance in Hoare triple specifications). Prove that the following two statements are equivalent in the above sense:

(a) `if` $g \wedge h$ `then` S `else` T

(b) `if` g `then` $\{$`if` h `then` S `else` $T\}$ `else` T

8. Give a sketch of a proof showing the validity of each of the specifications below (with total correctness interpretation). Come up with a minimalistic invariant.

(a)
```
{* true *}   while i>0 do i:=i-1   {* true *}
```

(b)
```
{* reds=100 ∧ blues=100 *}
  while reds>0 ∨ blues>0  do {
     if reds>0 then { reds := reds-1  ; blues := blues+2 }
     else blues := blues-1
{* true *}
```

(c)
```
{* x = 10 *}
while 0<x do
  if x=10 then x := x+1 else x := x−2
{* x = −1 *}
```

9. We want to extend uPL with a `for`-loop:

`for` var:=e_1 `to` e_2 `step` d `do` S

This statement initializes var to e_1 and repeatedly executes S while $var < e_2$. At the end of every iteration the value of var is increased by d.

(a) Give an inference rule to prove the correctness of a `for`-loop.
Hint: express the `for`-loop in terms of `while`-loop.

(b) Arguing the termination of a `for`-loop is often easier. Give a variant of your inference rule whose conditions are easier to prove.
Motivate the correctness of your rule.

10. We want to extend the `while`-loop in uPL to allow multiple bodies, which can be chosen non-deterministically:

```
while    g1   do   S1
       | ...
       | gn   do   Sn
```

This loop will iterate if there is at least one guard which is true. If multiple guards are true, one will be chosen non-deterministically, and the corresponding body will be executed. The loop terminates when all its guards are false.

Extend the inference rule for `while` so that it can deal with this non-deterministic version of `while`.

Consider the program below:

```
while   i>j do i:=i-1
      | i>j do j:=j+1
```

If initially `i`$\geq$`j`, the program will terminate with `i` = `j`. Give an invariant and a termination metric that can prove this. Motivate your answer.

11. Consider the specifcation of the program `test` below. Is the specification open or closed? Use the Program to Statement Reduction rule (page 101) to transform it to an equivalent specification in terms of its body.

```
{* true *}

test (b:[]bool, i:int) : bool
{ var r:bool ;
   r:=true ;
   while r ∧ i>0 do { i:=i-1; r:=b[i] } ;
   return ¬r
}

{* return = (∃j : 0≤j<i : ¬ b[j]) *}
```

12. Prove the following specification:

```
{* 0≤n *}

i:=0 ; c:=0 ;
while i < n do
  { if x[i]=0 then c:=c+1 else skip ;
    i:=i+1 }

{* c = COUNT [i | i from [0...n), x[i] = 0] *}
```

13. Section 6.5 discusses the correctness of a program to compute Fibonacci numbers. It discussed an incremental approach to construct an invariant. Section 6.6 sketches a variant of this approach. The approach is less intelligent, but on the other hand it can be automated. Redo the Fibonacci example, this time use the latter strategy to construct your invariant.

14. We want a program SUMPOWER with the following specification:

```
{* n ≥ 0 *}

SUMPOWER(x,n:int) : int

{* return = SUM [x^i | i from [0...n)] *}
```

Write SUMPOWER. You can avoid using the exponentiation operator, and thus getting a much faster program. Prove the correctness of your program.

15. Prove the following specification:

```
{* n > 1 *}

i:=0 ; s:=0 ;
while i < n do { s:=s+i; i:=i+1 }

{* s = n * (n − 1) div 2 *}
```

The program simply sums all integers from 0 up to $n - 1$. So, if the specification above is valid in a sense it also proves that $\mathtt{SUM}\,[0 \ldots n) = n * (n - 1)\ \mathtt{div}\ 2$.

16. Below is an algorithm to efficiently multiply two integers. It is a very old algorithm, written by an Egyptian A'h-mose, back in 1700 BC! It is the same algorithm is used in CPU. The $*2$ and `div` 2 operations used in the algorithm can be done efficiently by, essentially, shifting the bit strings representing the target integer. Specify the algorithm, and prove its correctness. The version below only works on non-negative `x`.

```
MUL (x,y:int) : int
{ var z ;
  z:=0 ;
  while x ≠ 0 do
       if odd x then { z:=z+y ; x:=x-1 }
                else { y:=y*2 ; x := x div 2 } ;
  return z
}
```

17. Let the record type `Item` is defined by:

$$\texttt{record Item} = \{\texttt{key} : \texttt{int},\ \texttt{name} : \texttt{string}\}$$

Calculate the weakest pre-condition of the statements below with respect to the given post-conditions; `r` and `s` are of the `Item` type.

(a) `j:=a[i]; a[a[i]]:=a[i] ; a[i]=0` $\{* \texttt{a[j]} = 0 *\}$

(b) `a[j]:=0; r.key:=a[i]` $\{* (\texttt{r.key} = 0) \ \vee \ (\texttt{r.item} = 0) *\}$

(c) `r.key:=0; s.item:=0` $\{* \texttt{r} = \texttt{s} *\}$

18. The program below swaps the contents of `a[i]` and `a[j]`. Specify it, and prove its correctness.

```
SWAP (a:[]int, i,j:int) : ()
{ var temp:int ;
  temp := a[i]  ;
  a[i] := a[j]  ;
  a[j] := temp }
```

19. Write a program to initialize an integer array by setting all elements in the domain `0≤i<n` to 0. Give a sufficient invariant; motivate your choice and prove its invariance.

20. Give a translation scheme so that we can handle assignments like:

$$a[e_1].fname := e_2 \quad \text{or} \quad r.fname[e_1] := e_2$$

Your translation scheme should translate these assignments to ordinary assignments that target variables. Can your scheme handle an assignment targeting an arbitrarily nested arrays and records?

21. The program below sorts an array. It is not the fastest algorithm. Its worst case run time is linear to $\texttt{n}^2$, where `n` is the assumed size of the array, compared to $\texttt{n} * \texttt{log n}$ of some other algoritms. However, its algorithm is very simple.

The outer loop gradually increases the region where the array is already sorted (elements with indices less than `i`). The inner loop keep swapping elements in the unsorted region (elements with indices at least `i`) in a certain direction, so that in the end a smallest element of the unsorted region can be found at the index `i`; then the counter `i` is advanced by 1. The algorithm is also known as *bubble sort* (from the way the inner loop 'bubbles' smaller elements to the 'top' of the array).

```
BUBBLESORT (a:[]int, n:int) : ()
{ var i,j:int ;
  i:=0 ;
  while i<n do
    { j:=n-1 ;
      while i<j do
         { j:=j-1
           if a[j+1]<a[j]
              then { tmp := a[j]    ;
                     a[j] := a[j+1] ;
                     a[j+1] := tmp  }
              else skip
         } ;
      i:=i+1 } }
```

Write a specification for the program. Give invariants that would prove its correctness; motivate your choice (are they sufficient? are they really invariant?)

22. Here is another sorting program. It takes an array `a` and implements an algoritm called *counting sort*. It is simple but very fast: linear to `a`'s size! However, it requires that the range of `a`'s elements to be *known* and that this range is *narrow*. An array with the latter property typically contains many duplicates.

    For simplicity, we will assume `a`'s elements to range from 0 to `max`. The program is also given two help arrays. The array `count` has been prepared such that `count`[`x`] is equal to the number of elements of `a` whose values are less than `x`. This array can be built in linear time.

    The array `b` contains the sorted elements. We still have to copy it back to `a`, but we leave this step out from the program. It can be done in linear time too.

```
COUNTSORT (a,b,count:[]int, n,max:int) : ()
{ var i,x:int ;
  i:=0 ;
  while i<n do
    { x:=a[i] ;
      b[count[x]] := x ;
      count[x]:=count[x]+1 ;
      i:=i+1 }
```

Write a specification for the program; to make it simple it is sufficient if your specification implies that upon termination `b` will be sorted. Give an invariant that would prove the program's correctness and motivate your choice.

CHAPTER 7

Program Call

A uPL program is comparable to what in other languages are called procedure or function. In particular, a uPL program can be called from another program. A possible way for our logic to handle a program call like $P(e)$ is to replace the call with a proper instantiation of P's body. Subsequently, we can handle it with the logic over ordinary statements from Chapter 6. This does not work if P is recursive. Moreover, this assumes we have P's source code. This is not always the case. Even if we do have the source code, if you recall the discussion about black box in Section 5.2, we may prefer not to rely on it. We will take the black box approach; however, in general it is also more complicated. Let us first show you the case where it is simple. Consider a program SUC with this specification:

$$\{*\ \texttt{true}\ *\}\ \texttt{SUC}(\texttt{x} : \texttt{int})\ \{*\ \texttt{return} = \texttt{x} + 1\ *\}$$

The program has no side effect; moreover, the specification *fully* specifies what the return value is. Such a specification is said to imply a 'functional behavior'. It allows a nice black box treatment. For example, a call like:

$$\texttt{x} := \texttt{SUC}(\texttt{x})$$

logically can be treated as:

$$\texttt{x} := \texttt{x} + 1$$

which is just a simple assignment; we already know how to handle it. Note that we do not need to know the code of SUC except for its header.

Things are more difficult if we are given a specification that is only partial, such as:

$$\{*\ \texttt{true}\ *\}\ \texttt{MORE}(\texttt{x} : \texttt{int})\ \{*\ \texttt{return} > \texttt{x}\ *\}$$

A call like $\mathtt{x} := \mathtt{MORE}(\mathtt{x})$ cannot be treated as an assignment $\mathtt{x}{:=}e$, because now we cannot infer what e is from the specification.

Before we proceed, let us first list down some restrictions imposed in order to simplify the logic of program call. Most of them have been discussed before. Keep in mind that these restrictions may be too stringent in practice (hence you would need to find a stronger logic).

7.1 Restrictions

Syntax of program call

In uPL a program can only be called as a statement, and the syntax is:

target `:=` *program-name* `(`*parameters*`)`
or
program-name `(`*parameters*`)`

The first will copy the return value to the variable targeted by the assignment; the second will simply ignore the return value. In particular, uPL does not allow a program to be called from an expression. So, if P is a program, this is not allowed:

$$\mathtt{x} := \mathtt{P}(\mathtt{y}) + \mathtt{x} * \mathtt{y}$$

If we allow this, the logic will have to take $\mathtt{P}(\mathtt{y})$'s side effect, and the exact order in which $\mathtt{P}(\mathtt{y})$ within the expression $\mathtt{P}(\mathtt{y}) + \mathtt{x} * \mathtt{y}$ is executed (is it before or after $\mathtt{x} * \mathtt{y}$?) into account.

Program's parameters

A parameter can be passed either by value or by copy-restore to a program. A uPL program cannot access a global variable, unless it is passed as a parameter to the program. See also Section 5.1. Parameters named in the definition of a program are usually called *formal parameters*. For example, in:

```
SUC(x:int) {return x+1}
```

x is a formal parameter of SUC. The parameters which are actually supplied to a program when it is called are called *actual parameters*. For example, in the call SUC(x+1), x+1 is an actual parameter of SUC.

If a program has multiple copy-restore parameters, they are restored in the same order as they are listed as the program's arguments.

Alias is excluded

Two variables with distinct names, e.g. x and y are *aliases* of each other if they refer to the same location in the memory. So, changing the content of one will implicitly affect the other. In uPL we cannot have aliases.

Aliasing complicates reasoning, and causes further complication in treating program call. To show you, consider the following program, where x and y are passed by reference. Passing parameters by reference is not possible in uPL, but suppose for a moment that it is possible.

$$\mathtt{TF}(\mathtt{REF}\ \mathtt{x}, \mathtt{y} : \mathtt{bool})\ \{\mathtt{x} := \mathtt{true};\ \mathtt{y} := \mathtt{false}\} \tag{7.1}$$

It may seem reasonable to expect the program to satisfy this specification:

$$\{*\ \mathtt{true}\ *\}\ \mathtt{TF}(\mathtt{x}, \mathtt{y})\ \{*\ \mathtt{x} \wedge \neg \mathtt{y}\ *\} \tag{7.2}$$

It is not valid however. It is satisfied by most calls, but not if we pass x twice as in $\mathtt{TF}(\mathtt{x}, \mathtt{x})$. This call would pass the same address in both parameters to TF. In other words, aliases are passed

to TF. The post-condition would then imply that the final value of x is equal to both true and false, which is a contradiction.

Specification (7.2) is valid if we e.g. strengthen the pre-condition by requiring that x and y are not aliases. But first the programmer will have to be aware of the more subtle situation arising from passing aliases. In practice this is quite easy to overlook; afterall x and y are two different names, thus we may be misled into thinking that they are really 'different'.

Anyway, as said in uPL this kind of subtleties will be kept away, since do not have pass-by-reference parameters nor aliases.

7.2 Formal Treatment of Program Call

A call replaces formal parameters with actual paremeters. You may expect that specifications can be also be inferred in the same way. For example, consider:

$$\{*\ \mathtt{true}\ *\}\ \mathtt{X} := \mathtt{x};\ \mathtt{INC}(\mathtt{OUT}\ \mathtt{x} : \mathtt{int})\ \{*\ \mathtt{x} = \mathtt{X} + 1\ *\} \tag{7.3}$$

From the above we can infer, simply by replacing x with a:

$$\{*\ \mathtt{true}\ *\}\ \mathtt{A} := \mathtt{a};\ \mathtt{INC}(\mathtt{a})\ \{*\ \mathtt{a} = \mathtt{A} + 1\ *\}$$

which at least give us some information to reason about the call INC(a). Another way to look at it is to see a specification like (7.3) as if it is implicitly ∀-quantified over all its formal parameters and auxilliary variables. Hence, we can infer a new specification simply by renaming the formal parameters and auxilliary variables. This is formally captured by the following rule.

Rule 7.2.1 : Renaming
Let $\overline{x}$, $\overline{y}$, $\overline{X}$ and $\overline{Y}$ be lists of distinct variables. The latter two are auxilliary variables.

$$\frac{\{*\ P\ *\}\quad \overline{X}{:=}\overline{x}\ ;\ Pr(\overline{x})\quad \{*\ Q\ *\}}{\{*\ P[\overline{y}/\overline{x}]\ *\}\quad \overline{Y}{:=}\overline{y}\ ;\ Pr(\overline{y})\quad \{*\ Q[\overline{y}/\overline{x}, \overline{Y}/\overline{X}]\ *\}}$$

□

Renaming (Rule 7.2.1) seems to be the natural way to handle program call. However, on its own it is not sufficient. Consider the call INC(a[i]). You may now expect, by replacing the formal parameter x in the INC's specification (7.3) with the actual parameter a[i], to infer:

$$\{*\ \mathtt{true}\ *\}\ \mathtt{X} := \mathtt{a[i]};\ \mathtt{INC}(\mathtt{a[i]})\ \{*\ \mathtt{a[i]} = \mathtt{X} + 1\ *\}$$

However, Rule 7.2.1 does not actually allow us to infer it. The Rule specifically requires that you can only rename formal parameters with variables. Instantiating the parameter x with a complex expression like a[i] is not covered by the Rule. One may propose to generalize the rule. For the above example it is not a problem. However, in general the rule will be incorrect. To show you the problem, consider the following variation of the program TF you saw before:

$$\mathtt{ZeroOne}(\mathtt{OUT}\ \mathtt{x}, \mathtt{y} : \mathtt{int})\ \{\ \mathtt{x} := 0;\ \mathtt{y} := 1;\} \tag{7.4}$$

The program satisfies this specification:

$$\{*\ \mathtt{true}\ *\}\ \ \mathtt{ZeroOne}(\mathtt{OUT}\ \mathtt{x}, \mathtt{y} : \mathtt{int})\ \ \{*\ (\mathtt{x} = 0) \wedge (\mathtt{y} = 1)\ *\} \tag{7.5}$$

If Rule 7.2.1 allows formal parameters to be instantiated with arbitrary expressions, it would allow this to be inferred:

$$\{*\ \mathtt{true}\ *\}\ \ \mathtt{ZeroOne}(\mathtt{a[i]}, \mathtt{i})\ \ \{*\ (\mathtt{a[i]} = 0) \wedge (\mathtt{i} = 1)\ *\} \tag{7.6}$$

However, this is not valid! Suppose i is initially 0. As the call exits, the used calling convention of uPL would first restore a[0], hence setting it to 0, and then restores i, which would set it to 1. There is nothing we can infer about the value of a[1], and yet the specification above claims that a[1] = 0.

To get around the above problem, we will first tranform a program call to an equivalent code where only variables are passed as actual paremeters. Subsequently, we can safely use Rule 7.2.1. The transformation is only done for our reasoning purpose. In particular, a compiler producing an executable code does not do it, because the transformed code is longer. Subsection 7.2.1 will show you the transformation. Subsection 7.2.2 will show you how to deal with the resulting code.

7.2.1 Transforming Program Call

Consider a program with this header:

$$Pr(x,\ \texttt{OUT}\ y)$$

Logically we can transform a call like $Pr(e_1, e_2)$ to the following, which has an equivalent effect:

$$@\texttt{x}, @\texttt{y} := e_1, e_2;\ Pr(@\texttt{x}, @\texttt{y});\ e_2 := @\texttt{y}$$

where @x and @y are fresh variables. The assignments before the call captures the passing of the actual parameters to the formal parameters. The assignment(s) after the call captures the copy-back action to the copy-restore parameters. Notice that after the transformation Pr is called in the form that can be handled by the Renaming Rule.

Be careful: if Pr has multiple copy-restore parameters, they are not restored simultaneously. Consistent to the agreement we made earlier (Section 7.1), these parameters are restored sequentially, and in the same order as they are listed as the program's arguments. The next example shows this.

Consider again the ZeroOne example; in particular, the specification below:

$$\{*\ \texttt{true}\ *\}\ \ \texttt{i}_0 := \texttt{i};\ \texttt{ZeroOne}(\texttt{a}[\texttt{i}], \texttt{i})\ \ \{*\ (\texttt{a}[\texttt{i}_0] = 0) \wedge (\texttt{i} = 1)\ *\}$$

Applying the above transformation scheme we obtain:

$$\{*\ \texttt{true}\ *\}$$

$$\texttt{i}_0 := \texttt{i};$$
$$@\texttt{x}, @\texttt{y} := \texttt{a}[\texttt{i}], \texttt{i};\ \texttt{ZeroOne}(@\texttt{x}, @\texttt{y});\ \texttt{a}[\texttt{i}] := @\texttt{x};\ \texttt{i} := @\texttt{y}$$

$$\{*\ (\texttt{a}[\texttt{i}_0] = 0) \wedge (\texttt{i} = 1)\ *\}$$

where @x and @y are fresh variables. Applying the Renaming Rule allows us to infer that $(@\texttt{x} = 0) \wedge (@\texttt{y} = 1)$ holds after the call ZeroOne(@x, @y). From this we will later be able to prove the specified post-condition, though we will need an additional formal mechanism to do so —we will return to this later.

Recall the example with the program TF (7.1). Below is a call which is similar to the problematic call TF(x, x): it would pass aliases if the parameters are passed by reference:

$$\texttt{ZeroOne}(\texttt{i}, \texttt{i})$$

However, it will have this meaning in uPL:

$$@\texttt{x}, @\texttt{y} := \texttt{i}, \texttt{i};\ \texttt{ZeroOne}(@\texttt{x}, @\texttt{y});\ \texttt{i} := @\texttt{x};\ \texttt{i} := @\texttt{y}$$

It will not lead to a contradiction. In fact, we will be able to infer that i = 1 holds after the call.

A uPL program also returns a value. We can handle return value by treating it as an implicit pass-by-reference parameter. For example, consider:

```
SUC(x:int) : int { return x+1 }
```

The program is translated to the equivalent one below:

```
SUC'(x:int, OUT return:int) {return := x+1}
```

Subsequently a call like y:=SUC(x) is translated to $\mathtt{SUC}'(\mathtt{x},\mathtt{y})$. With this translation scheme our logic will requires no special treatment to handle the return value. Having said this, in the subsequent discussion we will focus on programs without return value.

Note by the way that a `return` statement can only appear as the last statement of a uPL program.

7.2.2 Black Box Reduction

The Renaming Rule, even with the transformation scheme described in the previous subsection, only allows us to, essentially, instantiate a given black box specification. However, this is usually still not sufficient. To illustrate the problem, consider the (black box) specification below of a program INCR:

$$\{*\ \mathtt{x} \geq 0\ *\}\ \ \mathtt{X} := \mathtt{x};\ \mathtt{INCR}(\mathtt{OUT}\ \mathtt{x} : \mathtt{int})\ \ \{*\ \mathtt{x} > \mathtt{X}\ *\} \tag{7.7}$$

X is an auxiliary variable. The specification says that the program increases the value of x if it is initially non-negative. It does not specify how much x is increased.

Supposed we have another program that calls INCR and requires the following post-condition after the call:

$$\mathtt{INCR}(\mathtt{x})\ \{*\ \mathtt{x} > 0 \wedge \mathtt{y} > 0\ *\} \tag{7.8}$$

What would be a reasonable pre-condition to establish the post-condition?

The Renaming Rule cannot help us, at least not directly: there is no way we can obtain the post-condition in (7.8) by applying some renaming on (7.7).

We have to find another way. From (7.7) we conclude, assuming the pre-condition of (7.7) holds, that x>X will hold after the call INCR(x). Since this is not quite the required post-condition (x>0 ∧ y>0), the next best thing we can do is to require that x>X implies this required post-conditon. So, this has to hold in the state after the call:

$$\mathtt{x}>\mathtt{X}\ \Rightarrow\ \mathtt{x}>0 \wedge \mathtt{y}>0 \tag{7.9}$$

Given only (7.7) as our knowledge about INCR, the *best* fact that we can infer about the value of x in the state after the call is that it satisfies x>X; beyond this we do not know the exact value of x. With this observation we can just as well, without loss of information, put a ∀-quantification over x in the post-condition (7.8). That is, we replace (7.8) with:

$$(\forall \mathtt{x} :: \mathtt{x}>\mathtt{X}\ \Rightarrow\ \mathtt{x}>0 \wedge \mathtt{y}>0) \tag{7.10}$$

The call INCR(x) only affects the variable x. But x no longer occurs free in the post-condition above! Hence, INCR(x) cannot change the meaning of (7.10). So, for (7.10) to hold after the call, the same has to hold *before* the call. This is nice, because effectively we have now turned the post-condition requirement (7.9) into a pre-condition!

As further simplification, we replace the ∀-quantification over x in (7.10), which is now a pre-condition, with the use of a fresh variable, e.g. $\mathtt{x}'$, obtaining:

$$\mathtt{x}'>\mathtt{X}\ \Rightarrow\ \mathtt{x}'>0 \wedge \mathtt{y}>0 \tag{7.11}$$

With respect to the actual program variables, (7.11) and (7.10) are equivalent, because as a fresh variable we have no constraint on the value of $\mathtt{x}'$; thus, it is implicitly universally quantified.

In addition to (7.11), the call to INCR has to satisfy the pre-condition of (7.7); else we cannot use (7.7). So, the following pre-condition is needed:

$$\mathtt{x}\geq 0 \ \wedge \ (\mathtt{x}'>\mathtt{X} \ \Rightarrow \ \mathtt{x}'>0 \wedge \mathtt{y}>0) \tag{7.12}$$

The X above comes from the specification of INCR (7.7). Recall that in the specification it referes to the value of x before the call. The above condition (7.12) is interpreted on the state before the call, where $\mathtt{X} = \mathtt{x}$. We can therefore replace X with x, obtaining:

$$\mathtt{x}\geq 0 \wedge (\mathtt{x}'>\mathtt{x} \ \Rightarrow \ \mathtt{x}'>0 \wedge \mathtt{y}>0) \tag{7.13}$$

This is the final pre-condition. We conclude that INCR satisfies therefore the following specification:

$$\{*\ \mathtt{x}\geq 0 \ \wedge \ (\mathtt{x}'>\mathtt{x} \ \Rightarrow \ \mathtt{x}'>0 \wedge \mathtt{y}>0)\ *\} \tag{7.14}$$

$$\mathtt{INCR}(\mathtt{x})$$

$$\{*\ \mathtt{x}>0 \wedge \mathtt{y}>0\ *\}$$

By the way it is constructed, the pre-condition above is also the weakest one we can infer from INCR's black box specification in (7.7).

Let Pr be a uPL program. Let $\overline{x}$, $\overline{X}$, and $\overline{x'}$ be compatible lists of variables; the variables in $\overline{x'}$ are fresh and all distinct. Formally, we can capture the above line of reasoning that we did for INCR in a single inference rule:

$$\frac{\{*\ P\ *\} \quad \overline{X} := \overline{x};\ Pr(\overline{x}) \quad \{*\ Q\ *\}}{\{*\ P \wedge Q'\ *\} \quad Pr(\overline{x}) \quad \{*\ R\ *\}} \tag{7.15}$$

where $Q' \ = \ (Q \Rightarrow R)[\overline{x'}/\overline{x}][\overline{x}/\overline{X}]$.

Essentially Q' requires that the post-condition from the black box specification has to imply the actual post-condition R. The two successive substitutions in Q' adapt this after-call requirement to a pre-condition. If you now try to apply the rule to the specification of INCR (7.7) and the post-condition specified in (7.8), you will obtain (7.14).

The rule still has one defficiency however. If x is a pass-by-value parameter or a read-only parameter of Pr, we have agreed that occurrences of x in the post-condition of Pr's specification refer actually to the value of x when it is passed to Pr. In particular, it does not refer to x's final value just before Pr returns —see also the discussion in page 53. The rule above does not yet take this fact into account. As an example, consider the following specification:

$$\{*\ \mathtt{true}\ *\}$$

$$\mathtt{test}(\mathtt{READ}\ \mathtt{a} : \mathtt{int}[],\ \mathtt{x} : \mathtt{int}, \mathtt{OUT}\ \mathtt{return} : \mathtt{bool}) \tag{7.16}$$

$$\{*\ \mathtt{return} = (\mathtt{x}{=}\mathtt{a}[0])\ *\}$$

The x and a in the post-condition refer to the values of x and a when they are passed to test. So, we expect to be able to infer this from the above specification:

$$\{*\ \mathtt{x} = \mathtt{a}[0]\ *\} \ \ \mathtt{test}(\mathtt{a}, \mathtt{x}, \mathtt{b}) \ \ \{*\ \mathtt{b}\ *\} \tag{7.17}$$

Let us now apply the rule (7.15) on the specification of test and, as the R, the required post-condition above (7.17). We obtain:

$$\{*\ (\mathtt{b}' = (\mathtt{x}' = \mathtt{a}'[0])) \ \Rightarrow \ \mathtt{b}')\ *\}$$

$$\mathtt{test}(\mathtt{a}, \mathtt{x}, \mathtt{b}) \tag{7.18}$$

$$\{*\ \mathtt{b} = (\mathtt{a}[0] = 1)\ *\}$$

where $\mathtt{x}'$ is a fresh variable. The pre-condition above can be simplified to $\mathtt{x}' = \mathtt{a}'[0]$. However, there is no way we can prove this from $\mathtt{x} = \mathtt{a}[0]$. Hence, we fail to infer (7.17).

In applying rule (7.15) to the specification of `test`, we should not actually replace `x` and `a` in the post condition with fresh variables, since they refer to their respective values before the call. The rule below includes this revision. In the literature the rule is also called *Adaptation Rule* [12].

Rule 7.2.2 : Black Box Reduction
Let Pr be a uPL program. Let $\overline{x}$, $\overline{X}$, and $\overline{x'}$ be compatible lists of variables; variables in $\overline{x'}$ are fresh and distinct.

$$\frac{\{*\ P\ *\} \quad \overline{X} := \overline{x};\ \mathtt{Pr}(\overline{x}) \quad \{*\ Q\ *\}}{\{*\ P \wedge Q'\ *\} \quad \mathtt{Pr}(\overline{x}) \quad \{*\ R\ *\}}$$

where

$$Q' \quad = \quad (Q \Rightarrow R)\ [\overline{x'}//\overline{x}]\ [\overline{x}/\overline{X}]$$

and $\overline{x'}//\overline{x}$ is a substitution that replaces x_i with x'_i only if x_i is a pass-by-copy-restore parameter of `Pr`.
□

If we now use this Black Box Reduction Rule on the specification of `test` (7.16) and as the R the post-condition in (7.17), we obtain:

$$\begin{array}{l} \{*\ (\mathtt{b}' = (\mathtt{x} = \mathtt{a}[0]))\ \Rightarrow\ \mathtt{b}')\ *\} \\ \\ \mathtt{test}(\mathtt{a}, \mathtt{x}, \mathtt{b}) \\ \\ \{*\ \mathtt{b} = (\mathtt{a}[0] = 1)\ *\} \end{array} \tag{7.19}$$

The pre-condition can be simplified to $\mathtt{x} = \mathtt{a}[0]$ (was $\mathtt{x}' = \mathtt{a}'[0]$), which is exactly the pre-condition given in (7.17).

Notice that Black Box Reduction can only handle calls that pass variables as actual parameters. In the examples so far this is not a problem. In a more general case, we will have to combine its use with the transformation discussed in Subsection 7.2.1. We will show this in the next example.

Example

Consider this specification:

$$\begin{array}{c} \{*\ \mathtt{true}\ *\} \\ \\ \mathtt{X}, \mathtt{Y} := \mathtt{x}, \mathtt{y};\ \mathtt{UPPER}(\mathtt{x}, \mathtt{y} : \mathtt{int},\ \mathtt{OUT}\ \mathtt{return} : \mathtt{int}) \\ \\ \{*\ \mathtt{return} \geq \mathtt{x}\ \mathtt{max}\ \mathtt{y}\ *\} \end{array} \tag{7.20}$$

So, `UPPER` returns some integer which is greater or equal to the maximum of `x` and `y`. We want to prove that `UPPER` also satisfies the following specification:

$$\{*\ \mathtt{a}[\mathtt{i}] > 0\ *\}\ \mathtt{UPPER}(\mathtt{a}[\mathtt{i}], 0, \mathtt{i})\ \{*\ \mathtt{i} > 0\ *\} \tag{7.21}$$

Since we pass a complex expression ($\mathtt{a}[\mathtt{i}]$) in the call above, we cannot directly use the Black Box Reduction Rule. So, as we did in Subsection 7.2.1, we first transform the call the an equivalent one below. All variables whose name begin with `@` are assumed to be fresh.

$$\begin{array}{l} \{*\ \mathtt{a}[\mathtt{i}] > 0\ *\} \\ @\mathtt{x}, @\mathtt{y}, @\mathtt{r} := \mathtt{a}[\mathtt{i}], 0, \mathtt{i} \\ \mathtt{UPPER}(@\mathtt{x}, @\mathtt{y}, @\mathtt{r}) \\ \mathtt{i} := @\mathtt{r} \\ \{*\ \mathtt{i} > 0\ *\} \end{array} \tag{7.22}$$

By calculating wp we can equivalently prove this instead:

$$\begin{array}{l} \{*\ \mathtt{a[i]} > 0\ *\} \\ \mathtt{@x}, \mathtt{@y}, \mathtt{@r} := \mathtt{a[i]}, 0, \mathtt{i} \\ \mathtt{UPPER}(\mathtt{@x}, \mathtt{@y}, \mathtt{@r}) \\ \{*\ \mathtt{@r} > 0\ *\} \end{array} \tag{7.23}$$

The next step is to use the Black Box Reduction Rule. However, the actual paremeters passed above does not match the specification (7.20), which the Black Box Reduction Rule expects. We first has to adapt (7.20) to match the actual parameters. This is done by applying the Renaming Rule (Rule 7.2.1); we obtain:

$$\begin{array}{c} \{*\ \mathtt{true}\ *\} \\ \mathtt{X}, \mathtt{Y} := \mathtt{@x}, \mathtt{@y};\ \mathtt{UPPER}(\mathtt{@x}, \mathtt{@y},\ \mathtt{OUT}\ \mathtt{@r}) \\ \{*\ \mathtt{@r} \geq \mathtt{@x}\ \mathtt{max}\ \mathtt{@y}\ *\} \end{array} \tag{7.24}$$

Now we apply Black Box Reduction on the specification above and as the R the post-condition in (7.23). We obtain this as the pre-condition:

$$\mathtt{r}' \geq \mathtt{@x}\ \mathtt{max}\ \mathtt{@y} \ \Rightarrow\ \mathtt{r}'{>}0$$

where $\mathtt{r}'$ is a fresh variable. Notice that @r has been replaced by $\mathtt{r}'$, but @x and @y are left unchanged (can you explain why?). After the Black Box Reduction (7.23) can be reduced to just proving:

$$\begin{array}{l} \{*\ \mathtt{a[i]} > 0\ *\} \\ \mathtt{@x}, \mathtt{@y}, \mathtt{@r} := \mathtt{a[i]}, 0, \mathtt{i} \\ \{*\ \mathtt{r}' \geq \mathtt{@x}\ \mathtt{max}\ \mathtt{@y}\ \Rightarrow\ \mathtt{r}'{>}0\ *\} \end{array} \tag{7.25}$$

After calculating wp it comes down to proving this implication:

$$\mathtt{a[i]} > 0 \quad \Rightarrow \quad (\mathtt{r}' \geq \mathtt{a[i]}\ \mathtt{max}\ \mathtt{i}\ \Rightarrow\ \mathtt{r}'{>}0) \tag{7.26}$$

which is quite trivial to prove.

7.3 Functional Black Boxes

An imperative program that does not have a side effect is said to be *functional*, as it can be thought to implement a mathematical function. Functions are nice because their behavior are fully determined by the relation between the inputs and their return values, which make reasoning about them simpler.

For simplicity, we will further require that a functional uPL program takes no writable copy-restrore parameter. The post-condition in its specification must fully specify what the return value is. So, the post-condition must be of the form:

$$\{*\ P\ *\}\ Pr(\overline{x})\ \{*\ \mathtt{return} = e\ *\}$$

for some expression e. Notice that any occurence of a variable x from $\overline{x}$ in e refers thus to x's initial value. An example of such a specification is the specification of SUC:

$$\{*\ \mathtt{true}\ *\}\ \ \mathtt{SUC(x)}\ \ \{*\ \mathtt{return} = \mathtt{x} + 1\ *\}$$

The Black Box Reduction Rule can be specialized to handle functional programs:

Rule 7.3.1 : Functional Box Reduction I
Let Pr be a functional uPL program and $\overline{x}$ be a list of variables.

$$\frac{\{*\ P\ *\}\ \mathtt{Pr}(\overline{x})\ \{*\ \mathtt{return} = e\ *\}}{\{*\ P \wedge R[e/r]\ *\}\ r{:=}Pr(\overline{x})\ \{*\ R\ *\}}$$

□

Proof: for simplicity, assume Pr just have one parameter x. Treat Pr as if it has $Pr(x,\ \mathtt{OUT}\ return)$ as header, and treat the call $r{:=}Pr(x)$ as $Pr(x, r)$. Applying the Black Box Reduction we obtain $P \wedge ((r' = e) \Rightarrow R[r'/r])$ as a pre-condition. The second conjunct is equivalent to $R[e/r]$.
□

Since $R[e/r]$ is also the wp of $r{:=}e$ with respect to the post condition R, the Rule above essentially allows us to treat a call to Pr as an assignment. We do have to strengthen the resulting wp with P, since P specifies when the call is allowed. We can also prove the following corollary, which can handle arbitrary actual parameters (the above version of Functional Box Reduction require all actual parameters to be variables):

Rule 7.3.2 : Functional Box Reduction II
Let Pr be a functional uPL program, $\overline{x}$ be a list of variables, and $\overline{d}$ be a list of expressions.

$$\frac{\{*\ P\ *\}\ \mathtt{Pr}(\overline{x})\ \{*\ \mathtt{return} = e\ *\}}{\{*\ P[\overline{d}/\overline{x}] \wedge R[e'/r]\ *\}\ r{:=}Pr(\overline{d})\ \{*\ R\ *\}}$$

where $e' = e[\overline{d}/\overline{x}]$.
□

The proof is left out for you as an exercise.

Example

Recall the program `test`:

$$\{*\ \mathtt{true}\ *\}$$

$$\mathtt{test}(\mathtt{READ\ a} : \mathtt{int}[],\ \mathtt{x} : \mathtt{int}, \mathtt{OUT\ return} : \mathtt{bool}) \tag{7.27}$$

$$\{*\ \mathtt{return} = (\mathtt{x{=}a[0]})\ *\}$$

The program is functional. Moreover the post-condition is of the form $\mathtt{return} = e$. Earlier we have inferred by directly using Black Box Reduction that the call $\mathtt{test(a, x, b)}$, which we can also write as $\mathtt{b} := \mathtt{test(a, x)}$, satisfies:

$$\{*\ \mathtt{x} = \mathtt{a[0]}\ *\}\ \ \mathtt{b} := \mathtt{test(a, x)}\ \ \{*\ \mathtt{b}\ *\} \tag{7.28}$$

Using plain Black Box Reduction we first arrive in the specification (7.19) with a complicated looking pre-condition. Using the Functional Box Reduction we can infer (7.28) directly from (7.27).

7.4 Recursion

The logic of uPL so far cannot yet handle recursive programs. Consider the following simple example:

```
test(OUT n:int) : ()

    { if n=0 then skip else { n:=n-1; test(n) }}
```

Suppose we want to show that it satisfies this:

$\{*\ \mathtt{n}\geq 0\ *\}$ `test(n)` $\{*\ \mathtt{n} = 0\ *\}$

Since a recursion is a form of iteration, we can try to proceed along the same line used to handle the `while`-loop. To show that the recursion terminates we try to come up with an integer expression which is decreased by each recursion; `n` will do in the above case. This expression has to be bounded below by, for example, zero. This can be imposed by the pre-condition. That is, we require that the pre-condition implies $\mathtt{n}\geq 0$. In the above case this is trivially satisfied.

Next, we show that if `test(n)` is correct, then so is `test(n')`, where $\mathtt{n}' = \mathtt{n} + 1$. Suppose we can show that for the 'base case', namely $\mathtt{n} = 0$, `test` is correct. It follows that for $\mathtt{n} = 1$, `test` is correct. Then it follows that it is also correct for $\mathtt{n} = 2$, and so on. Hence, `test` is correct for all non-negative `n`. Formally, this is captured by the inference rule below. It is slightly more general, as it allows the termination metric `n` to decrease by any positive unit per recursion.

Rule 7.4.1 : REC Reduction
Let $\overline{x}$ be Pr's formal parameters. Let n be an integer expression.

BC1 : $P \Rightarrow n \geq 0$

BC2 : $\{*\ P \wedge (n = 0)\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\{*\ Q\ *\}$

RC : $\{*\ P \wedge n < K\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\ \ \ \{*\ Q\ *\}$
implies
$\{*\ P \wedge (n = K) \wedge K > 0\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\{*\ Q\ *\}$

$\{*\ P\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\ \ \{*\ Q\ *\}$

□

We leave the proof of `test` to you as an exercise.

The rule above can be specialized for functional uPL programs that recurse down an integer parameter n by 1 at a time. The specialized rule is shown below:

Rule 7.4.2 : REC Reduction II
Let Pr be a *functional* uPL program; $\overline{x}$ are its formal parameters, and $n \in \overline{x}$. Let $e(K)$ denote $e[K/n]$.

BC1 : $P \Rightarrow n\geq 0$

BC2 : $\{*\ P \wedge (n = 0)\ *\}\ Pr(\overline{x})\ \{*\ \mathtt{return} = e(0)\ *\}$

RC : $\{*\ P \wedge (n = K)\ *\}\ Pr(\overline{x})\ \{*\ \mathtt{return} = e(K)\ *\}$
implies
$\{*\ P \wedge (n = K{+}1) \wedge K\geq 0\ *\}\ Pr(\overline{x})\ \{*\ \mathtt{return} = e(K{+}1)\ *\}$

$\{*\ P\ *\}\ Pr(\overline{x})\ \{*\ \mathtt{return} = e\ *\}$

□

Example

Consider the following recursive program to sum all integers from 0 up to `n`:

```
SUMN(READ n:int)
 { var s:int ;
   if n=0 then s:=0 else s:=SUMN(n-1) ;
   return s+n
 }
```

Let us prove the following specification of `SUMN`:

$$\{*\ \mathtt{n}\geq 0\ *\}\ \ \mathtt{SUMN(n)}\ \ \{*\ \mathtt{return} = (\mathtt{n}^2+\mathtt{n})/2\ *\} \tag{7.29}$$

Since `SUMN` is functional and recurs down n with step 1, we can use the second REC Reduction rule, which says that it is sufficient to prove the following:

1. **BC1:** $\mathtt{n}\geq 0 \Rightarrow \mathtt{n}\geq 0$. Trivial.
2. **BC2:** $\{*\ \mathtt{n}\geq 0 \wedge (\mathtt{n}=0)\ *\}\ \mathtt{SUMN(n)}\{*\ \mathtt{return} = 0\ *\}$. This is quite easy to prove; we leave it to you.
3. The recursion step (**RC**). Assuming the following recursion hypothesis, call it **RH**:

   $$\{*\ \mathtt{n}\geq 0 \wedge (\mathtt{n}=\mathtt{K})\ *\}\ \mathtt{SUMN(n)}\ \{*\ \mathtt{return} = (\mathtt{K}^2+\mathtt{K})/2\ *\}$$

   show that:

   $$\{*\ \mathtt{n}\geq 0 \wedge (\mathtt{n}=\mathtt{K}+1) \wedge \mathtt{K}\geq 0\ *\}\ \mathtt{SUMN(n)}\ \{*\ \mathtt{return} = ((\mathtt{K}+1)^2+\mathtt{K}+1)/2\ *\}$$

For the last proof obligation, it comes down that we have to prove the following specification of `SUMN`'s body:

$$\{*\ \mathtt{n}\geq 0 \wedge (\mathtt{n}=\mathtt{K}+1) \wedge \mathtt{K}\geq 0\ *\} \tag{7.30}$$

$$\mathtt{if}\ \mathtt{n}=0\ \mathtt{then}\ \mathtt{s}:=0\ \mathtt{else}\ \mathtt{s}:=\mathtt{SUMN}(\mathtt{n}-1)$$

$$\{*\ \mathtt{s}+\mathtt{n} = ((\mathtt{K}+1)^2+\mathtt{K}+1)/2\ *\}$$

The pre-condition in (7.30) implies $\mathtt{n}\neq 0$. So, the `else`-branch will be taken; it is therefore sufficient to prove only the `else`-branch:

$$\{*\ \mathtt{n}\geq 0 \wedge (\mathtt{n}=\mathtt{K}+1) \wedge \mathtt{K}\geq 0\ *\} \tag{7.31}$$

$$\mathtt{s}:=\mathtt{SUMN}(\mathtt{n}-1)$$

$$\{*\ \mathtt{s}+\mathtt{n} = ((\mathtt{K}+1)^2+\mathtt{K}+1)/2\ *\}$$

By the Functional Box Reduction Rule, using the specification assumed in **RC**, we can equivallently prove:

$$\mathtt{n}\geq 0 \wedge (\mathtt{n}=\mathtt{K}+1) \wedge \mathtt{K}\geq 0 \quad\Rightarrow\quad (((\mathtt{n}-1)^2+\mathtt{n}-1)/2+\mathtt{n} = ((\mathtt{K}+1)^2+\mathtt{K}+1)/2)$$

Since $\mathtt{n}=\mathtt{K}+1$ is assumed, we can simplify the right hand side of the implication to:

$$(\mathtt{K}^2+\mathtt{K})/2+\mathtt{K}+1 \ = \ ((\mathtt{K}+1)^2+\mathtt{K}+1)/2$$

This is proven below:

$$((\mathtt{K}+\mathtt{1})^2+(\mathtt{K}+\mathtt{1}))/\mathtt{2}$$
$$=$$
$$(\mathtt{K}^2+\mathtt{2K}+\mathtt{1}+\mathtt{K}+\mathtt{1})/\mathtt{2}$$
$$=$$
$$(\mathtt{K}^2+\mathtt{K}+\mathtt{2K}+\mathtt{2})/\mathtt{2}$$
$$=$$
$$(\mathtt{K}^2+\mathtt{K})/\mathtt{2}+\mathtt{K}+\mathtt{1}$$

We have closed all proof obligations. So we conclude that the specification (7.29) is indeed valid.

7.5 Exercise

1. Consider again the `ZeroOne` program. The specification is in (7.5). In Subsection 7.2.1 we have discussed the following use of `ZeroOne`, but we have not finished its proof:

   $$\{*\ \mathtt{true}\ *\}\ \ \mathtt{i}_0 := \mathtt{i};\ \mathtt{ZeroOne}(\mathtt{a}[\mathtt{i}], \mathtt{i})\ \ \{*\ (\mathtt{a}[\mathtt{i}_0] = 0) \wedge (\mathtt{i} = 1)\ *\}$$

   Complete the proof.

2. Given the following program:

   ```
   CSWAP(n:int, OUT x,y:int) : ()
     { var tmp:int;
       if n>0 then {tmp:=x; x:=y; y:=tmp} else skip }
   ```

   (a) Give a formal black box specification for `CSWAP`.

   (b) Prove the following specification. You are not allowed to use `CSWAP` as a white box.

   $$\{*\ \mathtt{y} \geq 0\ *\}\ \ \mathtt{x} := \mathtt{y} + 1;\ \mathtt{CSWAP}(\mathtt{x}, \mathtt{x}, \mathtt{y})\ \ \{*\ \mathtt{x} \geq 0 \wedge \mathtt{y} > 0\ *\}$$

   (c) Now try this specification:

   $$\{*\ \mathtt{i} > 0\ *\}\ \ \mathtt{x}, \mathtt{y}{:=}\mathtt{a}[\mathtt{i}], \mathtt{i};\ \mathtt{CSWAP}(\mathtt{i}, \mathtt{a}[\mathtt{i}], \mathtt{i})\ \ \{*\ \mathtt{a}[\mathtt{y}] = \mathtt{y}\ *\}$$

   Is $\mathtt{a}[\mathtt{i}] = \mathtt{y}$ a valid post-condition for the call above?

3. Given a program `Pr` with the following black box specification:

   $$\{*\ 0{\leq}\mathtt{n}\ *\}\ \ \mathtt{EXP2}\ (\mathtt{n} : \mathtt{int})\ \ \{*\ \mathtt{return} = 2^{\mathtt{n}}\ *\}$$

   Prove this specification:

   $$\{*\ \mathtt{m}{\geq}0 \wedge \mathtt{n}{\geq}0\ *\}\ \ \mathtt{n} := \mathtt{EXP2}(\mathtt{n});\ \mathtt{m} := \mathtt{EXP2}(\mathtt{m})\ \ \{*\ \mathtt{m}{*}\mathtt{n} > 0\ *\}$$

   Use the Functional Box Reduction.

4. Here is the program `test` from Section 7.4:

   ```
   test(OUT n:int) : ()

     { if n=0 then skip else { n:=n-1; test(n) }}
   ```

   Prove this specification:

   $$\{*\ \mathtt{n}{\geq}0\ *\}\ \mathtt{test}(\mathtt{n})\ \{*\ \mathtt{n} = 0\ *\}$$

5. Given the following recursive program:

   ```
   allTrue(n:int, READ a:bool[]) : bool
     { var b:int ;
       if n=0 then b:=true
       else { b:=allTrue(n-1,a) ;
              b:=a[n-1]/\b        } ;
       return b
     }
   ```

   The program checks whether the array `a` contains only `true`'s in the domain $0{\leq}\mathtt{i}{<}\mathtt{n}$. Give a formal specification, and prove its validity.

CHAPTER 8

Abstract Data Type

ADT is an example of a more advanced program structure that can be built beyond the ordinary statements that we have seen in uPL so far. We will discuss here issues related to its verification and validation.

ADT stands for *Abstract Data Type*. It is a closed data structure together with a set of operations to access and manipulate the structure. An ADT is *closed* in two ways. Firstly, the only way to access (and modify) the data stored in an ADT is through the provided operations. This provides some security. For example, implementing a stack as an ADT would force access to the stack to go through its operations. This in turn would prevent the programmer from e.g. accidentally taking out an element from the wrong side of a stack. Secondly, an ADT does not expose the actual implementation of its data structure nor its operations. This is why it is called *abstract*: it hides implementation details. This allows an ADT to be implemented in different ways. Furthermore, any change in the implementation will not influence its use, since it never relies on any implementation knowledge.

A similar idea of abstraction approach is also found in the Object Oriented (OO) proramming and in Component Based Approach (CBA), e.g. CORBA. An *object* in OO hides its private fields. Like in ADT, access to them is only possible via the object's methods. A *component* in CBA are intended to be highly reusable. To achieve this, a component does not expose its code. Like ADT it only offers a partial information about its functionality, such as the methods it provides and their specifications.

Z [8] and B [2] are examples of industrial strength specification languages that rely a lot on the ADT idea. In Z it is called *schema* and in B it is called *abstract machine*. Both have been used for specifying large software.

```
ADT Queue {

  model Queue = [int]

  operation
    init()      :()
    size()      :int
    qin(x:int)  :()
    qout()      :int

  spec   // q below is of type Queue

    {* true *} q.init() {* q=[] *}

    {* true *} q.size() {* return=COUNT q *}

    {* true *} Q:=q; q.qin(x) {* q=Q++[x] *}

    {* q ≠ [] *} Q:=q; q.qout() {* (return=head Q) ∧ (q=tail Q) *}

}
```

Figure 8.1: *An example of an ADT.*

8.1 ADT with uPL

Let us extend our language uPL so that we can also specify an ADT. Figure 8.1 shows an example. Such an ADT declaration introduces a new type of data, in this case it is called `Queue`. As the name suggests, this will be an ADT representing queues.

The `model` part specifies that we will *model* the type `Queue` with the type integer list (denoted by [`Int`]). This does *not* say that we will implement a `Queue` with the said list. It just says that abstractly we will pretend as if it is a list. In particular an ADT's operations will be specified in terms of the ADT's model.

The `operation` section lists the operations that are available for this ADT. We see four operations are listed for `Queue`. It is illegal to access an ADT through any means other than its operations.

The `spec` section specifies the behavior of these operations with Hoare triple specifications. We use a little OO syntax flavour in the specifications. E.g. the specification of `init` looks like this:

$$\{*\ \texttt{true}\ *\}\quad \texttt{q.init()}\quad \{*\ \texttt{q} = [\,]\ *\}$$

The notation `q.init()` is new in uPL. What we mean here is that `init` actually has `q` as it first parameter, which is furthermore passed by *copy-restore*. Implicitly, `q` is assumed to be of type `Queue`. So, the header of `init` is actually:

$$\texttt{init}(\texttt{OUT q} : \texttt{Queue}) : ()$$

And a call like `q.init()` is actually the call `init(q)`. Similarly, `qin` actually has this header:

$$\texttt{qin}(\texttt{OUT q} : \texttt{Queue},\ \texttt{x} : \texttt{int}) : ()$$

and the call `q.qin(x)` is actually `qin(q, x)`.

The operation `init` is a bit special. It is used to initialize an ADT when it is instantiated. That is, each instantiation of an ADT, e.g. when we create a local variable of its type:

{`var a : Queue`; *...do something with a...*}

This block creates thus a fresh instance of `Queue` and binds it to the local variable `a`. Immediately after its creation, the operation `init` will be called on it to initialize its content.

From the specifications we can infer that `init` sets the queue to empty; `size` returns the length of the queue; `qin` is used to enter a value into the queue; and `qout` is used to retrieve the oldest value from the queue.

8.2 Extending ADT

In OO a new class can be introduced by extending (subclassing) an existing one. The new class inherits all the fields and methods of the old class. This is a useful construction method. We can also do it with ADTs: a new ADT can be introduced by extending an existing one. We will keep it simple here: an extension only adds new operations (and their specifications). In particular, we do not allow operations to be overriden (as in Java). An example is shown below. It defines an ADT for a queue with an additional operation `dequeue` to retrieve the last element entered to the queue. In the specification, `last` returns the last element of a list, and `lead` returns the entire list except the last element.

```
ADT DQueue extend Queue {

  operation
    dequeue()     :int

  spec
    {* q ≠ [] *} Q:=q; q.dequeue() {* (return=last Q) ∧ (q=lead Q) *}

}
```

The ADT `DQueue` inherits `Queue`'s data model (which models a queue as a list) and operations. Any correct implementation of `Queue` that satisfies the additional specification imposed by `DQueue` is therefore also an implementation of `Queue`.

8.3 Implementing ADT

An ADT can be implemented by providing a concrete data structure and code for its operations; this process is also called *data refinement*. We will extend `uPL` so that we can also express an ADT implementation. An example is shown in Figure 8.2.

The `rep` keyword stands for 'representation'. The line:

$$\texttt{Queue} = \texttt{record}\{\texttt{a} : \texttt{int}[] \; ; \; \texttt{m}, \texttt{n} : \texttt{int}\}$$

states that this implementation (of `Queue`) represents a queue with a value of the type specified after '='; so, a record of the above structure. If `r` is such a record, the idea here is to store the elements of the queue which `r` represents in the array `r.a`, more specifically in the segment `r.a`[`m...n`).

When an instance of `Queue` is created, we actually create a record of the above type. When an operation p is applied on a queue, we actually apply the operation on a record of the above type.

```
IMPL Queue {

  rep Queue = record { a:int[] ; m,n:int }

  absfn α r = r.a[r.m..r.n)

  inv  r.m≤r.n  // r ranges over the above record

  impl  // r below is a record of the above type

    r.init()    :()  { r.m,r.n:=0,0        }
    r.size()    :int { return (r.n-r.m)    }
    r.qin(x:int):()  { r.a[r.n]:=x; r.n:=r.n+1 }
    r.qout():int
       {var x:int;
        if r.n-r.m>0 then {x:=r.a[r.m]; r.m:=r.m+1 } else skip ;
        return x }
}
```

Figure 8.2: *An example of an ADT implementation.*

Remember that in its specification a queue was modelled by a list. We now also have to specify how a record of the above type should be mapped to the `Queue`'s model. This is specified in the `absfn` section, which stands for *abstraction function*. The line:

$$\alpha\,\mathtt{r} \;=\; \mathtt{r.a[r.m..r.n)}$$

defines a function α that maps records of the above type to `Queue`'s model, which is [`int`]. This function is called abstraction function. It defines the meaning of each record of the above type in terms of `Queue`'s model. As you can see, it says that each record `r` is to be interpreted as the list `r.a[r.m..r.n)`.

We will explain the `inv` later. The `impl` section specifies the implementation of each of `Queue` operations. However, instead of passing a 'queue' we now of course pass a concrete value representing it, which is a record of the type as specified in the `rep` section. So for example `init` would concretely have this type:

$$\mathtt{init(OUT\ r\ :\ record\{a : int[]\ ;\ m, n : int\})} : ()$$

This type of `init` is only known within the implementation. Externally, it would have the type as specified by the `ADT` to which it belongs. So, externally its type remains:

$$\mathtt{init(OUT\ r : Queue)} : ()$$

Verifying that the implementation is correct amounts to verifying each specification in `Queue`'s `spec` section. However, we have a slight mismatch here. The specification assumes a queue to be represented as a list, whereas the implementation uses a record. Fortunately we have the abstraction function α. Recall that its role was to provide an interpretation of records in terms of lists. Exactly what we need here! However, as the consequence we also have to adapt the original specifications of `Queue`. For example, this is the original specification of `init`:

$$\{*\ \mathtt{true}\ *\}\quad \mathtt{q.init()}\quad \{*\ \mathtt{q} = [\,]\ *\}$$

However q is now actually represented by a record, say r. Given such a record r, α specifies which list it represents, namely: r.a[r.m ... r.n). So, we have to adapt the specification above to:

$$\{*\ \mathtt{true}\ *\}\quad \mathtt{r.init}()\quad \{*\ \mathtt{r.a[r.m \ldots r.n)} = []\ *\}$$

The init above is now the 'concrete' init. This is the init as defined in the implementation in Figure 8.2. The above is thus the specification we have to verify for this concrete init.

Similarly, this was the original specification for qin:

$$\{*\ \mathtt{true}\ *\}\ \ \mathtt{Q} := \mathtt{q};\ \mathtt{q.qin(x)}\ \ \{*\ \mathtt{q{=}Q{+\!\!+}[x]}\ *\}$$

This has to be adapted to:

$$\{*\ \mathtt{true}\ *\}\ \ \mathtt{Q} := \mathtt{r.a[r.m \ldots r.n)};\ \mathtt{r.qin(x)}\ \ \{*\ \mathtt{r.a[r.m \ldots r.n){=}Q{+\!\!+}[x]}\ *\}$$

The specication for init can be proven without problem. However that of qin is only valid if r.m≤r.n holds in the pre-condition. The idea of the fields m and n is that they specify the left, respectively, right border of the array r.a that actually contains the elements of our queue. So it actually makes sense to expect that if r is not in the middle of being processed by one of the Queue's operations, then r.m≤r.n holds. Such a property is also called a *data invariant*.

We can require r.m≤r.n as the data invariant of the implementation in Figure 8.2. This is specified in the inv section. When proving that the implementation is correct, we are allowed to strengthen the pre-conditions with the data invariant. So, in the case of proving the specification of qin, we are allowed to assume r.m≤r.n:

$$\{*\ \mathtt{r.m{\leq}r.n}\ *\}\ \ \mathtt{Q} := \mathtt{r.a[r.m \ldots r.n)};\ \mathtt{r.qin(x)}\ \ \{*\ \mathtt{r.a[r.m \ldots r.n){=}Q{+\!\!+}[x]}\ *\}$$

Proving this specification poses now no problem.

More precisely, a predicate I is a data invariant of an implementation of ADT A if the implementation makes it so that I holds when an instance of A is created, and is restored by every operation of A. Note that this is not the same notion of 'invariant' as in the context of loops. A data invariant can be thought to express a constraint on the well formedness (integrity) of a data structure. It is analogous to what in JML/Java is called *class invariant*.

Of course now we also have the obligation to prove that the claimed data invariant is maintaied. Firstly, we have to show that it is established when an ADT is instantiated. It means we have to prove that (the implementation of) init satisfies:

$$\{*\ \mathtt{true}\ *\}\ \ \mathtt{r.init}()\ \ \{*\ \mathtt{r.m{\leq}r.n}\ *\}$$

Secondly, we have to show that (the implementation of) each operation maintains the invariant. For example, for qin we have to prove:

$$\{*\ \mathtt{r.m{\leq}r.n}\ *\}\ \ \mathtt{r.qin(x)}\ \ \{*\ \mathtt{r.m{\leq}r.n}\ *\}$$

Using ADT in Implementation

We can also use ADTs to implement another ADT. Consider the following stack ADT:

```
ADT Stack {

  model Stack = [int]

  operation
    init()      : ()
    push(x:int) : ()
    pop()       : int

  spec  // s below is of type Stack

    {* true *} s.init() {* s = [] *}
    {* true *} S:=s; s.push(x) {* s = x : S *}
    {* s ≠ [] *} S:=s; s.pop()    {* (s = tail S) ∧ (return = head S) *}
}
```

We can implement it using `DQueue` (from the previous section):

```
IMPL Stack {
  rep Stack = DQueue
  absfn ...
  inv   ...
  impl // q below is of type DQueue
    q.init()      { q.init() }
    q.push(x:int) { q.qin(x) }
    q.pop()       {var y; y:=q.dequeue() ; return y }
}
```

The abstraction function and data invariant are left out for exercise. The nice thing about the implementation above is that it is independent of the chosen implementation of the `DQueue`. Though on the other hand, when verifying the correctness of the above implementation we cannot use the information on how `DQueue` is implemented, because this may not be known. So, all we can use is the specification of `DQueue` itself. So, if it is weak, we may not be able to show the correctness of the above implementation.

8.4 Validation

So far we usually use a program's specification for (abstractly) expressing the program's functionality. Such a specification can be used, as discussed in Chapter 7, as a substitute for the program's code when reasoning about the program's effect when it is called by another program.

We can also write specifications purely for the purpose of validating a program. *Validation* is often contrasted to *verification*. These are popular terms, but unfortunately there does not seem to be a consensus on what they precisely mean, except perhaps that verification is more rigorous than validation. Here we will define verification as the act of proving that a program satisfies a *functional specification*. A functional specification is a specification that describes the program's functionality. Validation is the act of proving that the program satisfies a set of specifications. In particular, these specifications do not have to describe the program's functionality. However, failure to satisfy one of them would imply that the program is incorrect. In a sense we can also view validation as cross-checking a program against a set of partial (weaker) specifications. Verification would typically give a stronger result; but on the other hand validation may be cheaper —in practice it is a trade-off that a designer has to decide. We will also call specifications used for validation: *validation properties*. For example, consider the program:

```
SUC(x : int) { return; x+1 }
```

The following would make a good functional specification for SUC:

$$\{*\ \texttt{true}\ *\}\ \texttt{SUC(x)}\ \{*\ \texttt{return} = \texttt{x} + 1\ *\}$$

But this specification:

$$\{*\ \texttt{true}\ *\}\ \texttt{SUC(x)}\ \{*\ \texttt{true}\ *\}$$

is not very useful for describing SUC's functionality. However, it still states that SUC terminates; any correct implementation of SUC clearly must satisfy the specification. This is a typical example of a validation property.

It should be noted that the border between a functional specification and a validation property is a grey area. Consider this specification:

$$\{*\ \texttt{true}\ *\}\ \texttt{SUC(x)}\ \{*\ \texttt{return} > \texttt{x}\ *\}$$

It only partially describes what SUC does. It can be used as a validation property. Still, we can choose to use it as a functional specification. Though it is indeed a rather weak functional specification.

An interesting case of validation is that of ADT. Because an ADT can only be accessed through its operations, it makes the operations themselves suitable for capturing sophisticated validation properties. This leads to a possible application in unit testing and run-time checking. As an example, consider again the Queue ADT (Figure 8.1). The following specification says that the queue's size will increase by 1 after a qin operation:

$$\{*\ \texttt{COUNT q} = \texttt{N}\ *\}\ \texttt{q.qin(x)}\ \{*\ \texttt{COUNT q} = \texttt{N+1}\ *\} \tag{8.1}$$

As opposed to qin's specification in Queue's SPEC-section, the above specification only partially describes what qin does. We may want to have it as a validation property. Another way to express (8.1) is as follows, which relies on size to compute the queue's actual size:

$$\{*\ \texttt{true}\ *\}\ \texttt{m} := \texttt{q.size()};\ \texttt{q.qin(x)};\ \texttt{n} := \texttt{q.size()}\{*\ \texttt{n} = \texttt{m+1}\ *\} \tag{8.2}$$

Notice that the assertions in (8.2) are ordinary program expressions, whereas in (8.1) the function COUNT is used, which is only available in Form. Suppose we decide to first *test* a specification. Testing means running a program on some concrete inputs, and checks if it behaves according to its specification. Although testing is only partial, in practice it makes sense to combine testing with validation or verification. For example we may want to delay validation until ordinary testing cannot reveal further error.

During a test, an assertion which is also a program expression can be checked by evaluating the expression and checking if it results in true. A tool like JUnit [1] can automate testing and assertion checks during the test.

In contrast, checking an arbitrary Form formula as an assertion is more complicated. For example, to check the assertion COUNT q = N+1 in (8.1) at the run-time would require us to specify an implementation of COUNT. The same applies for all other operators and functions that are only available in Form, e.g. $\forall$, $\in$, or $+\!\!+$.

In some specification languages, e.g. JML [18], it is also allowed to have calls to functional programs (programs without any side effect) to appear in assertions. JML also allows a formula of the form old e to appear in the post-condition of a program; it refers to the value of e evaluated on the state when the program is called. With such extensions we can equivalently express (8.2) as follows:

$$\{*\ \texttt{true}\ *\}\ \texttt{q.qin(x)}\ \{*\ \texttt{q.size()} = \texttt{old(q.size())} + 1\ *\} \tag{8.3}$$

Strictly speaking, (8.2) specifies a call to qin when it is preceeded and followed by calls to size. In contrast, (8.3) above specifies an arbitrary call to qin. Except for the use of old the assertions

```
ADT Queue {

  model Queue = [int]

  operation ... // as before

  spec      ... // as before

  prop   // q below is of type Queue

    {* true *} m:=q.size(); q.qin(x); n:=q.size() {* n = m+1 *} ;

    {* true *}
      q.qin(x)    ;
      s:=q.size() ;
      while 0<s do { s:=s-1; y:=q.quot() }
    {* y = x *}

}
```

Figure 8.3: *Queue ADT with some validation properties.*

in (8.3) are in principle ordinary program expressions. As before, they can be staight forwardly converted to code fragments that checks at the run-time if the respective assertions are not violated. However now this code can be wrapped around `qin` so that a call to `qin` can be refused if the pre-condition is not met, and reported if it violates the post-condition. JML implements this idea of run-time assertion checks.

Most of the specifications you have seen so far are of the form of:

$$\{*\ P\ *\}\ \ X{:=}x;\ Pr(x)\ \ \{*\ Q\ *\}$$

In (8.2) you see a different pattern. It uses a more complex statement to specify a property of a single program (`qin`). This is quite powerful. For example, we may want to have a validation property that says that any element entered into the queue can be retrieved again after sufficient number of calls to `qout`. This cannot be expressed with a 'plain' Hoare triple. However, in the style of (8.2) we can capture it like this:

```
{* true *}
 q.qin(x)  ;
 s:=q.size() ;
 while 0<s do { s:=s-1; y:=q.quot() }
{* y = x *}
```

Figure 8.3 adds the two validation properties discussed above in the declaration of the `Queue` ADT; they are listed in the `prop` section. Like the specification in the `spec` sections, the specified validation properties do not refer to the implementation details of `Queue`.

If the specification of an ADT (the `spec` section) is strong enough, we may be able to prove its validation properties direction from it (the `spec` section). This gives an extra benefit that any correct implementation of the ADT would then automatically satisfy all its validation properties.

Validation properties can be used for a number of purposes:

1. They can complement the `spec` section. So they provide additional checks for an implementation.

2. Out of budget consideration, an implementator may decide to only verify validation properties.

3. They can be used to check if the specifications in the `spec` section are making sense. Note that if they contradict each other, then no implementation is possible. There is a contradiction if you can show that a value of the ADT exists that satisfies the `spec` section, but violates one of the validation properties.

4. They can be used to check if the specifications in the `spec` section are sufficient. They are sufficient if all validation properties can be proven from the `spec` section.

8.5 Exercise

1. Sketch how to proceed in order to prove that the implementation of the ADT `Queue` in Figure 8.2 is correct.

2. Let us write an ADT `Set` that will provide sets to uPL. To make it simple we will only provide sets of integers. For this exercise we also extend Form with sets, set comprehension, and the standard set operators like $\in$ and $/$. This is just for the purpose of writing specifications. Write now your `Set` ADT. It should support operations: to insert an element to the set, to delete an element, and to test membership. Write also your own uPL implementation of the ADT. What are your abstraction function and data invariant?

3. Consider again the implementation of the `Stack` ADT in page 156. The abstraction function and the data invariant are still missing. Specify them. Does the implementation satisfy your data invariant? Give a worked out proof for the correctness of `init`.

4. Suppose we extend the ADT `Queue` in Figure 8.1 with operations `isEmpty` to test if the queue is empty and `elem` to test if a value is in the queue. Write validation properties capturing the following:

   (a) A queue is empty after initialization.

   (b) A queue is empty if and only if `size` is 0.

   (c) An empty queue does not contain any element.

   (d) A value inserted with `qin` will be an element of the queue.

   (e) If we insert a value x to a queue that already has n elements, after n times `qout` x will still be in the queue.

   Are your specifications executable?

5. Consider again the `Queue` ADT in Figure 8.1 and the validation properties given in Figure 8.3. Are the functional specifications given in the `spec` section sufficient? Use the validation properties to check it. If you find out that `spec` is insufficient, give a proposal to strengthen it.

   As a side note: when you strengthen a specification, there is a danger that it becomes overly strong that it cannot be implemented. A trivial example of this is if you replace all the post conditions in `spec` with `false`.

CHAPTER 9

Incremental Proof

At some point we often want to optimize our program, usually by tweaking it at some points. Unfortunately, changing a program invalidates its correctness proof. Since constructing a proof is expensive, we prefer not to redo the proof from the the scratch. This chapter will discuss an incremental proof approach, where we will try to reuse old results. A potentially interesting application of this is to verify systematic optimization e.g. as done by compilers, or by a code refactoring tool. However that is beyond our scope; here we just introduce you to the basic concepts of incremental proofs.

Here are some other applications that we can mention. It has been found useful for the verification of complicated algorithms [27]. First, one builds a base algorithm, which is simple and easy to prove. Subsequently, it is refined in several steps to get to its final version. Each refinement is justified with an incremental proof. B method [2] uses the same idea to incrementally design programs. One starts with an abstract model of a program, and in several steps incrementally refines it until a model is obtained which is concrete enough to be compiled to an executable code. Incremental proof has also been suggested for speeding-up re-verification of integrated circuits [25].

More formally, we can express an incremental proof strategy as an inference rule. Let $\mathcal{T}$ be a transformation that changes a given program and its specification to a new one; we are looking for a rule of this form:

$$\frac{\begin{array}{c}\{*\ P\ *\}\ S\ \{*\ Q\ *\}\\ \textit{some conditions}\end{array}}{\mathcal{T}(\{*\ P\ *\}\ S\ \{*\ Q\ *\})}$$

So, from the validity of the old specification $\{*\ P\ *\}\ S\ \{*\ Q\ *\}$ we can infer the validity of the new specification just by proving the conditions! In the sequel we will discuss some simple examples of incremental proof strategies. For a more comprehensive discussion you can try [20].

9.1 Splitting Specifications

Rules like the Hoare triples Conjunction and Disjunction (Rules 6.1.8 and 6.1.7) allow us to split a specification into a set V of 'smaller' specifications, such that the validity of the smaller specifications implies the original one. This has the benefit that if we later change one of the specifications in V, we do not, as long as the code of the program being specified is unchanged, invalidate the others. So, only the changed specification need to be proven anew. We will discuss some examples.

Multiple Functionalities

Sometimes you have a program with a conjunctive post-condition. That is, a post-condition of the form $Q_1 \wedge \ldots \wedge Q_k$. This is typically the case when the program has multiple functionalities, with each Q_i specifying one functionality. We show below a simplified variant of the Hoare triples Conjunction Rule:

$$\frac{\{*\ P\ *\}\ S\ \{*\ Q_1\ *\} \quad \{*\ P\ *\}\ S\ \{*\ Q_2\ *\}}{\{*\ P\ *\}\ S\ \{*\ Q_1 \wedge Q_2\ *\}}$$

For example, consider a program `reset(x)` that resets `x` to some non-negative value and returns `x`'s old value:

$$\{*\ \texttt{true}\ *\}\ \texttt{X} := \texttt{x};\ \texttt{reset(OUT x : int)}\ \{*\ \texttt{x}{\geq}\texttt{0} \wedge (\texttt{return} = \texttt{X})\ *\} \tag{9.1}$$

Suppose that in the implementation `reset` just sets `x` to 0. Now suppose that later the programmer decides that the `x≥0` part of the post-condition is not good enough. He wants to change it to a stronger one below:

$$\{*\ \texttt{true}\ *\}\ \texttt{X} := \texttt{x};\ \texttt{reset(OUT x : int)}\ \{*\ (\texttt{x}{=}\texttt{0}) \wedge (\texttt{return}{=}\texttt{X})\ *\} \tag{9.2}$$

Since the new post-condition is stronger, it cannot be inferred from the old one (9.1); so the programmer has to redo the proof.

Alternatively, we could have used the Conjunction Rule to 'split' (9.1) over the two 'functionalities'. That is, we require these two specifications instead:

$$\{*\ \texttt{true}\ *\}\ \texttt{reset(OUT x : int)}\ \{*\ \texttt{x} \geq \texttt{0}\ *\} \tag{9.3}$$

$$\{*\ \texttt{true}\ *\}\ \texttt{X} := \texttt{x};\ \texttt{reset(OUT x : int)}\ \{*\ \texttt{return} = \texttt{X}\ *\} \tag{9.4}$$

Each of the above specifications is indeed partial. But together they are sufficient, because by the Conjunction Rule they imply the original specification (9.1).

If we later decide to change the post-condition of one partial specification, as long as we do not change the code of `reset`, it will not invalidate the others. In the case above, the programmer wants to change (9.3) to:

$$\{*\ \texttt{true}\ *\}\ \texttt{X} := \texttt{x};\ \texttt{reset(OUT x : int)}\ \{*\ \texttt{x} = \texttt{0}\ *\} \tag{9.5}$$

This change does not invalidate (9.4). Of course, (9.5) still has to be proven.

The original Conjunction Rule (Rule 6.1.8) allows you to alter the pre-condition as well. Below we also give another variant of the rule, which is perhaps more convenient to use. It follows from Rule 6.1.8 and Pre-condition Strengthening (Rule 6.1.6).

Rule 9.1.1 : Conjunction of Hoare Triples

$$\frac{\vdash P \Rightarrow P_1 \wedge P_2 \quad \{*\ P_1\ *\}\ S\ \{*\ Q_1\ *\} \quad \{*\ P_2\ *\}\ S\ \{*\ Q_2\ *\}}{\{*\ P\ *\}\ S\ \{*\ Q_1 \wedge Q_2\ *\}}$$

□

Multiple Scenarios

A conjunctive post-condition can also be used to capture different 'scenarios' under which a program is expected to operate. The program is to behave differently under different scenarios. For example, consider a program `xdiv` that returns the result of `x div y` if `y` is non-zero, else it just returns a non-positive value. It can be specified as follows:

$$\{*\ \texttt{true}\ *\}$$

$$\texttt{xdiv}(\texttt{x},\texttt{y}) \tag{9.6}$$

$$\{*\ (\texttt{y}\neq 0) \Rightarrow (\texttt{return} = \texttt{x div y}))\ \wedge\ ((\texttt{y} = 0) \Rightarrow \texttt{return} \leq 0\ *\}$$

Using the Conjunction Rule we can again split the specification to get a separate specification for each scenario:

$$\{*\ \texttt{true}\ *\}\ \ \texttt{xdiv}(\texttt{x},\texttt{y})\ \ \{*\ (\texttt{y}\neq 0) \Rightarrow (\texttt{return} = \texttt{x div y})\ *\} \tag{9.7}$$

$$\{*\ \texttt{true}\ *\}\ \ \texttt{xdiv}(\texttt{x},\texttt{y})\ \ \{*\ (\texttt{y}=0) \Rightarrow \texttt{return} \leq 0\ *\} \tag{9.8}$$

Note that since `x` and `y` are passed by value, their occurrences in the post-condition refer to their values when `xdiv` is called. So we can just as well express (9.7) and (9.8) as:

$$\{*\ \texttt{y}\neq 0\ *\}\ \ \texttt{xdiv}(\texttt{x},\texttt{y})\ \ \{*\ \texttt{return} = \texttt{x div y}\ *\} \tag{9.9}$$

$$\{*\ \texttt{y}=0\ *\}\ \ \texttt{xdiv}(\texttt{x},\texttt{y})\ \ \{*\ \texttt{return} \leq 0\ *\} \tag{9.10}$$

If we later decide to change the the pre- or post-condition of one scenario, e.g. we change (9.10) so that 0 is returned, rather than allowing an non-positive integer, it will not affect the correctness of other scenarios.

9.2 Superposition

A simple way to extend a program is by introducing fresh variables and adding assignments to the new variables. A special case of such an extension is called *superposition*. In superposition we are not allowed to use the new variables in the way that would change the program's old behavior[1]. For example, extending the guard of a loop in the old program with the new variables is not allowed. Superposition obviously preserves the validity of the program's old specification. To handle it formally we first need to introduce some notions and notations.

Refinement and Equivalence

Let us write $S \sqsubseteq T$ to mean that T satisfies *all* Hoare triples satisfied by S. It implies that T implements all S's functionalities. T can potentially do 'more': it may terminate when S does not, and it may be more deterministic than S. In any case, in terms of functionality it follows that T can be safely used to replace S, so we also say that T is a *refinement* of S. The other way around is not always true. That is, we cannot always replace T with S, since, for example, S may not terminate when T does.

Note that the above notion of refinement is only defined with respect to Hoare triples. In particular, the definition does not capture aspetcs such as performance. So, under the above definition $S \sqsubseteq T$ does not necessarily mean that T will be faster than S. Theoretically, you can strengthen the definition to capture more aspects, but here we will keep to the above defition.

[1]The idea is the same as with adding auxilliary variables, except that variables added in superposition are intended to be actual program variables, whereas auxilliary variables are only introduced for the purpose of specifying the program.

If S and T refine each other, they can be used to replace each other. So, in this sense they are *equivalent*. We will denote it with $S \equiv T$. For example:

$$\texttt{x} := \texttt{x+1}; \texttt{x} := \texttt{x-1} \quad \equiv \quad \texttt{skip}$$

The above definition of refinement is too strong however. It does not actually allow us to replace $\texttt{x} := \texttt{x+1}$ with, for example $\texttt{x}, \texttt{y} := \texttt{x+1}, \texttt{0}$, where $\texttt{y}$ is a fresh variable. Let us weaken it a bit. We parameterize it with a set of program variables V; we now write $P \sqsubseteq_V Q$ to mean that Q refines P, but only relative to what they do to the variables in V. Formally this is defined below.
If P is a predicate, let $\mathsf{free}(P)$ denote the set of all free variables in P.

Definition 9.2.1 : Refinement

$$\begin{array}{l} S \sqsubseteq_V T \\ = \\ (\forall P, Q : \ \mathsf{free}(P) \subseteq V \ \wedge \ \mathsf{free}(Q) \subseteq V \ : \\ \qquad\qquad \{* \, P \, *\} \, S \, \{* \, Q \, *\} \ \Rightarrow \ \{* \, P \, *\} \, T \, \{* \, Q \, *\}) \end{array}$$

□

For example, this is valid:

$$\texttt{x:=0} \ \sqsubseteq_{\{\texttt{x}\}} \ \texttt{x}, \texttt{y:=0}, \texttt{0}$$

However, this is *not*: $\texttt{x:=0} \sqsubseteq_{\{\texttt{x,y}\}} \texttt{x}, \texttt{y:=0}, \texttt{0}$. We can now define the corresponding notion of equivalence:

$$S \equiv_V T \quad = \quad S \sqsubseteq_V T \ \wedge \ T \sqsubseteq_V S$$

So, if $S \equiv_V T$ then both statements will behave the same way with respect to the variables in V, though they may behave differently with respect to other variables. This can be captured by the following rule, which follows straight forwardly from the definition of $\equiv_V$ and $\sqsubseteq_V$:

Theorem 9.2.2 :
If $\mathsf{free}(P) \subseteq V$ and $\mathsf{free}(Q) \subseteq V$ and $S \equiv_V T$, we have:

$$\{* \, P \, *\} \, S \, \{* \, Q \, *\} \quad = \quad \{* \, P \, *\} \, T \, \{* \, Q \, *\}$$

□

If two statements are equivalent with respect to some set V of variables, they are also equivalent with respect to any subset of V:

Theorem 9.2.3 :

$$S \equiv_V T \ \wedge \ U \subseteq V \ \Rightarrow \ S \equiv_U T$$

□

Let $\texttt{var}(S)$ denote the set of all variables used in S. Superpositioning a statement S produces a statement S' which is equivalent to S with respect to $\texttt{var}(S)$. Formally:

Theorem 9.2.4 : Equivalence by Superposition
Let S' be obtained from superpositioning S with assignments to fresh variables. Let Z be the set of these fresh variables. For any V such that $V \cap Z = \emptyset$, we have:

$$S \quad \equiv_{\texttt{var}(S) \cup V} \quad S'$$

□

Superpositioning a loop

Loop makes a quite interesting case of superposition. Consider a loop satisfying:

$$\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\} \tag{9.11}$$

Moreover, I is an invariant and its set of free variables is assumed to be a subset of the set of variables used in the loop. Next, we decide to superposition the body S to S', and obtain a new loop:

$$\texttt{while}\ g\ \texttt{do}\ S'$$

Note that the new loop is also a superposition of the old one.

Quite trivially, (9.11) implies $\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ \texttt{true}\ *\}$. By Theorem 9.2.2, the new loop satisfies the same specification, which implies that the new loop terminates. Notice that you can thus infer this for free (we do not have to prove the temination of the new loop from the scratch).

Now suppose we also change the post-condition from Q to $Q \wedge Q'$, where Q' specifies what we want to achieve with the new variables introduced in the superposition. For proving Q' we may need to strengthen the old invariant I with some I'; so, taking $I \wedge I'$ as the new invariant. For Q', it is now sufficient to prove that it is implied by $I \wedge I' \wedge \neg g$, which holds when the loop terminates. However, we also have the obligation to show that $I \wedge I'$ is indeed an invariant. By Theorem 9.2.2, we know that the new body S' will maintain old invariant I. So, it is sufficient to prove $\{*\ I \wedge I' \wedge g\ *\}\ S\ \{*\ I'\ *\}$. Formally this is captured by the rule below:

Rule 9.2.5 : Loop Superposition
Let S' be obtained from superpositioning S with assignments to fresh variables. Let Z be the set of these fresh variables. Z is assumed to have no intersection with $\mathsf{free}(g) \cup \texttt{var}(S) \cup \mathsf{free}(I) \cup \mathsf{free}(Q)$. We have:

$$\frac{\begin{array}{l}\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\} \\ \{*\ I \wedge g\ *\}\ S\ \{*\ I\ *\} \\ \{*\ I \wedge I' \wedge g\ *\}\ S'\ \{*\ I'\ *\} \\ \vdash I \wedge I' \wedge \neg g \Rightarrow Q'\end{array}}{\{*\ I \wedge I'\ *\}\ \texttt{while}\ g\ \texttt{do}\ S'\ \{*\ Q \wedge Q'\ *\}}$$

□

Notice that the rule requires the new loop to start from $I \wedge I'$, which implies that you may have to adapt the initialization code as well.

Example

Below is a program for counting the number of `true`'s in a Boolean array:

```
{* 0≤n *}

 i,c:=n,0 ;
 while 0<i do { i:=i-1; if b[i] then c:=c+1 else skip }

{* c = cnt0 b n *}
```

where:

$$\texttt{cnt0 b i} = \texttt{COUNT}\ [\texttt{v} \mid \texttt{v from b}[0 \ldots \texttt{i}),\ \texttt{v}]$$

In the following discussion we will just focus on the loop part. Suppose we have proven:

$$\{*\ \texttt{I}\ *\}\ \ \texttt{while 0<i do}\ \textit{body}\ \ \{*\ \texttt{c = cnt0 b n}\ *\} \tag{9.12}$$

where:

$$\mathtt{I} \;=\; (\mathtt{c} \;=\; \mathtt{cnt0\ b\ i}) \wedge 0{\le}\mathtt{i}{\le}\mathtt{n}$$

Furthermore, the proof uses I as as the invariant. So, additionally we also have this information:

$$\{*\ \mathtt{I} \wedge 0{<}\mathtt{i}\ *\}\ \ \{\mathtt{i} := \mathtt{i}{-}1;\ \mathtt{if\ b}[\mathtt{i}]\ \mathtt{then\ c} := \mathtt{c}{+}1\ \mathtt{else\ skip}\}\ \ \{*\ \mathtt{I}\ *\} \tag{9.13}$$

Later, we decide to extend the loop a little bit, so that it also calculate an index to some `true` element in b, if one exists. If there are more than one `true`'s, it does not matter which one is chosen. We can do this extension with supperposition. We introduce a fresh variable k and extend the code (of the loop) to:

$$\mathtt{while}\ 0 < \mathtt{i}\ \mathtt{do}\ \{\mathtt{i} := \mathtt{i}{-}1;\ \ \mathtt{if\ b}[\mathtt{i}]\ \mathtt{then\ c,k} := \mathtt{c}{+}1, \mathtt{i\ else\ skip}\} \tag{9.14}$$

First, Theorem 9.2.2 allows us to infer that the new loop satisfies the same specification as in (9.12). So it confirms that we did not do anything harmful.

Second, we now also have a new functionality. We expect the loop now to satisfy this specification:

$$\begin{array}{l} \{*\ \ldots\ *\} \\ \\ \mathtt{while}\ 0 < \mathtt{i}\ \mathtt{do}\ \{\mathtt{i} := \mathtt{i}{-}1;\ \ \mathtt{if\ b}[\mathtt{i}]\ \mathtt{then\ c,k} := \mathtt{c}{+}1, \mathtt{i\ else\ skip}\} \\ \\ \{*\ \textit{old post-cond}\ \wedge\ (\mathtt{c} > 0 \Rightarrow \mathtt{b}[\mathtt{k}])\ *\} \end{array} \tag{9.15}$$

The new post-condtion expresses what we want to compute with the new variable k.

Rather than proving the new specification from the scratch, we invoke Rule 9.2.5. The Rule's first and second conditions are fullfilled; they are (9.12) and (9.13) respectively. The Q' in the Rule is now $\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}]$. Quite obviously, the old invariant I is not sufficient to get $\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}]$. We take the naive solution that simply extends I with Q' (below, it turns out to be sufficient). So, the choice of I' in the Rule is just Q'. This trivially proves the Rule's 4th condition. So, the only remaining condition to prove is the 3rd one, which is:

$$\begin{array}{l} \{*\ \mathtt{I} \wedge 0{<}\mathtt{i} \wedge (\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}])\ *\} \\ \\ \{\mathtt{i} := \mathtt{i}{-}1;\ \ \mathtt{if\ b}[\mathtt{i}]\ \mathtt{then\ c,k} := \mathtt{c}{+}1, \mathtt{i\ else\ skip}\} \\ \\ \{*\ \mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}]\ *\} \end{array}$$

After computing the wp, we can equivallently prove:

$$\begin{array}{l} \mathtt{I} \wedge 0{<}\mathtt{i} \wedge (\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}]) \\ \Rightarrow \\ (\mathtt{b}[\mathtt{i}{-}1]\ \ \rightarrow\ \ (\mathtt{c}{+}1{>}0 \Rightarrow \mathtt{b}[\mathtt{i}{-}1])\ |\ (\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}])) \end{array}$$

which is quite trivial (and does not actually depend on what I is).
So, we conclude that the new code satisfies:

$$\begin{array}{l} \{*\ \mathtt{I} \wedge (\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}])\ *\} \\ \\ \mathtt{while}\ 0{<}\mathtt{i}\ \mathtt{do}\ \{\mathtt{i} := \mathtt{i}{-}1;\ \mathtt{if\ b}[\mathtt{i}]\ \mathtt{then\ c} := \mathtt{c}{+}1\ \mathtt{else\ skip}\} \\ \\ \{*\ (\mathtt{c}\ \ =\ \ \mathtt{cnt0\ b\ n}) \wedge (\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}])\ *\} \end{array} \tag{9.16}$$

This does require that the initialization code may have to be extended to realize the extra invariant $\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}]$. In this case this is not necessary, because c is initialized to 0, so $\mathtt{c}{>}0 \Rightarrow \mathtt{b}[\mathtt{k}]$ is trivially satisfied.

Notice that in concluding (9.16), Rule 9.2.5 only generates two proof obligations —one of which is trivial.

9.3 Breaking a Loop

A loop can sometimes be optimzed by breaking it if further iterations are not needed to get to the loop's specified post-condition. For example, consider this statement for computing the product of all elements in an array over the indices in [0 . . . n):

```
{* 0≤n *}

  i,r:=0,1; while i<n do { r:=r*a[i]; i:=i+1 }

{* r  =  PROD(a[0 . . . n)) *}
```

where for a list s, PROD s specifies the product of all elements of s. Suppose we have proven the specification's validity. Later on, we decide to optimize the statement: we break the loop as soon as r becomes 0 (which would happen after encountering a 0 in the array) because further computation will not change the value of r. So we have the following new program now:

```
i,r:=0,1;  while i<n ∧ r ≠ 0 do { r:=r*a[i]; i:=i+1 }
```

Let us see if it is possible to avoid redoing the entire proof. Consider a specification:

$$\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\} \tag{9.17}$$

Suppose we have proven it to be valid using I as the invariant. Next, we alter the loop by strengthening its guard to:

$$\texttt{while}\ g \wedge h\ \texttt{do}\ S$$

We can make the following observations. Firstly, (9.17) implies that from the pre-condition I the old loop terminates. The new loop either proceeds as the old one, or stops prematurely because h becomes false before g does. In either case we conclude that it terminates.

Secondly, since the body is still the same, any invariant (that is , a predicate preserved by every iteration) of the old loop is also an invariant of the new loop. In particular, I is also an invariant of the new loop. Consider an execution of the new loop; we have conlcuded that it will terminate. There are two possibilities:

1. it terminates 'normally', as in the old loop, because g becomes false. Since I is also an invariant of the new loop, we know that $I \wedge \neg g$ holds upon termination. It suffices now to prove that it implies the required post-condition Q. However, this has been proven in the old loop, so we do not have to do it again.

2. it terminates prematurely because h becomes false. The state would satisfy $I \wedge \neg h$. So, it is sufficient if we can show that $I \wedge \neg h \Rightarrow Q$. This is the only additional proof obligation needed to infer:

$$\{*\ I\ *\}\ \texttt{while}\ g \wedge h\ \texttt{do}\ S\ \{*\ Q\ *\} \tag{9.18}$$

Formally, the above reasoning is captured by the following inference rule:

$$\frac{\begin{array}{l}\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\}\\ \{*\ I \wedge g\ *\}\ S\ \{*\ I\ *\}\\ \vdash I \wedge \neg h \Rightarrow Q\end{array}}{\{*\ I\ *\}\ \texttt{while}\ g \wedge h\ \texttt{do}\ S\ \{*\ Q\ *\}}$$

The rule can be improved a bit. The third condition concerns the premature termination of the new loop. Since the first condition already implies that if $\neg g$ holds upon termination then Q also holds, it is sufficient to prove the third condition under a further assumption that g still holds. So, we get this rule:

Rule 9.3.1 : Loop Strengthening I

$$\frac{\begin{array}{l}\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\}\\ \{*\ I \wedge g\ *\}\ S\ \{*\ I\ *\}\\ \vdash I \wedge g \wedge \neg h \Rightarrow Q\end{array}}{\{*\ I\ *\}\ \texttt{while}\ g \wedge h\ \texttt{do}\ S\ \{*\ Q\ *\}}$$

□

Example

Consider again the program to compute the product over an array given in page 171. It can be proven it via:

$$\{*\ \mathtt{I}\ *\}\ \ \mathtt{while\ i{<}n\ do}\ \mathit{body}\ \ \{*\ \mathtt{r} = \mathtt{PROD}(\mathtt{a}[\mathtt{0}\ldots\mathtt{n}))\ *\} \tag{9.19}$$

where:

$$\mathtt{I}\ \ =\ \ (\mathtt{r} = \mathtt{PROD}(\mathtt{a}[\mathtt{0}\ldots\mathtt{i})))\ \wedge\ \mathtt{0{\leq}i{\leq}n} \tag{9.20}$$

Suppose we have proven (9.19) using `I` as an invariant; so, we also have:

$$\{*\ \mathtt{I} \wedge \mathtt{i{<}n}\ *\}\ \ \mathit{body}\ \ \{*\ \mathtt{I}\ *\} \tag{9.21}$$

We later change the loop to:

$$\{*\ \mathtt{I}\ *\}\ \ \mathtt{while\ i{<}n} \wedge \mathtt{r} \neq \mathtt{0\ do}\ \mathit{body}\ \ \{*\ \mathtt{r} = \mathtt{PROD}(\mathtt{a}[\mathtt{0}\ldots\mathtt{n}))\ *\} \tag{9.22}$$

Rather than using the standard Loop Reduction Rule, we now use the Loop Strengthening Rule to prove this. The Rule's first and second conditions have been proven; they are (9.19) and (9.21). So, we only have one proof obligation, namely the Rule's third condition:

$$\mathtt{I} \wedge \mathtt{i{<}n} \wedge (\mathtt{r} = \mathtt{0})\ \Rightarrow\ (\mathtt{r} = \mathtt{PROD}(\mathtt{a}[\mathtt{0}\ldots\mathtt{n})))\ \ \ \

9.4 Exercise

1. The rule below, called Direct Termination Rule, captures the scenario when a loop directly terminates because its guard is already false in the initial state:

$$\frac{-}{\{*\, Q \wedge \neg g \,*\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\, Q \,*\}}$$

   Derive the rule from the standard rule for `while`.

2. Consider the program below:

   $\{*\ \texttt{n} \geq 0\ *\}$

```
i,s:=0,0; while i<n do { s:=s+i; i:=i+1 }
```

   $\{*\ \texttt{s} \ = \ \texttt{SUM}[0\ldots\texttt{n})\ *\}$

   The specification can be easily proven, and suppose we have done so, using this as an invariant:

$$\texttt{I} \ = \ (\texttt{s} = \texttt{SUM}[0\ldots\texttt{i})) \wedge\ 0{\leq}\texttt{i}{\leq}\texttt{n}$$

   Notice that the $\texttt{i}{\leq}\texttt{n}$ part of the invariant is needed so that when the loop terminates ($\texttt{i}{\geq}\texttt{n}$) we can infer that $\texttt{i}{=}\texttt{n}$, and hence infer the loop's post-condition from the $\texttt{s} = \texttt{SUM}[0\ldots\texttt{i})$ part of `I`.

   We now weaken the pre-condition to `true`. So, a negative `n` is allowed. The claim is that the program can still realize its post-condition.

   Try to first argue the claim using the standard Loop Reduction rule. Find a suitable invariant, then check if it is sufficient. Note that $\texttt{i} \leq \texttt{n}$ is now no longer an invariant property.

   The second approach is to split the specification. Use the Hoare triple disjunction property to split the specification over two cases: (1) $0{\leq}\texttt{n}$ holds initially, and (2) $\texttt{n}{<}0$ holds instead. How to procced along this line? What do we gain?

3. The program below checks if a boolean array contains a `true`:

   $\{*\ \texttt{true}\ *\}$

```
i,r:=0,false ;
while i<n do { r:=b[i]∨r; i:=i+1 }
```

   $\{*\ \texttt{r} \ = \ (\exists \texttt{i} : 0{\leq}\texttt{i}{<}\texttt{n} : \texttt{b}[\texttt{i}])\ *\}$

   Give an invariant that will be sufficient to prove the correctness of the loop above. You may first want to argue the correctness with respect to a stronger pre-condition $0{\leq}\texttt{n}$, and then consider the $\texttt{n}{<}0$ case separately as in No. 2.

   Next, we optimize the program by stopping the `loop` as soon as `r` becomes `true`, because further iterations will not change the value of `r`. The new program:

```
i,r:=0,false ;
while i<n ∧ ¬ r do { r:=b[i]∨r; i:=i+1 }
```

   Note that the guard is now strengthened with $\neg\texttt{r}$. Can we build an incremental proof for the new code? How?

4. We can optimze the second code in No. 3 a bit further by replacing the assingment $\mathtt{r} := \mathtt{b}[\mathtt{i}] \vee \mathtt{r}$ with $\mathtt{r} := \mathtt{b}[\mathtt{i}]$. This is safe since `r` is false anyway when the body is entered. In our theory, refinement is used to formally express code replacement. However, the above kind of replacement, which depends on some assumptions on the states (namely that `r` is false at that moment), cannot be directly expressed in our notion of refinement. Can you propose an extension to it?

5. Consider again the optimized code in No. 3. We extend it further. We superposition it with a new variable `k` so that in the end `k` points to a `true` element, if such an element can be found. The code and new specification are given below:

   $\{*\ \mathtt{true}\ *\}$

   ```
   i,r:=0,false ;
   k:=0 ;  --new
   while i<n ∧ ¬ r do
     { r := b[i] ∨ r ;
       if r then k:=i else skip    ;  -- new
       i:=i+1 }
   ```

   $\{*\ (\mathtt{r} \ = \ (\exists \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}])) \ \wedge \ (\mathtt{r} \ \Rightarrow \ \mathtt{b}[\mathtt{k}] \wedge 0 \leq \mathtt{k} < \mathtt{n})\ *\}$

   How to prove this incerementally?

CHAPTER 10

Exercises Collection

1. Consider the following statement to test whether an array consists of only positive element.

```
i:=n ;
b:=T ;
while i > 0 do {i:=i-1; b := b ∧ (a[i] > 0) }
```

   Suppose n $\geq$ 0 holds initially. Moreover, the array a contains at least one zero element in the domain 0$\leq$j$<$n. Prove that the statement will terminate with b false.

2. A non-deterministic loop is a loop with multiple bodies. The syntax is:

   while g_1 do S_1
   ...
   g_n do S_n

   The loop terminates if none of the guards ($g_1 \ldots g_n$) is true. At each iteration, if only one guard is true, then the corresponding body will be executed. If multiple guards are true, then one will be selected *non-deterministically*, and the corresponding body is executed.

   As an example, we can use a non-deterministic loop to non-deterministically select a number from a certain range:

```
NDSELECT(READ n:int) : int
stop:bool ;
{ i:=0;
  stop:=F;
  while
    ¬stop ∧ i < n do i:=i+1
    ¬stop do stop:=T  ;
  return i
}
```

Generalize the reduction rule for the standard `while` loop to handle non-deterministic loops then prove the correctness of the above program.

3. The following statement is supposed to compute the sum of all elements in the array `a` in the domain `0≤k<n`, if started with `n ≥ 0`. This sum will be stored in `s`. Is the program correct (prove it)? If it is not, gives a minimum correction, and proves that it is then correct.

```
j:=n; i:=0; s:=0;

while
  i<j do { s:=s+a[i]; i:=i+1 }
  j ≥ i do { j:=j-1; s:=s+a[j] } ;
```

4. Write a program with the following header:

```
SumLess(READ a:int[], n,m:int) :  bool
```

The program checks if the sum of all elements of an array `a` in the domain $0 \leq i < n$ is less than `m`. It is known that the array contains only positive integers, so you can stop summing if you already know that the current sum already exceeds `m`. Give a formal specification for the program, and prove its correctness.

5. Consider the following program to compute the greatest element in a non-empty array:

```
{∗ n > 0 ∗}

MaxArray(READ a:int[], n:int) : int
  i:int;
  { i:=n-1 ; m:=a[i] ;
    while i>0 do
      {i := i-1;
       if a[i]>m then m:=a[i] else skip } ;
    return m
  }

{∗ return  =  MAX(a[0...n)) ∗}
```

Prove the correctness of the program (by now, this should be quite standard for you).

Let us now extend the program, so that not only we know what the greatest element is, but also where it can be found in `a`. Here is the new program:

```
MaxArray(READ a:int[], n:int, OUT k:int) : int
  i:int;
  { i:=n-1 ; k:=i ; m:=a[i] ;
    while i>0 do
      {i := i-1;
       if a[i]>m then { m:=a[i]; k:=i }
                 else skip
      }  ;
    return i
  }
```

Give a specification for this new program and the prove its correctness. You may want to investigate first, whether it is possible to reuse some of the proof of the old program so that you don't have to prove the new one from scratch.

6. Here is another program to compute the greatest element of a non-empty array.

```
MaxArray(READ a:int[],READ n:int) : int

i,j:int;

{* n > 0 *}

{ i:=0; j:=n-1;
  while i ≠ j do
    { if a[i] ≤ a[j] then i:=i+1 else j:=j-1 } ;
  return x[i]
}

{* return  =  MAX(a[0...n)) *}
```

Prove its correctness. (Hint: it may be easier to use a tail invariant)

7. Write an (efficient) imperative program to compute `fusc n`, where `fusc` is defined below. Prove the correctness of your program.

```
fusc 0      = 0
fusc 1      = 1
fusc 2n     = fusc n
fusc (2n+1) = fusc n + fusc (n+1)
```

`n` is assumed to be ≥ 0.

Hint: calculate `fusc 78` with hand.

8. Give an $O(\log n)$ program to compute $\mathtt{SUM}[\mathtt{x}^{\mathtt{i}} \mid \mathtt{i}\ \mathtt{from}\ [\mathtt{0}\ldots\mathtt{n})]$, where $\mathtt{n} \geq \mathtt{0}$, and then prove the correctness of your program.

9. Let us define this:

$$\mathtt{onlyOne\ b\ m\ n} \;=\; (\exists \mathtt{i} : \mathtt{m} \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}] \;\wedge\; (\forall \mathtt{j} : \mathtt{m} \leq \mathtt{j} < \mathtt{n} : \mathtt{b}[\mathtt{j}] \Rightarrow (\mathtt{i} = \mathtt{j})))$$

So, if `b` is a boolean array, `onlyOne b m n` means that the array contains exactly one true element in the domain $m \leq i < n$. The following program checks is an array satisfies this property. Prove its correctness.

```
CheckOnlyOne(READ b:int[],READ n:int) : bool
  i,j:int;
  { i:=0; j:=n-1;
    while i < j  ∧  ¬(b[i]) do i:=i+1
          i < j  ∧  ¬(b[j]) do j:=i-1 ;
    return ((i = j)  ∧  b[i])
  }
```

10. Let us define this:

$$\texttt{disected b m n} \;=\; (\exists \texttt{i} : \texttt{m} \leq \texttt{i} < \texttt{n} : \texttt{leftFalse b 0 i} \wedge \texttt{rightTrue b i n})$$

where:

$$\texttt{leftFalse b 0 i} \;=\; (\forall \texttt{k} : 0 \leq \texttt{k} < \texttt{i} : \neg(\texttt{b}[\texttt{k}]))$$

and

$$\texttt{rightTrue b i n} \;=\; (\forall \texttt{k} : \texttt{i} \leq \texttt{k} < \texttt{n} : \texttt{b}[\texttt{k}])$$

So, if `b` is a boolean array, `disected b m n` means that the array, in the domain $m \leq i < n$, can be split in two parts: the left part that consists only of `F`'s, and the right part that consists only of `T`'s. The following program checks is an array satisfies this property. Prove its correctness.

```
checkDisected(READ b:int[],READ n:int) : bool
  i,j:int;
  { i:=0; j:=n-1;
    while i < j  ∧  ¬(b[i]) do i:=i+1
          i < j  ∧  b[j] do j:=i-1 ;
    return (i=j)
  }
```

11. The function `f` is defined recursively as follows:

$$\begin{array}{lcl} \texttt{f 0} & = & 1 \\ \texttt{f (2n)} & = & 2 * \texttt{f n} \quad \text{, provided } n > 0 \\ \texttt{f (2n + 1)} & = & \texttt{n} + \texttt{f (2n)} \quad \text{, provided } n \geq 0 \end{array}$$

Write an imperative implementation of `f` with $O(\log n)$ run time.

12. Here is a program that will delete all 0 from an array. The program does not have to preserve the initial ordering of the elements. Give a formal specification, and prove it.

```
DEL0(a:int[]) : int
  i:int
  { i:=0
    while i < n do
      { if a[i]=0 then { a[i]:=a[n]; n:=n-1 } else i:=i+1 } ;
    return n
  }
```

13. We have assumed that arrays in uPL have unbounded size. In reality this is of course not the case. How can we adjust uPL logic if the arrays are now of bounded size? In particular, an expression like `a[i]` may cause a run time error if `i` turns out to be outside the domain of the array `a`. How can we deal with this?

14. Expressions in uPL do not have side effect. In C you have expressions like `x++` and `x--` which will increase (decrease) the value of `x` after evaluating its value. These expressions have side effect, because evaluating it also changes the state of the program.

    Let us assume the following simplified expression sub-language of uPL:

$$
\begin{array}{lcl}
\textit{Expr} & ::= & \textit{VariableName} \mid \textit{Constant} \\
 & \mid & \textit{Expr} \texttt{ + } \textit{Expr} \mid \text{Expr} \texttt{ = } \text{Expr} \\
 & \mid & \textit{VariableName}\texttt{++} \mid \text{VariableName}\texttt{--}
\end{array}
$$

How can we compute the `wp` of an assignment now? How to handle `if-then-else`?

15. We extend `uPL` with exceptions. We introduce two new constructs:

    `abort`
    `try` *Statement* `catch` *Statement*

    If in `try` S_1 `catch` S_2 the execution of S_1 executes `abort` the program jump to the closest encolsing `catch` statement. So if S_1 does not contain further `try` statement, the control will jump to S_2.

    How can we extend the logic for `uPL` to also handle exception?

Bibliography

[1] JUnit test infected: Programmers love writing tests. available at `junit.sourceforge.net`.

[2] J.-R. Abrial. *The B-Book: Assigning Programs to Meanings.* Cambridge University Press, 1996.

[3] Apt. Ten years of hoare's logic: A survey–part I. *ACM Transactions on Programming Languages and Systems*, 3, 1981.

[4] Apt. Ten years of hoare's logic: A survey–part II: Nondeterminism. *TCS: Theoretical Computer Science*, 28, 1984.

[5] P. Audebaud and E. Zucca. Deriving proof rules from continuation semantics. *Formal Asp. Comput*, 11(4):426–447, 1999.

[6] E. Cohen. *Programming in the 1990s, an Introduction to the Calculation of Programs.* Springer-Verlag, 1990.

[7] E.W. Dijkstra. *A discipline of programming.* Prentice-Hall, 1976.

[8] A. Diller. *Z: An Introduction to Formal Methods.* Wiley and Sons, 1994.

[9] R. W. Floyd. Assigning meaning to programs. In J. T. Schwartz, editor, *Mathematical aspects of computer science: Proc. American Mathematics Soc. symposia*, volume 19, pages 19–31, Providence RI, 1967. American Mathematical Society.

[10] J. Fokker, S.D. Swierstra, and H. Zantema. *Programmeren En Correctheid.* Academic Service, 1991.

[11] D. Gries. *The science of computer programming.* Springer-Verlag, 1981.

[12] C. A. R. Hoare. Procedures and parameters: An axiomatic approach. In Erwin Engeler, editor, *Symposium on Semantics of Algorithmic Languages*, volume 188 of *Lecture Notes in Mathematics*, pages 102–116. Springer-Verlag, Berlin, 1971.

[13] C.A.R. Hoare. An axiomatic basis for computers programs. *Commun. Ass. Comput. Mach.*, 12:576–583, 1969.

[14] Marieke Huisman. *Java Program Verification in Higher-Order Logic with PVS and Isabelle.* PhD thesis, University of Nijmegen, The Netherlands, 2001.

[15] M. Huth and M. Ryan. *Logic in Computer Science*. Cambridge, second edition, 2004.

[16] A. Kaldewaij. *Programming, the Derivation of Algorithms*. Prentice Hall, 1990.

[17] N. Klarlund and M.I. Schwartzbach. Verification of pointers, 1997.

[18] G.T. Leavens and Y. Cheon. Design by contract with JML. Draft, available from jmlspecs.org., 2005.

[19] K. Rustan M. Leino, James B. Saxe, and Raymie Stata. Checking java programs via guarded commands. In *Workshop on Object-Oriented Technology*, pages 110–111. Springer-Verlag, 1999.

[20] Carroll Morgan. *Programming from Specifications*. Prentice Hall, 1990.

[21] R.P. Nederpelt. *Logica En Verzamelingen*. Univ. Utrecht, 2001. Dictaat van college Beschrijven en Bewijzen bij Informatica in 2001.

[22] M. Norrish. Deterministic expressions in C. *Lecture Notes in Computer Science*, 1576:147–161, 1999.

[23] Michael Norrish. C formalised in HOL. Technical Report UCAM-CL-TR-453, University of Cambridge, Computer Laboratory, December 1998.

[24] I.S.W.B Prasetya, A. Azurat, T.E.J. Vos, A. van Leeuwen, and H. Suhartanto. Theorem prover supported logics for small imperative languages. Technical Report UU-CS-2005-046, Inst. of Information and Comp. Sciences, Utrecht Univ., 2005.

[25] G. M. Swamy and R. K. Brayton. Incremental formal design verification. In *International Conference on Computer-Aided Design (ICCAD '94)*, pages 458–467, Los Alamitos, Ca., USA, November 1994. IEEE Computer Society Press.

[26] Simon Thompson. *Haskell: The Craft of Functional Programming (2nd Edition)*. Addison-Wesley, 1999.

[27] T.E.J. Vos. *UNITY in Diversity: A Stratified Approach to the Verification of Distributed Algorithms*. PhD thesis, Dept. of Inf. & Comp. Sciences, Utrecht Univ., 2000.

[28] Jos Warmer and Anneke Kleppe. *The Object Constraint Language: Precise Modeling with UML*. Addison Wesley Longman, Reading, Mass., 1999.

Index

∀, 13
∃, 13
`repby`, 102

Abstract Data Type, 151
abstraction, 59
abstraction function, 154
actual parameter, 134
address, 49
ADT, 151
alias, 49, 68, 134
array, 13
assertion, 9, 51
assignment
    simultant, 99
associativity, 11
auxiliary variable, 52

bar notation, 100
black box specification, 52
bound variable, 14
bug, 51

common divisor, 8
compatible, 100
conclusion, 20
COND Conversion, 31
COND Split, 31
conditional, 13
conditional rewrite, 24
conjunction rule, 166
context, 19
correctness, 51
counter example, 51
critical software, 7
critical system, 7

data invariant, 155
deterministic program, 60
distribute over, 40
distributive, 40
divisor, 8
    common, 8
    greatest common, 8
domain
    empty, 13
    full, 13
domain expression, 13
Domain Merging, 30
domain shift, 14
Domain Split, 30

EBNF, 203
    meta symbol, 203
EC, 72
eliminating ∀, 26
eliminating ∃, 27
expression, 11
    array, 13
    conditional, 13
    domain, 13
    quantified
        without domain, 13

formal parameter, 134
formal proof, 10
formula, 11
    conditional, 13
    quantified, 13
        domain of, 13
free variable, 14
fresh name, 26, 28
fresh variable, 26, 28
functional specification, 156

GCD, 8

gcd, 8
goal, 18
grammar, 203
greatest common divisor, 8

Hoare triple, 9
Hoare triples conjunction, 63
Hoare triples disjunction, 63

IC, 72
IL shape, 73
ILF shape, 73
inference rule, 20
informal proof, 10
instantiating ∀, 20
introducing ∀, 26
introducing ∃, 26
invariant, 72

left associative, 11
list
  comprehension, 39
    condition, 39
    generator, 39
    nested, 39
    split, 39
    transformer, 39
  enumeration, 38
listed array, 40
logic, 10, 17
loop reduction rule, 72

model of program, 59

non-deterministic program, 60

parameter
  actual, 134
paramter
  formal, 134
partial correctness, 52
pass by copy-restore, 49
pass by reference, 49
pass by value, 49
pointer, 49
post-condition, 9, 51
post-condition weakening, 62
pre-condition, 9, 51
pre-condition strengthening, 62
predicate, 18
predicate logic, 17
premise, 20
production rule, 203
programming logic, 17
proof
  formal, 10
  informal, 10
proof plan, 78
proof-context, 19
proving, 18

quantifier, 13

Range Split, 30
reference, 49
reference passing, 49
refine, 167
refinement, 167
relation, 60
rewriting, 22
right associative, 11

satisfy, 51
set interpretation of predicate, 61
shifting domain, 14
simultant assignment, 99
simultant substitution, 99
specializing ∀, 20
specification, 8, 50
  black box, 52
    functional, 140
  formal, 50
  informal, 50
  language, 8
  white box, 52
specification language, 8
state, 60
state:final, 60
state:initial, 60
state:terminal, 60
stronger predicate, 18
subproof, 24
substitution, 21
  simultant, 99
superposition, 167, 168
syntax, 203

TC1, 72
TC2, 72
termination metric, 73
testing, 7
total correctness, 52

uEL, 11
universally true, 18
uPL, 47

valid, 18, 51
validation, 156
validation property, 156

value passing, 49
verification, 8, 156

weaker predicate, 18
weakest pre-condition, 63, 64
white box specification, 52
wp, 63

APPENDIX A

Standard Rules and Facts

A.1 Predicate Logic Inference Rules

Rule A.1.1 : Excluded Middle

$$\frac{-}{P \vee \neg P}$$

Rule A.1.2 : Modus Ponens ($\Rightarrow$ Elimination)

$$\frac{\begin{array}{l} P \\ P \Rightarrow Q \end{array}}{Q}$$

Rule A.1.3 : Contradiction

$$\frac{\begin{array}{l} P \\ \neg P \end{array}}{\texttt{false}} \qquad \frac{P \Rightarrow \texttt{false}}{\neg P}$$

Rule A.1.4 : True Consequence

$$\frac{P = \texttt{true}}{P} \qquad \frac{\texttt{true} \Rightarrow P}{P}$$

Rule A.1.5 : $\wedge$ Elimination

$$\frac{P \wedge Q}{Q} \qquad \frac{P \wedge Q}{P}$$

Rule A.1.6 : Conjunction ($\wedge$ Introduction)

$$\frac{\begin{array}{l} P \\ Q \end{array}}{P \wedge Q}$$

Rule A.1.7 : $\vee$ Elimination

$$\frac{\begin{array}{l} \neg P \\ P \vee Q \end{array}}{Q}$$

Rule A.1.8 : $\vee$ Introduction

1. $\dfrac{\neg P \Rightarrow Q}{P \vee Q}$
2. An easier version. But it is weaker, since you may fail to prove Q without the help of $\neg P$:

$$\frac{Q}{P \vee Q}$$

Rule A.1.9 : Case Split

$$\frac{\begin{array}{l} P \Rightarrow Q \\ \neg P \Rightarrow Q \end{array}}{Q} \qquad \frac{\begin{array}{l} P_1 \vee P_2 \\ P_1 \Rightarrow Q \\ P_2 \Rightarrow Q \end{array}}{Q}$$

Rule A.1.10 : Specialization ($\forall$-Elimination)

$$\frac{\begin{array}{l} P\ e \\ (\forall i : P\ i : Q\ i) \end{array}}{Q\ e} \qquad \frac{(\forall i : P\ i : Q\ i)}{P\ e\ \Rightarrow\ Q\ e}$$

Rule A.1.11 : $\exists$ Introduction

$$\frac{P\ e}{(\exists i :: P\ i)} \qquad \frac{\begin{array}{l} P\ e \\ Q\ e \end{array}}{(\exists i : P\ i : Q\ i)}$$

In the last two rules, you should be aware of conditions with regards to the subsitution which is implicit in the notation $P\ e$ —see Section 3.4.

A.2 Rewriting

As with subsitution, there are also similar conditions for rewriting —see Section 3.4. You should be aware of them when using the rules below.

Rule A.2.1 : Rewrite

$$\frac{e_1 \ = \ e_2 \quad P}{P[e_2/e_1]}$$

Rule A.2.2 : Conditional Rewrite

$$\frac{P \Rightarrow (e_1 \ = \ e_2) \quad P \quad Q}{Q[e_2/e_1]}$$

Rule A.2.3 : Rewrite with $\forall$

$$\frac{(\forall i : P\ i :\ e_1\ i = e_2\ i) \quad P\ d \quad Q}{Q[e_2\ d\ /\ e_1\ d]}$$

Rule A.2.4 : Rewrite with T and F

$$\frac{P \quad Q}{Q[\texttt{true}/P]} \qquad \frac{\neg P \quad Q}{Q[\texttt{false}/P]}$$

A.2.1 Rewrite with a Valid Equality

A theorem of the form:

$$\vdash\ e_1 = e_2$$

can be treated as if it is universally quantified over its free variable and is already derived in your current proof-context. Then you can use inference rules such as Rules A.2.1 - A.2.4 to perform rewrite.

Almost similarly, if we have a theorem of the form:

$$\vdash\ P\ \Rightarrow\ (e_1 = e_2)$$

stating a conditioned equality, it can be treated as if it is universally quantified over its free variable and is already derived in your current proof-context.

A.3 Inductions

Rule A.3.1 : Induction on Positive Integers

$$\frac{P\ 0 \quad (\forall n : n \geq 0 : P\ n\ \Rightarrow\ P\ (n+1))}{(\forall n : n \geq 0 : P\ n)}$$

where n is assumed to be of type integer.

Rule A.3.2 : List Induction

$$\frac{P\ [] \quad (\forall x, s :: P\ s \Rightarrow P\ (x : s))}{(\forall s :: P\ s)}$$

A.4 Predicate Logic Rewrite Theorems

A.4.1 Boolean Connectors

Theorem A.4.1 : Basic equalities of Boolean connectors

1. $\vdash \neg\neg P = P$
2. $\vdash P \vee Q = Q \vee P$
3. $\vdash \texttt{true} \vee Q = \texttt{true}$
4. $\vdash \texttt{false} \vee Q = Q$
5. $\vdash (P \vee Q) \vee R = P \vee (Q \vee R) = P \vee Q \vee R$
6. $\vdash P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R)$
7. $\vdash P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R)$
8. $\vdash P \wedge Q = Q \wedge P$
9. $\vdash \texttt{true} \wedge Q = Q$
10. $\vdash \texttt{false} \wedge Q = F$
11. $\vdash (P \wedge Q) \wedge R = P \wedge (Q \wedge R) = P \wedge Q \wedge R$
12. $\vdash P \Rightarrow Q = \neg P \vee Q$
13. $\vdash \neg(P \Rightarrow Q) = P \wedge \neg Q$
14. $\vdash P \Rightarrow Q \Rightarrow R = P \Rightarrow (Q \Rightarrow R) = P \wedge Q \Rightarrow R$
15. $\vdash (P = Q) = (P \Rightarrow Q) \wedge (Q \Rightarrow P)$

Theorem A.4.2 : de Morgan

1. $\vdash \neg(P \vee Q) = \neg P \wedge \neg Q$
2. $\vdash \neg(P \wedge Q) = \neg P \vee \neg Q$

Theorem A.4.3 : Contra Position

$\vdash P \Rightarrow Q = \neg Q \Rightarrow \neg P$

A.4.2 Conditional

Theorem A.4.4 : COND conversion

1. $\vdash \quad P \quad \Rightarrow \quad (P \rightarrow e_1 \mid e_2 \quad = \quad e_1)$
2. $\vdash \quad \neg P \quad \Rightarrow \quad (P \rightarrow e_1 \mid e_2 \quad = \quad e_2)$
3. $\vdash P \rightarrow e \mid e \;=\; e$
4. $\vdash f\ (P \rightarrow e_1 \mid e_2) \;=\; P \rightarrow f\ e_1 \mid f\ e_2$
5. $\vdash P \rightarrow e_1 \mid e_2 \;=\; \neg P \rightarrow e_2 \mid e_1$

Theorem A.4.5 : COND Split

$\vdash \quad P \rightarrow Q \mid R \quad = \quad (P \Rightarrow Q) \wedge (\neg P \Rightarrow R)$

P, Q, and R have to be predicates.

A.4.3 Quantification

Theorem A.4.6 : Renaming bound variables

1. $\vdash (\forall i : P : Q) = (\forall i' : P[i'/i] : Q[i'/i])$
2. $\vdash (\exists i : P : Q) = (\exists i' : P[i'/i] : Q[i'/i])$

i' should not occur free in P and Q.

Theorem A.4.7 : Negate $\forall$

$\vdash \neg(\forall i : P\ i : Q\ i) = (\exists i : P\ i : \neg(Q\ i))$

Theorem A.4.8 : Negate $\exists$

$\vdash \neg(\exists i : P\ i : Q\ i) = (\forall i : P\ i : \neg(Q\ i))$

Theorem A.4.9 : Nested quantifications

1. $\vdash (\forall i, j :: P\ i\ j) = (\forall i :: (\forall j :: P\ i\ j))$
2. $\vdash (\exists i, j :: P\ i\ j) = (\exists i :: (\exists j :: P\ i\ j))$

Theorem A.4.10 : Quantification over singleton domain

1. $\vdash (\forall i : i = e : P\ i) = P\ e$
2. $\vdash (\exists i : i = e : P\ i) = P\ e$

Theorem A.4.11 : Range Split

1. $\vdash (\forall i : P\ i : Q_1\ i \wedge Q_2\ i) = (\forall i : P\ i : Q_1\ i) \wedge (\forall i : P\ i : Q_2\ i)$
2. $\vdash (\exists i : P\ i : Q_1\ i \vee Q_2\ i) = (\exists i : P\ i : Q_1\ i) \vee (\exists i : P\ i : Q_2\ i)$

Theorem A.4.12 : Domain split

1. $\vdash (\forall i : P\ i \vee Q\ i : R\ i) = (\forall i : P\ i : R\ i) \wedge (\forall i : Q\ i : R\ i)$
2. $\vdash (\exists i : P\ i \vee Q\ i : R\ i) = (\exists i : P\ i : R\ i) \vee (\exists i : Q\ i : R\ i)$

Theorem A.4.13 : Quantification over empty domain

1. $\vdash (\forall i : \texttt{false} : P\ i) = \texttt{true}$
2. $\vdash (\exists i : \texttt{false} : P\ i) = \texttt{false}$

Theorem A.4.14 : Domain Shift

1. $\vdash (\forall i : P\ i : Q\ i) = (\forall i :: P\ i \Rightarrow Q\ i)$
2. $\vdash (\forall i : P_1\ i \wedge P_2\ i : Q\ i) = (\forall i : P_1\ i : P_2\ i \Rightarrow Q\ i)$
3. $\vdash (\exists i : P\ i : Q\ i) = (\exists i :: P\ i \wedge Q\ i)$
4. $\vdash (\exists i : P_1\ i \wedge P_2\ i : Q\ i) = (\exists i : P_1\ i : P_2\ i \wedge Q\ i)$

A.4.4 Domains

Theorem A.4.15 : +1 Conversion
Let a, b, and i be integers.

1. $i{\leq}b\ =\ i{<}b{+}1$
2. $a{+}1{\leq}i\ =\ a{<}i$

Theorem A.4.16 : Domain Merging
Let a, b, and i be integers.

1. $\vdash\ a \leq b\ \Rightarrow\ (a{\leq}i{<}b{+}1\ =\ a{\leq}i{<}b \vee (i = b))$
2. $\vdash\ a \leq b\ \Rightarrow\ (a{\leq}i{\leq}b\ =\ a{\leq}i{<}b \vee (i = b))$
3. $\vdash\ a \leq b\ \wedge\ b \leq c\ \Rightarrow\ (a{\leq}i{<}b \vee b{\leq}i{<}c\ =\ a{\leq}i{<}c)$

Theorem A.4.17 : Empty Domain Conversion
Let a, b, and i be integers.

1. $\vdash \neg(a{\leq}i{<}a)$
2. $\vdash \neg(a{\leq}i{<}a)$
3. $\vdash b \leq a\ \Rightarrow\ (a{\leq}i{<}b = \texttt{false})$
4. $\vdash b \leq a\ \Rightarrow\ (a{\leq}i{<}b = \texttt{false})$

A.5 Lists

Definition A.5.1 : $+\!\!+$ AND $\in$

$$\begin{array}{lcl} [] +\!\!+ t & = & t \\ (x : s) +\!\!+ t & = & x : (s +\!\!+ t) \\ \\ x \in [] & = & \texttt{false} \\ x \in (y : s) & = & (x = y) \vee x \in s \end{array}$$

Theorem A.5.2 : NON-EMPTINESS OF LIST

1. $\vdash\ (t \neq []) = (\exists x, s ::\ t = x : s)$
2. $\vdash\ (t \neq []) = (\exists s, y ::\ t = s +\!\!+ [y])$

Definition A.5.3 : SUM, COUNT, MAX, AND MIN

$$\begin{array}{lcl} \texttt{SUM}\ [] & = & 0 \\ \texttt{SUM}\ (x : s) & = & x + \texttt{SUM}\ s \\ \\ \texttt{COUNT}\ [] & = & 0 \\ \texttt{COUNT}\ (x : s) & = & 1 + \texttt{COUNT}\ s \\ \\ \texttt{MAX}\ [x] & = & x \\ \texttt{MAX}\ (x : s) & = & x\ \texttt{max}\ \texttt{MAX}\ s \qquad \text{, provided } s \text{ is non-empty.} \\ \\ \texttt{MIN}\ [x] & = & x \\ \texttt{MIN}\ (x : s) & = & x\ \texttt{min}\ \texttt{MIN}\ s \qquad \text{, provided } s \text{ is non-empty.} \end{array}$$

Definition A.5.4 : map AND filter

$$\begin{array}{lcl} \texttt{map}\ f\ [] & = & [] \\ \texttt{map}\ f\ (x : s) & = & f\ x : \texttt{map}\ f\ s \\ \\ \texttt{filter}\ p\ [] & = & [] \\ \texttt{filter}\ p\ (x : s) & = & (p\ x \rightarrow [x] \mid []) +\!\!+ \texttt{filter}\ p\ s \end{array}$$

Theorem A.5.5 : EMPTY ENUMERATION

1. $\vdash\ j \leq i\ \Rightarrow\ ([i \ldots j) = [])$
2. $\vdash\ [i \ldots i) = []$

Theorem A.5.6 : SINGLETON ENUMERATION

$\vdash\ [i \ldots i + 1) = [i]$

Theorem A.5.7 : ENUMERATION MEMBERSHIP

$\vdash\ j \in [i \ldots k)\ =\ i \leq j < k$

Theorem A.5.8 : ENUMERATION SPLIT

1. $i \leq j\ \Rightarrow\ ([i \ldots j + 1)\ =\ [i \ldots j) +\!\!+ [j])$
2. $i \leq j \leq k\ \Rightarrow\ ([i \ldots k)\ =\ [i \ldots j) +\!\!+ [j \ldots k))$

Theorem A.5.9 : EMPTY COMPREHENSION

1. $\vdash\ [e\ i \mid i\ \texttt{from}\ [],\ P\ i]\ =\ []$
2. $\vdash\ [e\ i \mid i\ \texttt{from}\ s,\ \texttt{false}]\ =\ []$
3. $\vdash\ ([e\ i \mid i\ \texttt{from}\ s,\ \texttt{P i}] = [])\ =\ (\forall i : i \in s : \neg(P\ i))$

Theorem A.5.10 : SINGLETON COMPREHENSION

1. $\vdash\ [e\ i \mid i\ \texttt{from}\ [x]]\ =\ [e\ x]$
2. $\vdash\ [e\ i \mid i\ \texttt{from}\ [x],\ P\ i]\ =\ P\ x \rightarrow [e\ x] \mid []$

Theorem A.5.11 : Comprehension Membership

1. $\vdash\ x \in [e\ i \mid i\ \texttt{from}\ s,\ P\ i]\ =\ (\exists i : i \in s \wedge P\ i : x = e\ i)$
2. $\vdash\ x \in [i \mid i\ \texttt{from}\ s,\ P\ i]\ =\ x \in s \wedge P\ x$

Theorem A.5.12 : Comprehension Split

$\vdash\ [e \mid i\ \texttt{from}\ (s +\!\!+\ t),\ P]\ =\ [e \mid i\ \texttt{from}\ s,\ P] +\!\!+ [e \mid i\ \texttt{from}\ t,\ P]$

Theorem A.5.13 : Nested Comprehension

1. $\vdash\ [e_1\ i \mid i\ \texttt{from}\ [e_2\ j \mid j\ \texttt{from}\ s,\ P\ j],\ Q\ i]$
   $=$
   $[e_1\ (e_2\ j) \mid j\ \texttt{from}\ s,\ P\ j\ \wedge\ Q\ (e_2\ j)]$
2. $\vdash\ [e_1\ i \mid i\ \texttt{from}\ [e_2\ j \mid j\ \texttt{from}\ s,\ P\ j]]\ =\ [e_1\ (e_2\ j) \mid j\ \texttt{from}\ s,\ P\ j]$

Definition A.5.14 : Listing Array Elements
Let s be a list (of integers) and $\texttt{a}$ be an array.

$$\texttt{a}\ s\ =\ [\texttt{a}[i] \mid i\ \texttt{from}\ s]$$

Theorem A.5.15 : Split over Array

1. $i \leq j\ \Rightarrow\ (a[i \ldots j+1)\ =\ a[i \ldots j) +\!\!+ [a[j]])$
2. $i \leq j \leq k\ \Rightarrow\ (a[i \ldots k)\ =\ a[i \ldots j) +\!\!+ a[j \ldots k))$

Theorem A.5.16 : Homomorphism of List Functions

1. $\vdash\ x \in (s +\!\!+\ t)\ =\ x \in s\ \vee\ x \in t$
2. $\vdash\ \texttt{SUM}\ (s +\!\!+\ t)\ =\ \texttt{SUM}\ s + \texttt{SUM}\ t$
3. $\vdash\ \texttt{COUNT}\ (s +\!\!+\ t)\ =\ \texttt{COUNT}\ s + \texttt{COUNT}\ t$
4. $\vdash\ \texttt{map}\ f\ (s +\!\!+\ t)\ =\ \texttt{map}\ f\ s +\!\!+ \texttt{map}\ f\ t$
5. $\vdash\ \texttt{filter}\ p\ (s +\!\!+\ t)\ =\ \texttt{filter}\ p\ s +\!\!+ \texttt{filter}\ p\ t$

6. $\vdash\ s \neq [] \wedge t \neq []\ \Rightarrow\ (\texttt{MAX}\ (s +\!\!+\ t)\ =\ \texttt{MAX}\ s\ \texttt{max}\ \texttt{MAX}\ t)$
7. $\vdash\ s \neq [] \wedge t \neq []\ \Rightarrow\ (\texttt{MIN}\ (s +\!\!+\ t)\ =\ \texttt{MIN}\ s\ \texttt{min}\ \texttt{MIN}\ t)$

APPENDIX B

The uPL Logic

B.1 Inference Rules

Rule B.1.1 : Post-condition Weakening

$$\frac{\models Q \Rightarrow Q' \quad \{*\ P\ *\}\ S\ \{*\ Q\ *\}}{\{*\ P\ *\}\ S\ \{*\ Q'\ *\}}$$

Rule B.1.2 : Pre-condition Strengthening

$$\frac{\models P \Rightarrow P' \quad \{*\ P'\ *\}\ S\ \{*\ Q\ *\}}{\{*\ P\ *\}\ S\ \{*\ Q\ *\}}$$

Rule B.1.3 : Hoare Triple Disjunction

$$\frac{\{*\ P_1\ *\}\ S\ \{*\ Q_1\ *\} \quad \{*\ P_2\ *\}\ S\ \{*\ Q_2\ *\}}{\{*\ P_1 \vee P_2\ *\}\ S\ \{*\ Q_1 \vee Q_2\ *\}}$$

Rule B.1.4 : Hoare Triple Conjunction

$$\frac{\{*\ P_1\ *\}\ S\ \{*\ Q_1\ *\} \quad \{*\ P_2\ *\}\ S\ \{*\ Q_2\ *\}}{\{*\ P_1 \wedge P_2\ *\}\ S\ \{*\ Q_1 \wedge Q_2\ *\}}$$

Rule B.1.5 : If-Then-Else

$$\frac{\{*\ P \wedge g\ *\}\ S_1\ \{*\ Q\ *\} \quad \{*\ P \wedge \neg g\ *\}\ S_2\ \{*\ Q\ *\}}{\{*\ P\ *\}\quad \texttt{if}\ g\ \texttt{then}\ S_1\ \texttt{else}\ S_2 \quad \{*\ Q\ *\}}$$

Rule B.1.6 : SEQ

$$\frac{\{*\ P\ *\}\ S_1\ \{*\ P'\ *\} \quad \{*\ P'\ *\}\ S_2\ \{*\ Q\ *\}}{\{*\ P\ *\}\quad (S_1;\ S_2) \quad \{*\ Q\ *\}}$$

Rule B.1.7 : Loop Reduction

$$\frac{\begin{array}{lcl} \textbf{TC1} & : & \{*\ I \wedge g\ *\}\ (C := m; S)\ \{*\ m < C\ *\} \\ \textbf{TC2} & : & \models I \wedge g \Rightarrow m > 0 \\ \textbf{IC} & : & \{*\ I \wedge g\ *\}\ S\ \{*\ I\ *\} \\ \textbf{EC} & : & \models I \wedge \neg g \Rightarrow Q \end{array}}{\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\}}$$

Rule B.1.8 : Loop Reduction

$$\frac{\begin{array}{lcl} \textbf{InitC} & : & \models P \Rightarrow I \\ \textbf{TC1} & : & \{*\ I \wedge g\ *\}\ \ (C := m; S)\ \ \{*\ m < C\ *\} \\ \textbf{TC2} & : & \models I \wedge g \Rightarrow m > 0 \\ \textbf{EC} & : & \models I \wedge \neg g \Rightarrow Q \\ \textbf{IC} & : & \{*\ I \wedge g\ *\}\ \ S\ \ \{*\ I\ *\} \end{array}}{\{*\ P\ *\}\ \texttt{while}\ \ g\ \texttt{do}\ S\ \ \{*\ Q\ *\}}$$

Rule B.1.9 : Direct Termination

$$\frac{-}{\{*\ Q \wedge \neg g\ *\}\ \ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\}}$$

B.2 Weakest Pre-condition

Definition B.2.1 : Characterization of wp

$$\models P \Rightarrow \mathsf{wp}\ S\ Q \quad = \quad \{*\ P\ *\}\ S\ \{*\ Q\ *\}$$

Corollary B.2.2 :

$$\{*\ \mathsf{wp}\ S\ Q\ *\}\ S\ \{*\ Q\ *\}$$

Theorem B.2.3 : wp Distributivity

1. $\models \mathsf{wp}\ S\ (Q_1 \wedge Q_2)\ =\ (\mathsf{wp}\ S\ Q_1) \wedge (\mathsf{wp}\ S\ Q_2)$
2. If S is deterministic, we have:

$$\models \mathsf{wp}\ S\ (Q_1 \vee Q_2)\ =\ (\mathsf{wp}\ S\ Q_1) \vee (\mathsf{wp}\ S\ Q_2)$$

Theorem B.2.4 : wp of skip

$$\mathsf{wp}\ \texttt{skip}\ Q\ =\ Q$$

Theorem B.2.5 : wp of if-then-else

$$\mathsf{wp}\ (\texttt{if}\ g\ \texttt{then}\ S_1\ \texttt{else}\ S_2)\ Q \quad = \quad g \rightarrow \mathsf{wp}\ S_1\ Q \mid \mathsf{wp}\ S_2\ Q$$

Theorem B.2.6 : wp of Statements Sequence

$$\mathsf{wp}\ (S_1;S_2)\ Q \quad = \quad \mathsf{wp}\ S_1\ (\mathsf{wp}\ S_2\ Q)$$

Theorem B.2.7 : wp of Assignment

1. Assignment targeting an identifier (a variable name):

$$\mathsf{wp}\ (v{:=}E)\ Q \quad = \quad Q[E/v]$$

2. Assignment targeting an array's element:

$$\mathsf{wp}\ (a[i]{:=}e)\ Q\ =\ Q[a(i\ \texttt{repby}\ e)/a]$$

where $a(i\ \texttt{repby}\ e)$ is defined as follows:

$$a(i\ \texttt{repby}\ e)[j]\ =\ (i = j) \rightarrow e \mid a[j]$$

3. Assignment targeting a record's field:

$$\texttt{wp}\ (r.fn{:=}e)\ Q \ = \ Q[r(fn\ \texttt{repby}\ e)/r]$$

where `repby` is defined as follows:

$$\begin{array}{lcll} r(fn\ \texttt{repby}\ e).fn & = & e & \\ r(fn\ \texttt{repby}\ e).gn & = & r.gn & \text{, if } fn \neq gn \end{array}$$

Theorem B.2.8 : Program to Statement Reduction
Let Pr be defined as below; x, y are its formal parameters; x is passed by-value and y by copy-restore; X, Y are auxilliary paremeters used to record x, y's initial values. Let this be a *closed* specification:

$$\{*\ P\ *\}\ \ X, Y{:=}x, y;\ Pr(x, \texttt{OUT}\ y)\ \{\texttt{var}\ z;\ S;\ \texttt{return}\ e\ \}\ \ \{*\ Q\ *\}$$

It is valid if and only if the following statement is valid:

$$\{*\ P\ *\}\ \ \{\ X, Y{:=}x, y;\ S;\ \texttt{return}{:=}e\ \}\ \ \{*\ Q[X/x]\ *\}$$

B.3 Program Call

uPL Calling Convention

An array parameter is passed by copy-restore unless it is marked by a `READ` keyword. Parameters of other types are passed by value unless they are marked by an `OUT` keyword. Copy-restore parameters are restored in the same order as they appear in the parameters list.

Rule B.3.1 : Renaming
Let $\overline{x}$, $\overline{y}$, $\overline{X}$ and $\overline{Y}$ be lists of distinct variables. The latter two are auxilliary variables.

$$\frac{\{*\ P\ *\}\quad \overline{X}{:=}\overline{x}\ ;\ Pr(\overline{x})\quad \{*\ Q\ *\}}{\{*\ P[\overline{y}/\overline{x}]\ *\}\quad \overline{Y}{:=}\overline{y}\ ;\ Pr(\overline{y})\quad \{*\ Q[\overline{y}/\overline{x}, \overline{Y}/\overline{X}]\ *\}}$$

Rule B.3.2 : Black Box Reduction
Let Pr be a uPL program. Let $\overline{x}$, $\overline{X}$, and $\overline{x'}$ be compatible lists of variables; variables in $\overline{x'}$ are fresh and distinct.

$$\frac{\{*\ P\ *\}\quad \overline{X} := \overline{x};\ \texttt{Pr}(\overline{x})\quad \{*\ Q\ *\}}{\{*\ P \wedge Q'\ *\}\quad \texttt{Pr}(\overline{x})\quad \{*\ R\ *\}}$$

where

$$Q' \ = \ (Q \Rightarrow R)\ [\overline{x'}//\overline{x}]\ [\overline{x}/\overline{X}]$$

and $\overline{x'}//\overline{x}$ is a substitution that replaces x_i with x'_i only if x_i is a pass-by-copy-restore parameter of `Pr`.
□

Rule B.3.3 : Functional Box Reduction I
Let Pr be a functional uPL program and $\overline{x}$ be a list of variables.

$$\frac{\{*\ P\ *\}\ \texttt{Pr}(\overline{x})\ \{*\ \texttt{return} = e\ *\}}{\{*\ P \wedge R[e/r]\ *\}\ r{:=}Pr(\overline{x})\ \{*\ R\ *\}}$$

Rule B.3.4 : Functional Box Reduction II
Let Pr be a functional uPL program, $\overline{x}$ be a list of variables, and $\overline{d}$ be a list of expressions.

$$\frac{\{*\ P\ *\}\ \texttt{Pr}(\overline{x})\ \{*\ \texttt{return} = e\ *\}}{\{*\ P[\overline{d}/\overline{x}] \wedge R[e'/r]\ *\}\ r{:=}Pr(\overline{d})\ \{*\ R\ *\}}$$

where $e' = e[\overline{d}/\overline{x}]$.

Rule B.3.5 : REC REDUCTION
Let $\overline{x}$ be Pr's formal parameters. Let n be an integer expression.

$$\frac{\begin{array}{ll}\textbf{BC1}: & P \Rightarrow n \geq 0 \\ \textbf{BC2}: & \{*\ P \wedge (n=0)\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\{*\ Q\ *\} \\ \textbf{RC}: & \{*\ P \wedge n < K\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\ \ \{*\ Q\ *\} \\ & \text{implies} \\ & \{*\ P \wedge (n=K) \wedge K > 0\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\{*\ Q\ *\}\end{array}}{\{*\ P\ *\}\ \ \overline{X} := \overline{x};\ Pr(\overline{x})\ \ \{*\ Q\ *\}}$$

Rule B.3.6 : REC REDUCTION II
Let Pr be a *functional* uPL program; $\overline{x}$ are its formal parameters, and $n \in \overline{x}$. Let $e(K)$ denote $e[K/n]$.

$$\frac{\begin{array}{ll}\textbf{BC1}: & P \Rightarrow n{\geq}0 \\ \textbf{BC2}: & \{*\ P \wedge (n=0)\ *\}\ Pr(\overline{x})\ \{*\ \texttt{return} = e(0)\ *\} \\ \textbf{RC}: & \{*\ P \wedge (n=K)\ *\}\ Pr(\overline{x})\ \{*\ \texttt{return} = e(K)\ *\} \\ & \text{implies} \\ & \{*\ P \wedge (n=K{+}1) \wedge K{\geq}0\ *\}\ Pr(\overline{x})\ \{*\ \texttt{return} = e(K{+}1)\ *\}\end{array}}{\{*\ P\ *\}\ Pr(\overline{x})\ \{*\ \texttt{return} = e\ *\}}$$

B.4 Program Transformation

Definition B.4.1 : REFINEMENT

$$\begin{array}{l} S \sqsubseteq_V T \\ = \\ (\forall P, Q: \ \ \mathsf{free}(P) \subseteq V \ \ \wedge \ \ \mathsf{free}(Q) \subseteq V \ \ : \\ \qquad\qquad \{*\ P\ *\}\ S\ \{*\ Q\ *\} \ \ \Rightarrow \ \ \{*\ P\ *\}\ T\ \{*\ Q\ *\}) \end{array}$$

Definition B.4.2 : EQUIVALENCE

$$S \equiv_V T \quad = \quad S \sqsubseteq_V T \ \wedge \ T \sqsubseteq_V S$$

Theorem B.4.3 :
If $\mathsf{free}(P) \subseteq V$ and $\mathsf{free}(Q) \subseteq V$ and $S \equiv_V T$, we have:

$$\{*\ P\ *\}\ S\ \{*\ Q\ *\} \quad = \quad \{*\ P\ *\}\ T\ \{*\ Q\ *\}$$

Theorem B.4.4 :

$$S \equiv_V T \ \ \wedge \ \ U \subseteq V \ \ \Rightarrow \ \ S \equiv_U T$$

Theorem B.4.5 : EQUIVALENCE BY SUPERPOSITION
Let S' be obtained from superpositioning S with assignments to fresh variables. Let Z be the set of these fresh variables. For any V such that $V \cap Z = \emptyset$, we have:

$$S \quad \equiv_{\texttt{var}(S) \cup V} \quad S'$$

Rule B.4.6 : LOOP SUPERPOSITION
Let S' be obtained from superpositioning S with assignments to fresh variables. Let Z be the set of these fresh variables. Z is assumed to have no intersection with $\mathsf{free}(g) \cup \texttt{var}(S) \cup \mathsf{free}(I) \cup \mathsf{free}(Q)$. We have:

$$\frac{\begin{array}{l}\{*\ I\ *\}\ \ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\} \\ \{*\ I \wedge g\ *\}\ S\ \{*\ I\ *\} \\ \{*\ I \wedge I' \wedge g\ *\}\ S'\ \{*\ I'\ *\} \\ \vdash I \wedge I' \wedge \neg g \Rightarrow Q'\end{array}}{\{*\ I \wedge I'\ *\}\ \ \texttt{while}\ g\ \texttt{do}\ S'\ \{*\ Q \wedge Q'\ *\}}$$

Rule B.4.7 : LOOP STRENGTHENING I

$$\frac{\begin{array}{l}\{*\ I\ *\}\ \ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\} \\ \{*\ I \wedge g\ *\}\ S\ \{*\ I\ *\} \\ \models I \wedge g \wedge \neg h \Rightarrow Q\end{array}}{\{*\ I\ *\}\ \ \texttt{while}\ g \wedge h\ \texttt{do}\ S\ \{*\ Q\ *\}}$$

Rule B.4.8 : Loop Strengthening II

$$\frac{\begin{array}{l}\{*\ I\ *\}\ \texttt{while}\ g\ \texttt{do}\ S\ \{*\ Q\ *\} \\ \{*\ I \wedge g\ *\}\ S\ \{*\ I\ *\} \\ \{*\ I \wedge I' \wedge g\ *\}\ S\ \{*\ I'\ *\} \\ \models I \wedge I' \wedge g \wedge \neg h \Rightarrow Q\end{array}}{\{*\ I \wedge I'\ *\}\ \texttt{while}\ g \wedge h\ \texttt{do}\ S\ \{*\ Q\ *\}}$$

APPENDIX C

uPL Grammar

The grammar (syntax) of uPL is shown in Figures C.1 and C.2. The grammar in written in the EBNF notation.

EBNF

EBNF stands for *Extended Backus-Naur Form*. It is a widely used notation invented by John Backus and Peter Naur to precisely describe computer languages. Below is a brief summary of the notation —you can Google "EBNF" to find a more extensive introduction to it.

A language is described by a *grammar*, which in turns consists of *production rules*. For example, the following production rule is part of uPL's grammar:

$\langle$*program*$\rangle$ ::= $\langle$*header*$\rangle$ $\langle$*body*$\rangle$

It tells us how to write a uPL program. It says that a program consists of a header, followed by a body.

The following production rule describes the syntax of statement in uPL:

$\langle$*statement*$\rangle$ ::= `skip`
| $\langle$*assignment*$\rangle$
| $\langle$*if-then-else*$\rangle$

It says that a statement is either a `skip` or an assignment, or an if-then-else construct. The above rule is not complete though. In uPL we also have while-loop, program call, etc.

The symbol | above is a *meta symbol*: it is part of the EBNF notation for describing languages, and is not part of the language being described (in this case uPL). If A and B are two EBNF fractions describing some language constructs, then $A \mid B$ means that in the place where it apears you can either use A or B.

There are more meta symbols: $*$, $+$, and ?. If C is a construct, $C*$ means zero or more repetition of C is allowed; $C+$ means one or more repetition of C is allowed; and C? means zero or one use of C.

For example the production rules below say that a *formal-parameter-list* is either empty, or a non-empty list of formal-patameters separated by commas.

$\langle$*formal-parameter-list*$\rangle$ ::= $\langle$*empty*$\rangle$
| $\langle$*formal-parameter*$\rangle$ (',' $\langle$*formal-parameter*$\rangle$)*

Binary Operators

Avaiable binary operators are shown in the table below, listed in the decreasing order of their priority (so, the top row lists the highest priority operators). The L flag means that the corresponding operator is left-associative; R means that it is right associative; and LR means that the operator is both left and right associative. Operators with no L/R flag is not associative.

`^` (L)
`*` (LR)
`+` (LR), `-` (L)
`mod`, `div`, `max` (LR), `min` (LR)
$<$, $>$, $\neq$, $\leq$, $\geq$
$\wedge$ (LR)
$\vee$ (LR)
`=`

⟨*program*⟩ ::= ⟨*header*⟩ ⟨*body*⟩

⟨*header*⟩ ::= ⟨*program-name*⟩ '(' ⟨*formal-parameter-list*⟩ ')' ':' ⟨*type*⟩

⟨*formal-parameter-list*⟩ ::= ⟨*empty*⟩
| ⟨*formal-parameter*⟩ (',' ⟨*formal-parameter*⟩)*

⟨*formal-parameter*⟩ ::= `OUT`? `READ`? ⟨*var-decl*⟩

⟨*var-decl*⟩ ::= ⟨*var-name*⟩ (',' ⟨*var-name*⟩)* ':' ⟨*type*⟩

⟨*body*⟩ ::= '{' ⟨*locvar-decl-list*⟩ ⟨*statement-sequence*⟩ ⟨*return*⟩ '}'

⟨*locvar-decl*⟩ ::= `var` ⟨*var-decl*⟩

⟨*locvar-decl-list*⟩ ::= (⟨*locvar-decl*⟩ ';')*

⟨*return*⟩ ::= ⟨*empty*⟩ | ';' `return` ⟨*expr*⟩

⟨*statement*⟩ ::= `skip`
| ⟨*assignment*⟩
| '{' ⟨*locvar-decl-list*⟩ ⟨*statement-sequence*⟩ '}'
| ⟨*if-then-else*⟩
| ⟨*while-loop*⟩
| ⟨*program-call*⟩

⟨*assignment*⟩ ::= ⟨*target*⟩ `:=` ⟨*expr*⟩

⟨*target*⟩ ::= ⟨*variable*⟩ | ⟨*array-element*⟩ | ⟨*record-element*⟩

⟨*statement-sequence*⟩ ::= ⟨*statement*⟩ (';' ⟨*statement*⟩)*

⟨*if-then-else*⟩ ::= `if` ⟨*expr*⟩ `then` ⟨*statement*⟩ `else` ⟨*statement*⟩

⟨*while-loop*⟩ ::= `while` ⟨*expr*⟩ `do` ⟨*statement*⟩

⟨*program-call*⟩ ::= ⟨*call*⟩ | ⟨*target*⟩ `:=` ⟨*call*⟩

⟨*call*⟩ ::= ⟨*program-name*⟩ '(' ⟨*actual-parameter-list*⟩ ')'

⟨*actual-parameter-list*⟩ ::= ⟨*empty*⟩ | ⟨*expr*⟩ (',' ⟨*expr*⟩)*

Figure C.1: *The grammar of* uPL*: program and statement.*

⟨*type*⟩ ::= ⟨*simple-type*⟩ | ⟨*record-type*⟩ | ⟨*array-type*⟩ | ⟨*type-name*⟩

⟨*simple-type*⟩ ::= `()` | `bool` | `int` | `char` | `string`

⟨*record-type*⟩ ::= `record` '{' ⟨*field-def*⟩ (';' ⟨*field-def*⟩)* '}'

⟨*field-def*⟩ ::= ⟨*field-name*⟩ (',' ⟨*field-name*⟩)* ':' ⟨*simple-type*⟩

⟨*array-type*⟩ ::= (⟨*simple-type*⟩ | ⟨*record-type*⟩)`[]`+

⟨*expr*⟩ ::= ⟨*constant*⟩
| ⟨*variable-name*⟩
| ⟨*array-element*⟩
| ⟨*record-element*⟩
| ¬⟨*expr*⟩
| ⟨*expr*⟩ ⟨*binary-operator*⟩ ⟨*expr*⟩
| '(' ⟨*expr*⟩ ')'

⟨*array-element*⟩ ::= ⟨*variable-name*⟩ ('[' ⟨*expr*⟩ ']')+

⟨*record-element*⟩ ::= ⟨*expr*⟩ '.' ⟨*field-name*⟩

Figure C.2: uPL *types and expressions*